



Carteira de Identidade Profissional - CFT
Lei nº 13.639, de 26 de MARÇO de 2018

CRT SP

Conselho Federal dos Técnicos Industriais



CONTROLADOR
CNC
3EIXOS
CLP
ST- STRUCTURED
TEXT
ABNT NBR IEC
61131



Carteira de Identidade Profissional - CFT
Lei nº 13.639, de 26 de MARÇO de 2018

CRT SP

Conselho Federal dos Técnicos Industriais

República Federativa do Brasil
Serviço Público Federal
Conselho Federal dos Técnicos Industriais
Conselho Regional dos Técnicos Industriais

CRT SP

2024

Nome
ALEJANDRO GARCIA DE GRACIA

Data de Registro
25/10/2022

Título Profissional
TÉCNICO EM MECÂNICA

Registro Nacional
0010004000

Data de Emissão
05/01/2024

Assinatura do Profissional

República Federativa do Brasil
Serviço Público Federal
Conselho Federal dos Técnicos Industriais
Conselho Regional dos Técnicos Industriais

CRT SP

Filiação
DOLORES GARCIA VILCHES
JOSE LUIZ DE GRACIA CRUZ

CPF
851.186.649-34

Doc. de Identidade
20512267

Nascimento
13/11/1981

Nacionalidade
BRASILEIRA

Naturalidade
BARCELONETA/SP

Assinatura do Profissional
RUBERTO NUNO SACANETO

Assinatura do Conselho
RUBERTO NUNO SACANETO

Assinatura do Conselho
RUBERTO NUNO SACANETO

CONTROLADOR

CNC

3 EIXOS



APRESENTAÇÃO

O presente trabalho aborda o **conceito de programação aplicado aos objetos de controle do perfil de dispositivos CiA 402 em CANopen**.

O CiA 402 representa um **vocabulário padronizado** para o controle de servoacionamentos, contando com um **dicionário comum** ao qual todos os drives compatíveis se adequam. Cada objeto possui o mesmo **índice (Object Index)** definido pela especificação, garantindo que o **significado dos dados** seja preservado independentemente do fabricante. Trata-se, portanto, de um **modelo padronizado de controle para drives industriais**, mantendo a **semântica** consistente mesmo em diferentes protocolos de comunicação, por meio de objetos padronizados, cada um definido por um **tipo de dado específico**.

Os objetos do CiA 402 encontram-se implementados no **firmware do drive**, sendo utilizados internamente durante a parametrização e a operação do equipamento. Esses objetos podem ser acessados externamente por meio de **protocolos de comunicação**, como CANopen e EtherCAT.

A programação utilizada no presente projeto é o **Texto Estruturado (Structured Text – ST)**, conforme a norma **IEC 61131-3**, empregando a semântica própria da linguagem para manipular **bits individuais e demais tipos de dados**, adequando-os às características específicas de cada objeto do CiA 402.

O projeto contempla o **conceito de Controle Numérico Computadorizado (CNC)**, utilizando **servoacionamentos** e funções de programação capazes de produzir dados organizados em trajetórias nos eixos **X, Y e Z**.

A execução do controle é intermediada por uma função interpretadora denominada **GRACIA_code**, desenvolvida neste trabalho, que permite escrever instruções sequenciais de movimentação e funções utilitárias destinadas ao processo de usinagem. Diversos programas, armazenados em arquivos **.txt**, podem ser enviados ao **CLP** por meio da interface gráfica do software de engenharia, responsável pela configuração do **mestre da rede EtherCAT** e de seus periféricos.

No presente projeto, o **CLP atua como mestre da rede EtherCAT**, configurando **três servoacionadores** como escravos, diretamente integrados às lógicas de programação implementadas nas diversas funções desenvolvidas para o controle numérico computadorizado.



8.3 POSITION CONTROL FUNCTION

Este grupo de objetos é utilizado para descrever o funcionamento do controlador de posição em malha fechada.

8.3.1 Objeto 6063h – Position internal actual value

Representa a posição atual do eixo do motor em incrementos. Uma volta completa representa 65536 incrementos.

Índice	Subíndice	Nome	Tipo	Acesso	PDO Mapping
6063h	0	Position actual value	INT32	ro	Sim

O valor deste objeto representa sempre a posição de eixo em uma volta apenas. O número de voltas não é controlado por este objeto.

8.3.2 Objeto 6064h – Position Actual Value /RESOLVER P360 _65537 608F pulsos/volta

Representa a posição atual do eixo do motor. O valor deste objeto pode ser transformado de unidades internas para valores definidos pelo usuário, de acordo com o programado nos objetos 608Fh, 6091h e 6092h, conforme tabela

8.7.

Índice	Subíndice	Nome	Tipo	Acesso	PDO Mapping
6064h	0	Position Actual Value in User Units	INT32	ro	Sim

O valor deste objeto representa sempre a posição de eixo em uma volta apenas. O número de voltas não é controlado por este objeto.

Tabela 8.7: Programação dos objetos Factor Group

Objeto	608Fh	608Fh	6091h	6091h	6092h	6092h
Unidade	sub-índice 1	sub-índice 2	sub-índice 1	sub-índice 2	sub-índice 1	sub-índice 2
Graus	41h	1	1	1	41h	1
Minutos	42h	1	1	1	42h	1
Segundos	43h	1	1	1	43h	1
Unidade interna	FFh	1	1	1	FFh	1

8.4 PROFILE POSITION MODE

Este modo de operação permite o controle do servoconversor SCA06 através do ajuste de set-point de posição, que podem ser executados seguindo dois métodos:

-
- single set-point. set of set-points.

Independente do método utilizado, o ajuste de um set-point é realizado da seguinte maneira: primeiramente deve-se escrever no objeto Target Position (607Ah) o set-point desejado. Após deve-se escrever 1 no bit NEW SET POINT no objeto de controle (ControlWord – 6040h). O bit SET-POINT ACKNOWLEDGE no objeto de status (StatusWord – 6041h) será setado indicando que um novo set-point foi recebido. Se o set-point for aceito o bit é resetado. Quando o set-point for alcançado, o bit TRAGET REACHED no objeto de status será setado. A figura 8.2 ilustra um exemplo de escrita de set-point.

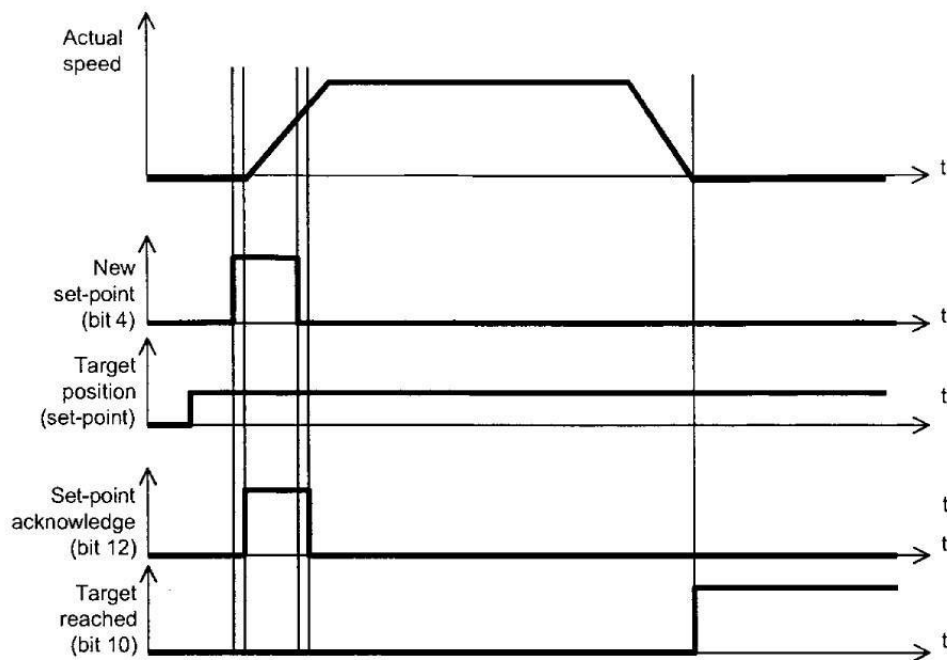


Figura 8.2: Ajuste de set-point de posição (Fonte: IEC 61800-7-201)

Single set-point

O método set-point único é utilizado quando deseja-se executar um novo set-point imediatamente. A figura 8.3 ilustra o funcionamento do método.

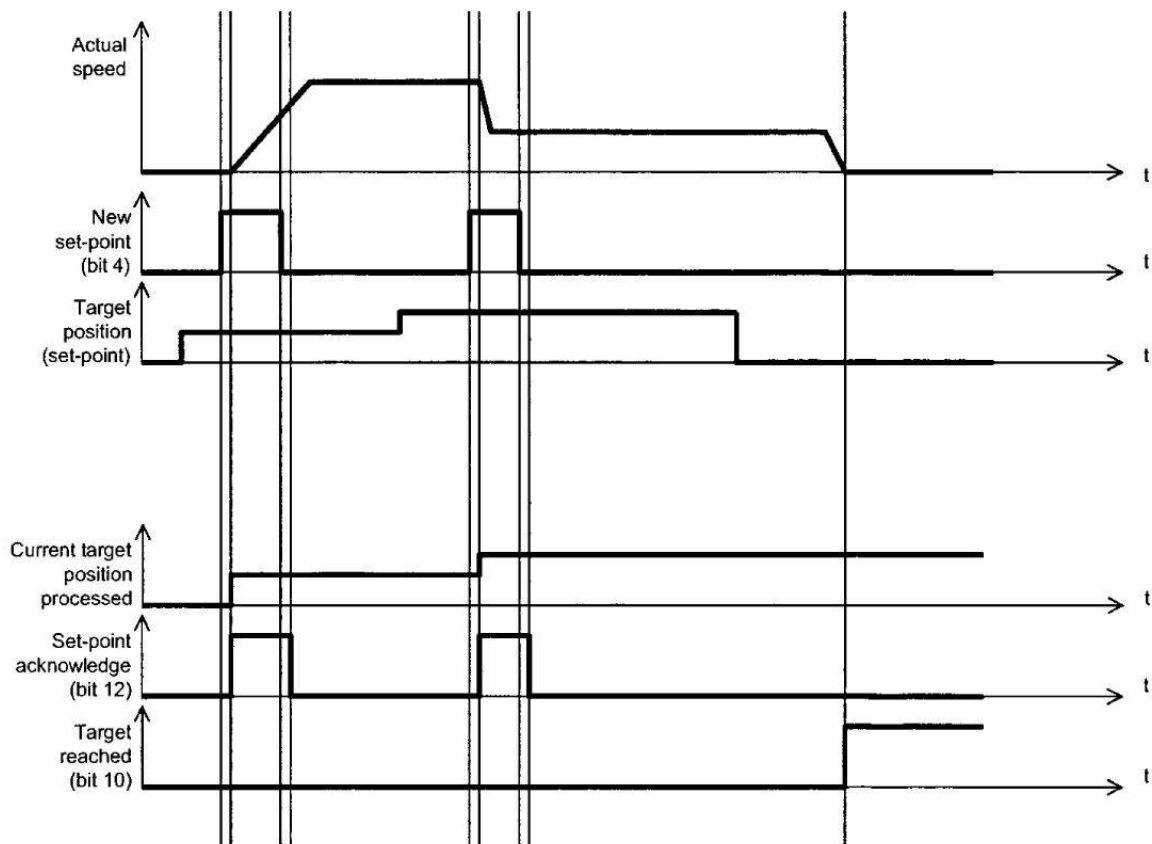


Figura 8.3: Método single set-point (Fonte: IEC 61800-7-201)

Set of set-point

O método conjunto de set-point é utilizado quando deseja-se executar um novo set-point somente após a finalização do anterior. A figura 8.4 ilustra o funcionamento do método.

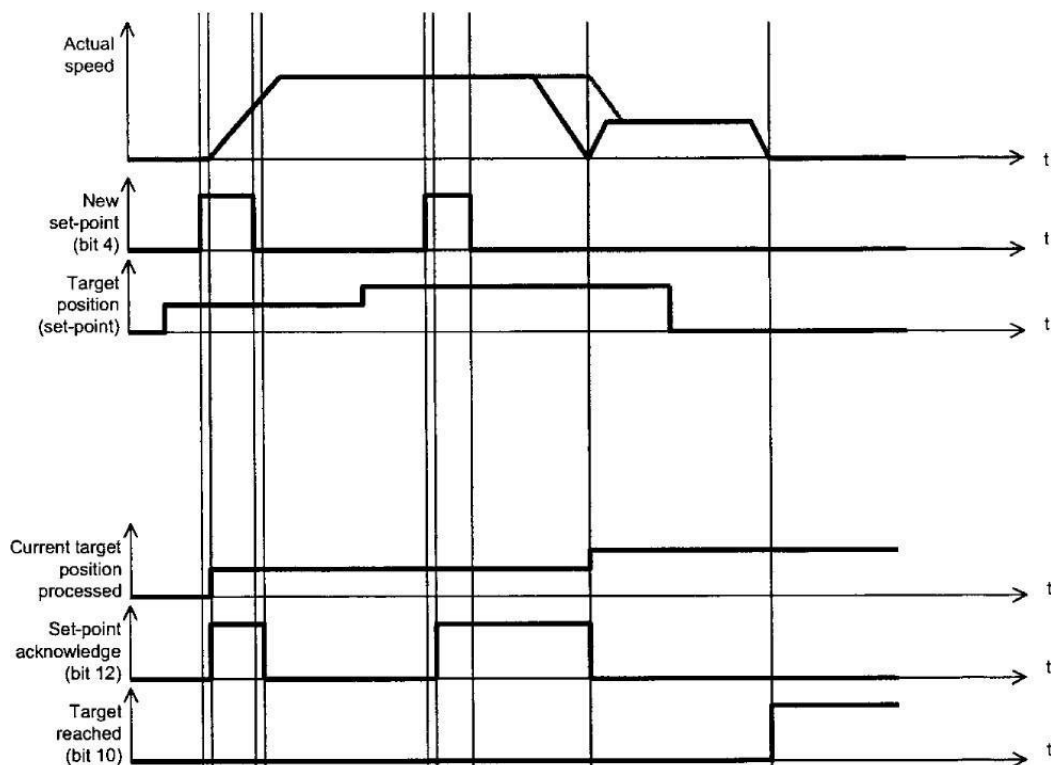


Figura 8.4: Método set of set-point (Fonte: IEC 61800-7-201)

O servoconversor SCA06 pode armazenar dois set-points, o que está em execução e o que será executado, como ilustra a figura 8.5.

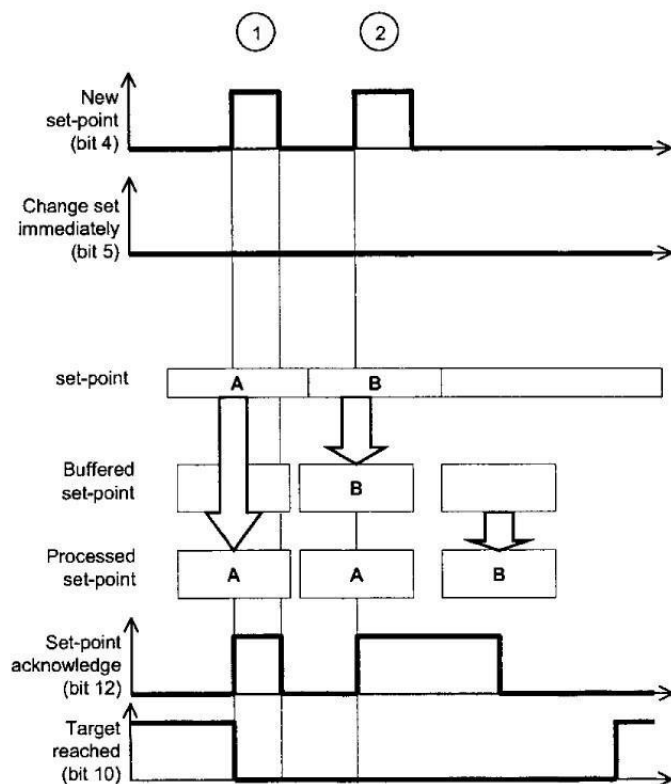


Figura 8.5: Armazenamento de set-point (Fonte: IEC 61800-7-201)

8.4.1 Bits de Controle e Estado

O profile mode position utiliza alguns bits dos objetos ControlWord e StatusWord para controlar e monitorar seu funcionamento.

Para o objeto ControlWord (6040h) são utilizados os seguintes bits:

- Bit 4 – New set-point.
- Bit 5 – Change set immediately.
- Bit 6 – absolute/relative.
- Bit 8 – Halt.
- Bit 9 – Change on set-point.

As tabela 8.8 e tabela 8.9 informam a definição dos bits de controle.

Tabela 8.8: Modo Posicionamento – definição dos bits 4, 5 e 9

Bit 9	Bit 5	Bit 4	Definição
0	0	0 → 1	Posição deve ser concluída antes de a próxima iniciar.
X	1	0 → 1	Próxima posição deve ser iniciada imediatamente.
1	0	0 1	Opção não implementada no SCA06.

→

Tabela 8.9: Modo Posicionamento – definição dos bits 6 e 8

Bit	Valor	Definição
6	0	Referência de posição deve ser um valor absoluto.
	1	Referência de posição deve ser um valor relativo.
8	0	Posicionamento deve ser executado ou continuado.
	1	Eixo deve ser parado conforme objeto 605Dh.

Para o objeto StatusWord (6041h) são utilizados os seguintes bits:

- Bit 10 – Target reached.
- Bit 12 – Set-point acknowledgeBit
- 13Following error.
Parametro 1032 lag error /14 bits 16383 pulsos .

A tabela 8.10 informa a definição dos bits de status.

Tabela 8.10: Modo Posicionamento – definição dos bits 10, 12 e 13

Bit	Valor	Definição
10	0	Referência de posição não alcançada.
	1	Referência de posição alcançada.
12	0	Referência de posição anterior já processada, aguardando nova referência de posição.
	1	Referência de posição anterior em processamento, substituição de referência de posição será aceita.
13	0	Sem erro de Following.
	1	Erro de Following.

Permite programar a referência de posição para o servoconversor SCA06 no modo posicionamento. Os 16 bits mais significativos informam o número de voltas e os 16 bits menos significativos informam a fração de volta. A escala utilizada neste objeto é 65536 para número de voltas e 65536 incrementos para uma volta do eixo. O valor deste objeto deve ser interpretado como absoluto ou relativo, conforme estado do Bit 6 do objeto ControlWord (6040h).

Índice	Subíndice	Nome	Tipo	Acesso	PDO Mapping
607Ah	0	Target Position	INT32	RW	Sim

8.4.3 Objeto 6081h – Profile Velocity **BUFFER X Y RESOLVER–65537 pulsos/ volta/segundo 608F**

Permite programar a velocidade normalmente atingida no final da rampa de aceleração durante um perfil de movimento. O valor a ser programado neste objeto deve estar entre 0 a 9999 rpm.

Índice	Subíndice	Nome	Tipo	Acesso	PDO Mapping
6081h	0	Profile Velocity	UINT32	RW	Sim

8.4.4 Objeto 6083h – Profile Acceleration **RESOLVER– 65537 pulsos/ volta² /volta³ jerk 608F**

Permite programar a rampa de aceleração até que o eixo do motor atinja a velocidade programada. A escala utilizada é a escala ms/krpm e os valores devem estar entre 1 a 32767.

Índice	Subíndice	Nome	Tipo	Acesso	PDO Mapping
6083h	0	Profile Acceleration	UINT32	RW	Sim

8.4.5 Objeto 6084h – Profile Deceleration **RESOLVER 65537 pulsos/ volta²/volta³ Jerk 608F**

Permite programar a rampa de desaceleração até que o eixo do motor atinja a velocidade zero. A escala utilizada neste objeto é a mesma do objeto 6083h.

Índice	Subíndice	Nome	Tipo	Acesso	PDO Mapping
6084h	,0	Profile Deceleration	UINT32	RW	Sim

8.4.6

8.4.6 Objeto 6086h – Motion Profile Type

Permite programar o perfil da rampa de aceleração e desaceleração para o drive.

Índice	Subíndice	Nome	Tipo	Acesso	PDO Mapping
6086h	0	Motion Profile Type	INT16	RW	Sim

Tabela 8.11: Valores para o Motion Profile Type

Valor	Perfil
0000h	Rampa linear

FFFFh	Sem rampa
-------	-----------

INTERFACE OPERACIONAL

BOTÕES E GRÁFICOS DINÂMICOS

PROGRAMA PHYTON / ANDROID HMI CNC

Carregamento e envio >>>> de Programa GRACIA-CODE TXT .
HMI ANDROID .

Interface Homem-Máquina (HMI) para Sistema CNC

A HMI foi desenvolvida em Python utilizando a biblioteca Tkinter, com arquitetura modular baseada em páginas independentes. O sistema estabelece comunicação bidirecional com o CLP através de uma interface USB serial, permitindo tanto o envio de comandos quanto a aquisição contínua de variáveis de processo para supervisão em tempo real.

A aplicação possui uma tela inicial de apresentação e cinco páginas funcionais dedicadas às principais operações da máquina.

A primeira página é destinada à transferência de programas para o CLP. O operador seleciona um arquivo de texto contendo o programa CNC, e o sistema realiza a transmissão sequencial das linhas, controlando os sinais de validação (rxValid) e término (rxDone), garantindo a integridade da carga do programa.

A segunda página concentra as funções de operação manual. São disponibilizados comandos de JOG para os três eixos, acionamento de Homing, visualização das posições absolutas e relativas, indicação do sentido de movimento (CW/CCW), monitoramento dos estados de Following Error e Fault de cada servoacionamento e ajuste do potenciômetro eletrônico utilizado como referência de velocidade durante a operação manual. Todas as informações são sincronizadas continuamente com o CLP.

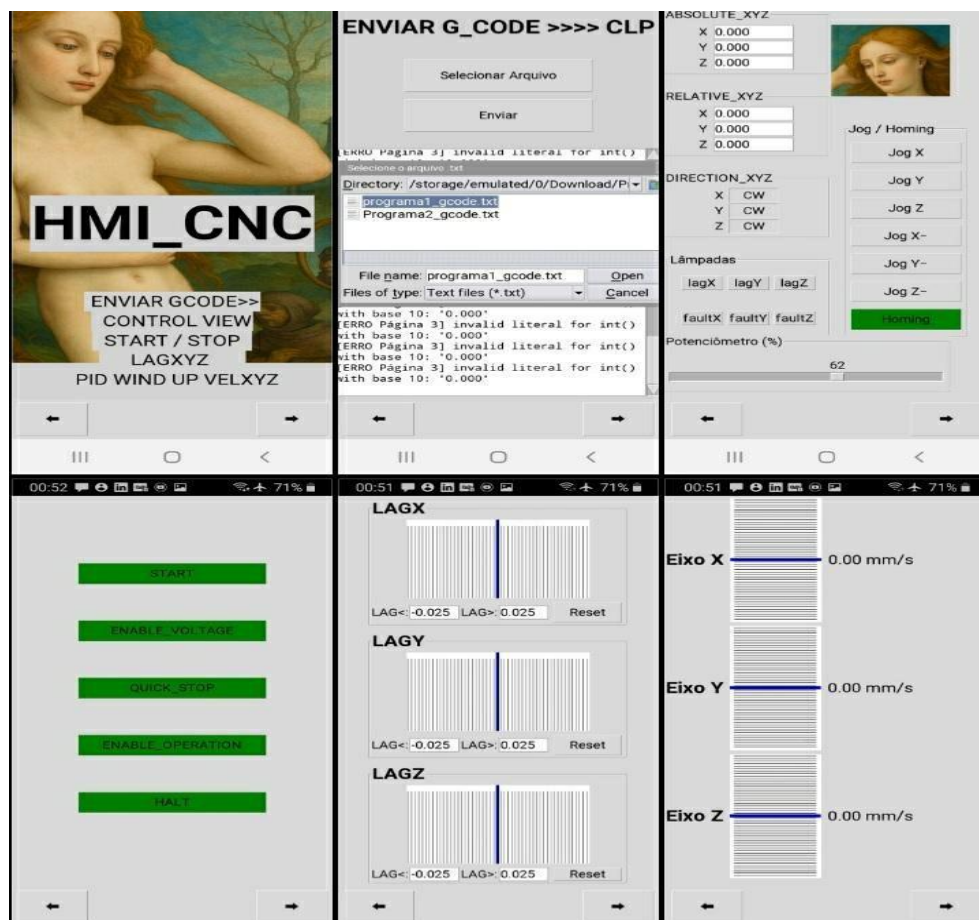
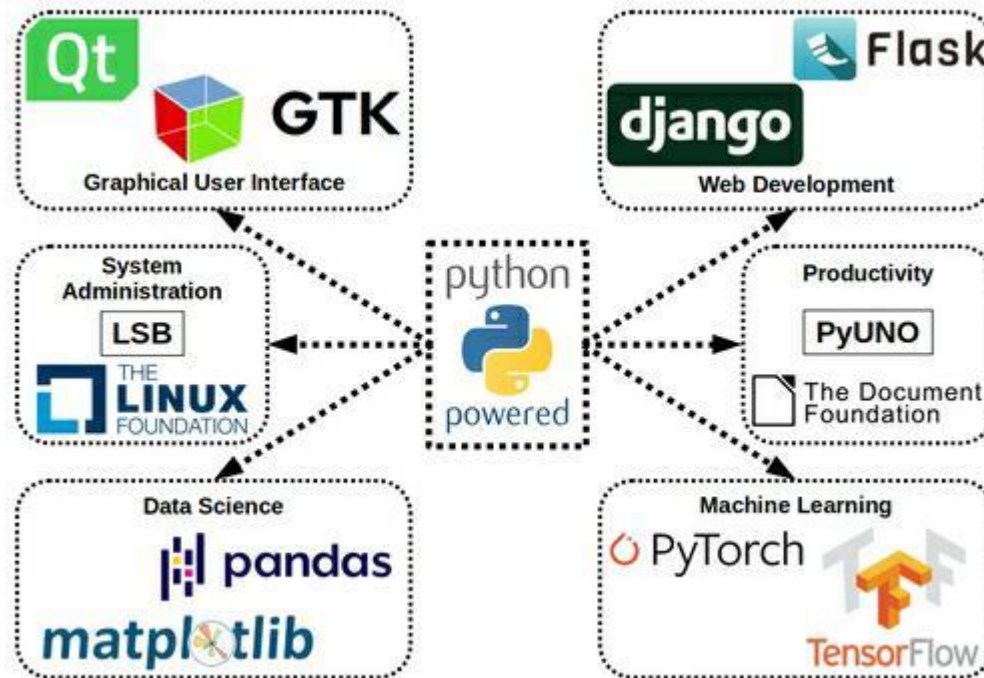
A terceira página implementa o painel de controle operacional dos servoacionadores baseado na máquina de estados CiA 402. O operador pode comandar os bits de Start, Enable Voltage, Quick Stop, Enable Operation e Halt, além dos respectivos comandos de limpeza de falhas. O sistema incorpora intertravamentos de segurança para impedir partidas quando as condições dos sensores de segurança não forem satisfeitas e apresenta mensagens de estado recebidas do CLP.

A quarta página disponibiliza três gráficos dedicados ao acompanhamento do Following Error dos eixos X, Y e Z. Cada gráfico apresenta uma escala ajustável pelo operador, permitindo visualizar continuamente a posição do erro em relação aos limites configurados, facilitando o ajuste fino da dinâmica dos servoacionamentos.

A quinta página realiza o monitoramento das velocidades dos três eixos. Os valores recebidos do servoacionador são convertidos para unidades físicas (mm/s), sendo apresentados simultaneamente a velocidade real, a velocidade nominal calculada, o desvio entre ambas (Delta) e os indicadores STP individuais dos eixos, incluindo o valor máximo global (STPMax).

Essa visualização permite avaliar, em tempo real, o desempenho cinemático do sistema durante a execução dos movimentos.

Toda a aplicação foi estruturada para operar em atualização contínua, utilizando filas de comunicação, processos assíncronos e leitura permanente das variáveis do CLP, mantendo sincronismo entre a interface gráfica e o sistema de controle sem interromper a operação da máquina. A arquitetura modular permite a expansão da HMI com novas páginas e funções sem alterar a estrutura principal da aplicação.



```
import os
import time
import tkinter as tk
from tkinter import filedialog, messagebox, scrolledtext

# --- Constantes ---
PASTA = "/storage/emulated/0/Download/ProjetosGracia"
```

```

USB_DEVICE = "/dev/ttyGS0"
DEFAULT_BG = "lightgray" # cor padrão para Android

# --- Variável global para arquivo selecionado ---
arquivo_txt = None

# --- Função de envio para o CLP ---

def enviar_para_clp_usb(linha, fim=False):
    try:
        with open(USB_DEVICE, "w") as usb:
            usb.write(linha + "\n")
            usb.write("rxValid=True\n")
            if fim:
                usb.write("rxDone=True\n")
            usb.flush()
    except PermissionError:
        log_text.insert(tk.END, f"Erro: permissão negada para {USB_DEVICE}\n")
    except Exception as e:
        log_text.insert(tk.END, f"Erro: {e}\n")

# --- Selecionar arquivo ---
def selecionar_arquivo():
    global arquivo_txt
    arquivo_path = filedialog.askopenfilename(
        initialdir=PASTA,
        title="Selecione o arquivo .txt",
        filetypes=(("Text files", "*.txt"),)
    )
    if arquivo_path:
        arquivo_txt = arquivo_path
        log_text.insert(tk.END, f"Arquivo selecionado: {arquivo_txt}\n")

# --- Enviar arquivo (VERSÃO ROBUSTA INTEGRADA) ---

def enviar_arquivo():
    if not arquivo_txt:
        messagebox.showwarning("Aviso", "Nenhum arquivo selecionado!")
        return
    try:
        with open(arquivo_txt, "r", encoding="utf-8") as f:
            linhas = [linha.strip() for linha in f.readlines()]
    except FileNotFoundError:
        log_text.insert(tk.END, f"Arquivo não encontrado: {arquivo_txt}\n")
        return
    except Exception as e:
        log_text.insert(tk.END, f"Erro ao abrir arquivo: {e}\n")
        return

    try:
        with open(USB_DEVICE, "w") as usb: # Abre USB apenas uma vez
            for i, linha in enumerate(linhas):
                fim = (i == len(linhas) - 1)
                log_text.insert(tk.END, f"Enviando linha {i+1}: {linha}\n")
                log_text.see(tk.END)

                usb.write(linha + "\n")
                usb.write("rxValid=True\n")
                if fim:
                    usb.write("rxDone=True\n")
                usb.flush()
                time.sleep(0.05) # Delay conservador para CLP
    except PermissionError:
        log_text.insert(tk.END, f"Erro: permissão negada para {USB_DEVICE}\n")
    except Exception as e:
        log_text.insert(tk.END, f"Erro ao enviar para USB: {e}\n")
        return

    log_text.insert(tk.END, "Envio concluído!\n")

```

```

# --- Janela principal ---
root = tk.Tk()
root.title("HMI Gracia CNC")
root.geometry("480x640") # layout para Samsung J8

# --- Criar frames para páginas ---
frames = []
for i in range(5):
    frame = tk.Frame(root)
    frames.append(frame)

# =====
# Página : Capa com LOGO
# =====

page6= tk.Frame(root)
frames.append(page6) # adiciona à lista de frames

# Cor de fundo opcional
page6.config(bg=DEFAULT_BG)

# Inserir LOGO
try:
    logo_capa = tk.PhotoImage(file="/storage/emulated/0/Download/ProjetosGracia/logo.png")
    label_logo_capa = tk.Label(page6, image=logo_capa, bg=DEFAULT_BG)
    label_logo_capa.image = logo_capa
    label_logo_capa.place(relx=0.5, rely=0.4, anchor="center") # centraliza horizontalmente
except Exception as e:
    log_text.insert(tk.END, f"[ERRO LOGO CAPA] {e}\n")

# Texto de boas-vindas (opcional)
tk.Label(page6, text=" HMI_CNC", font=("Arial", 35, "bold"),
        bg=DEFAULT_BG).place(relx=0.5, rely=0.6, anchor="center")


# -----
# --- Página 1: Envio de arquivo ---
# -----

page1 = frames[0]
btn_selecionar = tk.Button(page1, text="Selecionar Arquivo", command=selecionar_arquivo, height=2, width=20)
btn_selecionar.pack(pady=10)
btn_enviar = tk.Button(page1, text="Enviar", command=enviar_arquivo, height=2, width=20)
btn_enviar.pack(pady=10)
log_text = scrolledtext.ScrolledText(page1, width=60, height=25)
log_text.pack(pady=10)

# --- Função de log central ---
# --- Função de log central --
def log(msg):
    log_text.insert(tk.END, msg + "\n")
    log_text.see(tk.END)

# --- Página 2: Controle Jog/Homing, Lâmpadas e Absolutos ---
import tkinter as tk
import serial
import threading
import queue
import time
import os

page2 = frames[1]

# --- Constantes e layout ---
LEFT_X = 0
RIGHT_X = 400
Y_START_LEFT = 0

```



```

DY_LEFT = 250
Y_START_RIGHT = 350
BUTTON_SPACING = 10
DEFAULT_BG = "lightgray"

# --- Estados globais ---
BtnXYZ_HOMING = False
LAG_LAMP_X = LAG_LAMP_Y = LAG_LAMP_Z = False
FAULT_LAMP_X = FAULT_LAMP_Y = FAULT_LAMP_Z = False
ABSOLUTOX = ABSOLUTOY = ABSOLUTOZ = 0.0
pos_rel_X_mm = pos_rel_Y_mm = pos_rel_Z_mm = 0.0
sentidoX_CW = sentidoY_CW = sentidoZ_CW = True
Potenciometro1 = 0

# --- Serial Manager ---
USB_DEVICE = "/dev/ttyGS0"
BAUDRATE = 115200
TIMEOUT = 0.1

ser = None
modo_simulacao = False
fila_recebimento = queue.Queue()

# --- Inicializa USB ---
if os.path.exists(USB_DEVICE):
    try:
        ser = serial.Serial(USB_DEVICE, BAUDRATE, timeout=TIMEOUT)
        modo_simulacao = False
        fila_recebimento.put("[USB] CLP conectado via USB.")
    except Exception as e:
        ser = None
        modo_simulacao = True
        fila_recebimento.put(f"[ERRO USB] Falha ao abrir: {e}")
else:
    modo_simulacao = True
    fila_recebimento.put("[USB] USB não encontrada. Modo simulação ativado.")

# --- Função de envio USB ---
def enviar_para_clp_usb(linha, fim=False):
    if modo_simulacao:
        fila_recebimento.put(f"[SIMULAÇÃO] Enviado: {linha}")
    else:
        if ser:
            try:
                ser.write((linha + "\n").encode('ascii'))
                if fim:
                    ser.flush()
                fila_recebimento.put(f"[USB OK] {linha}")
            except Exception as e:
                fila_recebimento.put(f"[ERRO USB] Falha ao enviar: {e}")
        else:
            fila_recebimento.put(f"[ERRO USB] Sem conexão: {linha}")

# --- Thread de leitura serial ---
def thread_leitura_serial():
    while True:
        if modo_simulacao:
            fila_recebimento.put(f"ABSOLUTOX={ABSOLUTOX:.3f}")
            fila_recebimento.put(f"ABSOLUTOY={ABSOLUTOY:.3f}")
            fila_recebimento.put(f"ABSOLUTOZ={ABSOLUTOZ:.3f}")
            fila_recebimento.put(f"pos_rel_X_mm={pos_rel_X_mm:.3f}")
            fila_recebimento.put(f"pos_rel_Y_mm={pos_rel_Y_mm:.3f}")
            fila_recebimento.put(f"pos_rel_Z_mm={pos_rel_Z_mm:.3f}")
            fila_recebimento.put(f"sentidoX_CW={int(sentidoX_CW)}")
            fila_recebimento.put(f"sentidoY_CW={int(sentidoY_CW)}")
            fila_recebimento.put(f"sentidoZ_CW={int(sentidoZ_CW)}")
            fila_recebimento.put(f"LAG_LAMP_X={int(LAG_LAMP_X)}")
            fila_recebimento.put(f"LAG_LAMP_Y={int(LAG_LAMP_Y)}")
            fila_recebimento.put(f"LAG_LAMP_Z={int(LAG_LAMP_Z)}")
            fila_recebimento.put(f"FAULT_LAMP_X={int(FAULT_LAMP_X)}")
            fila_recebimento.put(f"FAULT_LAMP_Y={int(FAULT_LAMP_Y)}")

```

```

        fila_recebimento.put(f"FAULT_LAMP_Z={int(FAULT_LAMP_Z)}")
        fila_recebimento.put(f"Potenciometro1={Potenciometro1}")
        time.sleep(0.05)
    else:
        try:
            linha = ser.readline().decode('ascii').strip()
            if linha:
                fila_recebimento.put(linha)
        except Exception as e:
            fila_recebimento.put(f"[ERRO USB] Leitura falhou: {e}")
            time.sleep(0.05)

threading.Thread(target=thread_leitura_serial, daemon=True).start()

# --- Recebe dados do CLP e atualiza variáveis ---
def receber_dados_clp():
    global ABSOLUTOX, ABSOLUTOY, ABSOLUTOZ
    global pos_rel_X_mm, pos_rel_Y_mm, pos_rel_Z_mm
    global sentidoX_CW, sentidoY_CW, sentidoZ_CW
    global LAG_LAMP_X, LAG_LAMP_Y, LAG_LAMP_Z
    global FAULT_LAMP_X, FAULT_LAMP_Y, FAULT_LAMP_Z
    global Potenciometro1

    try:
        while not fila_recebimento.empty():
            linha = fila_recebimento.get()

            # --- Exibição segura no log_text ---
            if linha.startswith("["):
                log_text.insert(tk.END, linha + "\n")
                log_text.see(tk.END)
                continue

            # --- Processamento das variáveis ---
            if "=" in linha:
                var, val = linha.split("=")
                val = val.strip()
                if var == "ABSOLUTOX": ABSOLUTOX = float(val)
                elif var == "ABSOLUTOY": ABSOLUTOY = float(val)
                elif var == "ABSOLUTOZ": ABSOLUTOZ = float(val)
                elif var == "pos_rel_X_mm": pos_rel_X_mm = float(val)
                elif var == "pos_rel_Y_mm": pos_rel_Y_mm = float(val)
                elif var == "pos_rel_Z_mm": pos_rel_Z_mm = float(val)
                elif var == "sentidoX_CW": sentidoX_CW = bool(int(val))
                elif var == "sentidoY_CW": sentidoY_CW = bool(int(val))
                elif var == "sentidoZ_CW": sentidoZ_CW = bool(int(val))
                elif var == "LAG_LAMP_X": LAG_LAMP_X = bool(int(val))
                elif var == "LAG_LAMP_Y": LAG_LAMP_Y = bool(int(val))
                elif var == "LAG_LAMP_Z": LAG_LAMP_Z = bool(int(val))
                elif var == "FAULT_LAMP_X": FAULT_LAMP_X = bool(int(val))
                elif var == "FAULT_LAMP_Y": FAULT_LAMP_Y = bool(int(val))
                elif var == "FAULT_LAMP_Z": FAULT_LAMP_Z = bool(int(val))
                elif var == "Potenciometro1":
                    Potenciometro1 = int(val)
                    slider_pot.set(Potenciometro1)
    except Exception as e:
        log_text.insert(tk.END, f"[ERRO] Página 2: {e}\n")

page2.after(50, receber_dados_clp)

# --- Funções de controle ---
def toggle_homing():
    global BtnXYZ_HOMING
    BtnXYZ_HOMING = not BtnXYZ_HOMING
    btn_homing.config(bg="green" if BtnXYZ_HOMING else DEFAULT_BG)
    enviar_para_clp_usb(f"HOMING={'TRUE' if BtnXYZ_HOMING else 'FALSE'}")
    fila_recebimento.put(f"[LOG] Homing {'Ativado' if BtnXYZ_HOMING else 'Desativado'}")

```

```

def press_jog(btn_name):
    enviar_para_clp_usb(f"{btn_name}=TRUE")
    fila_recebimento.put(f"[LOG] {btn_name} pressionado")

def release_jog(btn_name):
    enviar_para_clp_usb(f"{btn_name}=FALSE")
    fila_recebimento.put(f"[LOG] {btn_name} liberado")

def atualizar_sentidos():
    chk_x.config(text="CW" if sentidoX_CW else "CCW")
    chk_y.config(text="CW" if sentidoY_CW else "CCW")
    chk_z.config(text="CW" if sentidoZ_CW else "CCW")

def atualizar_absolutos():
    for entry, val in zip([entry_x, entry_y, entry_z], [ABSOLUTOX, ABSOLUTOY, ABSOLUTOZ]):
        entry.delete(0, tk.END)
        entry.insert(0, f"{val:.3f}")
    for entry, val in zip([entry_rx, entry_ry, entry_rz], [pos_rel_X_mm, pos_rel_Y_mm, pos_rel_Z_mm]):
        entry.delete(0, tk.END)
        entry.insert(0, f"{val:.3f}")

def atualizar_lampadas():
    for lamp, state in zip([lamp_lag_x, lamp_lag_y, lamp_lag_z], [LAG_LAMP_X, LAG_LAMP_Y, LAG_LAMP_Z]):
        lamp.config(bg="green" if state else DEFAULT_BG)
    for lamp, state in zip([lamp_fault_x, lamp_fault_y, lamp_fault_z], [FAULT_LAMP_X, FAULT_LAMP_Y, FAULT_LAMP_Z]):
        lamp.config(bg="green" if state else DEFAULT_BG)
    atualizar_sentidos()
    atualizar_absolutos()
    page2.after(100, atualizar_lampadas)

# --- Frame Jog/Homing ---
frame_jog = tk.LabelFrame(page2, text="Jog / Homing", padx=10, pady=10)
frame_jog.place(x=RIGHT_X, y=Y_START_RIGHT)

for btn_name in ["Jog X", "Jog Y", "Jog Z", "Jog X-", "Jog Y-", "Jog Z-"]:
    btn = tk.Button(frame_jog, text=btn_name, width=10)
    btn.pack(pady=BUTTON_SPACING)
    btn.bind("<ButtonPress-1>", lambda e, n=btn_name: press_jog(n))
    btn.bind("<ButtonRelease-1>", lambda e, n=btn_name: release_jog(n))

btn_homing = tk.Button(frame_jog, text="Homing", width=10, command=toggle_homing, bg=DEFAULT_BG)
btn_homing.pack(pady=BUTTON_SPACING)

# --- Frames ABSOLUTOS e RELATIVOS ---
frame_abs = tk.LabelFrame(page2, text="ABSOLUTE_XYZ", padx=70, pady=10)
frame_abs.place(x=LEFT_X, y=Y_START_LEFT)

tk.Label(frame_abs, text="X").grid(row=1, column=0, padx=5, pady=2, sticky="e")
entry_x = tk.Entry(frame_abs, width=10); entry_x.grid(row=1, column=1, padx=5, pady=2)
tk.Label(frame_abs, text="Y").grid(row=2, column=0, padx=5, pady=2, sticky="e")
entry_y = tk.Entry(frame_abs, width=10); entry_y.grid(row=2, column=1, padx=5, pady=2)
tk.Label(frame_abs, text="Z").grid(row=3, column=0, padx=5, pady=2, sticky="e")
entry_z = tk.Entry(frame_abs, width=10); entry_z.grid(row=3, column=1, padx=5, pady=2)

frame_rel = tk.LabelFrame(page2, text="RELATIVE_XYZ", padx=70, pady=10)
frame_rel.place(x=LEFT_X, y=Y_START_LEFT + 1*DY_LEFT)

tk.Label(frame_rel, text="X").grid(row=1, column=0, padx=5, pady=2, sticky="e")
entry_rx = tk.Entry(frame_rel, width=10); entry_rx.grid(row=1, column=1, padx=5, pady=2)
tk.Label(frame_rel, text="Y").grid(row=2, column=0, padx=5, pady=2, sticky="e")
entry_ry = tk.Entry(frame_rel, width=10); entry_ry.grid(row=2, column=1, padx=5, pady=2)
tk.Label(frame_rel, text="Z").grid(row=3, column=0, padx=5, pady=2, sticky="e")
entry_rz = tk.Entry(frame_rel, width=10); entry_rz.grid(row=3, column=1, padx=5, pady=2)

# --- Frame DIREÇÃO ---
frame_dir = tk.LabelFrame(page2, text="DIRECTION_XYZ", padx=105, pady=10)
frame_dir.place(x=LEFT_X, y=Y_START_LEFT + 2*DY_LEFT)

tk.Label(frame_dir, text="X").grid(row=1, column=0, padx=5, pady=2, sticky="e")
chk_x = tk.Label(frame_dir, text="CW", width=6, relief="sunken"); chk_x.grid(row=1, column=1, padx=5, pady=2)

```

```

tk.Label(frame_dir, text="Y").grid(row=2, column=0, padx=5, pady=2, sticky="e")
chk_y = tk.Label(frame_dir, text="CW", width=6, relief="sunken"); chk_y.grid(row=2, column=1, padx=5, pady=2)
tk.Label(frame_dir, text="Z").grid(row=3, column=0, padx=5, pady=2, sticky="e")
chk_z = tk.Label(frame_dir, text="CW", width=6, relief="sunken"); chk_z.grid(row=3, column=1, padx=5, pady=2)

# --- Frame Lâmpadas ---
frame_lamps = tk.LabelFrame(page2, text="Lâmpadas", padx=25, pady=0)
frame_lamps.place(x=LEFT_X+10, y=Y_START_LEFT + 3*DY_LEFT)

frame_lag = tk.Frame(frame_lamps); frame_lag.pack(pady=25)
lamp_lag_x = tk.Label(frame_lag, text="lagX", width=5, relief="raised", bg=DEFAULT_BG);
lamp_lag_x.grid(row=1, column=1, padx=5, pady=5)
lamp_lag_y = tk.Label(frame_lag, text="lagY", width=5, relief="raised", bg=DEFAULT_BG);
lamp_lag_y.grid(row=1, column=2, padx=5, pady=5)
lamp_lag_z = tk.Label(frame_lag, text="lagZ", width=5, relief="raised", bg=DEFAULT_BG);
lamp_lag_z.grid(row=1, column=3, padx=5, pady=5)

frame_fault = tk.Frame(frame_lamps); frame_fault.pack(pady=30)
lamp_fault_x = tk.Label(frame_fault, text="faultX", width=5, relief="raised", bg=DEFAULT_BG);
lamp_fault_x.grid(row=1, column=1, padx=5, pady=5)
lamp_fault_y = tk.Label(frame_fault, text="faultY", width=5, relief="raised", bg=DEFAULT_BG);
lamp_fault_y.grid(row=1, column=2, padx=5, pady=5)
lamp_fault_z = tk.Label(frame_fault, text="faultZ", width=5, relief="raised", bg=DEFAULT_BG);
lamp_fault_z.grid(row=1, column=3, padx=5, pady=5)

# --- Potenciômetro ---
frame_pot = tk.LabelFrame(page2, text="Potenciômetro (%)", padx=10, pady=30)
frame_pot.place(x=LEFT_X, y=Y_START_LEFT + 4*DY_LEFT)

def atualizar_pot(val):
    global Potenciometro1
    Potenciometro1 = int(val)
    enviar_para_clp_usb(f"Potenciometro1={Potenciometro1}")
    fila_recebimento.put(f"[LOG] Potenciometro1={Potenciometro1}%")

slider_pot = tk.Scale(frame_pot, from_=0, to=100, orient="horizontal", length=600, command=atualizar_pot)
slider_pot.set(Potenciometro1)
slider_pot.pack()

# --- Inicializa atualização contínua ---
atualizar_lampadas()
receber_dados_clp()

# --- Inserir logo ---
try:
    imagem_logo = tk.PhotoImage(file="/storage/emulated/0/Download/ProjetosGracia/logo.png")
    label_logo = tk.Label(page2, image=imagem_logo, bd=0)
    label_logo.image = imagem_logo
    label_logo.place(x=370, y=50)
except Exception as e:
    fila_recebimento.put(f"[ERRO LOGO] {e}")
# FIM PAG 2

```

```

# --- Página 3: Controle de Bits CLP ---
import tkinter as tk
import queue

```

```

page3 = frames[2]

```

```

# Toast para Quick Stop

```

```

toast_label = tk.Label(page3, text="", bg="yellow", fg="black")

# --- Constantes dos bits ---
START_BIT      = 1 << 0
ENABLE_VOLTAGE_BIT = 1 << 1
QUICK_STOP_BIT  = 1 << 2
ENABLE_OP_BIT   = 1 << 3
HALT_BIT        = 1 << 4

# --- Estados dos botões ---
btn_states = {
    "START": False,
    "ENABLE_VOLTAGE": False,
    "QUICK_STOP": False,
    "ENABLE_OPERATION": False,
    "HALT": False,
    "QS_CLEAR": False,
    "FAULT_CLEAR": False,
    "HALT_CLEAR": False # <--- adicionado
}

# --- Palavras de controle dos eixos ---
vars_to_clp = {"WR_PDO_6040_X":0, "WR_PDO_6040_Y":0, "WR_PDO_6040_Z":0}

# --- Fila de logs ---
fila_logs = queue.Queue()

def log_msg(msg):
    fila_logs.put(msg)

# --- Atualiza palavras WR_PDO ---
def atualizar_wr_pdo_xyz():
    WR = 0

    # Bits normais (ativo em nível alto)
    if btn_states["START"]:
        WR |= START_BIT

    if btn_states["ENABLE_VOLTAGE"]:
        WR |= ENABLE_VOLTAGE_BIT

    if btn_states["ENABLE_OPERATION"]:
        WR |= ENABLE_OP_BIT

    # -----
    # QUICK STOP (bit 2)
    # ativo em nível BAIXO
    # -----
    if not btn_states["QUICK_STOP"]:
        WR |= QUICK_STOP_BIT
    else:
        WR &= ~QUICK_STOP_BIT

    # -----
    # HALT (bit 4)
    # ativo em nível BAIXO
    # -----
    if not btn_states["HALT"]:
        WR |= HALT_BIT
    else:
        WR &= ~HALT_BIT

    # Atualiza todos os eixos
    for var in vars_to_clp:
        vars_to_clp[var] = WR

# --- Envio USB ---
def enviar_para_clp_usb(linha):
    if modo_simulacao:
        log_msg(f"[SIMULAÇÃO] {linha}")
    else:

```

```

if ser:
    try:
        ser.write((linha + "\n").encode("ascii"))
        log_msg(f"[USB OK] {linha}")
    except Exception as e:
        log_msg(f"[ERRO USB] {e}")
    else:
        log_msg(f"[ERRO USB] Sem conexão")

# --- Alterna botão ---
def toggle_bit_xyz(bit_name):

    # --- INTERTRAVAMENTO DE SEGURANÇA DO START ---
    if bit_name == "START":
        if not (
            vars_to_clp.get("MICRO1", False)
            and vars_to_clp.get("MICRO2", False)
            and vars_to_clp.get("MICRO3", False)
        ):
            log_msg("START bloqueado: linha de segurança aberta")
            return

    btn_states[bit_name] = not btn_states[bit_name]
    atualizar_wr_pdo_xyz()
    status = "ATIVADO" if btn_states[bit_name] else "DESATIVADO"
    log_msg(f"{bit_name} {status}")
    enviar_dados_clp_p3()

# --- Recebe dados do CLP ---
def receber_dados_clp_p3():
    try:
        while not fila_recebimento.empty():
            linha = fila_recebimento.get()
            if "=" in linha:
                var, val = linha.split("=")

                # Tratamento de MSG como string
                if var == "MSG":
                    vars_to_clp["MSG"] = val.strip()
                else:
                    val = int(val.strip())
                    if var in vars_to_clp:
                        vars_to_clp[var] = val

                # Atualiza estados dos botões a partir dos bits
                if var != "MSG": # só faz para variáveis numéricas
                    btn_states["START"] = bool(vars_to_clp.get("WR_PDO_6040_X", 0) & START_BIT)
                    btn_states["ENABLE_VOLTAGE"] = bool(vars_to_clp.get("WR_PDO_6040_X", 0) & ENABLE_VOLTAGE_BIT)
                    btn_states["QUICK_STOP"] = bool(vars_to_clp.get("WR_PDO_6040_X", 0) & QUICK_STOP_BIT)
                    btn_states["ENABLE_OPERATION"] = bool(vars_to_clp.get("WR_PDO_6040_X", 0) & ENABLE_OP_BIT)
                    btn_states["HALT"] = bool(vars_to_clp.get("WR_PDO_6040_X", 0) & HALT_BIT)

    except Exception as e:
        log_msg(f"[ERRO Página 3] {e}")

    page3.after(50, receber_dados_clp_p3)

# --- Envio contínuo ---
def enviar_dados_clp_p3():
    for var, val in vars_to_clp.items():
        enviar_para_clp_usb(f"{var}={val}")

# =====
# Linha de sensores indutivos
# =====

frame_indutivos = tk.Frame(page3)
frame_indutivos.pack(pady=10)

btn_ind1 = tk.Button(frame_indutivos, text="HOMEX", width=10, bg="red")
btn_ind2 = tk.Button(frame_indutivos, text="HOMEY", width=10, bg="red")
btn_ind3 = tk.Button(frame_indutivos, text="HOMEZ", width=10, bg="red")

```

```

btn_ind1.pack(side="left", padx=5)
btn_ind2.pack(side="left", padx=5)
btn_ind3.pack(side="left", padx=5)

frame_indutivos2 = tk.Frame(page3)
frame_indutivos2.pack(pady=5)

btn_indx1 = tk.Button(frame_indutivos2, text="MICRO1", width=10, bg="red")
btn_indy1 = tk.Button(frame_indutivos2, text="MICRO2", width=10, bg="red")
btn_indz1 = tk.Button(frame_indutivos2, text="MICRO3", width=10, bg="red")

btn_indx1.pack(side="left", padx=5)
btn_indy1.pack(side="left", padx=5)
btn_indz1.pack(side="left", padx=5)

# =====
# Atualização visual
# =====

# =====
# Quick Stop e HALT
# =====
quickstop_prev = False
halt_prev = False # nova flag para HALT

def atualizar_pagina3():
    global quickstop_prev, halt_prev

    # Atualiza cores dos botões
    btn_start.config(bg="green" if btn_states["START"] else DEFAULT_BG)
    btn_enable_voltage.config(bg="green" if btn_states["ENABLE_VOLTAGE"] else DEFAULT_BG)
    btn_quick_stop.config(bg="green" if btn_states["QUICK_STOP"] else DEFAULT_BG)
    btn_enable_op.config(bg="green" if btn_states["ENABLE_OPERATION"] else DEFAULT_BG)
    btn_halt.config(bg="green" if btn_states["HALT"] else DEFAULT_BG)
    btn_halt_clear.config(bg="green" if btn_states.get("HALT_CLEAR", False) else DEFAULT_BG)
    btn_qs_clear.config(bg="green" if btn_states["QS_CLEAR"] else DEFAULT_BG)
    btn_fault_clear.config(bg="green" if btn_states["FAULT_CLEAR"] else DEFAULT_BG)

    # Sensores de indicação
    btn_ind1.config(bg="green" if vars_to_clp.get("HOMEX", False) else "red")
    btn_ind2.config(bg="green" if vars_to_clp.get("HOMEY", False) else "red")
    btn_ind3.config(bg="green" if vars_to_clp.get("HOMEZ", False) else "red")

    # Sensores de segurança
    btn_indx1.config(bg="green" if vars_to_clp.get("MICRO1", False) else "red")
    btn_indy1.config(bg="green" if vars_to_clp.get("MICRO2", False) else "red")
    btn_indz1.config(bg="green" if vars_to_clp.get("MICRO3", False) else "red")

    # --- Quick Stop ---
    if btn_states["QUICK_STOP"] and not quickstop_prev:
        toast_label.config(text="PARADA NÃO PROGRAMADA", wraplength=500)
        toast_label.pack(side="top")
    elif btn_states["QS_CLEAR"]:
        btn_states["QUICK_STOP"] = False # reseta QUICK_STOP
        btn_states["QS_CLEAR"] = False # reseta botão QS_CLEAR
        toast_label.pack_forget() # limpa toast

    quickstop_prev = btn_states["QUICK_STOP"]

    # --- HALT ---
    if btn_states["HALT"] and not halt_prev:
        if vars_to_clp.get("MSG"):
            toast_label.config(text=vars_to_clp["MSG"], wraplength=500)
            toast_label.pack(side="top")
    elif btn_states.get("HALT_CLEAR", False):
        btn_states["HALT"] = False # reseta HALT
        btn_states["HALT_CLEAR"] = False # reseta botão HALT_CLEAR
        toast_label.pack_forget() # limpa toast do HALT

    halt_prev = btn_states["HALT"]

```

```

# Processa fila de logs
processar_logs()

# Loop contínuo
page3.after(100, atualizar_pagina3)

# --- Processa fila de logs ---
def processar_logs():
    while not fila_logs.empty():
        log_text.insert(tk.END, fila_logs.get() + "\n")
        log_text.see(tk.END)

# --- Layout dos botões ---
frame_bits = tk.Frame(page3)
frame_bits.pack(pady=50)

btn_start = tk.Button(frame_bits, text="START", width=20,
                      command=lambda: toggle_bit_xyz("START"), bg=DEFAULT_BG)
btn_enable_voltage = tk.Button(frame_bits, text="ENABLE_VOLTAGE", width=20,
                              command=lambda: toggle_bit_xyz("ENABLE_VOLTAGE"), bg=DEFAULT_BG)
btn_quick_stop = tk.Button(frame_bits, text="QUICK_STOP", width=20,
                           command=lambda: toggle_bit_xyz("QUICK_STOP"), bg=DEFAULT_BG)
btn_enable_op = tk.Button(frame_bits, text="ENABLE_OPERATION", width=20,
                          command=lambda: toggle_bit_xyz("ENABLE_OPERATION"), bg=DEFAULT_BG)
btn_halt = tk.Button(frame_bits, text="HALT", width=20,
                    command=lambda: toggle_bit_xyz("HALT"), bg=DEFAULT_BG)
btn_qs_clear = tk.Button(frame_bits, text="QS_CLEAR", width=20,
                        command=lambda: toggle_bit_xyz("QS_CLEAR"), bg=DEFAULT_BG)
btn_fault_clear = tk.Button(frame_bits, text="FAULT_CLEAR", width=20,
                            command=lambda: toggle_bit_xyz("FAULT_CLEAR"), bg=DEFAULT_BG)
btn_halt_clear = tk.Button(frame_bits, text="HALT_CLEAR", width=20,
                           command=lambda: toggle_bit_xyz("HALT_CLEAR"), bg=DEFAULT_BG)

# --- Empacota os botões ---
btn_start.pack(pady=15)
btn_enable_voltage.pack(pady=25)
btn_quick_stop.pack(pady=25)
btn_enable_op.pack(pady=25)
btn_halt.pack(pady=25)
btn_qs_clear.pack(pady=25)
btn_fault_clear.pack(pady=25)
btn_halt_clear.pack(pady=25)

# --- Inicializa loops ---
atualizar_pagina3()
receber_dados_clp_p3()

```

Página 4: Gráficos LAG X/Y/Z (linhas de escala apenas)

```

# -----
import tkinter as tk

page4 = frames[3]

# =====
# Configuração genérica
# =====
canvas_width = 400
canvas_height = 240
barra_largura = 6
UPDATE_INTERVAL = 50 # ms (20 Hz)
ESCALA_DIV = 25      # número de divisões da escala

# =====
# Estrutura de cada eixo
# =====
class LagFrame:
    def __init__(self, master, nome):
        self.nome = nome

```



```

self.lag_esq = -0.025
self.lag_dir = 0.025
self.lag_valor = 0
self.barra_id = None

# Frame principal
self.frame = tk.LabelFrame(master, text=nome, font=("Arial", 12, "bold"))
self.frame.pack(pady=10)

# Canvas
self.canvas = tk.Canvas(self.frame, width=canvas_width, height=canvas_height, bg="white")
self.canvas.pack(pady=5)
self.meio = canvas_width // 2

# Entradas LAG< e LAG>
input_frame = tk.Frame(self.frame)
input_frame.pack(pady=5)

tk.Label(input_frame, text="LAG<:").pack(side="left")
self.entry_esq = tk.Entry(input_frame, width=6)
self.entry_esq.pack(side="left")
self.entry_esq.insert(0, str(self.lag_esq))
self.entry_esq.bind("<Return>", lambda e: self.set_lag_esq(self.entry_esq.get()))

tk.Label(input_frame, text="LAG>:").pack(side="left")
self.entry_dir = tk.Entry(input_frame, width=6)
self.entry_dir.pack(side="left")
self.entry_dir.insert(0, str(self.lag_dir))
self.entry_dir.bind("<Return>", lambda e: self.set_lag_dir(self.entry_dir.get()))

tk.Button(input_frame, text="Reset", command=self.reset_barra).pack(side="left", padx=10)

# Desenha elementos fixos
self._desenhar_escala()
self._desenhar_linha_central()

# Reseta barra no centro (sempre visível)
self.reset_barra()

# Inicia loop de atualização
self.atualizar_barra()

# =====
# Funções internas
# =====
def escalar_lag(self, valor):
    if valor >= 0:
        escala = (valor / self.lag_dir) * (canvas_width // 2)
    else:
        escala = (valor / abs(self.lag_esq)) * (canvas_width // 2)
    return self.meio + int(escala)

def _desenhar_escala(self):
    for i in range(-ESCALA_DIV, ESCALA_DIV + 1):
        x = int(self.meio + (i / ESCALA_DIV) * (canvas_width / 2.05))
        self.canvas.create_line(x, canvas_height - 220, x, canvas_height, fill="black", tags="escala")

def _desenhar_linha_central(self):
    self.canvas.create_line(self.meio, 0, self.meio, canvas_height, fill="gray", dash=(4,2), tags="linha_central")

def atualizar_barra(self):
    x = self.escalar_lag(self.lag_valor)
    if self.barra_id is None:
        # Cria barra azul central se ainda não existir
        self.barra_id = self.canvas.create_rectangle(
            x-barra_largura//2, 0, x+barra_largura//2, canvas_height, fill="blue"
        )
    else:
        self.canvas.coords(self.barra_id, x-barra_largura//2, 0, x+barra_largura//2, canvas_height)

# Loop contínuo

```

```

        self.frame.after(UPDATE_INTERVAL, self.atualizar_barra)

def set_lag_esq(self, val):
    try:
        self.lag_esq = float(val)
    except:
        pass

def set_lag_dir(self, val):
    try:
        self.lag_dir = float(val)
    except:
        pass

def reset_barra(self):
    x = self.meio
    if self.barra_id is None:
        # Cria barra azul central se ainda não existir
        self.barra_id = self.canvas.create_rectangle(
            x-barra_largura//2, 0, x+barra_largura//2, canvas_height, fill="blue"
        )
    else:
        self.canvas.coords(self.barra_id, x-barra_largura//2, 0, x+barra_largura//2, canvas_height)
    self.lag_valor = 0

# =====
# Criação dos 3 gráficos
# =====
LagX = LagFrame(page4, "LAGX")
LagY = LagFrame(page4, "LAGY")
LagZ = LagFrame(page4, "LAGZ")

# =====
# Leitura contínua via USB
# =====
def receber_dados_clp_p4():
    try:
        while not fila_recebimento.empty():
            linha = fila_recebimento.get()
            if "=" in linha:
                var, val = linha.split("=")
                val = float(val.strip())

                if var.strip().upper() == "LAGX":
                    LagX.lag_valor = val
                elif var.strip().upper() == "LAGY":
                    LagY.lag_valor = val
                elif var.strip().upper() == "LAGZ":
                    LagZ.lag_valor = val
    except Exception as e:
        print("Erro Página 4:", e)
    page4.after(UPDATE_INTERVAL, receber_dados_clp_p4)

# Inicia leitura contínua
receber_dados_clp_p4()
# Página 5: Velocidade 6081 convertida em mm/s
import tkinter as tk

page5 = frames[4]

# =====
# Configuração genérica
# =====
canvas_width_v = 200
canvas_height_v = 380
UPDATE_INTERVAL = 50 # ms
PASSO_MM = 6 # passo do eixo em mm
CONTROL_LAGXYZ = False # controle global do delta

# =====
# Estrutura de cada eixo vertical

```

```

# =====
class VelocidadeFrame:
    def __init__(self, master, nome):
        self.nome = nome
        self.valor_atual = 0.0
        self.velBase = 0.0    # linha central fixa quando controle desligado
        self.delta = 0.0     # oscilação em torno da base
        self.ativo = False
        self.barra_id = None

        self.frame = tk.Frame(master)
        self.frame.pack(pady=5, anchor="w")

        tk.Label(self.frame, text=nome, font=("Arial", 12, "bold")).pack(side="left", padx=5)
        self.canvas = tk.Canvas(self.frame, width=canvas_width_v, height=canvas_height_v, bg="white")
        self.canvas.pack(side="left", padx=5)
        self.valor_label = tk.Label(self.frame, text="0.0 mm/s", font=("Arial", 10), width=80, anchor="w")
        self.valor_label.pack(side="left", padx=5)

        self._desenhar_escala()
        self._desenhar_linha_central()
        self.atualizar_barra()

    def _desenhar_escala(self):
        for i in range(50):
            y = int(i * canvas_height_v / 50)
            self.canvas.create_line(10, y, canvas_width_v-10, y, fill="black", width=2)

    def _desenhar_linha_central(self):
        meio = canvas_height_v // 2
        self.canvas.create_line(0, meio, canvas_width_v, meio, fill="blue", width=10)

    def atualizar_barra(self):
        meio = canvas_height_v // 2
        # altura proporcional ao delta
        altura = int(self.delta * canvas_height_v / 100) if self.ativo else 0
        y_top = meio - altura if altura >= 0 else meio
        y_bottom = meio + abs(altura) if altura < 0 else meio

        if self.barra_id is None:
            self.barra_id = self.canvas.create_rectangle(
                10, y_top, canvas_width_v-10, y_bottom, fill="red"
            )
        else:
            self.canvas.coords(self.barra_id, 10, y_top, canvas_width_v-10, y_bottom)

        self.valor_label.config(text=f"{self.valor_atual:.2f} mm/s")
        self.frame.after(UPDATE_INTERVAL, self.atualizar_barra)

# =====
# Criação dos gráficos por eixo
# =====
VelX_Frame = VelocidadeFrame(page5, "Eixo X")
VelY_Frame = VelocidadeFrame(page5, "Eixo Y")
VelZ_Frame = VelocidadeFrame(page5, "Eixo Z")

# =====
# Recepção de dados do CLP
# =====
def receber_dados_clp_p5():
    global axisX, axisY, axisZ, CONTROL_LAGXYZ # objetos com WR_PDO_6081
    try:
        while not fila_recebimento.empty():
            linha = fila_recebimento.get()
            if "=" in linha:
                var, val = linha.split("=")
                val = val.strip()

            # Ativação do ramo
            if var.strip().upper() == "RAMOX":
                VelX_Frame.ativo = val in ["1", "TRUE"]

```

```

elif var.strip().upper() == "RAMOY":
    VelY_Frame.ativo = val in ["1", "TRUE"]
elif var.strip().upper() == "RAMOZ":
    VelZ_Frame.ativo = val in ["1", "TRUE"]

# Controle global
if var.strip().upper() == "CONTROL_LAGXYZ":
    CONTROL_LAGXYZ = val in ["1", "TRUE"]

# Atualiza a velocidade convertida (mm/s) e delta
for frame, axis in zip([VelX_Frame, VelY_Frame, VelZ_Frame], [axisX, axisY, axisZ]):
    frame.valor_atual = axis.WR_PDO_6081 * PASSO_MM / 60
    if CONTROL_LAGXYZ:
        # oscilação em torno da linha base
        frame.delta = frame.valor_atual - frame.velBase
    else:
        # base fixa = valor atual, sem oscilação
        frame.velBase = frame.valor_atual
        frame.delta = 0

except Exception as e:
    print("Erro Página 5:", e)

page5.after(UPDATE_INTERVAL, receber_dados_clp_p5)

# Inicia leitura contínua
receber_dados_clp_p5()

# --- Navegação entre páginas ---
current_page = 5
frames[current_page].pack(fill="both", expand=True)

def mostrar_pagina(index):
    global current_page
    frames[current_page].pack_forget()
    current_page = index
    frames[current_page].pack(fill="both", expand=True)

def proxima_pagina():
    global current_page
    if current_page < len(frames) - 1:
        mostrar_pagina(current_page+1)

def pagina_anterior():
    global current_page
    if current_page > 0:
        mostrar_pagina(current_page-1)

nav_frame = tk.Frame(root)
btn_prev = tk.Button(nav_frame, text="◀", command=pagina_anterior, width=5, height=2)
btn_next = tk.Button(nav_frame, text="▶", command=proxima_pagina, width=5, height=2)
btn_prev.pack(side="left", padx=20, pady=10)
btn_next.pack(side="right", padx=20, pady=10)
nav_frame.pack(side="bottom", fill="x")

# --- Inicializa a janela ---
root.mainloop()

```

@

Declarações Gerais de Variáveis, Structs e Endereços

O programa inicia com a definição de variáveis globais, estruturas de dados e endereços de comunicação, que constituem a base para toda a operação do sistema.

- Variáveis globais: armazenam estados, posições, velocidades e parâmetros compartilhados entre funções, permitindo a integração de módulos sem necessidade de passagem explícita de parâmetros.

- Structs: utilizadas para agrupar dados relacionados a eixos, buffers de interpolação e parâmetros de movimento, garantindo organização e consistência no acesso e atualização das informações.

- Endereços de comunicação: mapeiam variáveis e structs para o controle externo via rede, permitindo interação direta com os drives CiA 402 e com o interpretador, que gerencia a geração de buffers e a execução dos movimentos.

Esta estrutura centralizada assegura consistência na troca de dados entre funções, módulos e o controle externo, servindo como referência única para todo o programa.



FUNÇÕES E VARIÁVEIS GLOBAIS RELACIONADAS_

FUNÇÕES:

1. FUNCTION ProcessarLinha : BOOL
2. FUNCTION PrepararLinhaTransmissao : STRING
3. FUNCTION HMI_USB_Receiver_Transmitter
4. FUNCTION Loader_ProgGCODE : BOOL
5. FUNCTION Interpretador_GRACIA_CODE
6. FUNCTION VarTipoEsperado : INT
7. FUNCTION ExecuteFunc : BOOL
8. FUNCTION StrToReal : REAL
9. FUNCTION StrToInt : INT
10. FUNCTION StrSplit : INT
11. FUNCTION VarExiste : BOOL
12. FUNCTION PosVar : INT
13. FUNCTION JOG_X_Y_Z_PULSADO
14. FUNCTION ControlPushButtons
15. FUNCTION INTERPOLACAO_LINEAR_CIRCULAR
16. FUNCTION INTERPOLACAO_BI_PLANAR
17. FUNCTION INTERPOLACAO_BI_PLANAR_PUMPKIN
18. FUNCTION INTERPOLACAO_CONICA
19. FUNCTION ReferenceRPM : UDINT
20. FUNCTION VELOCIDADE_X_Y_Z
21. FUNCTION CalcAccelDecelFromProfile
22. FUNCTION ConvertRealDiffToTargetDINT : DINT
23. FUNCTION DRIVE_INPUT_OUTPUT
24. FUNCTION LAG_6064_TORQUE_FORCA_VELOCIDADE := BOOL
25. PROGRAM Main

VARIÁVEIS

CONSTANTES BITWISE CiA 402 STATUS / CONTROLE

CONST

```
// CONTROLWORD (6040H)
START_BIT      : INT := 1 << 0; // bit 1
ENABLE_VOLTAGE : INT := 1 << 1; // bit 2
QUICK_STOP     : INT := 1 << 2; // bit 3
ENABLE_OPERATION : INT := 1 << 3; // bit 4
NEW_SETPOINT   : INT := 1 << 4; // bit 5
IMMEDIATE      : INT := 1 << 5; // bit 6
RELATIVE       : INT := 1 << 6; // bit 7
HALT_BIT       : INT := 1 << 7; // bit 8

// STATUSWORD (6041H)
FAULT_BIT      : INT := 1 << 3; // bit 4
TARGET_REACHED : INT := 1 << 10; // bit 11
ACKNOWLEDGE    : INT := 1 << 12; // bit 13
FOLLOWING_ERROR : INT := 1 << 13; // bit 14
```

END_CONST

// --- Controle RELATIVE BIT 7 ---

```
RelativeX    : BOOL := FALSE;
RelativeY    : BOOL := FALSE;
RelativeZ    : BOOL := FALSE;
Relative     : BOOL := FALSE;
```

```
WR_PDO_6040_X := (WR_PDO_6040_X AND NOT RELATIVE) OR (RELATIVE * INT(RelativeX));
WR_PDO_6040_Y := (WR_PDO_6040_Y AND NOT RELATIVE) OR (RELATIVE * INT(RelativeY));
WR_PDO_6040_Z := (WR_PDO_6040_Z AND NOT RELATIVE) OR (RELATIVE * INT(RelativeZ));
```

```
RelativeX := Relative OR RelativeX;
RelativeY := Relative OR RelativeY;
RelativeZ := Relative OR RelativeZ;
```

// --- Controle IMMEDIATE BIT 6 ---

```
ImmediateX : BOOL := FALSE;
ImmediateY : BOOL := FALSE;
ImmediateZ : BOOL := FALSE;
Immediate  : BOOL := FALSE;
```

```
WR_PDO_6040_X := (WR_PDO_6040_X AND NOT IMMEDIATE) OR (IMMEDIATE * INT(ImmediateX));
WR_PDO_6040_Y := (WR_PDO_6040_Y AND NOT IMMEDIATE) OR (IMMEDIATE * INT(ImmediateY));
WR_PDO_6040_Z := (WR_PDO_6040_Z AND NOT IMMEDIATE) OR (IMMEDIATE * INT(ImmediateZ));
```

```
ImmediateX := Immediate OR ImmediateX;
ImmediateY := Immediate OR ImmediateY;
ImmediateZ := Immediate OR ImmediateZ;
```

VARIÁVEIS GLOBAIS / BUFFERS

```
indexBuffer_JOG : DINT := 0;
maxBuffer_JOG : DINT := 1256000;
```

```
bufferFilled_JOG : BOOL := FALSE;
bufferFilling_JOG : BOOL := FALSE;
```

```
bufferX_JOG : ARRAY[0..1256000] OF REAL;
bufferY_JOG : ARRAY[0..1256000] OF REAL;
bufferZ_JOG : ARRAY[0..1256000] OF REAL;
```

```
bufferVelX_JOG : ARRAY[0..1256000] OF UDINT;
bufferVelY_JOG : ARRAY[0..1256000] OF UDINT;
bufferVelZ_JOG : ARRAY[0..1256000] OF UDINT;
```

```
bufferTargetX_H_JOG : ARRAY[0..1256000] OF INT; // high word eixo X
bufferTargetX_L_JOG : ARRAY[0..1256000] OF INT; // low word eixo X
bufferTargetY_H_JOG : ARRAY[0..1256000] OF INT; // high word eixo Y
bufferTargetY_L_JOG : ARRAY[0..1256000] OF INT; // low word eixo Y
bufferTargetZ_H_JOG : ARRAY[0..1256000] OF INT;
bufferTargetZ_L_JOG : ARRAY[0..1256000] OF INT;
```

```

indexBuffer : DINT := 0;
maxBuffer : DINT := 1256000;
bufferFilled : BOOL := FALSE ;
bufferFilling : BOOL := FALSE;

bufferX      : ARRAY[0..1256000] OF REAL;
bufferY      : ARRAY[0..1256000] OF REAL;
bufferZ      : ARRAY[0..1256000] OF REAL;

bufferVelX   : ARRAY[0..1256000] OF UDINT;
bufferVelY   : ARRAY[0..1256000] OF UDINT;
bufferVelZ   : ARRAY[0..1256000] OF UDINT;

bufferTargetX_H : ARRAY[0..1256000] OF INT; // high word eixo X
bufferTargetX_L : ARRAY[0..1256000] OF INT; // low word eixo X
bufferTargetY_H : ARRAY[0..1256000] OF INT; // high word eixo Y
bufferTargetY_L : ARRAY[0..1256000] OF INT; // low word eixo Y
bufferTargetZ_H : ARRAY[0..1256000] OF INT;
bufferTargetZ_L : ARRAY[0..1256000] OF INT;

```

ENDEREÇOS IO CLP _ PDO ETHERCAT CiA 402

```

// ----- TARGET 607A -----
WR_PDO_607A_X_H AT %QW00 : INT;
WR_PDO_607A_X_L AT %QW01 : INT;
WR_PDO_607A_Y_H AT %QW02 : INT;
WR_PDO_607A_Y_L AT %QW03 : INT;
WR_PDO_607A_Z_H AT %QW04 : INT;
WR_PDO_607A_Z_L AT %QW05 : INT;

targetDINT_X : DINT;
targetDINT_Y : DINT;
targetDINT_Z : DINT;

// ----- CONTROLWORD 6040 -----
WR_PDO_6040_X AT %QW06 : INT;
WR_PDO_6040_Y AT %QW07 : INT;
WR_PDO_6040_Z AT %QW08 : INT;

// ----- STATUSWORD 6041 -----
RD_PDO_6041_X AT %IW09 : INT;
RD_PDO_6041_Y AT %IW10 : INT;
RD_PDO_6041_Z AT %IW11 : INT;

// ----- PROFILE VELOCITY 6081 -----
WR_PDO_6081_X AT %QW12 : UDINT;
WR_PDO_6081_Y AT %QW14 : UDINT;
WR_PDO_6081_Z AT %QW16 : UDINT;

```



```
// ----- ACCEL/DECEL 6083/6084 -----
WR_PDO_6083_X AT %QW18 : UDINT;
WR_PDO_6084_X AT %QW20 : UDINT;
WR_PDO_6083_Y AT %QW22 : UDINT;
WR_PDO_6084_Y AT %QW24 : UDINT;
WR_PDO_6083_Z AT %QW26 : UDINT;
WR_PDO_6084_Z AT %QW28 : UDINT;

// ----- RESOLVER 6064 -----
RD_PDO_6064_X AT %IW30 : DINT;
RD_PDO_6064_Y AT %IW32 : DINT;
RD_PDO_6064_Z AT %IW34 : DINT;
```

@

(*

HMI USB RECEIVER TRANSMITTER CLP

1. FUNCTION ProcessarLinha : BOOL
2. FUNCTION PrepararLinhaTransmissao : STRING
3. FUNCTION HMI_USB_Receiver_Transmitter
4. FUNCTION Loader_ProgGCODE : BOOL

A comunicação entre a Interface Homem-Máquina (HMI), o computador e o Controlador Lógico Programável (CLP) é realizada por um conjunto de funções desenvolvidas em Texto Estruturado (Structured Text – ST), responsáveis pelo intercâmbio bidirecional de informações de operação e configuração do sistema.

A função **ProcessarLinha** implementa o mecanismo de recepção de comandos enviados ao controlador. Cada mensagem é transmitida no formato textual **nome=valor**, sendo interpretada e convertida automaticamente para o respectivo tipo de dado da variável de destino. Essa abordagem permite o envio de comandos de operação, parâmetros de configuração e sinais de controle, incluindo funções de jog dos eixos, acionamento de homing, escrita de objetos de controle dos servoacionadores e demais variáveis utilizadas pelo sistema.

Em sentido oposto, a função **PrepararLinhaTransmissao** organiza as informações produzidas pelo controlador para envio à interface gráfica. Os dados são convertidos para representação textual, preservando o formato **nome=valor**, permitindo a atualização contínua da HMI com informações como posições absolutas e relativas dos eixos, velocidades, estados dos servoacionadores, indicadores de falha, monitoramento de atraso (lag), parâmetros de movimento e demais variáveis de supervisão.

A função **HMI_USB_Receiver_Transmitter** integra os mecanismos de recepção e transmissão em uma única rotina de comunicação. O sistema realiza a leitura assíncrona de dados provenientes da interface USB ou de um módulo ESP32, reconstruindo as mensagens recebidas caractere por caractere até a identificação do delimitador de fim de linha. Após a recepção completa da mensagem, seu conteúdo é encaminhado automaticamente ao interpretador de comandos. Paralelamente, a rotina transmite ciclicamente ao computador e ao módulo ESP32 o conjunto de variáveis de monitoramento, proporcionando sincronização permanente entre o controlador e a interface de operação.

O carregamento de programas é realizado pela função **Loader_ProgGCODE**, responsável por receber, validar e armazenar sequencialmente as linhas de um arquivo de programa no buffer utilizado pelo interpretador. Cada linha recebida é posicionada no vetor correspondente até que seja sinalizado o término da transmissão. Após a confirmação da recepção completa, a função aciona automaticamente o **Interpretador GRACIA_code**, iniciando a etapa de interpretação e organização das informações necessárias ao controle numérico.

Essa arquitetura estabelece uma separação entre comunicação, armazenamento e interpretação dos programas, permitindo que a transferência de dados ocorra de forma independente da execução determinística do sistema de controle. Como resultado, o controlador permanece disponível para as tarefas de tempo real, enquanto as operações de comunicação e carregamento de programas são executadas em uma camada lógica dedicada, aumentando a modularidade, a confiabilidade e a escalabilidade da aplicação.

```
(*  
  
VAR  
    HMI_ativo      : BOOL := TRUE;  
  
    JogX           : BOOL := FALSE;  
    JogY           : BOOL := FALSE;  
    JogZ           : BOOL := FALSE;  
    JogNX          : BOOL := FALSE;  
    JogNY          : BOOL := FALSE;  
    JogNZ          : BOOL := FALSE;  
  
    Potenciometro1 : UINT := 0;  
    HOMING         : BOOL := FALSE;  
END_VAR  
  
FUNCTION ProcessarLinha : BOOL  
  
VAR_INPUT  
    linha : STRING[50];  
    length : INT;  
END_VAR  
VAR  
    nome : STRING[20];  
    valor : STRING[20];  
    eq_pos : INT;  
END_VAR  
  
eq_pos := FIND('=', linha);
```

```

IF eq_pos > 0 THEN
    nome := LEFT(linha, eq_pos - 1);
    valor := RIGHT(linha, length - eq_pos);
ELSE
    RETURN(FALSE);
END_IF

CASE nome OF
    'JogX':    JogX := (valor = 'TRUE');
    'JogY':    JogY := (valor = 'TRUE');
    'JogZ':    JogZ := (valor = 'TRUE');
    'JogNX':   JogNX := (valor = 'TRUE');
    'JogNY':   JogNY := (valor = 'TRUE');
    'JogNZ':   JogNZ := (valor = 'TRUE');
    'Potenciometro1': Potenciometro1 := STRING_TO_INT(valor);
    'WR_PDO_6040_X': WR_PDO_6040_X := STRING_TO_INT(valor);
    'WR_PDO_6040_Y': WR_PDO_6040_Y := STRING_TO_INT(valor);
    'WR_PDO_6040_Z': WR_PDO_6040_Z := STRING_TO_INT(valor);
    'HOMING':   HOMING := (valor = 'TRUE');
ELSE
END_CASE

RETURN(TRUE);
END_FUNCTION

```

```

VAR
    ABSOLUTOX    : REAL := 0.0;
    ABSOLUTOY    : REAL := 0.0;
    ABSOLUTOZ    : REAL := 0.0;

    pos_rel_X_mm : REAL := 0.0;
    pos_rel_Y_mm : REAL := 0.0;
    pos_rel_Z_mm : REAL := 0.0;

    sentidoX_CW   : BOOL := FALSE;
    sentidoY_CW   : BOOL := FALSE;
    sentidoZ_CW   : BOOL := FALSE;

    LAG_LAMP_X    : BOOL := FALSE;
    LAG_LAMP_Y    : BOOL := FALSE;
    LAG_LAMP_Z    : BOOL := FALSE;

    FAULT_LAMP_X  : BOOL := FALSE;
    FAULT_LAMP_Y  : BOOL := FALSE;
    FAULT_LAMP_Z  : BOOL := FALSE;

    LagX          : REAL := 0.0;
    LagY          : REAL := 0.0;
    LagZ          : REAL := 0.0;

```

END_VAR

FUNCTION PrepararLinhaTransmissao : STRING

VAR_INPUT

step : INT; // índice da variável a enviar

END_VAR

VAR

linha : STRING[50];

END_VAR

CASE step OF

```
1: linha := CONCAT('ABSOLUTOX=', REAL_TO_STRING(ABSOLUTOX));
2: linha := CONCAT('ABSOLUTOY=', REAL_TO_STRING(ABSOLUTOY));
3: linha := CONCAT('ABSOLUTOZ=', REAL_TO_STRING(ABSOLUTOZ));
4: linha := CONCAT('pos_rel_X_mm=', REAL_TO_STRING(pos_rel_X_mm));
5: linha := CONCAT('pos_rel_Y_mm=', REAL_TO_STRING(pos_rel_Y_mm));
6: linha := CONCAT('pos_rel_Z_mm=', REAL_TO_STRING(pos_rel_Z_mm));
7: linha := CONCAT('sentidoX_CW=', BOOL_TO_STRING(sentidoX_CW));
8: linha := CONCAT('sentidoY_CW=', BOOL_TO_STRING(sentidoY_CW));
9: linha := CONCAT('sentidoZ_CW=', BOOL_TO_STRING(sentidoZ_CW));
10: linha := CONCAT('LAG_LAMP_X=', BOOL_TO_STRING(LAG_LAMP_X));
11: linha := CONCAT('LAG_LAMP_Y=', BOOL_TO_STRING(LAG_LAMP_Y));
12: linha := CONCAT('LAG_LAMP_Z=', BOOL_TO_STRING(LAG_LAMP_Z));
13: linha := CONCAT('FAULT_LAMP_X=', BOOL_TO_STRING(FAULT_LAMP_X));
14: linha := CONCAT('FAULT_LAMP_Y=', BOOL_TO_STRING(FAULT_LAMP_Y));
15: linha := CONCAT('FAULT_LAMP_Z=', BOOL_TO_STRING(FAULT_LAMP_Z));
16: linha := CONCAT('WR_PDO_6040_X=', INT_TO_STRING(WR_PDO_6040_X));
17: linha := CONCAT('WR_PDO_6040_Y=', INT_TO_STRING(WR_PDO_6040_Y));
18: linha := CONCAT('WR_PDO_6040_Z=', INT_TO_STRING(WR_PDO_6040_Z));
19: linha := CONCAT('LagX=', REAL_TO_STRING(LagX));
20: linha := CONCAT('LagY=', REAL_TO_STRING(LagY));
21: linha := CONCAT('LagZ=', REAL_TO_STRING(LagZ));
22: linha := CONCAT('VELX=', UDINT_TO_STRING(UDINT(VELX)));

23: linha := CONCAT('VELY=', UDINT_TO_STRING(UDINT(VELY)));

24: linha := CONCAT('VELZ=', UDINT_TO_STRING(UDINT(VELZ)));

25: linha := CONCAT('CONTROL_LAGXYZ=', BOOL_TO_STRING(CONTROL_LAGXYZ));

26: linha := CONCAT('VELX0=', UDINT_TO_STRING(UDINT(VELX0)));

27: linha := CONCAT('VELY0=', UDINT_TO_STRING(UDINT(VELY0)));

28: linha := CONCAT('VELZ0=', UDINT_TO_STRING(UDINT(VELZ0)));

29: linha := CONCAT('STPX=', REAL_TO_STRING(STPX));

30: linha := CONCAT('STPY=', REAL_TO_STRING(STPY));

31: linha := CONCAT('STPZ=', REAL_TO_STRING(STPZ));

32: linha := CONCAT('STPMAX=', REAL_TO_STRING(STPMax));
```

```

ELSE
    linha := ";
END_CASE

RETURN(linha);
END_FUNCTION

```

```

VAR
    tx_step      : INT := 1;
    rx_buffer     : STRING[50] := "";
    LastLineReceived : STRING[50] := "";
END_VAR

```

FUNCTION HMI_USB_Receiver_Transmitter

```

VAR_IN_OUT
    tx_step      : INT;           // estado do transmissor
    rx_buffer     : STRING[50];   // buffer de recepção
END_VAR

```

```

VAR_INPUT
    Enable       : BOOL;         // habilita execução
END_VAR

```

```

VAR_OUTPUT
    LastLineReceived : STRING[50]; // linha recebida completa
END_VAR

```

```

VAR
    total_steps : INT := 25;      // número de variáveis transmitidas
    linha_tx    : STRING[50];
    rx_byte     : STRING[1];      // leitura temporária de 1 byte
END_VAR

```

```
(* --- RECEIVER --- *)
```

```

// --- Recepção USB ---
IF Enable AND USB_ByteAvailable() THEN
    rx_byte := USB_ReadByteAsString(); // retorna STRING[1]

    IF rx_byte <> '\n' THEN
        IF LEN(rx_buffer) < 50 THEN
            rx_buffer := CONCAT(rx_buffer, rx_byte); // acumula byte no buffer
        ELSE
            rx_buffer := ""; // buffer cheio, descarta
        END_IF
    ELSE
        // linha completa
        LastLineReceived := rx_buffer;
        ProcessarLinha(rx_buffer, LEN(rx_buffer));
        rx_buffer := ""; // limpa buffer para próxima linha
    END_IF
END_IF

```

```

// --- Recepção ESP32 Wi-Fi ---
IF Enable AND ESP32_ByteAvailable() THEN
    rx_byte := ESP32_ReadByteAsString(); // retorna STRING[1]

    IF rx_byte <> '\n' THEN
        IF LEN(rx_buffer) < 50 THEN
            rx_buffer := CONCAT(rx_buffer, rx_byte); // acumula byte no buffer
        ELSE
            rx_buffer := ""; // buffer cheio, descarta
        END_IF
    ELSE
        // linha completa
        LastLineReceived := rx_buffer;
        ProcessarLinha(rx_buffer, LEN(rx_buffer));
        rx_buffer := ""; // limpa buffer para próxima linha
    END_IF
END_IF

(* --- TRANSMITTER --- *)
linha_tx := PrepararLinhaTransmissao(tx_step);

IF linha_tx <> "" THEN
    USB_WriteLine(linha_tx); // já existente
    ESP32_WriteLine(linha_tx); // adiciona envio para ESP32
END_IF

;tx_step := tx_step + 1;
IF tx_step > total_steps THEN
    tx_step := 1; // reinicia ciclo
END_IF

END_FUNCTION

(*)

```

Exemplo de Função de Carregamento de Programa GRACIA-CODE TXT >>>>> CLP

No contexto do sistema geral, a função Loader_ProgramaGCODE recebe arquivos ou linhas de texto (TXT) enviados pelo PC/HMI via rede e os armazena no array do interpretador, preparando-os para execução pelo sistema de controle.

```

*)
VAR

    rxString    : STRING[100];
    rxValid     : BOOL;
    rxDone      : BOOL;

```

```
ProgramaGCODE : ARRAY[1..200] OF STRING[100];  IndexWrite  : INT;  
  rxAck      : BOOL;  
  rxDoneAck  : BOOL;
```

```
END_VAR
```

FUNCTION Loader_ProgGCODE : BOOL

```
VAR_INPUT
```

```
  rxString  : STRING[100];  
  rxValid   : BOOL;  
  rxDone    : BOOL;
```

```
END_VAR
```

```
VAR_IN_OUT
```

```
  ProgramaGCODE : ARRAY[1..200] OF STRING;  
  IndexWrite : INT;  
  rxAck      : BOOL;  
  rxDoneAck  : BOOL;
```

```
END_VAR
```

```
IF rxValid THEN
```

```
  IF (IndexWrite >= 1) AND (IndexWrite <= UPPER_BOUND(ProgramaGCODE,1)) THEN  
    ProgramaGCODE[IndexWrite] := rxString;  
    IndexWrite := IndexWrite + 1;  
    rxAck := TRUE;
```

```
  ELSE
```

```
    rxAck := FALSE;
```

```
  END_IF;
```

```
ELSE
```

```
  rxAck := FALSE;
```

```
END_IF;
```

```
IF rxDone THEN
```

```
  Interpretador_GRACIA_CODE(ProgramaGCODE := ProgramaGCODE);
```

```
  rxDoneAck := TRUE;
```

```
ELSE
```

```
  rxDoneAck := FALSE;
```

```
END_IF;
```

```
Loader_ProgGCODE := TRUE;
```

```
(*
```

ProgramaGCODE: array de strings enviado pelo PC/HMI/ANDROID, com funções, coordenadas e parâmetros de movimento. Serve como fonte e executável para o interpretador.

O ProgramaGCODE é a estrutura de armazenamento utilizada pelo sistema para representar integralmente um programa de usinagem ou movimentação. Ele é composto por um vetor de strings recebido do PC, HMI ou dispositivo Android, onde cada linha corresponde a um comando, parâmetro ou dado de configuração interpretado sequencialmente pelo controlador. O programa é organizado em blocos delimitados pelo caractere \$, permitindo ao interpretador identificar o início e o término de cada conjunto de instruções. Cada bloco pode conter

parâmetros tecnológicos, comandos de posicionamento, ciclos de interpolação, funções especiais ou configurações de execução.

As instruções são descritas em formato textual, utilizando pares nome: valor, o que facilita a edição, transmissão e armazenamento sem dependência de estruturas binárias. O interpretador converte essas informações em movimentos e operações internas do sistema.

O arquivo suporta diferentes funções operacionais, identificadas por comandos como Função: F1, F2, F3, F4 e F5, cada uma responsável por um tipo específico de processamento, incluindo posicionamento, configuração tecnológica e geração de trajetórias geométricas.

Entre os parâmetros suportados encontram-se modos de operação, coordenadas absolutas e relativas, velocidades, potência, sentidos de interpolação, ângulos inicial e final, raios, centros geométricos, incrementos de passo, quantidades de repetições, quadrantes, offsets, massas, limites de torque, fatores de overdrive e demais variáveis necessárias à execução do processo.

O formato também permite a definição de movimentos simultâneos entre eixos, interpolações nos planos XY, ZX e ZY, além da combinação de múltiplas operações dentro de um mesmo programa, preservando a ordem exata de execução.

Ao término do carregamento, o conteúdo permanece armazenado no vetor ProgramaGCODE, tornando-se a fonte de dados utilizada pelo interpretador durante toda a execução. O processamento ocorre de forma sequencial, respeitando a organização dos blocos e a ordem das instruções até a identificação dos comandos de finalização, como DONE e Função: Fim, que encerram o programa.

```
ProgramaGCODE : ARRAY[1..200] OF STRING := [
```

```
'Função: F2',  
  'J_X: 0.000135',  
  'J_Y: 0.000135',  
  'J_Z: 0.000135',  
  'Mass_X_kg: 0',  
  'Mass_Y_kg: 0',  
  'Mass_Z_kg: 0',  
  'TorqueMax_X_Nm: 0',  
  'TorqueMax_Y_Nm: 0',  
  'TorqueMax_Z_Nm: 0',  
  'OverdriveFactor_X: 0',  
  'OverdriveFactor_Y: 0',  
  'OverdriveFactor_Z: 0',  
  '$',
```

```
Função : F1  
POT% : 1  
Mode : 3  
X : 162.5  
Y : 308.2107  
Z : 50  
$  
mode : 2
```


sentido : S0
maxLX : 175
minLX : 170
maxLY : 116.42
minLY : 111.42
passoDZ : 5
QEX : 1
QEY : 1
\$
mode : 3
X : 0,337.5006
Y : 0,191.7896
Z : 50
\$
mode : 2
sentido : S0A
maxLX : 175
minLX : 170
maxLY : 116.42
minLY : 111.42
passoDZ : 5
QEX : 1
QEY : 1
\$
\$
POT% : 1
\$
mode : 3
Z : 50
X : 0,317.6777
Y : 0,232.3223
RelativeZ : TRUE
\$
mode : 3
Z : -1
\$
POT% : 1
NI : TRUE
PXY : TRUE
SENTIDO : 0
ANGULOINICIAL : 45
ANGULOFINAL: 135
deltaX : 0.45
r : 95.71
XC : 345.71
YC : 300
\$
ANGULOINICIAL : 135
ANGULOFINAL : 225
r : 25
XC : 200
YC : 225
\$
ANGULOINICIAL : 225
ANGULOFINAL : 315
deltaX : 0.45
r : 95.71

XC : 154.2893
YC : 200
\$
ANGULOINICIAL : 315
ANGULOFINAL: 45
deltaX : 0.45
r : 25
XC : 300
YC : 275
\$
N : 24
\$
mode : 3
Z : 25
\$
mode : 3
X : 329.035
Y : 211.08
\$
NI := TRUE
mode : 3
Z : -1
\$
POT% : 1
PXY : TRUE
SENTIDO : 0
ANGULOINICIAL : 0
ANGULOFINAL : 360
deltaX : 0.45
r : 14.29
XC : 329.035
YC : 211.08
N : 24
\$
mode : 3
X : 0,288.6253
Y : 0,204.29
Z : 25,0,-25,25
\$
NI := TRUE
mode : 3
Z : -1
\$
PXY : TRUE
SENTIDO : 0
ANGULOINICIAL : 0
ANGULOFINAL : 360
deltaX : 0.45
r : 14.29
XC : 199.54
YC : 211.08
N : 24
\$
mode : 3
X : 0,211.3747
Y : 0,204.29
Z : 25,0,-25,25

\$
NI := TRUE
mode : 3
Z : -1
PXY : TRUE
SENTIDO : 0
ANGULOINICIAL : 0
ANGULOFINAL : 360
deltaX : 0.45
r : 14.29
XC : 199.54
YC : 288.92
N : 24

\$
mode : 3
X : 0,211.3747
Y : 0,295.71
Z : 25,0,-25,25
\$

\$
POT% : 1
mode : 3
X : 300.84,0,322.26
Y : 303.2
Z : 0,-25,
\$
PXY : TRUE
SENTIDO : 0
ANGULOINICIAL : 270
ANGULOFINAL : 360
deltaX : 0.45
r : 10.24
XC : 322.26
YC : 303.2

\$
mode : 3
X : 332.5
Y : 277.49
\$

PXY : TRUE
SENTIDO : 0
ANGULOINICIAL : 0
ANGULOFINAL : 133
deltaX : 0.45
r : 10.24
XC : 332.5
YC : 277.49
\$

PXY : TRUE
SENTIDO : 1
ANGULOINICIAL : 47
ANGULOFINAL : 61.33
deltaX : 0.45

r : 95.71
XC : 345.71
YC : 200
\$

PXY : TRUE
SENTIDO : 0
ANGULOINICIAL : 90
ANGULOFINAL : 270
deltaX : 0.45
r : 10.24
XC : 300.83
YC : 282.73
\$

\$
mode : 3
X : 300.84
Y : 303.21
Z : 22.26
\$
NI := TRUE
PZY : TRUE
RelativeZ : TRUE
SENTIDO : 1
ANGULOINICIAL : 0
ANGULOFINAL : 90
deltaX : 0.45
r : 2.74
ZC : 47.26
YC : 295.71
\$
P2 : TRUE
PR : TRUE
PX : 0.4371
PY : 0
PZ : 0
N : 49
\$

mode : 3
X : 0,322.26
Y : 0,300.47
Z : 50
\$
Função : F3
PXY : TRUE
ZY : TRUE
XY : TRUE
SENTIDO : 0
SENTIDO1 : 0
ANGULOINICIAL : 270
ANGULOFINAL : 360
ANGULOINICIAL1 : 270
ANGULOFINAL1 : 360
deltaX : 0.45
deltaX1 : 0.45

Q1 : TRUE
Q2 : TRUE
r1 : 2.74
ZC1 : 50
YC1 : 292.97
r : 7.5
rmax : 10.24
XC : 322.26
YC : 300.47
X0C : 322.26
Y0C : 300.47
X1C : 329.76
Y1C : 290.26
\$

Função : F1
mode : 3
X : 332.5
Y : 292.97
Z : 47.26
\$
NI := TRUE
PZX : TRUE
SENTIDO : 1
ANGULOINICIAL : 0
ANGULOFINAL : 90
deltaX : 0.45
r : 2.74
ZC : 47.26
XC : 325
\$
P2 : TRUE
PR : TRUE
PX : 0
PY : - 0.4422
PZ : 0
N : 35
\$

mode : 3
X : 0,329.76
Y : 0,277.49
Z : 50
\$

Função : F3
PXY : TRUE
XY : TRUE
ZY : TRUE
SENTIDO : 0
SENTIDO1 : 0
ANGULOINICIAL : 0
ANGULOFINAL : 133
ANGULOINICIAL1 : 270
ANGULOFINAL1 : 360
deltaX : 0.45
deltaX1 : 0.45

Q1 : TRUE
Q3 : TRUE
r1 : 2.74
XC1 : 322.26
ZC1 : 50
r : 7.5
rmax : 10.24
XC : 329.76
YC : 277.49
X0C : 329.76
Y0C : 277.49
X1C : 314.76
Y1C : 277.49

\$
Função : F1
Mode : 3
X : 0,315.2748
Y : 0,269.9965
Z : 50,0,46.24

\$
Função : F3
PXY : TRUE
XY : TRUE
ZY : TRUE
SENTIDO : 1
SENTIDO1 : 0
ANGULOINICIAL : 47
ANGULOFINAL : 61.33
ANGULOINICIAL1 : 180
ANGULOFINAL1 : 270
deltaX : 0.45
deltaX1 : 0.45
Q1 : TRUE
Q2 : TRUE
r1 : 2.74
YC1 : 274.75
ZC1 : 46.24
r : 95.71
rmax : 98.45
XC : 345.71
YC : 200
X0C : 250
Y0C : 295.71
X1C : 345.71
Y1C : 200

\$
\$
Função : F1
mode : 3
X : 0,322.245
Y : 0,211.08
Z : 50
\$

Função : F3
PXY : TRUE
XY : TRUE
ZX : TRUE
SENTIDO : 0
SENTIDO1 : 0
ANGULOINICIAL : 0
ANGULOFINAL : 360
ANGULOINICIAL1 : 270
ANGULOFINAL1 : 360
deltaX : 0.45
deltaX1 : 0.45
Q1 : TRUE
r1 : 6.79
XC1 : 314.745
ZC1 : 50
r : 7.5
rmax : 14.29
XC : 322.245
YC : 211.08
X0C : 322.245
Y0C : 211.08
X1C : 322.245
Y1C : 211.08
\$
Função : F1
mode : 3
X : 0,193.2236 offset 7.5
Y : 0,211.08
Z : 49.9835
\$

Função : F4
EXTERNO : TRUE
PXY : TRUE
XY : TRUE
ZX : TRUE
SENTIDO : 0
SENTIDO1 : 0
ANGULOINICIAL : 0
ANGULOFINAL : 360
ANGULOINICIAL1 : 274
ANGULOFINAL1 : 360
passoR : 0
deltaX : 0.45
deltaX1 : 0.45
r : 7.9865
rmax : 7.9865
XC : 193.2236
YC : 211.08
Q1 : TRUE / r1
Q2 : TRUE / r1
r1 : 6.79 +offset 0
r1max : 6.79+offset 0
XC1 : 185.25 + offset 7.5 _ 192.75
ZC1 : 50
X0C : 192.75

Z0C : 50
X1C : 199.54
Z1C : 43.21
r2 : 14.29
r2max : 14.29
XC2 : 199.54
YC2 : 211.08
\$
\$
Função : F1
mode : 3
X : 0,192.75
Y : 0,288.92
Z : 50
\$

Função : F5
PXY : TRUE
SENTIDO : 0
ANGULOINICIAL : 0
ANGULOFINAL : 360
angulocone : 90
hcone : - 6.79
deltaX : 0.45
r0 : 7.5
rmax : 14.29
Q1 : TRUE
XC : 192.75
YC : 288.92
X0C : 192.75
Y0C : 288.92
X1C : 192.75
Y1C : 288.92
\$
DONE
\$
 'Função: Fim'
];
*)

@

(*

INTERPRETADOR DE FUNÇÕES

5. FUNCTION Interpretador_GRACIA_CODE
6. FUNCTION VarTipoEsperado : INT
7. FUNCTION ExecuteFunc : BOOL
8. FUNCTION StrToReal : REAL
9. FUNCTION StrToInt : INT
10. FUNCTION StrSplit : INT

11. FUNCTION VarExiste : BOOL

12. FUNCTION PosVar : INT

*)

O núcleo de execução do presente trabalho é composto pelo **Interpretador GRACIA_code**, desenvolvido em **Texto Estruturado (Structured Text – ST)** conforme a norma **IEC 61131-3**. Sua função consiste em interpretar programas armazenados em arquivos de texto, convertendo suas instruções em estruturas de dados organizadas para posterior execução pelo sistema de controle.

O interpretador realiza inicialmente a leitura sequencial de cada linha do programa, identificando funções, parâmetros, variáveis e marcadores de controle. Para cada função reconhecida, é consultado um **dicionário interno**, responsável por definir as variáveis esperadas, seus respectivos tipos de dados e a rotina de processamento correspondente. Esse mecanismo permite que novas funções sejam incorporadas ao sistema sem alterar a estrutura geral do interpretador, bastando registrá-las no dicionário e implementar sua rotina específica.

Durante a interpretação, os valores textuais são convertidos automaticamente para seus respectivos tipos de dados (**REAL**, **INT** ou **BOOL**), incluindo também o tratamento de vetores numéricos utilizados na descrição de trajetórias dos eixos **X**, **Y** e **Z**. Após a validação dos parâmetros, cada função é executada, produzindo informações geométricas e operacionais que são armazenadas em buffers destinados ao processamento em tempo real.

Além da interpretação das instruções de movimento, o sistema organiza automaticamente blocos de usinagem, identifica eventos de início e término de geometrias, registra segmentos de trajetória, armazena parâmetros de interpolação, limites de atuação, regiões parciais de processamento, informações de elevação, mudanças de sentido, inversão de percurso e demais atributos necessários ao controle numérico desenvolvido neste trabalho.

Toda a etapa de interpretação é executada previamente à operação da máquina, permitindo que os dados sejam completamente processados e armazenados antes do início da execução. Dessa forma, durante o ciclo determinístico do controlador, as rotinas responsáveis pelo acionamento dos servoacionamentos realizam apenas a leitura dos buffers previamente preparados, reduzindo significativamente a carga computacional da tarefa de tempo real e garantindo compatibilidade com ciclos de controle da ordem de **1 ms**.

Essa arquitetura estabelece uma separação entre a fase de compilação e organização dos dados e a fase de execução determinística, permitindo que algoritmos complexos de processamento geométrico coexistam com os rigorosos requisitos temporais exigidos pelo controle de movimento em redes industriais como o **EtherCAT**.

```
ONE_DIR : BOOL;  
ANTI_COLISAO : BOOL;  
ELEVATION : BOOL
```

```
TYPE BlocoN :
```

```
STRUCT
```

```
indexAntiColisao : DINT;
```

```
totalScalePasses : INT := 1;
```

```
LimitStart1 : REAL;
```

```
LimitEnd1 : REAL;
```

```
LimitStart2 : REAL;
```

```
LimitEnd2 : REAL;
```

```
LimitStart3 : REAL;
```

```
LimitEnd3 : REAL;
```

```
LimitStart4 : REAL;
```

```
LimitEnd4 : REAL;
```

```
LimitStart5 : REAL;
```

```
LimitEnd5 : REAL;
```

```
ElevationX : ARRAY[1..5] OF REAL;
```

```
ElevationY : ARRAY[1..5] OF REAL;
```

```
ElevationZ : ARRAY[1..5] OF REAL;
```

```
Elevation : ARRAY[1..5, 1..MAX_PTS] OF tElevationPoint;
```

```
ElevationCount : ARRAY[1..5] OF INT;
```

```
SENTIDO : INT;
```

```
PartialStart1 : REAL;
```

```
PartialEnd1 : REAL;
```

```
PartialStart2 : REAL;
```

```
PartialEnd2 : REAL;
```

```
PartialStart3 : REAL;
```

```
PartialEnd3 : REAL;
```

```
PartialStart4 : REAL;
```

```
PartialEnd4 : REAL;
```

```
PartialStart5 : REAL;
```

```
PartialEnd5 : REAL;
```

```
NFtrigger : BOOL;
```

```
NFcount : INT;
```

```
N : INT;
```

```
StartBuffer: DINT;
```

```
EndBuffer : DINT;
```

```
PR : BOOL;
```

```
PR1 : BOOL;
```

```
P2 : BOOL;
```

```
PX : REAL;
PY : REAL;
PZ : REAL;
END_STRUCT
END_TYPE
```

```
TYPE VarValue :
STRUCT
  Nome   : STRING[50]; // Nome da variável
  Tipo   : INT;        // 0=REAL, 1=INT, 2=BOOL
  RealVal : REAL;
  IntVal  : INT;
  BoolVal : BOOL;
END_STRUCT
END_TYPE
```

```
TYPE tEventoParada :
STRUCT
  indexBuffer : UDINT; // posição do buffer principal
  halt        : BOOL;  // bit Halt ativo
END_STRUCT
END_TYPE
```

```
VAR
  UltimoInvertGravado : BOOL := FALSE;
  UltimoAnguloGravado : REAL := -1.0;
  UltimoSentidoGravado : INT := -999;
  IndiceBloco : UINT := 1;
  BufferN      : ARRAY[1..10000] OF BlocoN;
```

```
MaxVars : INT
FunAtivaVar : ARRAY[1..MaxVars] OF STRING[50];
  ValoresVar : ARRAY[1..MaxVars] OF VarValue;
  NumVarAtiva : INT;
  FunAtivaNome : STRING[50];
```

```
FuncoesOriginais : ARRAY[1..50] OF STRING[50];
FuncoesGCODE     : ARRAY[1..50] OF STRING[5];
Variaveis        : ARRAY[1..150] OF STRING[50];
TiposVariaveis   : ARRAY[1..150] OF STRING[50];
NumFuncoes       : INT := 50 ;
```

```
TempArray : ARRAY[1..50] OF STRING[50]; // auxiliar para StrSplit
BBufferX : ARRAY[1..100] OF REAL;
BBufferY : ARRAY[1..100] OF REAL;
BBufferZ : ARRAY[1..100] OF REAL;
CountX   : INT := 0;
  CountY   : INT := 0;
  CountZ   : INT := 0;
```

```
END_VAR;
```

```
FUNCTION Interpretador_GRACIA_CODE
```

```
VAR_INPUT
```

```
ProgramaGCODE : ARRAY[1..200] OF STRING[100];  
END_VAR
```

```
VAR
```

```
    IndexLinha : INT := 1;  
    FuncaoFim : BOOL := FALSE;  
    v : INT;  
    LinhaAtual : STRING[100];
```

```
END_VAR
```

```
BEGIN
```

```
WHILE (FuncaoFim = FALSE) AND (IndexLinha <= 200) DO
```

```
    LinhaAtual := ProgramaGCODE[IndexLinha];
```

```
IF F6 AND ELEVATION THEN
```

```
    axis.scalePassIndex := 1;  
    BufferN[IndexBloco].totalScalePasses := axis.scalePassIndex;  
    BufferN[IndexBloco].scalePassIndex := axis.scalePassIndex;  
    FOR i := 1 TO 5 DO  
        FOR j := 1 TO ElevationCount[i] DO
```

```
            BufferN[IndexBloco].Elevation[i, j].X := ElevationX[i, j];  
            BufferN[IndexBloco].Elevation[i, j].Y := ElevationY[i, j];  
            BufferN[IndexBloco].Elevation[i, j].Z := ElevationZ[i, j];
```

```
        END_FOR;
```

```
    END_FOR;
```

```
END_IF;
```

```
IF PARTIAL THEN
```

```
IF F6 AND INVERTT THEN
```

```
IF invert <> UltimoInvertGravado THEN
```

```
IF IndiceBloco > 0 THEN
```

```
    BufferN[IndexBloco - 1].EndBuffer := indexBuffer - 1;
```

```
END_IF;
```

```
    BufferN[IndexBloco].StartBuffer := indexBuffer;
```

```
    UltimoInvertGravado := invert;
```

```
    IndiceBloco := IndiceBloco + 1;
```

```
END_IF;
```

```
END_IF;
```

```
END_IF;
```

```
IF PARTIAL THEN
```

```
IF F3 AND PHELICAL AND (PHmax > 0) THEN
```

```
IF SENTIDO <> UltimoSentidoGravado THEN
```

```
IF IndiceBloco > 0 THEN
```

```
    BufferN[IndexBloco - 1].EndBuffer := indexBuffer - 1;
```

```
END_IF;
```

```
    BufferN[IndexBloco].SENTIDO := SENTIDO;
```

```
    BufferN[IndexBloco].StartBuffer := indexBuffer;
```

```
    UltimoSentidoGravado := SENTIDO;
```

```
    IndiceBloco := IndiceBloco + 1;
```

```
END_IF;
```

```
END_IF;
```

```
END_IF;
```

```

// --- Fim do programa ---
IF LinhaAtual = 'Função: Fim' THEN
    FuncaoFim := TRUE;
    EXIT;
END_IF;

// --- Início de função ---
IF LEFT(LinhaAtual,7) = 'Função:' THEN

    FunAtivaNome := MID(LinhaAtual,9,LEN(LinhaAtual)-8);

    // Busca função no dicionário
    FOR v := 1 TO NumFuncoes DO
        IF FuncoesGCODE[v] = FunAtivaNome THEN
            NumVarAtiva := StrSplit(Variaveis[v], ',', FunAtivaVar);
            EXIT;
        END_IF;
    END_FOR;

    // --- Leitura de blocos de variáveis ---
    v := 1;
    IndexLinha := IndexLinha + 1;

    WHILE (IndexLinha <= 200) DO
        LinhaAtual := ProgramaGCODE[IndexLinha];

        // Encontrou delimitador $ → executa bloco imediatamente
        IF LinhaAtual = '$' THEN

            ExecuteFunc(
                FunAtivaNome,
                ValoresVar,
                v-1,
                BBufferX, CountX,
                BBufferY, CountY,
                BBufferZ, CountZ);
            Reset := TRUE;

            ExecuteFunc(
                FunAtivaNome,
                ValoresVar,
                0,
                BBufferX, CountX,
                BBufferY, CountY,
                BBufferZ, CountZ);
            Reset := FALSE;

            v := 1; // prepara próximo bloco
            IndexLinha := IndexLinha + 1;
            CONTINUE;
        END_IF;

    IF PARTIAL THEN
    IF ((F3 AND PHELICAL AND (PHmax > 0)) OR (F6 AND INVERTT)) THEN

        BufferN[IndexBloco].LimitStart1 := LimitStart1;
        BufferN[IndexBloco].LimitEnd1 := LimitEnd1;

```

```
BufferN[IndiceBloco].LimitStart2 := LimitStart2;  
BufferN[IndiceBloco].LimitEnd2 := LimitEnd2;
```

```
BufferN[IndiceBloco].LimitStart3 := LimitStart3;  
BufferN[IndiceBloco].LimitEnd3 := LimitEnd3;
```

```
BufferN[IndiceBloco].LimitStart4 := LimitStart4;  
BufferN[IndiceBloco].LimitEnd4 := LimitEnd4;
```

```
BufferN[IndiceBloco].LimitStart5 := LimitStart5;  
BufferN[IndiceBloco].LimitEnd5 := LimitEnd5;
```

```
BufferN[IndiceBloco].PartialStart1 := PartialStart1;    BufferN[IndiceBloco].PartialEnd1 := PartialEnd1;  
BufferN[IndiceBloco].PartialStart2 := PartialStart2;    BufferN[IndiceBloco].PartialEnd2 := PartialEnd2;  
BufferN[IndiceBloco].PartialStart3 := PartialStart3;    BufferN[IndiceBloco].PartialEnd3 := PartialEnd3;  
BufferN[IndiceBloco].PartialStart4 := PartialStart4;    BufferN[IndiceBloco].PartialEnd4 := PartialEnd4;  
BufferN[IndiceBloco].PartialStart5 := PartialStart5;    BufferN[IndiceBloco].PartialEnd5 := PartialEnd5;  
END_IF;  
END_IF;
```

```
total_validos:=0;  
FOR i:=1 TO N DO  
    IF Buffer[i]<>7777 THEN  
        total_validos:=total_validos+1;  
    END_IF;  
END_FOR;
```

```
k:=0;  
pos:=0;  
  
FOR i:=1 TO N DO  
    IF Buffer[i]<>7777 THEN  
        pos:=pos+1;  
    ELSE  
        k:=k+1;  
        PctMinus[k]:=100.0*pos/total_validos;  
        PctPlus[k]:=100.0*(pos+1)/total_validos;  
    END_IF;  
END_FOR;
```

```
T1_Start:=0.0;  
T1_End:=PctMinus[1];
```

```
T2_Start:=PctPlus[1];  
T2_End:=PctMinus[2];
```

```
T3_Start:=PctPlus[2];  
T3_End:=PctMinus[3];
```

```
T4_Start:=PctPlus[3];  
T4_End:=PctMinus[4];
```

```
T5_Start:=PctPlus[4];  
T5_End:=100.0;
```

```
IF PartialStart1 < T1_Start THEN PartialStart1 := T1_Start; END_IF;
```

```
IF PartialStart1 > T1_End THEN PartialStart1 := T1_End; END_IF;  
IF PartialEnd1 < T1_Start THEN PartialEnd1 := T1_Start; END_IF;  
IF PartialEnd1 > T1_End THEN PartialEnd1 := T1_End; END_IF;  
IF LimitStart1 < T1_Start THEN LimitStart1 := T1_Start; END_IF;  
IF LimitStart1 > T1_End THEN LimitStart1 := T1_End; END_IF;  
IF LimitEnd1 < T1_Start THEN LimitEnd1 := T1_Start; END_IF;  
IF LimitEnd1 > T1_End THEN LimitEnd1 := T1_End; END_IF;
```

```
IF PartialStart2 < T2_Start THEN PartialStart2 := T2_Start; END_IF;  
IF PartialStart2 > T2_End THEN PartialStart2 := T2_End; END_IF;  
IF PartialEnd2 < T2_Start THEN PartialEnd2 := T2_Start; END_IF;  
IF PartialEnd2 > T2_End THEN PartialEnd2 := T2_End; END_IF;  
IF LimitStart2 < T2_Start THEN LimitStart2 := T2_Start; END_IF;  
IF LimitStart2 > T2_End THEN LimitStart2 := T2_End; END_IF;  
IF LimitEnd2 < T2_Start THEN LimitEnd2 := T2_Start; END_IF;  
IF LimitEnd2 > T2_End THEN LimitEnd2 := T2_End; END_IF;
```

```
IF PartialStart3 < T3_Start THEN PartialStart3 := T3_Start; END_IF;  
IF PartialStart3 > T3_End THEN PartialStart3 := T3_End; END_IF;  
IF PartialEnd3 < T3_Start THEN PartialEnd3 := T3_Start; END_IF;  
IF PartialEnd3 > T3_End THEN PartialEnd3 := T3_End; END_IF;  
IF LimitStart3 < T3_Start THEN LimitStart3 := T3_Start; END_IF;  
IF LimitStart3 > T3_End THEN LimitStart3 := T3_End; END_IF;  
IF LimitEnd3 < T3_Start THEN LimitEnd3 := T3_Start; END_IF;  
IF LimitEnd3 > T3_End THEN LimitEnd3 := T3_End; END_IF;
```

```
IF PartialStart4 < T4_Start THEN PartialStart4 := T4_Start; END_IF;  
IF PartialStart4 > T4_End THEN PartialStart4 := T4_End; END_IF;  
IF PartialEnd4 < T4_Start THEN PartialEnd4 := T4_Start; END_IF;  
IF PartialEnd4 > T4_End THEN PartialEnd4 := T4_End; END_IF;  
IF LimitStart4 < T4_Start THEN LimitStart4 := T4_Start; END_IF;  
IF LimitStart4 > T4_End THEN LimitStart4 := T4_End; END_IF;  
IF LimitEnd4 < T4_Start THEN LimitEnd4 := T4_Start; END_IF;  
IF LimitEnd4 > T4_End THEN LimitEnd4 := T4_End; END_IF;
```

```
IF PartialStart5 < T5_Start THEN PartialStart5 := T5_Start; END_IF;  
IF PartialStart5 > T5_End THEN PartialStart5 := T5_End; END_IF;  
IF PartialEnd5 < T5_Start THEN PartialEnd5 := T5_Start; END_IF;  
IF PartialEnd5 > T5_End THEN PartialEnd5 := T5_End; END_IF;  
IF LimitStart5 < T5_Start THEN LimitStart5 := T5_Start; END_IF;  
IF LimitStart5 > T5_End THEN LimitStart5 := T5_End; END_IF;  
IF LimitEnd5 < T5_Start THEN LimitEnd5 := T5_Start; END_IF;  
IF LimitEnd5 > T5_End THEN LimitEnd5 := T5_End; END_IF;
```

```
PartialStart1 :=  
  T1_Start +  
  (PartialStart1 / 100.0) *  
  (T1_End - T1_Start);
```

```
PartialEnd1 :=  
  T1_Start +  
  (PartialEnd1 / 100.0) *  
  (T1_End - T1_Start);
```

```
LimitStart1 :=  
  T1_Start +  
  (LimitStart1 / 100.0) *
```

$(T1_End - T1_Start);$

LimitEnd1 :=
T1_Start +
 $(LimitEnd1 / 100.0) * (T1_End - T1_Start);$

PartialStart2 :=
T2_Start +
 $(PartialStart2 / 100.0) * (T2_End - T2_Start);$

PartialEnd2 :=
T2_Start +
 $(PartialEnd2 / 100.0) * (T2_End - T2_Start);$

LimitStart2 :=
T2_Start +
 $(LimitStart2 / 100.0) * (T2_End - T2_Start);$

LimitEnd2 :=
T2_Start +
 $(LimitEnd2 / 100.0) * (T2_End - T2_Start);$

PartialStart3 :=
T3_Start +
 $(PartialStart3 / 100.0) * (T3_End - T3_Start);$

PartialEnd3 :=
T3_Start +
 $(PartialEnd3 / 100.0) * (T3_End - T3_Start);$

LimitStart3 :=
T3_Start +
 $(LimitStart3 / 100.0) * (T3_End - T3_Start);$

LimitEnd3 :=
T3_Start +
 $(LimitEnd3 / 100.0) * (T3_End - T3_Start);$

PartialStart4 :=
T4_Start +
 $(PartialStart4 / 100.0) * (T4_End - T4_Start);$

PartialEnd4 :=
T4_Start +
 $(PartialEnd4 / 100.0) * (T4_End - T4_Start);$


```
LimitStart4 :=  
    T4_Start +  
    (LimitStart4 / 100.0) *  
    (T4_End - T4_Start);
```

```
LimitEnd4 :=  
    T4_Start +  
    (LimitEnd4 / 100.0) *  
    (T4_End - T4_Start);
```

```
PartialStart5 :=  
    T5_Start +  
    (PartialStart5 / 100.0) *  
    (T5_End - T5_Start);
```

```
PartialEnd5 :=  
    T5_Start +  
    (PartialEnd5 / 100.0) *  
    (T5_End - T5_Start);
```

```
LimitStart5 :=  
    T5_Start +  
    (LimitStart5 / 100.0) *  
    (T5_End - T5_Start);
```

```
LimitEnd5 :=  
    T5_Start +  
    (LimitEnd5 / 100.0) *  
    (T5_End - T5_Start);
```

```
// topo de bloco gcode  
// --- NI como evento de linha ---  
IF LinhaAtual = 'NI' THEN  
    BufferN[IndexBloco].StartBuffer := indexBuffer;  
END_IF;
```

```
// --- Detectar fechamento da geometria ---  
// final de geometria vem de cada função  
IF NF THEN
```

```
    BufferN[IndexBloco].EndBuffer := indexBuffer;  
    BufferN[IndexBloco].NFtrigger := TRUE;  
    BufferN[IndexBloco].NFcount := BufferN[IndexBloco].NFcount + 1;
```

```
    NF := FALSE;
```

```
END_IF;
```

```
// --- N como evento de linha ---  
IF (LEFT(LinhaAtual,1) = 'N') AND (LinhaAtual <> 'NI') THEN
```

```

BufferN[IndiceBloco].N :=
    StrToInt(MID(LinhaAtual,2,LEN(LinhaAtual)-1));

BufferN[IndiceBloco].PR := PR;
BufferN[IndiceBloco].PR1 := PR1;

BufferN[IndiceBloco].P2 := axis.P2;

BufferN[IndiceBloco].PX := axis.PX;
BufferN[IndiceBloco].PY := axis.PY;
BufferN[IndiceBloco].PZ := axis.PZ;

IndiceBloco := IndiceBloco + 1;

END_IF;

// --- Sincroniza eixo ativo com o último bloco gravado ---
axis.N      := BufferN[IndiceBloco - 1].N;
axis.StartBuffer := BufferN[IndiceBloco - 1].StartBuffer;
axis.EndBuffer  := BufferN[IndiceBloco - 1].EndBuffer;
axis.PR := BufferN[IndiceBloco - 1].PR;
axis.PR1 := BufferN[IndiceBloco - 1].PR1;
axis.P2 := BufferN[IndiceBloco - 1].P2;
axis.PX := BufferN[IndiceBloco - 1].PX;
axis.PY := BufferN[IndiceBloco - 1].PY;
axis.PZ := BufferN[IndiceBloco - 1].PZ;

v := 1; // reinicia índice para novo bloco

WHILE (IndexLinha <= 200) DO
    LinhaAtual := ProgramaGCODE[IndexLinha];

    IF LinhaAtual = '$' THEN
        ExecuteFunc(
            FunAtivaNome,
            ValoresVar,
            v-1,
            BBufferX, CountX,
            BBufferY, CountY,
            BBufferZ, CountZ);
        v := 1;
        IndexLinha := IndexLinha + 1;
        EXIT;
    END_IF;

    // Converte linha para variável imediatamente
    ValoresVar[v].Nome := FunAtivaVar[v];
    ValoresVar[v].Tipo := VarTipoEsperado(FunAtivaVar[v], FunAtivaNome);

    CASE ValoresVar[v].Tipo OF
        0: ValoresVar[v].RealVal := StrToReal(LinhaAtual);
        1: ValoresVar[v].IntVal := StrToInt(LinhaAtual);

```

```

        2: ValoresVar[v].BoolVal := StrToBool(LinhaAtual);
    END_CASE;

    v := v + 1;
    IndexLinha := IndexLinha + 1;
END_WHILE;
CONTINUE;
END_IF;

```

// --- Conversão de variável X/Y/Z ou normal ---

```
IF (FunAtivaVar[v] = 'X') OR (FunAtivaVar[v] = 'Y') OR (FunAtivaVar[v] = 'Z') THEN
```

```
IF FIND(LinhaAtual, ',') > 0 THEN
```

```
// Array detectado
```

```
CASE FunAtivaVar[v] OF
```

```
'X':
```

```
    CountX := StrSplit(LinhaAtual, ',', TempArray);
```

```
    FOR i := 1 TO CountX DO
```

```
        BBufferX[i] := StrToReal(TempArray[i]);
```

```
    END_FOR
```

```
'Y':
```

```
    CountY := StrSplit(LinhaAtual, ',', TempArray);
```

```
    FOR i := 1 TO CountY DO
```

```
        BBufferY[i] := StrToReal(TempArray[i]);
```

```
    END_FOR
```

```
'Z':
```

```
    CountZ := StrSplit(LinhaAtual, ',', TempArray);
```

```
    FOR i := 1 TO CountZ DO
```

```
        BBufferZ[i] := StrToReal(TempArray[i]);
```

```
    END_FOR
```

```
END_CASE;
```

```
ELSE
```

```
// Valor único → mantém comportamento antigo
```

```
CASE ValoresVar[v].Tipo OF
```

```
    0: ValoresVar[v].RealVal := StrToReal(LinhaAtual);
```

```
    1: ValoresVar[v].IntVal := StrToInt(LinhaAtual);
```

```
    2: ValoresVar[v].BoolVal := StrToBool(LinhaAtual);
```

```
END_CASE;
```

```
END_IF;
```

```
ELSE
```

```
// Variável normal → comportamento atual
```

```
CASE ValoresVar[v].Tipo OF
```

```
    0: ValoresVar[v].RealVal := StrToReal(LinhaAtual);
```

```
    1: ValoresVar[v].IntVal := StrToInt(LinhaAtual);
```

```
    2: ValoresVar[v].BoolVal := StrToBool(LinhaAtual);
```

```
END_CASE;
```

```
END_IF;
```

```
v := v + 1;
```

```
IndexLinha := IndexLinha + 1;
```

```
END_WHILE; // fim leitura bloco função
```

```
END_IF; // fim IF função
```

```
IndexLinha := IndexLinha + 1;
```

```
END_WHILE;
```

END_FUNCTION

// --- Retorna tipo da variável ---

FUNCTION VarTipoEsperado : INT

VAR_INPUT

NomeVar : STRING[50];

NomeFunc : STRING[50];

END_VAR

VAR

i,t : INT;

VarNames : ARRAY[1..50] OF STRING;

Tipos : ARRAY[1..50] OF STRING;

NumVars : INT;

END_VAR

BEGIN

VarTipoEsperado := 0;

FOR i := 1 TO NumFuncoes DO

IF FuncoesGCODE[i] = NomeFunc THEN

NumVars := StrSplit(Variaveis[i], ',', VarNames);

StrSplit(TiposVariaveis[i], ',', Tipos);

FOR t := 1 TO NumVars DO

IF VarNames[t] = NomeVar THEN

IF Tipos[t] = 'REAL' THEN VarTipoEsperado := 0;

ELSIF Tipos[t] = 'INT' THEN VarTipoEsperado := 1;

ELSIF Tipos[t] = 'BOOL' THEN VarTipoEsperado := 2;

END_IF

EXIT;

END_IF

END_FOR

END_IF

END_FOR

END_FUNCTION

// --- Executa função (placeholder robusto) ---

FUNCTION ExecuteFunc : BOOL

VAR_INPUT

NomeFunc : STRING[50];

Valores : ARRAY[1..50] OF VarValue;

BBufferX : ARRAY[1..100] OF REAL;

CountX : INT;

BBufferY : ARRAY[1..100] OF REAL;

CountY : INT;

BBufferZ : ARRAY[1..100] OF REAL;

CountZ : INT;

END_VAR

BEGIN

IF NomeFunc = 'F1' THEN

F1(Valores, BBufferX, CountX, BBufferY, CountY, BBufferZ, CountZ);

ELSIF NomeFunc = 'F2' THEN

F2(Valores);

ELSIF NomeFunc = 'F3' THEN

F3(Valores, BBufferX, CountX, BBufferY, CountY, BBufferZ, CountZ);

ELSIF NomeFunc = 'F4' THEN

```

    F4(Valores , BBufferX, CountX, BBufferY, CountY, BBufferZ, CountZ);
ELSIF NomeFunc = 'F5' THEN
    F5(Valores , BBufferX, CountX, BBufferY, CountY, BBufferZ, CountZ);
ELSIF NomeFunc = 'F6' THEN
    F6(Valores, BBufferX, CountX,BBufferY, CountY, BBufferZ, CountZ);
END_IF;ExecuteFunc := TRUE;
END_FUNCTION

```

// --- Funções utilitárias ---

FUNCTION StrToReal : REAL

```

VAR_INPUT
    S : STRING[50];
END_VAR
BEGIN
    StrToReal := REAL(STRING_TO_REAL(S));
END_FUNCTION

```

FUNCTION StrToInt : INT

```

VAR_INPUT
    S : STRING[50];
END_VAR
BEGIN
    StrToInt := STRING_TO_INT(S);
END_FUNCTION

```

FUNCTION StrToBool : BOOL

```

VAR_INPUT
    S : STRING[50];
END_VAR
BEGIN
    IF (UPPER(S)='1') OR (UPPER(S)='TRUE') THEN
        StrToBool := TRUE;
    ELSE
        StrToBool := FALSE;
    END_IF
END_FUNCTION

```

// --- Divide texto por delimitador ---

FUNCTION StrSplit : INT

```

VAR_INPUT
    Texto : STRING[255];
    Delimiter : STRING[1];
END_VAR
VAR_IN_OUT
    Partes : ARRAY[1..50] OF STRING[50];
END_VAR
VAR
    i, lastPos, count, len : INT;
    token : STRING[50];

```

```

END_VAR
BEGIN
len := LEN(Texto);
lastPos := 1;
count := 0;
FOR i := 1 TO len DO
    IF MID(Texto,i,1) = Delimiter THEN
        count := count + 1;
        token := MID(Texto,lastPos,i-lastPos);
        Partes[count] := TRIM(token);
        lastPos := i+1;
    END_IF
END_FOR
IF lastPos <= len THEN
    count := count + 1;
    token := MID(Texto,lastPos,len-lastPos+1);
    Partes[count] := TRIM(token);
END_IF
StrSplit := count;
END_FUNCTION

```

// --- Verifica se a variável existe ---

FUNCTION VarExiste : BOOL

```

VAR_INPUT
    nomeVar : STRING[50];
END_VAR
VAR
    i : INT;
END_VAR
BEGIN
    VarExiste := FALSE;
    FOR i := 1 TO NumVarAtiva DO
        IF ValoresVar[i].Nome = nomeVar THEN
            VarExiste := TRUE;
            EXIT;
        END_IF;
    END_FOR;
END_FUNCTION

```

// --- Retorna o índice da variável ---

FUNCTION PosVar : INT

```

VAR_INPUT
    nomeVar : STRING[50];
END_VAR
VAR
    i : INT;
END_VAR
BEGIN
    PosVar := -1; // -1 indica que não foi encontrada
    FOR i := 1 TO NumVarAtiva DO
        IF ValoresVar[i].Nome = nomeVar THEN
            PosVar := i;

```

```
EXIT;  
END_IF;  
END_FOR;  
END_FUNCTION
```

@

(*

A operação manual da máquina é implementada por um conjunto de funções responsáveis pela leitura dos dispositivos físicos do painel de comando, pela geração de movimentos incrementais dos eixos e pelo gerenciamento dos sinais de controle dos servoacionamentos.

A função **JOG_X_Y_Z_PULSADO** implementa o modo de deslocamento manual dos eixos **X**, **Y** e **Z**. Sua lógica interpreta os comandos provenientes dos botões físicos de avanço e retorno de cada eixo, permitindo deslocamentos incrementais durante o acionamento momentâneo dos pulsadores. Quando o operador mantém um botão pressionado, a função utiliza temporizadores internos para produzir incrementos sucessivos após um intervalo inicial de espera, reproduzindo o comportamento típico de repetição automática empregado em interfaces industriais.

Cada incremento de posição é convertido para a representação numérica utilizada pelos servoacionadores, sendo posteriormente armazenado em buffers específicos de posição e velocidade. Durante esse processo, os valores em milímetros são transformados para o formato de posição absoluta requerido pelo sistema de controle, incluindo a separação dos registradores de 32 bits em palavras de alta e baixa ordem destinadas à comunicação com os drives. Simultaneamente, a velocidade correspondente a cada deslocamento é calculada e registrada nos respectivos buffers, permitindo que a execução ocorra de forma determinística pela tarefa de controle em tempo real.

A função também implementa a sequência de **homing** da máquina. Inicialmente, o eixo **Z** é conduzido individualmente à posição de referência, reduzindo o risco de colisões durante o procedimento. Após a confirmação do estado **Target Reached** pelo servoacionador, os eixos **X** e **Y** são sincronizados e encaminhados às respectivas posições de referência, concluindo automaticamente o processo de referenciamento da máquina.

O gerenciamento dos buffers de movimento constitui outro aspecto importante dessa função. À medida que novos pontos são produzidos, os índices de armazenamento são atualizados até que o conjunto de dados esteja completamente preparado para execução. Ao término do preenchimento, a função sinaliza que o buffer encontra-se disponível para leitura, atualiza os

índices finais de cada eixo e libera as rotinas responsáveis pela interpolação e envio das trajetórias aos servoacionadores.

Complementando a operação manual, a função **ControlPushButtons** realiza a interface direta entre os dispositivos físicos do painel elétrico e os objetos de controle definidos pelo perfil **CiA 402**. Os estados dos botões são convertidos em comandos destinados ao objeto **Controlword (0x6040)**, permitindo acionar funções como **Enable Voltage**, **Quick Stop**, **Enable Operation**, **Halt**, **Fault Reset** e demais comandos previstos pela máquina de estados dos servoacionadores.

Além do envio dos comandos de controle, a função realiza continuamente a leitura do objeto **Statusword (0x6041)** de cada servoacionador para monitorar condições operacionais e de falha. A partir dessas informações são atualizados os indicadores luminosos do painel, sinalizando estados como erro de seguimento (*Following Error*) e condição de falha (*Fault*), proporcionando ao operador uma supervisão imediata do estado de cada eixo.

A separação entre a geração dos movimentos manuais, o gerenciamento dos buffers de trajetória e a manipulação dos objetos CiA 402 contribui para uma arquitetura modular, na qual a interação do operador ocorre de forma independente das tarefas determinísticas responsáveis pelo controle de movimento em tempo real. Essa organização reduz a carga computacional das rotinas críticas e facilita a manutenção, expansão e reutilização dos módulos desenvolvidos.

13. FUNCTION JOG_X_Y_Z_PULSADO

14. FUNCTION ControlPushButtons

*)

FUNCTION JOG_X_Y_Z_PULSADO

```
VAR_INPUT
    // --- Variáveis internas do pulsador ---
    // Mapear os endereços físicos para os botões internos
    BtnX_JOG : BOOL ; // botão físico X
    BtnY_JOG : BOOL ; // botão físico Y
    BtnZ_JOG : BOOL ; // botão físico Z
    BtnZ_JOGN : BOOL ; // bbotão físico Z
    BtnX_JOGN : BOOL ; // botão físico X
    BtnY_JOGN : BOOL ; // botão físico Y
    BtnXYZ_HOMING : BOOL ; // botão físico homing
```

```
rpmX , rpmY , rpmZ : UDINT ;
```

```
END_VAR
VAR_OUTPUT
```

```
diffX_mm := X;
diffY_mm := Y;
diffZ_mm := Z;
END_VAR
```



```

VAR
  BtnX_Last   : BOOL := FALSE;
  BtnY_Last   : BOOL := FALSE;
  BtnZ_Last   : BOOL := FALSE;
  BtnXN_Last  : BOOL := FALSE;
  BtnYN_Last  : BOOL := FALSE;
  BtnZN_Last  : BOOL := FALSE;

  BtnTimerYN  : REAL := 0.0;
  BtnTimerXN  : REAL := 0.0;
  BtnTimerZN  : REAL := 0.0;
  BtnTimerX   : REAL := 0.0;
  BtnTimerY   : REAL := 0.0;
  BtnTimerZ   : REAL := 0.0;

  // Parâmetros de tempo
  PulseInterval : REAL := 0.5; // incremento contínuo a cada 0.5 s
  InitialDelay  : REAL := 1.0; // espera 1 s antes do contínuo
  CycleTime     : REAL := 0.01; // tempo real do ciclo do PLC
END_VAR

BEGIN

  IF NOT MANUAL THEN
    RETURN;
  END_IF;

  indexBuffer_JOG := 0;

  // --- Botão Homing ---
  IF BtnXYZ_HOMING THEN
    // --- Etapa 1: Força Z para home e salva apenas Z ---
    RelativeX := FALSE ;
    RelativeY := FALSE ;
    RelativeZ := FALSE ;
    Z := 500; // modelo de sinais usa home alto em Z
    bufferZ_JOG[indexBuffer_JOG] := Z;
    indexBuffer_JOG := indexBuffer_JOG + 1; axisZ.flags.singlePoint := TRUE;

    bufferFilled_JOG := TRUE;
    // --- Verificação se Z chegou na posição ---
    IF (RD_PDO_6041_Z AND TARGET_REACHED) <> 0
      AND (RD_PDO_6041_Z AND ACKNOWLEDGE) = 0 THEN

      // --- Etapa 2: Z chegou, sincroniza X e Y ---
      X := 0;
      Y := 0;

      // --- Atualiza buffers X e Y ---
      bufferX_JOG[indexBuffer_JOG] := X;
      bufferY_JOG[indexBuffer_JOG] := Y;

      indexBuffer_JOG := indexBuffer_JOG + 1;

      axisX.flags.singlePoint := TRUE;

```

```
axisY.flags.singlePoint := TRUE;
```

```
bufferFilled_JOG := TRUE;
```

```
// --- Pulsador X ---
```

```
IF BtnX_JOG AND NOT BtnX_Last THEN
```

```
axisX.ready := TRUE;
```

```
    bufferFilling := TRUE;
```

```
    X := X + 1.0; // incremento inicial
```

```
    BtnTimerX := 0.0;
```

```
ELSIF BtnX_JOG AND BtnX_Last THEN
```

```
    BtnTimerX := BtnTimerX + CycleTime;
```

```
    IF BtnTimerX >= InitialDelay THEN
```

```
        IF ((BtnTimerX - InitialDelay) MOD PulseInterval) < CycleTime THEN
```

```
            X := X + 1.0;
```

```
        END_IF;
```

```
    END_IF;
```

```
ELSIF NOT BtnX_JOG AND BtnX_Last THEN
```

```
    bufferFilling_JOG := FALSE; // botão solto
```

```
    bufferFilled_JOG := TRUE; // libera leitura do buffer
```

```
END_IF;
```

```
BtnX_Last := BtnX_JOG ;
```

```
// --- Pulsador X Negativo ---
```

```
IF BtnX_JOGR AND NOT BtnXN_Last THEN
```

```
axisX.ready := TRUE;
```

```
    bufferFilling_JOGR := TRUE;
```

```
    X := X - 1.0; // decremento inicial
```

```
    BtnTimerXN := 0.0;
```

```
ELSIF BtnX_JOGR AND BtnXN_Last THEN
```

```
    BtnTimerXN := BtnTimerXN + CycleTime;
```

```
    IF BtnTimerXN >= InitialDelay THEN
```

```
        IF ((BtnTimerXN - InitialDelay) MOD PulseInterval) < CycleTime THEN
```

```
            X := X - 1.0; // decremento contínuo
```

```
        END_IF;
```

```
    END_IF;
```

```
ELSIF NOT BtnX_JOGR AND BtnXN_Last THEN
```

```
    bufferFilling_JOGR := FALSE; // botão solto
```

```
    bufferFilled_JOGR := TRUE; // libera leitura do buffer
```

```
END_IF;
```

```
BtnXN_Last := BtnX_JOGR;
```

```
// --- Pulsador Y ---
```

```
IF BtnY_JOG AND NOT BtnY_Last THEN
```

```
axisY.ready := TRUE;
```

```
bufferFilling_JOG := TRUE;
```

```
Y := Y + 1.0; // incremento inicial
```

```
BtnTimerY := 0.0;
```

```

ELSIF BtnY_JOG AND BtnY_Last THEN
    BtnTimerY := BtnTimerY + CycleTime;
    IF BtnTimerY >= InitialDelay THEN
        IF ((BtnTimerY - InitialDelay) MOD PulseInterval) < CycleTime THEN
            Y := Y + 1.0;
        END_IF;
    END_IF;

ELSIF NOT BtnY_JOG AND BtnY_Last THEN
    bufferFilling_JOG := FALSE; // botão solto
    bufferFilled_JOG := TRUE; // libera leitura do buffer
END_IF;
BtnY_Last := BtnY_JOG ;

// --- Pulsador Y Negativo ---
IF BtnY_JOGN AND NOT BtnYN_Last THEN
    axisY.ready := TRUE;
    bufferFilling_JOG := TRUE;
    Y := Y - 1.0; // decremento inicial
    BtnTimerYN := 0.0;

ELSIF BtnY_JOGN AND BtnYN_Last THEN
    BtnTimerYN := BtnTimerYN + CycleTime;
    IF BtnTimerYN >= InitialDelay THEN
        IF ((BtnTimerYN - InitialDelay) MOD PulseInterval) < CycleTime THEN
            Y := Y - 1.0; // decremento contínuo
        END_IF;
    END_IF;

ELSIF NOT BtnY_JOGN AND BtnYN_Last THEN
    bufferFilling_JOG := FALSE;
    bufferFilled_JOG := TRUE;
END_IF;
BtnYN_Last := BtnY_JOGN;

// --- Pulsador Z ---
IF BtnZ_JOG AND NOT BtnZ_Last THEN
    axisZ.ready := TRUE;
    bufferFilling_JOG := TRUE;
    Z := Z + 1.0; // incremento inicial
    BtnTimerZ := 0.0;

ELSIF BtnZ_JOG AND BtnZ_Last THEN
    BtnTimerZ := BtnTimerZ + CycleTime;
    IF BtnTimerZ >= InitialDelay THEN
        IF ((BtnTimerZ - InitialDelay) MOD PulseInterval) < CycleTime THEN
            Z := Z + 1.0; // incremento contínuo
        END_IF;
    END_IF;

NOT BtnZ_JOG AND BtnZ_Last THEN
    bufferFilling_JOG := FALSE; // botão solto
    bufferFilled_JOG := TRUE; // libera leitura do buffer
END_IF;
BtnZ_Last := BtnZ_JOG;

```

```

// --- Pulsador Z Negativo ---
IF BtnZ_JOGN AND NOT BtnZN_Last THEN
    axisZ.ready := TRUE;
    bufferFilling_JOG := TRUE;
    Z := Z - 1.0;    // decremento inicial
    BtnTimerZN := 0.0;

ELSIF BtnZ_JOGN AND BtnZN_Last THEN
    BtnTimerZN := BtnTimerZN + CycleTime;
    IF BtnTimerZN >= InitialDelay THEN
        IF ((BtnTimerZN - InitialDelay) MOD PulseInterval) < CycleTime THEN
            Z := Z - 1.0; // decremento contínuo
        END_IF;
    END_IF;

ELSIF NOT BtnZ_JOGN AND BtnZN_Last THEN
    bufferFilling_JOG := FALSE;
    bufferFilled_JOG := TRUE;
END_IF;
BtnZN_Last := BtnZ_JOGN;

// --- Atualiza buffers do drive ---
bufferX_JOG[indexBuffer_JOG] := X;
bufferY_JOG[indexBuffer_JOG] := Y;
bufferZ_JOG[indexBuffer_JOG] := Z;

targetDINT_X := ConvertRealDiffToTargetDINT(X);
targetDINT_Y := ConvertRealDiffToTargetDINT(Y);
targetDINT_Z := ConvertRealDiffToTargetDINT(Z);

bufferTargetX_H_JOG[indexBuffer_JOG] := INT(SHR(UDINT(targetDINT_X),16) AND 65535);
bufferTargetX_L_JOG[indexBuffer_JOG] := INT(UDINT(targetDINT_X) AND 65535);

bufferTargetY_H_JOG[indexBuffer_JOG] := INT(SHR(UDINT(targetDINT_Y),16) AND 65535);
bufferTargetY_L_JOG[indexBuffer_JOG] := INT(UDINT(targetDINT_Y) AND 65535);

bufferTargetZ_H_JOG[indexBuffer_JOG] := INT(SHR(UDINT(targetDINT_Z),16) AND 65535);
bufferTargetZ_L_JOG[indexBuffer_JOG] := INT(UDINT(targetDINT_Z) AND 65535);

diffX_mm := X;
diffY_mm := Y;
diffZ_mm := Z;

VELOCIDADE_X_Y_Z(diffX_mm, diffY_mm, diffZ_mm, rpmX, rpmY, rpmZ);
bufferVelX_JOG[indexBuffer_JOG] := rpmX;
bufferVelY_JOG[indexBuffer_JOG] := rpmY;
bufferVelZ_JOG[indexBuffer_JOG] := rpmZ;

// --- Incrementa buffer ---
indexBuffer_JOG := indexBuffer_JOG + 1;

IF indexBuffer_JOG >= maxBuffer_JOG THEN
    bufferFilled_JOG := TRUE;
END_IF;

```

```

// --- Finaliza status ---

// --- Finaliza status para todos os eixos ---
IF bufferFilled_JOG THEN

    // Atualiza índice final de cada eixo
    axisX.lastIndex := indexBuffer_JOG - 1;
    axisY.lastIndex := indexBuffer_JOG - 1;
    axisZ.lastIndex := indexBuffer_JOG - 1;

    // Desliga ready para todos os eixos
    axisX.ready := FALSE;
    axisY.ready := FALSE;
    axisZ.ready := FALSE;

    // Reseta índice do buffer
    indexBuffer_JOG := 0;

ELSE
    // Caso buffer não esteja cheio, limpa lastIndex
    axisX.lastIndex := -1;
    axisY.lastIndex := -1;
    axisZ.lastIndex := -1;

END_IF;
END_FUNCTION;

```

(*

ControlPushButtons – lê botões do painel/HMI, , e acende indicadores de erro/falha.

*)

```

VAR
MANUAL : BOOL := %IX54.0 ; // Endereço provisório
...
Potenciometro2 : UINT := %IW48;

LAG_LAMP_X : BOOL := %Q50.0;
FAULT_LAMP_X : BOOL := %Q50.1;

LAG_LAMP_Y : BOOL := %Q52.4;
FAULT_LAMP_Y : BOOL := %Q52.5;

LAG_LAMP_Z : BOOL := %Q52.6;
FAULT_LAMP_Z : BOOL := %Q52.7;

START_BIT_BUTTON : BOOL := %IX51.0;
ENABLE_VOLTAGE_BUTTON : BOOL := %IX51.1;
QUICK_STOP_BUTTON : BOOL := %IX51.2;
ENABLE_OPERATION_BUTTON : BOOL := %IX51.3;
HALT_BIT_BUTTON : BOOL := %IX51.4;

```

```

BtnX_JOG    : BOOL := %IX51.5;
BtnY_JOG    : BOOL := %IX51.6;
BtnXYZ_HOMING : BOOL := %IX51.7;

BtnX_JOGN : BOOL := %IX52.0;
BtnY_JOGN : BOOL := %IX52.1;
BtnZ_JOGN : BOOL := %IX52.2;
BtnZ_JOG  : BOOL := %IX52.3;

QS_CLEAR_BUTTON : BOOL := %IX53.0;
FAULT_CLEAR_BUTTON : BOOL := %IX53.1;
HALT_CLEAR_BUTTON : BOOL := %IX53.2;

HOMEX : BOOL := %IX53.2;
HOMEY : BOOL := %IX53.3;
HOMEZ : BOOL := %IX53.4;

MICRO1 : BOOL := %IX53.5;
MICRO2 : BOOL := %IX53.6;
MICRO3 : BOOL := %IX53.7;
END_VAR

```

FUNCTION ControlPushButtons

```
BEGIN
```

```

WR_PDO_6040_X :=
  (WR_PDO_6040_X AND NOT
    (START_BIT OR ENABLE_VOLTAGE OR QUICK_STOP OR ENABLE_OPERATION OR HALT_BIT)
  )
  OR (INT_TO_INT(%IX51.0 AND MICRO1 AND MICRO2 AND MICRO3) * START_BIT)
  OR (INT_TO_INT(%IX51.1) * ENABLE_VOLTAGE)
  OR (INT_TO_INT(%IX51.2) * QUICK_STOP)
  OR (INT_TO_INT(%IX51.3) * ENABLE_OPERATION)
  OR (INT_TO_INT(%IX51.4) * HALT_BIT);

```

```

// =====
// Quick Stop Clear — BOTÃO FÍSICO
// =====
IF QS_CLEAR_BUTTON THEN
  WR_PDO_6040_X := WR_PDO_6040_X AND NOT QUICK_STOP;
END_IF;

```

```

// =====
// Halt Clear — BOTÃO FÍSICO
// =====
IF HALT_CLEAR_BUTTON THEN
  WR_PDO_6040_X := WR_PDO_6040_X AND NOT HALT_BIT;
END_IF;

```

```

// =====
// Fault Clear
// =====

```

```

IF FAULT_CLEAR_BUTTON THEN
    WR_PDO_6040_X := 0;
    WR_PDO_6040_Y := 0;
    WR_PDO_6040_Z := 0;
END_IF;

// Sensores físicos (não usados nesta função):
// HOMEX, HOMEY, HOMEZ, MICRO1, MICRO2, MICRO3

WR_PDO_6040_Y := WR_PDO_6040_X; // espelha Y
WR_PDO_6040_Z := WR_PDO_6040_X; // espelha Z

// Lampadas
LAG_LAMP_X := (RD_PDO_6041_X AND FOLLOWING_ERROR) <> 0;
FAULT_LAMP_X := (RD_PDO_6041_X AND FAULT_BIT) <> 0;

// Para eixo Y
LAG_LAMP_Y := (RD_PDO_6041_Y AND FOLLOWING_ERROR) <> 0;
FAULT_LAMP_Y := (RD_PDO_6041_Y AND FAULT_BIT) <> 0;

// Para eixo Z
LAG_LAMP_Z := (RD_PDO_6041_Z AND FOLLOWING_ERROR) <> 0;
FAULT_LAMP_Z := (RD_PDO_6041_Z AND FAULT_BIT) <> 0;

END_FUNCTION

@
(*)

```

Função INTERPOLAÇÃO_LINEAR_CIRCULAR:

Este bloco Structured Text implementa um gerador de trajetória multi-eixo baseado em seleção de modo operacional, com variação de comportamento entre modos 1, 2, 3 e 4, utilizando estados de fase e vetores direcionais sinallX, sinallY e sinallZ associados a diferentes configurações S0 a S13 e suas variantes. A execução ocorre em estrutura de repetição enquanto indexBuffer for menor que maxBuffer e bufferFilled não estiver ativo, com atualização contínua de variáveis de posição X, Y e Z conforme regras condicionais dependentes de modo, sentido e flags de operação relativa.

No modo 3, as coordenadas são ajustadas por incrementos passoDX, passoDY e passoDZ com restrições de limites máximos e mínimos por eixo, além de controle de progresso por variáveis PROFX, PROFY e PROFZ, com detecção de condição geométrica para reinicialização do perfil baseado em erro de aproximação. No modo 4, há alternância entre expansão e retração de passos com controle de cavidade, utilizando variáveis de controle aumentar e aumentar_old, além de contadores de fase que determinam a progressão dos estados e ajustes incrementais por QEX, QEY e QEZ.

A discretização do percurso é derivada de cálculos baseados em proporções entre extensões máximas e incrementos unitários, com ajuste para múltiplos inteiros e correções por tolerância.

A estrutura de fase evolui ciclicamente com incremento de fase e contagem de ciclos totais, com diferentes comportamentos conforme modo ativo.

Cada ponto gerado é armazenado em buffers de posição e convertido para formato de comunicação através de função de conversão para DINT, separando valores em palavras alta e baixa para transmissão via protocolo de controle de movimento. A velocidade correspondente é calculada por função dedicada e armazenada em buffers paralelos.

O ramo circular executa geração de trajetória baseada em parâmetros de raio, ângulo e incremento angular, com suporte a variação por quadrante, sentido de rotação e planos de interpolação XY, ZX e ZY, incluindo possibilidade de trajetória helicoidal. As diferenças diferenciais entre posições atuais e anteriores são calculadas para cada eixo conforme modo relativo ou absoluto.

Eventos de parada são registrados quando condições de halt são detectadas, armazenando índices de buffer e estado de interrupção. Ao final da execução, o sistema marca buffers como preenchidos, registra índices finais, ativa flags de prontidão dos eixos e reinicializa estruturas de controle.

15. FUNCTION INTERPOLACAO_LINEAR_CIRCULAR

16. FUNCTION INTERPOLACAO_BI_PLANAR

17. FUNCTION INTERPOLACAO_BI_PLANAR_PUMPKIN

18. FUNCTION INTERPOLACAO_CONICA

18A.. FUNCTION_REVOLUTION_POINTS

*)

TYPE eSentido : (S0, S0A, S0B, S0AB, S1, S1A, S1B, S1AB, S2, S3, S4, S5, S6, S7, S8, S9, S10, S11, S12, S13);

END_TYPE

sentido : eSentido;

VAR

```
(*
  BOOL TRUE / FALSE *) NI ,NF,Reset,
  CAVIDADE,CONTROL_LAGXYZ,EXTERNO ,FLAT,Q1,Q2,Q3,Q4,
  Q1r,Q2r,Q3r,Q4r,Q1r2,Q2r2,Q3r2,Q4r2,initBuffers, P2, PHELICAL, DONE, aumentar, vaivem , RETURN_0,
  PR, PR1, RelativeX, RelativeY, RelativeZ,Relative, PXY,PZY,PZX,XY,ZY,ZX ,MODE1,MODE2 : BOOL;
  (* USINT. 8 BITS SEM SINAL 0- 255*)
  N,mode, fase,segmentos, SENTIDO,SENTIDO1 : USINT;
  (* UINT 16 BITS SEM SINAL 0 - 65535 *)
  N, contador, contador1,contador2, contadorfasesC ,contadorfasesD , contadorRETURN_0
,TESTE_DOWN, TESTE : UINT;
eventoldx : UINT := 0;
  (* UDINT 32 BITS SEM SINAL 0 – 4.29 9 *) VELX , VELY , VELZ ,VELX0,VELY0,VELZ0,
totalCycles, totalCycles1, totalPassos, totalPassos1,
  contadorCiclos, contadorCiclos1 : UDINT;
  totalPassosTotal : UDINT := 0;
```



```

(* REAL 32 BITS IEEE 754*)
r_ant,r2_ant , STPX , STPY,STPZ,STPMax,A, B, C, D,DEN,
LIM_POS, LIM_NEG, POT,
PX, PY, PZ, PH,PHmax,PHmin,
X0C,Y0C,Z0C,X1C,Y1C,Z1C ,PROF, PROF_XYZ,
ANGULOINICIAL, ANGULOFINAL,
r, r1,r2,r0,ratual, deltaX, deltaX1,
passoC ,passoR,
X, Y, Z,
XC1, YC1, ZC1,
XC, YC, ZC,XC2,YC2,ZC2,
passoLX, passoLY, passoLZ,
passoDX, passoDY, passoDZ,
maxLX, maxLY, maxLZ,
minLX, minLY, minLZ,
QEX, QEY, QEZ,HIPOTENUSA,
XD, YD, ZD
, rmax,r1max,r2max,
ANGULOINICIAL1, ANGULOFINAL1,
passosQuadrante, passosQuadrante1,
anguloInc, anguloInc1,
rad, rad1,
angulo, angulo1,
ANGGFINAL, ANGGFINAL1,
ANGULOTOTAL, ANGULOTOTAL1,
ANG1, ANG11,
X_ant, Y_ant, Z_ant,
X_ant1, Y_ant1, Z_ant1,
incremento_maximo, incremento_maximo1,
,deltaR,hcone,prof,angulocone,
match,
profX, profY, profZ ,J_X, J_Y, J_Z,
Mass_X_kg, Mass_Y_kg, Mass_Z_kg,
TorqueMax_X_Nm, TorqueMax_Y_Nm, TorqueMax_Z_Nm,
OverdriveFactor_X, OverdriveFactor_Y, OverdriveFactor_Z : REAL;

```

```

(* STRING *)
MSG : STRING[100];

```

```

(* ARRAY OF SINT -128 a 127 8 BITS SIGNED *)
sinallX, sinallY, sinallZ : ARRAY[0..5] OF SINT;
eventosParada : ARRAY[0..19] OF tEventoParada;
END_VAR

```

```

FuncoesOriginais[1] := 'INTERPOLACAO_LINEAR_CIRCULAR';

```

```

FuncoesGCODE[1] := 'F1';

```

```

Variaveis[1] :=

```

```

'P2,PHELICAL,DONE,MODE1,MODE2,PR,PR1,Relative,RelativeX,RelativeY,RelativeZ,SENTIDO,SENTIDO1,X
Y,ZY,ZX,PXY,PZX,PZY,N,mode,LIM_POS,LIM_NEG,POT,PH,PROF,PROF_XYZ,ANGULOINICIAL,ANGULOFIN
AL,sendido,r,ratual,deltaX,passoC,X,Y,Z,passoLX,passoLY,passoLZ,passoDX,passoDY,passoDZ,maxLX,maxLY,
maxLZ,minLX,minLY,minLZ,QEX,QEY,QEZ,XD,YD,ZD,XC,YC,ZC';

```

```

TiposVariaveis[1] :=

```

```
'BOOL,BOOL,BOOL,BOOL,BOOL,BOOL,BOOL,BOOL,BOOL,BOOL,BOOL,BOOL,BOOL,BOOL,BOOL,BOOL,
BOOL,BOOL,BOOL,USINT,USINT,UINT,USINT,REAL,REAL,REAL,REAL,REAL,REAL,eSentido,REAL,REAL,R
EAL,REAL,REAL,REAL,REAL,REAL,REAL,REAL,REAL,REAL,REAL,REAL,REAL,REAL,REAL,REAL,R
EAL,REAL,REAL,REAL,REAL,REAL,REAL,REAL,REAL,REAL,REAL';
```

FUNCTION INTERPOLACAO_LINEAR_CIRCULAR

VAR

```
  x_tmp, y_tmp, z_tmp : REAL := 0.0;
  PQ_min   : INT;
  theta    : REAL;
  tolerancia : REAL;
  PQ       : INT;
  teste0    : REAL;
  arred    : REAL;
  erro      : REAL;
  sx : SINT;
  sy : SINT;
  ativos : INT;
PROFX , PROFY , PROFZ , PROFFF : REAL;
deltaS : REAL;
PI : REAL := 3.14159265;
END_VAR
```

VAR_OUTPUT

```
  // --- Diferenças de posição ---
  diffX_mm, diffY_mm, diffZ_mm : REAL;
END_VAR
```

VAR_INPUT

```
  // --- Velocidades ---
  rpmX, rpmY, rpmZ : UDINT;
END_VAR
```

BEGIN

contador := 0;contadorCiclos := 0;

```
IF Reset THEN
  N := 0;
  PartialStart0 := 0.0;
  PartialEnd0   := 100.0;
  mode := 0;
  maxLX := 0.0; minLX := 0.0;
  maxLY := 0.0; minLY := 0.0;
  maxLZ := 0.0; minLZ := 0.0;
  QEX := 0.0; QEY := 0.0; QEZ := 0.0;
  XD := 0.0;
  PROFX := 0.0; PROFY := 0.0; PROFZ := 0.0;
  PROF := 0.0; PROF_XYZ := 0.0;
  passoDX := 0.0; passoDY := 0.0; passoDZ := 0.0;
  PXY := FALSE; PZX := FALSE; PZY := FALSE;
  XY := FALSE; ZX := FALSE; ZY := FALSE;
  MODE1 := FALSE; MODE2 := FALSE;
  PHELICAL := FALSE; FLAT := FALSE;
```

```
bufferFilled := FALSE; DONE := FALSE;
fase := 0; totalCycles := 0;
END_IF;
```

// RAMO LINEAR

CASE mode OF

```
1: aumentar := TRUE;
2: aumentar := FALSE;
4: aumentar := FALSE;
   aumentar_old := TRUE;
END_CASE;
```

CASE sentido OF

S0:

```
sinallX := [1, 0, -1, 0];
sinallY := [0, -1, 0, 1];
sinallZ := [0, -1, 0, 1];
```

IF mode = 4 THEN

```
sinallX := [0, -1, 0, 1, 0, 1];
sinallY := [1, 0, 1, 0, -1, 0];
sinallZ := [1, 0, 1, 0, -1, 0];
```

END_IF;

S0B:

```
sinallX := [-1, 0, -1, 0, 1, 0];
sinallY := [0, 1, 0, -1, 0, -1];
sinallZ := [0, 1, 0, -1, 0, -1];
```

S0A:

```
sinallX := [-1, 0, 1, 0];
sinallY := [0, 1, 0, -1];
sinallZ := [0, 1, 0, -1];
```

IF mode = 4 THEN

```
sinallX := [0, -1, 0, 1, 0, 1];
sinallY := [-1, 0, -1, 0, 1, 0];
sinallZ := [-1, 0, -1, 0, 1, 0];
```

END_IF;

S0AB:

```
sinallX := [-1, 0, -1, 0, 1, 0];
sinallY := [0, -1, 0, 1, 0, -1];
sinallZ := [0, -1, 0, 1, 0, -1];
```

S1:

```
sinallX := [1, 0, -1, 0];
sinallY := [0, 1, 0, -1];
sinallZ := [0, 1, 0, -1];
```

IF mode = 4 THEN

```
sinallX := [0, 1, 0, -1, 0, -1];
sinallY := [1, 0, 1, 0, -1, 0];
```

```
    sinallZ := [1, 0, 1, 0, -1, 0];  
END_IF;
```

S1B:

```
    sinallX := [1, 0, 1, 0, -1, 0];  
    sinallY := [0, 1, 0, -1, 0, -1];  
    sinallZ := [0, 1, 0, -1, 0, -1];
```

S1A:

```
    sinallX := [-1, 0, 1, 0];  
    sinallY := [0, -1, 0, 1];  
    sinallZ := [0, -1, 0, 1];
```

IF mode = 4 THEN

```
    sinallX := [0, 1, 0, -1, 0, -1];  
    sinallY := [-1, 0, -1, 0, 1, 0];  
    sinallZ := [-1, 0, -1, 0, 1, 0];
```

END_IF;

S1AB:

```
    sinallX := [1, 0, 1, 0, -1, 0];  
    sinallY := [0, -1, 0, 1, 0, 1];  
    sinallZ := [0, -1, 0, 1, 0, 1];
```

S2:

```
    sinallX := [1, 0, -1, 0];  
    sinallY := [0, 1, 0, 1];  
    sinallZ := [0, 1, 0, 1]; // sobe y / sobe z
```

S3:

```
    sinallX := [1, 0, -1, 0];  
    sinallY := [0, -1, 0, -1];  
    sinallZ := [0, -1, 0, -1]; // desce y / desce z
```

S4:

```
    sinallX := [1, 0, -1, 0];  
    sinallY := [0, 1, 0, 1];  
    sinallZ := [0, -1, 0, -1]; // combina subida y / descida z
```

S5:

```
    sinallX := [1, 0, -1, 0];  
    sinallY := [0, -1, 0, -1];  
    sinallZ := [0, 1, 0, 1]; // combina descida y / subida z
```

S6:

```
    sinallX := [-1, 0, -1, 0];  
    sinallY := [0, 1, 0, -1];  
    sinallZ := [-1, 0, -1, 0]; // esquerda x / desce z
```

S7:

```
    sinallX := [1, 0, 1, 0];  
    sinallY := [0, 1, 0, -1];  
    sinallZ := [1, 0, 1, 0]; // direita x / sobe z
```

S8:

```
    sinallX := [-1, 0, -1, 0];  
    sinallY := [0, 1, 0, -1];  
    sinallZ := [1, 0, 1, 0]; // esquerda x / sobe z
```

S9:

```
sinallX := [1, 0, 1, 0];  
sinallY := [0, 1, 0, -1];  
sinallZ := [-1, 0, -1, 0]; // direita x / desce z
```

S10:

```
sinallX := [0, -1, 0, -1];  
sinallY := [0, -1, 0, -1];  
sinallZ := [1, 0, -1, 0]; // esquerda x / desce y
```

S11:

```
sinallX := [0, 1, 0, 1];  
sinallY := [0, 1, 0, 1];  
sinallZ := [1, 0, -1, 0]; // direita x / sobe y
```

S12:

```
sinallX := [0, 1, 0, 1];  
sinallY := [0, -1, 0, -1];  
sinallZ := [1, 0, -1, 0]; // combinação x / desce y
```

S13:

```
sinallX := [0, -1, 0, -1];  
sinallY := [0, 1, 0, 1];  
sinallZ := [1, 0, -1, 0]; // combinação x / sobe y
```

END_CASE;

WHILE (indexBuffer < maxBuffer) AND (NOT bufferFilled) DO

IF mode = 3 THEN

```
IF Relative OR RelativeX OR (X <> 0) THEN X := X; ELSE X := X_ant; END_IF;  
IF Relative OR RelativeY OR (Y <> 0) THEN Y := Y; ELSE Y := Y_ant; END_IF;  
IF Relative OR RelativeZ OR (Z <> 0) THEN Z := Z; ELSE Z := Z_ant; END_IF;
```

IF (sentido = S2) OR (sentido = S3) OR (sentido = S4) OR (sentido = S5) THEN

IF sinallX[fase]=1 THEN X:=X+XD; ELSIF sinallX[fase]=-1 THEN X:=X-XD; END_IF;

IF maxLY>0 THEN IF (PROFY+passoDY)<=maxLY THEN IF sinallY[fase]=1 THEN Y:=Y+passoDY; ELSIF sinallY[fase]=-1 THEN Y:=Y-passoDY; END_IF; PROFY:=PROFY+passoDY; END_IF; END_IF;

IF maxLZ>0 THEN IF (PROFZ+passoDZ)<=maxLZ THEN IF sinallZ[fase]=1 THEN Z:=Z+passoDZ; ELSIF sinallZ[fase]=-1 THEN Z:=Z-passoDZ; END_IF; PROFZ:=PROFZ+passoDZ; END_IF; END_IF;

ELSIF (sentido = S6) OR (sentido = S7) OR (sentido = S8) OR (sentido = S9) THEN

IF sinallY[fase]=1 THEN Y:=Y+YD; ELSIF sinallY[fase]=-1 THEN Y:=Y-YD; END_IF;

IF maxLX>0 THEN IF (PROFX+passoDX)<=maxLX THEN IF sinallX[fase]=1 THEN X:=X+passoDX; ELSIF sinallX[fase]=-1 THEN X:=X-passoDX; END_IF; PROFX:=PROFX+passoDX; END_IF; END_IF;

IF maxLZ>0 THEN IF (PROFZ+passoDZ)<=maxLZ THEN IF sinallZ[fase]=1 THEN Z:=Z+passoDZ; ELSIF sinallZ[fase]=-1 THEN Z:=Z-passoDZ; END_IF; PROFZ:=PROFZ+passoDZ; END_IF; END_IF;

ELSIF (sentido = S10) OR (sentido = S11) OR (sentido = S12) OR (sentido = S13) THEN

IF sinallZ[fase]=1 THEN Z:=Z+ZD; ELSIF sinallZ[fase]=-1 THEN Z:=Z-ZD; END_IF;

IF maxLX>0 THEN IF (PROFX+passoDX)<=maxLX THEN IF sinallX[fase]=1 THEN X:=X+passoDX; ELSIF sinallX[fase]=-1 THEN X:=X-passoDX; END_IF; PROFX:=PROFX+passoDX; END_IF; END_IF;

```
IF maxLY>0 THEN IF (PROFY+passoDY)<=maxLY THEN IF sinallY[fase]=1 THEN Y:=Y+passoDY; ELSIF  
sinallY[fase]=-1 THEN Y:=Y-passoDY; END_IF; PROFY:=PROFY+passoDY; END_IF; END_IF;  
END_IF;
```

```
IF (PROFX + passoDX > maxLX) OR (PROFY + passoDY > maxLY) OR (PROFZ + passoDZ > maxLZ) THEN
```

```
ativos := BOOL_TO_INT(maxLX>0) + BOOL_TO_INT(maxLY>0) + BOOL_TO_INT(maxLZ>0);
```

```
IF ( (ativos>=2) AND (ABS(SQRT(PROFX*PROFX+PROFY*PROFY+PROFZ*PROFZ)-HIPOTENUSA)<=0.5) )  
OR ( (ativos=1) AND ( (ABS(maxLX-PROFX)<=0.5) OR (ABS(maxLY-PROFY)<=0.5) OR (ABS(maxLZ-  
PROFZ)<=0.5) ) ) THEN
```

```
PROF:=PROF_XYZ+1; PROFX:=0.0; PROFY:=0.0; PROFZ:=0.0;  
passoDX:=0.0; passoDY:=0.0; passoDZ:=0.0;  
passoLX:=0.0; passoLY:=0.0; passoLZ:=0.0;  
XD:=0.0; YD:=0.0; ZD:=0.0; PROF:=0;
```

```
END_IF;
```

```
END_IF;// fim mode 3
```

```
IF (mode = 4) AND (totalCycles = 0) AND (CAVIDADE = FALSE) THEN
```

```
IF maxLX > 0 THEN passoLX := maxLX - QEX; END_IF; IF maxLY > 0 THEN passoLY := maxLY - QEY; END_IF;  
IF maxLZ > 0 THEN passoLZ := maxLZ - QEZ; END_IF; END_IF;
```

```
IF (mode = 4) AND (totalCycles = 0) AND (CAVIDADE = TRUE) THEN
```

```
IF maxLX > 0 THEN passoLX := maxLX; END_IF; IF maxLY > 0 THEN passoLY := maxLY; END_IF; IF maxLZ >  
0 THEN passoLZ := maxLZ; END_IF;
```

```
IF (mode = 4 ) AND ((aumentar = FALSE AND (fase = 3)) OR (fase = 0 AND CAVIDADE = FALSE) OR (aumentar  
= TRUE AND ((fase = 2) OR (fase = 5)))) THEN
```

```
A := passoLX;
```

```
B := passoLY;
```

```
C := passoLZ;
```

```
passoLX := QEX;
```

```
passoLY := QEY;
```

```
passoLZ := QEZ;
```

```
END_IF;
```

```
IF (mode = 4) AND ((aumentar = FALSE AND ((fase = 1) OR (fase = 4))) OR (aumentar = TRUE AND ((fase = 0)  
OR (fase = 3)))) THEN
```

```
passoLX := A;
```

```
passoLY := B;
```

```
passoLZ := C;
```

```
IF (mode = 4) AND (totalCycles = 0) THEN
```

```
D := TRUE;
```

```
C := FALSE;
```

```
contadorfasesD := 0;
```

```
END_IF;
```

```
IF (mode = 4) AND (aumentar = FALSE AND aumentar_old = TRUE) THEN
```

```
D := TRUE;
```

```
C := FALSE;
```

```
contadorfasesD := 0;
```

END_IF;

IF D THEN

 contadorfasesD := contadorfasesD + 1;

END_IF;

IF (mode = 4) AND (aumentar = TRUE AND aumentar_old = FALSE) THEN

 C := TRUE;

 D := FALSE;

 contadorfasesC := 0;

END_IF;

IF C THEN

 contadorfasesC := contadorfasesC + 1;

END_IF;

IF aumentar AND contadorfasesC > 3 THEN

 IF sinalIX[fase] <> 0 THEN passoLX := passoLX + QEX; END_IF;

 IF sinalIY[fase] <> 0 THEN passoLY := passoLY + QEY; END_IF;

 IF sinalIZ[fase] <> 0 THEN passoLZ := passoLZ + QEZ; END_IF;

END_IF;

IF NOT aumentar AND contadorfasesD > 4 THEN

 IF sinalIX[fase] <> 0 THEN passoLX := passoLX - QEX; END_IF;

 IF sinalIY[fase] <> 0 THEN passoLY := passoLY - QEY; END_IF;

 IF sinalIZ[fase] <> 0 THEN passoLZ := passoLZ - QEZ; END_IF;

END_IF;

IF fase = 0 THEN

IF

 (mode = 4 AND CAVIDADE = TRUE AND (aumentar = FALSE AND aumentar_old = TRUE))

 OR

 (mode = 4 AND CAVIDADE = TRUE AND totalCycles = 0)

THEN

 A := passoLX;

 B := passoLY;

 C := passoLZ;

 passoLX := 0;

 passoLY := 0;

 passoLZ := 0;

END_IF;

END_IF;

aumentar_old := aumentar;

// XY

IF (totalCycles = 0) AND (CAVIDADE = FALSE) THEN IF (maxLX > minLX) AND (QEX > 0) THEN TESTE := INT((((maxLX - minLX) / QEX) / 2) * 6) + 0.5; TESTE := INT((TESTE / 6.0) + 0.5) * 6; DEN := (TESTE / 6.0) * 2; IF (DEN > 0) THEN IF ((maxLX - minLX) / DEN) >= D THEN TESTE := TESTE + 6; END_IF; END_IF; QEX := (maxLX - minLX) / ((TESTE / 6.0) * 2); END_IF; IF (maxLY > minLY) AND (QEY > 0) THEN QEY := (maxLY - minLY) / ((TESTE / 6.0) * 2); END_IF; TESTE_DOWN := TESTE - 1; END_IF;

```

IF (totalCycles = 0) AND (CAVIDADE = TRUE) THEN IF (maxLX > minLX) AND (QEX > 0) THEN TESTE :=
INT((((maxLX - minLX) / QEX) / 2) * 6) + 0.5); TESTE := INT((TESTE / 6.0) + 0.5) * 6; DEN := ((TESTE - 3) / 6.0)
* 2; IF (DEN > 0) THEN IF ((maxLX - minLX) / DEN) >= D THEN TESTE := TESTE + 6; END_IF; END_IF; QEX
:= (maxLX - minLX) / (((TESTE - 3) / 6.0) * 2); END_IF; IF (maxLY > minLY) AND (QEY > 0) THEN QEY := (maxLY
- minLY) / (((TESTE - 3) / 6.0) * 2); END_IF; TESTE_DOWN := TESTE - 1; END_IF;
// ZY
// ZY
IF (totalCycles = 0) AND (CAVIDADE = FALSE) THEN IF (maxLZ > minLZ) AND (QEZ > 0) THEN TESTE :=
INT((((maxLZ - minLZ) / QEZ) / 2) * 6) + 0.5); TESTE := INT((TESTE / 6.0) + 0.5) * 6; DEN := (TESTE / 6.0) * 2;
IF (DEN > 0) THEN IF ((maxLZ - minLZ) / DEN) >= D THEN TESTE := TESTE + 6; END_IF; END_IF; QEZ :=
(maxLZ - minLZ) / ((TESTE / 6.0) * 2); END_IF; IF (maxLY > minLY) AND (QEY > 0) THEN QEY := (maxLY -
minLY) / ((TESTE / 6.0) * 2); END_IF; TESTE_DOWN := TESTE - 1; END_IF;
// ZY
IF (totalCycles = 0) AND (CAVIDADE = TRUE) THEN IF (maxLZ > minLZ) AND (QEZ > 0) THEN TESTE :=
INT((((maxLZ - minLZ) / QEZ) / 2) * 6) + 0.5); TESTE := INT((TESTE / 6.0) + 0.5) * 6; DEN := ((TESTE - 3) / 6.0)
* 2; IF (DEN > 0) THEN IF ((maxLZ - minLZ) / DEN) >= D THEN TESTE := TESTE + 6; END_IF; END_IF; QEZ
:= (maxLZ - minLZ) / (((TESTE - 3) / 6.0) * 2); END_IF; IF (maxLY > minLY) AND (QEY > 0) THEN QEY := (maxLY
- minLY) / (((TESTE - 3) / 6.0) * 2); END_IF; TESTE_DOWN := TESTE - 1; END_IF;

// ZX
// ZX
IF (totalCycles = 0) AND (CAVIDADE = FALSE) THEN IF (maxLZ > minLZ) AND (QEZ > 0) THEN TESTE :=
INT((((maxLZ - minLZ) / QEZ) / 2) * 6) + 0.5); TESTE := INT((TESTE / 6.0) + 0.5) * 6; DEN := (TESTE / 6.0) * 2;
IF (DEN > 0) THEN IF ((maxLZ - minLZ) / DEN) >= D THEN TESTE := TESTE + 6; END_IF; END_IF; QEZ :=
(maxLZ - minLZ) / ((TESTE / 6.0) * 2); END_IF; IF (maxLX > minLX) AND (QEX > 0) THEN QEX := (maxLX -
minLX) / ((TESTE / 6.0) * 2); END_IF; TESTE_DOWN := TESTE - 1; END_IF;
// ZX
IF (totalCycles = 0) AND (CAVIDADE = TRUE) THEN IF (maxLZ > minLZ) AND (QEZ > 0) THEN TESTE :=
INT((((maxLZ - minLZ) / QEZ) / 2) * 6) + 0.5); TESTE := INT((TESTE / 6.0) + 0.5) * 6; DEN := ((TESTE - 3) / 6.0)
* 2; IF (DEN > 0) THEN IF ((maxLZ - minLZ) / DEN) >= D THEN TESTE := TESTE + 6; END_IF; END_IF; QEZ
:= (maxLZ - minLZ) / (((TESTE - 3) / 6.0) * 2); END_IF; IF (maxLX > minLX) AND (QEX > 0) THEN QEX := (maxLX
- minLX) / (((TESTE - 3) / 6.0) * 2); END_IF; TESTE_DOWN := TESTE - 1; END_IF;

IF (contadorfasesD = TESTE) THEN
    aumentar := TRUE; END_IF;
IF (contadorfasesC = TESTE) THEN
    aumentar := FALSE; END_IF;

IF mode = 4 THEN
IF (mode = 4) AND (aumentar = TRUE AND aumentar_old = FALSE) THEN
    CASE sentido OF
        S0: sentido := S0B;
        S1: sentido := S1B;
        S0A: sentido := S0AB;
        S1A: sentido := S1AB;
    END_CASE; END_IF;

IF (mode = 4) AND (aumentar = FALSE AND aumentar_old = TRUE) THEN
CASE sentido OF
    S0B: sentido := S0;
    S1B: sentido := S1;
    S0AB: sentido := S0A;
    S1AB: sentido := S1A;
END_CASE;
END_IF;
END_IF;

```



```

IF (mode = 1) OR (mode = 2) THEN
IF (mode = 2) AND (totalCycles = 0) THEN
    IF sinallX[fase] <> 0 THEN passoLX := maxLX + QEX; END_IF;
    IF sinallY[fase] <> 0 THEN passoLY := maxLY + QEY; END_IF;
    IF sinallZ[fase] <> 0 THEN passoLZ := maxLZ + QEZ; END_IF;
END_IF;

```

IF fase = 0 AND (mode = 1 OR mode = 2) THEN

```

    IF (mode = 2) AND aumentar = FALSE THEN
        IF sinallX[fase] <> 0 THEN passoLX := passoLX - QEX; IF passoLX <= minLX THEN passoLX := minLX; aumentar := TRUE; END_IF; END_IF;
        IF sinallY[fase] <> 0 THEN passoLY := passoLY - QEY; IF passoLY <= minLY THEN passoLY := minLY; aumentar := TRUE; END_IF; END_IF;
        IF sinallZ[fase] <> 0 THEN passoLZ := passoLZ - QEZ; IF passoLZ <= minLZ THEN passoLZ := minLZ; aumentar := TRUE; END_IF; END_IF;
    END_IF;
END_IF;
IF aumentar THEN
    IF sinallX[fase] <> 0 THEN passoLX := passoLX + QEX; END_IF;
    IF sinallY[fase] <> 0 THEN passoLY := passoLY + QEY; END_IF;
    IF sinallZ[fase] <> 0 THEN passoLZ := passoLZ + QEZ; END_IF;
END_IF;
// Verifica se atingiu o máximo
IF passoLX > maxLX THEN IF sinallX[fase] <> 0 THEN IF mode = 1 THEN passoLX := QEX; ELSE
aumentar := FALSE; passoLX := maxLX; END_IF; END_IF; END_IF;
IF passoLY > maxLY THEN IF sinallY[fase] <> 0 THEN IF mode = 1 THEN passoLY := QEY; ELSE
aumentar := FALSE; passoLY := maxLY; END_IF; END_IF; END_IF;
IF passoLZ > maxLZ THEN IF sinallZ[fase] <> 0 THEN IF mode = 1 THEN passoLZ := QEZ; ELSE
aumentar := FALSE; passoLZ := maxLZ; END_IF; END_IF; END_IF; // if aumentar
END_IF; // FASE 0

```

```

IF ((mode = 4 AND totalCycles = 0 AND aumentar = FALSE AND aumentar_old = TRUE)
OR (mode = 4 AND totalCycles = 0 AND aumentar = TRUE AND aumentar_old = FALSE)
OR (mode = 4 AND aumentar = FALSE AND aumentar_old = TRUE)
OR (mode = 4 AND aumentar = TRUE AND aumentar_old = FALSE)
OR (fase = 0 AND (mode = 1 OR mode = 2))) THEN

```

```

IF ( (sinallX[fase] <> 0 AND ABS(passoLX - maxLX) <= 0.5) OR
(sinallY[fase] <> 0 AND ABS(passoLY - maxLY) <= 0.5) OR
(sinallZ[fase] <> 0 AND ABS(passoLZ - maxLZ) <= 0.5) OR
(mode = 4 AND totalCycles = 0 AND aumentar = FALSE AND aumentar_old = TRUE) ) THEN

```

```

// Passo finalizador (aplica apenas se não ultrapassar)
IF (PROF + passoDZ + passoDY + passoDX) <= PROF_XYZ THEN
    PROF := PROF + passoDZ + passoDY + passoDX;
    PROFFF := TRUE;
    IF passoDZ > 0 THEN Z := Z + passoDZ; END_IF;
    IF passoDY > 0 THEN Y := Y + passoDY; END_IF;
    IF passoDX > 0 THEN X := X + passoDX; END_IF;
END_IF;
END_IF; // fecha if ABS

```

```
IF ((sinallX[fase] <> 0 AND ABS(passoLX - minLX) <= 0.5) OR (sinallY[fase] <> 0 AND ABS(passoLY - minLY) <= 0.5) OR (sinallZ[fase] <> 0 AND ABS(passoLZ - minLZ) <= 0.5)) OR ((mode = 1) AND (fase = 0) AND (PROF = 0)) THEN
```

```
    aumentar := TRUE;
    // Passo finalizador (aplica apenas se não ultrapassar)
    IF (PROF + passoDZ + passoDY + passoDX) <= PROF_XYZ THEN
        PROF := PROF + passoDZ + passoDY + passoDX;
        PROFFF := TRUE;
        IF passoDZ > 0 THEN Z := Z + passoDZ; END_IF;
        IF passoDY > 0 THEN Y := Y + passoDY; END_IF;
        IF passoDX > 0 THEN X := X + passoDX; END_IF;
    END_IF;
    END_IF; // fecha if ABS
END_IF; // fase 0 tripla
```

```
IF (PROF + passoDX > PROF_XYZ) OR (PROF + passoDY > PROF_XYZ) OR (PROF + passoDZ > PROF_XYZ) THEN
```

```
    IF ABS(PROF - PROF_XYZ) < 0.5 THEN
        PROF := PROF_XYZ + 1 ;
    END_IF;
    PROF := 0;
    passoDX := 0.0;
    passoDY := 0.0;
    passoDZ := 0.0;
    passoLX := 0.0;
    passoLY := 0.0;
    passoLZ := 0.0;
    XD := 0.0;
    YD := 0.0;
    ZD := 0.0;
    totalCycles := 0;
    maxLX := 0;
    maxLY := 0;
    maxLZ := 0;
END_IF;
```

```
IF (contadorfasesC = TESTE ) AND (CAVIDADE = TRUE) THEN
```

```
    sinallX[fase] := 0;
    sinallY[fase] := 0;
    sinallZ[fase] := 0;
END_IF;
```

```
IF (contadorfasesD = TESTE_DOWN) AND (minLX = 0) AND (minLY = 0) AND (minLZ = 0) AND (CAVIDADE = TRUE) THEN
```

```
    sinallX[fase] := 0;
    sinallY[fase] := 0;
    sinallZ[fase] := 0;
END_IF;
```

```
IF (contadorfasesC = 2 ) AND (minLX = 0) AND (minLY = 0) AND (minLZ = 0) AND (CAVIDADE = TRUE) THEN
```

```
    sinallX[fase] := 0;
    sinallY[fase] := 0;
    sinallZ[fase] := 0;
END_IF;
```

```
IF ((sentido = S0) OR (sentido = S0A) OR (sentido = S0B) OR (sentido = S0AB)) OR
```

```

((sentido = S1) OR (sentido = S1A) OR (sentido = S1B) OR (sentido = S1AB)) THEN // Atualiza X se maxLX
> 0
IF maxLX > 0 THEN
    IF sinalX[fase] = 1 THEN
        X := X + passoLX;
    ELSIF sinalX[fase] = -1 THEN
        X := X - passoLX;
    END_IF;
END_IF;

// Atualiza Y se maxLY > 0
IF maxLY > 0 THEN
    IF sinalY[fase] = 1 THEN
        Y := Y + passoLY;
    ELSIF sinalY[fase] = -1 THEN
        Y := Y - passoLY;
    END_IF;
END_IF;

// Atualiza Z se maxLZ > 0
IF maxLZ > 0 THEN
    IF sinalZ[fase] = 1 THEN
        Z := Z + passoLZ;
    ELSIF sinalZ[fase] = -1 THEN
        Z := Z - passoLZ;
    END_IF;
END_IF;

END_IF;
END_IF;

(* X *)
IF RelativeX THEN
    diffX_mm := 0.0;
    IF X>0 THEN diffX_mm := X; END_IF;
    IF passoDX>0 THEN diffX_mm := passoDX; X := passoDX; END_IF;
    IF passoLX>0 THEN diffX_mm := passoLX; X := passoLX; END_IF;
    IF XD>0 THEN diffX_mm := XD; X := XD; END_IF;
    IF sinalX[fase] = -1 THEN diffX_mm := -diffX_mm; X := -X; END_IF;
ELSE
    diffX_mm := X;
END_IF;

(* Y *)
IF RelativeY THEN
    diffY_mm := 0.0;
    IF Y>0 THEN diffY_mm := Y; END_IF;
    IF passoDY>0 THEN diffY_mm := passoDY; Y := passoDY; END_IF;
    IF passoLY>0 THEN diffY_mm := passoLY; Y := passoLY; END_IF;
    IF YD>0 THEN diffY_mm := YD; Y := YD; END_IF;
    IF sinalY[fase] = -1 THEN diffY_mm := -diffY_mm; Y := -Y; END_IF;
ELSE
    diffY_mm := Y;
END_IF;

(* Z *)
IF RelativeZ THEN

```

```

diffZ_mm := 0.0;
IF Z>0 THEN diffZ_mm := Z; END_IF;
IF passoDZ>0 THEN diffZ_mm := passoDZ; Z := passoDZ; END_IF;
IF passoLZ>0 THEN diffZ_mm := passoLZ; Z := passoLZ; END_IF;
IF ZD>0 THEN diffZ_mm := ZD; Z := ZD; END_IF;
IF sinallZ[fase] = -1 THEN diffZ_mm := -diffZ_mm; Z := -Z; END_IF;
ELSE
    diffZ_mm := Z;
END_IF;

```

// X-YZ intertravados

```

IF (WR_PDO_6040_X AND HALT_BIT) = 0 THEN
    eventosParada[eventoldx].indexBuffer := indexBuffer;
    eventosParada[eventoldx].halt := TRUE;
    eventoldx := eventoldx + 1;
    IF eventoldx > 19 THEN eventoldx := 0; END_IF;
END_IF;

```

IF fase = 0 AND PROFF THEN

```

    IF passoDX > 0 THEN bufferX[indexBuffer] := X; END_IF;
    IF passoDY > 0 THEN bufferY[indexBuffer] := Y; END_IF;
    IF passoDZ > 0 THEN bufferZ[indexBuffer] := Z; END_IF;

```

```

targetDINT_X := ConvertRealDiffToTargetDINT(diffX_mm);
targetDINT_Y := ConvertRealDiffToTargetDINT(diffY_mm);
targetDINT_Z := ConvertRealDiffToTargetDINT(diffZ_mm);

```

```

    bufferTargetX_H[indexBuffer] := INT(SHR(UDINT(targetDINT_X),16) AND 65535);
    bufferTargetX_L[indexBuffer] := INT(UDINT(targetDINT_X) AND 65535);
    bufferTargetY_H[indexBuffer] := INT(SHR(UDINT(targetDINT_Y),16) AND 65535);
    bufferTargetY_L[indexBuffer] := INT(UDINT(targetDINT_Y) AND 65535);
bufferTargetZ_H[indexBuffer] := INT(SHR(UDINT(targetDINT_Z),16) AND 65535);
bufferTargetZ_L[indexBuffer] := INT(UDINT(targetDINT_Z) AND 65535);

```

```

VELOCIDADE_X_Y_Z(diffX_mm, diffY_mm, diffZ_mm, rpmX, rpmY, rpmZ);
bufferVelX[indexBuffer] := rpmX;
bufferVelY[indexBuffer] := rpmY;
bufferVelZ[indexBuffer] := rpmZ;

```

indexBuffer := indexBuffer + 1;

```

IF passoLX>0 OR XD>0 THEN bufferX[indexBuffer]:=X; END_IF;
IF passoLY>0 OR YD>0 THEN bufferY[indexBuffer]:=Y; END_IF;
IF passoLZ>0 OR ZD>0 THEN bufferZ[indexBuffer]:=Z; END_IF;
END_IF;

```

```

targetDINT_X := ConvertRealDiffToTargetDINT(diffX_mm);
targetDINT_Y := ConvertRealDiffToTargetDINT(diffY_mm);
targetDINT_Z := ConvertRealDiffToTargetDINT(diffZ_mm);

```

```

    bufferTargetX_H[indexBuffer] := INT(SHR(UDINT(targetDINT_X),16) AND 65535);
    bufferTargetX_L[indexBuffer] := INT(UDINT(targetDINT_X) AND 65535);
    bufferTargetY_H[indexBuffer] := INT(SHR(UDINT(targetDINT_Y),16) AND 65535);
    bufferTargetY_L[indexBuffer] := INT(UDINT(targetDINT_Y) AND 65535);
bufferTargetZ_H[indexBuffer] := INT(SHR(UDINT(targetDINT_Z),16) AND 65535);
bufferTargetZ_L[indexBuffer] := INT(UDINT(targetDINT_Z) AND 65535);

```

```
VELOCIDADE_X_Y_Z(diffX_mm, diffY_mm, diffZ_mm, rpmX, rpmY, rpmZ);  
bufferVelX[indexBuffer] := rpmX;  
bufferVelY[indexBuffer] := rpmY;  
bufferVelZ[indexBuffer] := rpmZ;
```

```
indexBuffer := indexBuffer + 1;
```

```
ELSE
```

```
    bufferX[indexBuffer] := X;  
    bufferY[indexBuffer] := Y;  
    bufferZ[indexBuffer] := Z;
```

```
targetDINT_X := ConvertRealDiffToTargetDINT(diffX_mm);  
    targetDINT_Y := ConvertRealDiffToTargetDINT(diffY_mm);  
    targetDINT_Z := ConvertRealDiffToTargetDINT(diffZ_mm);
```

```
    bufferTargetX_H[indexBuffer] := INT(SHR(UDINT(targetDINT_X),16) AND 65535);  
    bufferTargetX_L[indexBuffer] := INT(UDINT(targetDINT_X) AND 65535);  
    bufferTargetY_H[indexBuffer] := INT(SHR(UDINT(targetDINT_Y),16) AND 65535);  
    bufferTargetY_L[indexBuffer] := INT(UDINT(targetDINT_Y) AND 65535);  
bufferTargetZ_H[indexBuffer] := INT(SHR(UDINT(targetDINT_Z),16) AND 65535);  
bufferTargetZ_L[indexBuffer] := INT(UDINT(targetDINT_Z) AND 65535);
```

```
VELOCIDADE_X_Y_Z(diffX_mm, diffY_mm, diffZ_mm, rpmX, rpmY, rpmZ);  
bufferVelX[indexBuffer] := rpmX;  
bufferVelY[indexBuffer] := rpmY;  
bufferVelZ[indexBuffer] := rpmZ;
```

```
indexBuffer := indexBuffer + 1;
```

```
END_IF; // IF FASE = 0 and profff  
PROFFF := FALSE;  
END_IF;
```

```
IF indexBuffer > maxBuffer THEN  
    indexBuffer := 0;  
    bufferFilled := TRUE;  
END_IF;
```

```
IF mode = 1 OR mode = 2 THEN
```

```
    fase := fase + 1;  
    IF fase > 3 THEN  
        fase := 0;  
        totalCycles := totalCycles + 1;  
    END_IF;  
END_IF;
```

```
IF mode = 4 THEN
```

```
    fase := fase + 1;  
    IF fase > 5 THEN  
        fase := 0;  
        totalCycles := totalCycles + 1;  
    END_IF;  
END_IF;
```

```
IF Relative THEN
  X := 0; Y := 0; Z := 0;
ELSIF RelativeX THEN X := 0;
ELSIF RelativeY THEN Y := 0;
ELSIF RelativeZ THEN Z := 0;
END_IF;
```

```
IF ((mode = 1) OR (mode = 2) OR (mode = 4)) AND
  (ABS(PROF - PROF_XYZ) <= 0.0005) THEN
```

```
  NF := TRUE;
```

```
END_IF;
```

```
IF (mode = 3) AND (sentido >= S0) AND
  (ABS(PROF - PROF_XYZ) <= 0.0005) THEN
```

```
  NF := TRUE;
```

```
END_IF;
```

```
IF (mode = 3) AND (N > 0) THEN
```

```
  NF := TRUE;
```

```
END_IF;
```

```
END_WHILE;
```

// RAMO CIRCULAR

```
WHILE (indexBuffer < maxBuffer) AND
(
  (MODE1 AND (ratual <= r) AND (totalCycles1 >= totalPassosTotal)) OR
  (MODE2 AND (ratual >= r) AND (totalCycles1 >= totalPassosTotal)) OR
  ((NOT MODE1) AND (NOT MODE2) AND (totalPassos >= contadorCiclos))
)
```

DO

```
IF contadorCiclos = 0 THEN
(* atualizar antigos *)
X_ant := X;
Y_ant := Y;
Z_ant := Z;
contador := 1;
END_IF;
```

```
IF contadorCiclos = 0 OR totalPassosTotal = contadorCiclos THEN
  IF MODE1 OR MODE2 THEN
    totalCycles1 := 0;
    theta := 360.0;
    tolerancia := 3.0;
    WHILE (MODE1 AND (ratual <= r)) OR (MODE2 AND (ratual >= r)) DO
      s := 3.14159265 * ratual / 2.0;
      PQ_min := CEIL(s / deltaX);
      totalPassosTotal := 0;
      FOR PQ := PQ_min TO 2000 DO
        teste0 := (theta * REAL(PQ)) / 90.0;
        arred := REAL(ROUND(teste0));
        erro := ABS(teste0 - arred);
        IF (erro < tolerancia) AND (INT(arred) MOD 2 = 0) THEN
          totalPassosTotal := INT(arred);
          EXIT;
        END_IF;
      END_FOR;
      totalCycles1 := totalCycles1 + totalPassosTotal;
      IF MODE1 THEN
        ratual := ratual + passoC;
      ELSE
        ratual := ratual - passoC;
      END_IF;
    END_WHILE;
  ELSE
    s := 3.14159265 * r / 2.0;
    PQ_min := CEIL(s / deltaX);
    theta := ANGULOFINAL - ANGULOINICIAL;
    IF theta < 0 THEN
      theta := theta + 360;
    END_IF;
    passosQuadrante := 0;
    deltaS := 0.0;
    anguloInc := 0.0;
    totalPassos := 0;
    tolerancia := 3.0;
    FOR PQ := PQ_min TO 2000 DO
      teste0 := (theta * REAL(PQ)) / 90.0;
      arred := REAL(ROUND(teste0));
      erro := ABS(teste0 - arred);
```

```

    IF (erro < tolerancia) AND (INT(arred) MOD 2 = 0) THEN
        passosQuadrante := PQ;
        deltaS := s / REAL(PQ);
        anguloInc := 90.0 / REAL(PQ);
        totalPassos := INT(arred);
        totalCycles1 := totalCycles1 + totalPassos;
        EXIT;
    END_IF;
END_FOR;
END_IF;
END_IF;

```

```

ANGGFINAL := ANGULOFINAL - ANGULOINICIAL;
IF ANGGFINAL < 0 THEN ANGGFINAL := ANGGFINAL + 360;

```

```

END_IF ; // CICLO 0 FECHADO

```

```

IF ((MODE1) OR (MODE2)) AND
((contadorCiclos = 0) OR (contadorCiclos = totalPassosTotal)) THEN
    totalPassosTotal := totalPassosTotal + totalPassos;
END_IF;
rad := angulo * 3.14159265 / 180.0;

```

```

//deltaS max 0.45mm // 3000 RPM // fuso 6mm// 1/2at²
// limites 1RPM

```

IF contadorCiclos = 0 THEN

```

RPM := deltaS * 6666.7;

```

```

incremento_maximo := RPM * WR_PDO_6083_X * 0.00005;

```

```

ANG1 := (incremento_maximo / r) * 57.32 ;

```

```

IF ANGGFINAL <= 2 * ANG1 THEN
    ANG1 := ANGGFINAL * 0.5 ;
END_IF;

```

```

END_IF ; // CICLO 0 FECHADO

```

```

ANGULOTOTAL := contador * anguloInc ;
angulo := ANGULOINICIAL + contador * anguloInc ;

```

```

IF ANGULOTOTAL >= ANGGFINAL - ANG1 THEN

```

```

    MAX_RPM := RPM - 30.0 * SQRT( RPM * (incremento_maximo - deltaS) / (0.045 * WR_PDO_6083_X) );

```

```

IF ANGULOTOTAL >= ANGGFINAL THEN
    ANGULOTOTAL := 0 ;
END_IF;

```



```

IF PZY THEN // ZY
IF PHELICAL THEN X := (PH / (2 * PI)) * (anguloInc * PI / 180); END_IF;
z_tmp := r * (1 - COS(rad));
y_tmp := r * SIN(rad);
IF SENTIDO = 1 THEN
Z := ZC - z_tmp; Y := YC + y_tmp;
ELSE
Z := ZC - z_tmp;
Y := YC - y_tmp;
END_IF; END_IF;

```

```

IF PZX THEN // ZX
IF PHELICAL THEN Y := (PH / (2 * PI)) * (anguloInc * PI / 180); END_IF;
x_tmp := r * (1 - COS(rad));
z_tmp := r * SIN(rad);
IF SENTIDO = 1 THEN
X := XC - x_tmp; Z := ZC + z_tmp;
ELSE
X := XC - x_tmp;
Z := ZC - z_tmp;
END_IF; END_IF;

```

```

IF PXY THEN
IF PHELICAL THEN Z := (PH / (2 * PI)) * (anguloInc * PI / 180); END_IF;
x_tmp := r * (1 - COS(rad));
y_tmp := r * SIN(rad);
IF SENTIDO = 1 THEN
X := XC - x_tmp; Y := YC + y_tmp;
ELSE
X := XC - x_tmp;
Y := YC - y_tmp;
END_IF; END_IF;

```

(* ----- DIFERENCIAIS ----- *)

```

IF Relative THEN
diffX_mm := 0;
diffY_mm := 0;
diffZ_mm := 0;
END_IF;

```

```

IF SENTIDO = 1 THEN
IF rad < PI/2 THEN
sx := -1; sy := +1;
ELSIF rad < PI THEN
sx := -1; sy := -1;
ELSIF rad < 3*PI/2 THEN
sx := +1; sy := -1;
ELSE
sx := +1; sy := +1;
END_IF;
ELSE

```

```

IF rad < PI/2 THEN
    sx := -1; sy := -1;
ELSIF rad < PI THEN
    sx := -1; sy := +1;
ELSIF rad < 3*PI/2 THEN
    sx := +1; sy := +1;
ELSE
    sx := +1; sy := -1;
END_IF;
END_IF;

```

```

(* X *)
IF PZY THEN
IF Relative THEN
diffZ_mm := sy*ABS(Z - Z_ant);
Z1 := diffZ_mm ;
diffY_mm := sx*ABS(Y - Y_ant);
Y1 := diffY_mm;
IF PHELICAL THEN
diffX_mm := X;
X1 := X ;
END_IF;
END_IF;
ELSE
diffZ_mm := ABS(Z - Z_ant); // absoluto sem sinal
Z1 := Z ;
diffY_mm := ABS(Y - Y_ant); // absoluto sem sinal
Y1 := Y ;
IF PHELICAL THEN
diffX_mm := X ; // absoluto
X1 := X + X_ant ;
END_IF;
END_IF;
END_IF;

```

```

(* Y *)

IF PZX THEN
IF Relative THEN
diffZ_mm := sy*ABS(Z - Z_ant);
Z1 := diffZ_mm;
diffX_mm := sx*ABS(X - X_ant);
X1 := diffX_mm;
IF PHELICAL THEN
diffY_mm := Y;
Y1 := Y ;
END_IF;
END_IF;
ELSE
diffZ_mm := ABS(Z - Z_ant); // absoluto sem sinal
Z1 := Z ;
diffX_mm := ABS(X - X_ant); // absoluto sem sinal
X1 := X ;
IF PHELICAL THEN
diffY_mm := Y ;
Y1 := Y + Y_ant ;

```

```
END_IF;  
END_IF;  
END_IF;
```

```
(* Z *)
```

```
IF PXY THEN  
IF Relative THEN  
diffY_mm := sy*ABS(Y - Y_ant);  
Y1 := diffY_mm;  
diffX_mm := sx*ABS(X - X_ant);  
X1 := diffX_mm;  
IF PHELICAL THEN  
diffZ_mm := Z;  
Z1 := Z ;  
END_IF;  
END_IF;  
ELSE  
diffY_mm := ABS(Y - Y_ant); // absoluto sem sinal  
Y1 := Y;  
diffX_mm := ABS(X - X_ant); // absoluto sem sinal  
X1 := X;  
IF PHELICAL THEN  
diffZ_mm := Z ;  
Z1 := Z + Z_ant ;  
END_IF;  
END_IF;  
END_IF;
```

```
// --- Buffers ---
```

```
IF (WR_PDO_6040_X AND HALT_BIT) = 0 THEN  
eventosParada[eventoldx].indexBuffer := indexBuffer;  
eventosParada[eventoldx].halt := (WR_PDO_6040_X AND HALT_BIT) = 0;  
eventoldx := eventoldx + 1;  
IF eventoldx > 19 THEN eventoldx := 0; END_IF;  
END_IF;
```

```
bufferX[indexBuffer] := X1; // absoluto / relativo  
bufferY[indexBuffer] := Y1;  
bufferZ[indexBuffer] := Z1;
```

```
targetDINT_X := ConvertRealDiffToTargetDINT(diffX_mm);  
targetDINT_Y := ConvertRealDiffToTargetDINT(diffY_mm);  
targetDINT_Z := ConvertRealDiffToTargetDINT(diffZ_mm);
```

```
bufferTargetX_H[indexBuffer] := INT(SHR(UDINT(targetDINT_X),16) AND 65535);  
bufferTargetX_L[indexBuffer] := INT(UDINT(targetDINT_X) AND 65535);  
bufferTargetY_H[indexBuffer] := INT(SHR(UDINT(targetDINT_Y),16) AND 65535);  
bufferTargetY_L[indexBuffer] := INT(UDINT(targetDINT_Y) AND 65535);  
bufferTargetZ_H[indexBuffer] := INT(SHR(UDINT(targetDINT_Z),16) AND 65535);  
bufferTargetZ_L[indexBuffer] := INT(UDINT(targetDINT_Z) AND 65535);
```

```

VELOCIDADE_X_Y_Z(diffX_mm, diffY_mm, diffZ_mm, rpmX, rpmY, rpmZ);
    bufferVelX[indexBuffer] := rpmX;
    bufferVelY[indexBuffer] := rpmY;
    bufferVelZ[indexBuffer] := rpmZ;

    indexBuffer := indexBuffer + 1;
    contador := contador + 1;
    contadorCiclos := contadorCiclos + 1 ;

// --- Atualiza o raio nos modos 1 e 2 ---

IF MODE1 THEN
    IF (ratual < r1) AND (contadorCiclos = totalPassosTotal) THEN
        ratual := ratual + passoC;
    END_IF;

ELSIF MODE2 THEN
    IF (ratual > r1) AND (contadorCiclos = totalPassosTotal) THEN
        ratual := ratual - passoC;
    END_IF;
END_IF;

IF (contadorCiclos = 1)
    OR (NOT FLAT AND (contadorCiclos = totalPassosTotal) AND (totalCycles1 > contadorCiclos)) THEN

IF PXY THEN
Z := Z + passoDZ; // passo + ou -
IF RelativeZ THEN
    diffZ_mm := (Z - Z_ant); // individual ou XYZ
Z1 := diffZ_mm;
ELSE
    diffZ_mm := Z - Z_ant ; // absoluto
Z1 := Z ;
END_IF;

IF PZX THEN
    Y := Y + passoDY; // passo + ou -
IF RelativeY THEN
    diffY_mm := (Y - Y_ant); // individual ou XYZ
Y1 := diffY_mm ;
ELSE
    diffY_mm := Y - Y_ant ; // absoluto
Y1 := Y;
END_IF;

IF PZY THEN
    X := X + passoDX; // passo + ou -
IF RelativeX THEN
    diffX_mm := (X - X_ant); // individual ou XYS
X1 := diffX_mm ;
ELSE
    diffX_mm := X - X_ant; // absoluto
X1 := X;
END_IF;

```

```

bufferX[indexBuffer] := X1;
bufferY[indexBuffer] := Y1;
bufferZ[indexBuffer] := Z1;

targetDINT_X := ConvertRealDiffToTargetDINT(diffX_mm);
targetDINT_Y := ConvertRealDiffToTargetDINT(diffY_mm);
targetDINT_Z := ConvertRealDiffToTargetDINT(diffZ_mm);

bufferTargetX_H[indexBuffer] := INT(SHR(UDINT(targetDINT_X),16) AND 65535);
bufferTargetX_L[indexBuffer] := INT(UDINT(targetDINT_X) AND 65535);
bufferTargetY_H[indexBuffer] := INT(SHR(UDINT(targetDINT_Y),16) AND 65535);
bufferTargetY_L[indexBuffer] := INT(UDINT(targetDINT_Y) AND 65535);
bufferTargetZ_H[indexBuffer] := INT(SHR(UDINT(targetDINT_Z),16) AND 65535);
bufferTargetZ_L[indexBuffer] := INT(UDINT(targetDINT_Z) AND 65535);

VELOCIDADE_X_Y_Z(diffX_mm, diffY_mm, diffZ_mm, rpmX, rpmY, rpmZ);
bufferVelX[indexBuffer] := rpmX;
bufferVelY[indexBuffer] := rpmY;
bufferVelZ[indexBuffer] := rpmZ;

indexBuffer := indexBuffer + 1;

```

```
END_IF;
```

```

IF contadorCiclos > 0 THEN
(* atualizar antigos *)
X_ant := X;
Y_ant := Y;
Z_ant := Z;
END_IF;

```

```

IF contadorCiclos >= totalCycles1 THEN
NF := TRUE;
END_IF;

```

```
END_WHILE;
```

```

contadorCiclos := 0;
contador := 0;
passoDX := 0;
passoDY := 0;
passoDZ := 0;

```

```
IF DONE THEN
```

```

bufferFilled := TRUE;

// Marca os índices finais dos eixos
axisX.lastIndex := indexBuffer - 1;
axisY.lastIndex := indexBuffer - 1;
axisZ.lastIndex := indexBuffer - 1;

// Marca eixos como prontos
axisX.ready := TRUE;
axisY.ready := TRUE;
axisZ.ready := TRUE;

// Zera índices ou ciclos se necessário
indexBuffer := 0;
totalCycles := 0;

END_IF;
END_FUNCTION ;

```

@

16. FUNCTION INTERPOLACAO_BI_PLANAR

A função INTERPOLACAO_BI_PLANAR , realiza reset completo de estados, seguido de cálculo do coeficiente de acoplamento radial-angular (k) entre PH e raio. O loop principal gera a trajetória enquanto respeita limites de buffer, raio e ciclos.

A interpolação é construída por discretização geométrica do arco (via PQ otimizado por erro angular), definindo incremento angular e passo linear. Se o modo helicoidal estiver ativo, o raio evolui em degraus (deltaSS), acoplado à progressão axial PH com limitação por PHmin e dinâmica de desaceleração baseada em RPM e incremento máximo.

A posição é atualizada por trigonometria circular (sen/cos), com inversão de sinais por quadrante e sentido de rotação. Em modos combinados (XY/ZX/ZY), o sistema projeta o movimento em diferentes planos, recombina componentes de X, Y e Z conforme o par de eixos ativo.

O avanço helicoidal ocorre com acoplamento entre rotação e translação axial, mantendo continuidade cinemática entre ciclos e controle de sentido com inversão simétrica de ângulos e centros.

As diferenças de posição são calculadas em modo absoluto ou relativo, convertidas para formato de transmissão (DINT high/low) e armazenadas em buffers de posição e velocidade. Eventos de HALT são registrados por índice. O processo encerra com marcação de DONE, fechamento de buffers e reset de contadores.

```
FuncoesOriginais[3] := 'INTERPOLACAO_BI_PLANAR';
```

```
FuncoesGCODE[3] := 'F3';
```

```
Variaveis[3]
```

```

:=
'NI,NF,RETURN_0,PHELICAL,XY,ZX,ZY,PXY,PZX,PZY, Q1,Q2,Q3,Q4,
SENTIDO,SENTIDO1, LIM_POS,LIM_NEG,PH,PHmax,PHmin,
X0C,Y0C,Z0C,X1C,Y1C,Z1C,
XC,YC,ZC,XC1,YC1,ZC1,
r,r1,r0,rmax,
deltaX,deltaX1,
ANGULOINICIAL,ANGULOFINAL,ANGULOINICIAL1,ANGULOFINAL1';

```

TiposVariaveis[3]

```

:=
'BOOLBOOL,BOOL,BOOL,BOOL,BOOL,BOOL,BOOL,BOOL,BOOL,BOOL,BOOL,BOOL,BOOL,
USINT,USINT,
REAL,REAL,REAL,
REAL,REAL,REAL,REAL,REAL,REAL,
REAL,REAL,REAL,REAL,REAL,REAL,
REAL,REAL,REAL,REAL,
REAL,REAL,
REAL,REAL,REAL,REAL,REAL,REAL';

```

FUNCTION INTERPOLACAO_BI_PLANAR

VAR

```

    k : REAL ;
    D : INT;
    deltaSS : REAL;
    deltaS : REAL;
    deltaS1 : REAL;
    ang_i_tmp : REAL;
    ang_f_tmp : REAL;
    PI : REAL := 3.14159265;
    PQ_min    : INT;
    PQ        : INT;
    theta     : REAL;
    teste0    : REAL;
    arred     : REAL;
    erro      : REAL;
    tolerancia : REAL;
    PQ_min1   : INT;
    PQ1       : INT;
    theta1    : REAL;
    teste1    : REAL;
    arred1    : REAL;
    erro1     : REAL;
    tolerancia1 : REAL;
    x_tmp     : REAL;
    y_tmp     : REAL;
    z_tmp     : REAL;
    sx        : SINT;
    sy        : SINT;
END_VAR

```

VAR_OUTPUT

```

// --- Diferenças de posição ---
diffX_mm, diffY_mm, diffZ_mm : REAL;

```

END_VAR

VAR_INPUT

// --- Velocidades ---

rpmX, rpmY, rpmZ : UDINT;

END_VAR

BEGIN

SENTIDO_ant := SENTIDO;

IF Reset THEN

PartialStart0 := 0.0; PartialEnd0 := 100.0;

N := 0;

RETURN_0 := FALSE; PHELICAL := FALSE;

XY := FALSE; ZX := FALSE; ZY := FALSE;

PXY := FALSE; PZX := FALSE; PZY := FALSE;

Q1 := FALSE; Q2 := FALSE; Q3 := FALSE; Q4 := FALSE;

LIM_POS := 0.0; LIM_NEG := 0.0;

PH := 0.0; PHmax := 0.0; PHmin := 0.0;

X0C := 0.0; Y0C := 0.0; Z0C := 0.0;

X1C := 0.0; Y1C := 0.0; Z1C := 0.0;

XC := 0.0; YC := 0.0; ZC := 0.0;

XC1 := 0.0; YC1 := 0.0; ZC1 := 0.0;

r := 0.0; r1 := 0.0; r0 := 0.0; rmax := 0.0;

contadorCiclos := 0; contadorCiclos1 := 0;

contador := 1; contador1 := 1;

END_IF;

$k := (PHmax - PHmin) / (rmax - r);$

WHILE (indexBuffer < maxBuffer) AND

((totalPassos1 >= contadorCiclos1) OR (r <= rmax))

DO

IF (contadorCiclos = 0) AND (contadorCiclos1 = 0) THEN

X_ant1 := X; // posição segura de cálculo

Y_ant1 := Y; // posição de cálculo

Z_ant1 := Z; // posição de cálculo

END_IF;

IF contadorCiclos = 1 THEN

$s := 3.14159265 * r / 2.0;$

PQ_min := CEIL(s / deltaX);

theta := ANGULOFINAL - ANGULOINICIAL;

IF theta < 0 THEN theta := theta + 360;

passosQuadrante := 0;

deltaS := 0.0;

anguloInc := 0.0;

totalPassos := 0;

tolerancia := 3 ;

FOR PQ := PQ_min TO 2000 DO


```

teste0 := (theta * REAL(PQ)) / 90.0;
arred := REAL(ROUND(teste0));
erro := ABS(teste0 - arred);
IF (erro < tolerancia) AND (INT(arred) MOD 2 = 0) THEN      passosQuadrante := PQ;
    deltaS := s / REAL(PQ);
    anguloInc := 90.0 / REAL(PQ);
    totalPassos := INT(arred);    EXIT;
END_IF;
END_FOR;

```

```

IF PHELICAL THEN
    D := CEIL((rmax - r0) / deltaS);
    deltaSS := (rmax - r0) / D;
END_IF;

```

```

ANGGFINAL := ANGULOFINAL - ANGULOINICIAL;
IF ANGGFINAL < 0 THEN ANGGFINAL := ANGGFINAL + 360;

```

```

//deltaS max 0.45mm // 3000 RPM // fuso 6mm// 1/2at²
// limites 1RPM

```

```

RPM := deltaS * 6666.7;

```

```

incremento_maximo := RPM * WR_PDO_6083_X * 0.00005;

```

```

ANG1 := (incremento_maximo / r) * 57.32 ;

```

```

IF ANGGFINAL <= 2 * ANG1 THEN
ANG1 := ANGGFINAL * 0.5 ;
END_IF;

```

```

END_IF; // IF contadorCiclos = 1

```

```

angulo := ANGULOINICIAL + contador * anguloInc ;
ANGULOTOTAL := contador * anguloInc;
IF ANGULOTOTAL >= ANGGFINAL - ANG1 THEN
MAX_RPM := RPM - 30.0 * SQRT( RPM * (incremento_maximo - deltaS) / (0.045 * WR_PDO_6083_X) );
IF ANGULOTOTAL >= ANGGFINAL THEN
ANGULOTOTAL := 0 ;
END_IF;

```

```

IF contadorCiclos1 = 0 THEN

```

```

s1 := 3.14159265 * r1 / 2.0;
PQ_min1 := CEIL(s1 / deltaX1);
theta1 := ANGULOFINAL1 - ANGULOINICIAL1;
IF theta1 < 0 THEN theta1 := theta1 + 360;
passosQuadrante1 := 0;

```

```

deltaS1 := 0.0;
anguloInc1 := 0.0;
totalPassos1 := 0;
tolerancia1 := 3;
FOR PQ1 := PQ_min1 TO 2000 DO
teste1 := (theta1 * REAL(PQ1)) / 90.0;
arred1 := REAL(ROUND(teste1));
erro1 := ABS(teste1 - arred1);
IF (erro1 < tolerancia1) AND (INT(arred1) MOD 2 = 0) THEN
passosQuadrante1 := PQ1;
deltaS1 := s1 / REAL(PQ1);
anguloInc1 := 90.0 / REAL(PQ1);
totalPassos1 := INT(arred1);
EXIT;
END_IF;
END_FOR;

ANGGFINAL1 := ANGULOFINAL1 - ANGULOINICIAL1;
IF ANGGFINAL1 < 0 THEN ANGGFINAL1 := ANGGFINAL1 + 360;

```

```

RPM := deltaS1 * 6666.7;
incremento_maximo1 := RPM * WR_PDO_6083_X * 0.00005;
ANG11 := (incremento_maximo1 / r1) * 57.32;
IF ANGGFINAL1 <= 2 * ANG11 THEN
ANG11 := ANGGFINAL1 * 0.5;
END_IF;
END_IF; // contadorCiclos1 = 0

```

```

Angulo1 := ANGULOINICIAL1 + contador1 * anguloInc1;
ANGULOTOTAL1 := contador1 * anguloInc1;
IF ANGULOTOTAL1 >= ANGGFINAL1 - ANG11 THEN
MAX_RPM := RPM - 30.0 * SQRT(RPM * (incremento_maximo1 - deltaS1) / (0.045 * WR_PDO_6083_X));
END_IF;
IF ANGULOTOTAL1 >= ANGGFINAL1 THEN
ANGULOTOTAL1 := 0;
END_IF;

```

```

rad := angulo * 0.0174533;
rad1 := angulo1 * 0.0174533;

```

```

IF Relative THEN
diffX_mm := 0;
diffY_mm := 0;
diffZ_mm := 0;
END_IF;

```

```

IF SENTIDO = 1 THEN
IF rad < PI/2 THEN
sx := -1; sy := +1;
ELSIF rad < PI THEN
sx := -1; sy := -1;
ELSIF rad < 3*PI/2 THEN
sx := +1; sy := -1;
ELSE

```

```

        sx := +1; sy := +1;
    END_IF;
ELSE
    IF rad < PI/2 THEN
        sx := -1; sy := -1;
    ELSIF rad < PI THEN
        sx := -1; sy := +1;
    ELSIF rad < 3*PI/2 THEN
        sx := +1; sy := +1;
    ELSE
        sx := +1; sy := -1;
    END_IF;
END_IF;

(* ----- MODE 3= 6 (ZY) ----- *)
IF PZY THEN

    IF (r1 > 0.0) AND
        ((XY AND ZY) OR (ZX AND ZY)) AND
        (contadorCiclos = 0) THEN
// Incremento de contadores
        contadorCiclos1 := contadorCiclos1 + 1;
        contador1 := contador1 + 1; // total passos

        // Bloco original: mode1 AND mode3
    IF (XY AND ZY) THEN
        x_tmp := r1 * (1 - COS(rad1));
        y_tmp := r1 * SIN(rad1);
        IF SENTIDO = 1 THEN
            X := XC1 - x_tmp;
            Y := YC1 + x_tmp;
        ELSE
            X := XC1 - x_tmp;
            Y := YC1 - y_tmp;
        END_IF;
    END_IF;

        // Bloco original: mode2 AND mode3
    IF (ZX AND ZY) THEN
        x_tmp := r1 * (1 - COS(rad1));
        z_tmp := r1 * SIN(rad1);
        IF SENTIDO1 = 1 THEN
            Z := ZC1 - z_tmp;
            X := XC1 + x_tmp;
        ELSE
            Z := ZC1 - z_tmp;
            X := XC1 - x_tmp;
        END_IF;
    END_IF;
    END_IF;

        // Bloco original: contadorCiclos <> 0 AND contadorCiclos <> 0
    IF (NOT RETURN_0) OR (SENTIDO = SENTIDO_ant) THEN

        IF contadorCiclos >= 1 THEN
            IF SENTIDO = 0 THEN
                IF PHELICAL THEN
                    IF contadorCiclos = 1 THEN

```

```

IF r1 = 0 THEN
r := r + deltaSS;
ELSE
r := r + match ;
END_IF;
PH := PHmax - k * (r - r0);
IF PH < PHmin THEN
PH := PHmin;
END_IF;
ELSE
X := X - (PH / (2 * PI)) * (anguloInc * PI / 180);
END_IF;
END_IF;
z_tmp := r * (1 - COS(rad));
y_tmp := r * SIN(rad);
Z := ZC - z_tmp;
Y := YC - y_tmp;
ELSE
IF PHELICAL THEN
IF contadorCiclos = 1 THEN
IF r1 = 0 THEN
r := r + deltaSS;
ELSE
r := r + match ;
END_IF;
PH := PHmax - k * (r - r0);
IF PH < PHmin THEN
PH := PHmin;
END_IF;
ELSE
X := X + (PH / (2 * PI)) * (anguloInc * PI / 180);
END_IF;
END_IF;

z_tmp := r * (1 - COS(rad));
y_tmp := r * SIN(rad);
Z := ZC - z_tmp;
Y := YC + y_tmp;
IF PHELICAL THEN
IF Relative THEN
diffX_mm := X;
X1 := X ;
ELSE
diffX_mm := X ;
X1 := X + X_ant ;
END_IF;
END_IF;

IF Relative THEN
diffY_mm := sx * ABS(Y - Y_ant);
diffZ_mm := sy * ABS(Z - Z_ant);
Y1 := diffY_mm;
Z1 := diffZ_mm;

```

```

ELSE
    diffY_mm := ABS(Y - Y_ant);
    diffZ_mm := ABS(Z - Z_ant);
Y1 := Y;
Z1 := Z;
END_IF;

END_IF;
END_IF;
END_IF;
END_IF;
END_IF; // RETURN0

IF PZX THEN

    IF (r1 > 0.0) AND
        ((XY AND ZX) OR (ZY AND ZX)) AND
        (contadorCiclos = 0) THEN
        // Incremento de contadores
        contadorCiclos1 := contadorCiclos1 + 1;
        contador1 := contador1 + 1; // total passos

        // Bloco original: mode1 AND mode2
        IF (XY AND ZX) THEN
            x_tmp := r1 * (1 - COS(rad1));
            y_tmp := r1 * SIN(rad1);
            IF SENTIDO THEN
                X := XC1 - x_tmp;
                Y := YC1 + x_tmp;
            ELSE
                X := XC1 - x_tmp;
                Y := YC1 - y_tmp;
            END_IF; END_IF;

            // Bloco original: mode3 AND mode2
            IF (ZY AND ZX) THEN
                z_tmp := r1 * (1 - COS(rad1));
                y_tmp := r1 * SIN(rad1);
                IF SENTIDO1 THEN
                    Z := ZC1 - z_tmp;
                    Y := YC1 + y_tmp;
                ELSE
                    Z := ZC1 - z_tmp;
                    Y := YC1 - y_tmp;
                END_IF; END_IF; END_IF;

                // Bloco original: contadorCiclos <> 0 AND contadorCiclos <> 0
                IF (NOT RETURN_0) OR (SENTIDO = SENTIDO_ant) THEN

                    IF contadorCiclos >= 1 THEN
                        IF SENTIDO = 0 THEN
                            IF PHELICAL THEN
                                IF contadorCiclos = 1 THEN
                                    IF r1 = 0 THEN
                                        r := r + deltaSS;
                                    ELSE

```

```

r := r + match ;
END_IF;
PH := PHmax - k * (r - r0);
IF PH < PHmin THEN
PH := PHmin;
END_IF;
ELSE
Y := Y - (PH / (2 * PI)) * (anguloInc * PI / 180);
END_IF;
END_IF;
x_tmp := r * (1 - COS(rad));
z_tmp := r * SIN(rad);
Z := ZC - z_tmp;
X := XC - x_tmp;
ELSE
IF PHELICAL THEN
IF contadorCiclos = 1 THEN
IF r1 = 0 THEN
r := r + deltaSS;
ELSE
r := r + match ;
END_IF;
PH := PHmax - k * (r - r0);
IF PH < PHmin THEN
PH := PHmin;
END_IF;
ELSE
Y := Y + (PH / (2 * PI)) * (anguloInc * PI / 180);
END_IF;
END_IF;
x_tmp := r * (1 - COS(rad));
z_tmp := r * SIN(rad);
Z := ZC - z_tmp;
X := XC + x_tmp;

IF PHELICAL THEN
IF Relative THEN
diffY_mm := Y;
Y1 := Y ;
ELSE
diffY_mm := Y;
Y1 := Y + Y_ant ;
END_IF;
END_IF;

IF Relative THEN
diffX_mm := sx * ABS(X - X_ant);
diffZ_mm := sy * ABS(Z - Z_ant);
Z1 := diffZ_mm ;
X1 := diffX_mm ;
ELSE
diffX_mm := ABS(X - X_ant) ;
diffZ_mm := ABS(Z - Z_ant) ;

```

```

Z1 := Z ;
X1 := X ;
END_IF;

END_IF;
END_IF;
END_IF;
END_IF;
END_IF; // RETURN0
(* ----- MODE1 4 (XY) ----- *)
IF PXY THEN
IF (r1 > 0.0) AND
  ((XY AND ZX) OR (XY AND ZY)) AND
  (contadorCiclos = 0)
THEN // Incremento de contadores
  contadorCiclos1 := contadorCiclos1 + 1;
  contador1 := contador1 + 1; // total passos

  // Bloco original: mode1 AND mode2
IF (XY AND ZX) THEN
x_tmp := r1 * (1 - COS(rad1));
z_tmp := r1 * SIN(rad1);
IF SENTIDO1 = 1 THEN
Z := ZC1 - x_tmp;
X := XC1 + x_tmp;
ELSE
Z := ZC1 - z_tmp;
X := XC1 - x_tmp;
END_IF; END_IF;

  // Bloco original: mode1 AND mode3
IF (XY AND ZY) THEN
z_tmp := r1 * (1 - COS(rad1));
y_tmp := r1 * SIN(rad1);
IF SENTIDO1 = 1 THEN
Z := ZC1 - z_tmp;
Y := YC1 + y_tmp;
ELSE
Z := ZC1 - z_tmp;
Y := YC1 - y_tmp;
END_IF; END_IF;END_IF;

  // Bloco original: contadorCiclos <> 0 AND contadorCiclos <> 0
IF (NOT RETURN_0) OR (SENTIDO = SENTIDO_ant) THEN

  IF contadorCiclos >= 1 THEN
    IF SENTIDO = 0 THEN

IF PHELICAL THEN
IF contadorCiclos = 1 THEN
IF r1 = 0 THEN
r := r + deltaSS;
ELSE
r := r + match ;
END_IF;

```

```

PH := PHmax - k * (r - r0);
IF PH < PHmin THEN
PH := PHmin;
END_IF;
ELSE
Z := Z - (PH / (2 * PI)) * (anguloInc * PI / 180);
END_IF;
END_IF;

```

```

x_tmp := r * (1 - COS(rad));
y_tmp := r * SIN(rad);
X := XC - x_tmp;
Y := YC - y_tmp;

```

```

ELSE

```

```

IF PHELICAL THEN
IF contadorCiclos = 1 THEN
IF r1 = 0 THEN
r := r + deltaSS;
ELSE
r := r + match ;
END_IF;
PH := PHmax - k * (r - r0);
IF PH < PHmin THEN
PH := PHmin;
END_IF;
ELSE
Z := Z + (PH / (2 * PI)) * (anguloInc * PI / 180);
END_IF;
END_IF;

```

```

z_tmp := r * (1 - COS(rad));
x_tmp := r * SIN(rad);
X := XC - x_tmp;
Y := YC + y_tmp;

```

```

IF PHELICAL THEN
IF Relative THEN
diffZ_mm := Z;
ELSE
diffZ_mm := Z + Z_ant ;
END_IF;
END_IF;
IF Relative THEN
diffX_mm := sx * ABS(X - X_ant);
diffY_mm := sy * ABS(Y - Y_ant);
Y1 := diffY_mm ;
X1 := diffX_mm ;
ELSE
diffX_mm := ABS(X - X_ant) ;
diffY_mm := ABS(Y - Y_ant) ;
Y1 := Y;
X1 := X;
END_IF;

```



```
    END_IF;  
    END_IF;  
END_IF;  
END_IF;  
END_IF; // return 0
```

```
IF contadorCiclos = 0 AND r1 > 0 THEN  
IF PZY THEN  
IF (NOT RETURN_0) OR (SENTIDO = SENTIDO_ant) THEN
```

```
IF XY AND ZY THEN  
match := sy * ABS(Y - Y_ant1);  
IF Relative THEN  
profX := sx * ABS(X - X_ant1);  
X1 := profX ;  
ELSE  
profX := ABS(X - X_ant1);  
X1 := X ;  
END_IF;  
diffX_mm := profX;  
ELSIF ZX AND ZY THEN  
match := sx * ABS(Z - Z_ant1);  
IF Relative THEN  
profX := sy * ABS(X - X_ant1);  
X1 := profX ;  
ELSE  
profX := ABS(X - X_ant1);  
X1 := X;  
END_IF;  
diffX_mm := profX;  
END_IF;  
END_IF;  
END_IF;
```

```
IF PZX THEN  
IF (NOT RETURN_0) OR (SENTIDO = SENTIDO_ant) THEN
```

```
IF XY AND ZX THEN  
match := sx * ABS(X - X_ant1);  
IF Relative THEN  
profY := sy * ABS(Y - Y_ant1);  
Y1 := profY ;  
ELSE  
profY := ABS(Y - Y_ant1 );  
Y1 := Y ;  
END_IF;  
diffY_mm := profY;  
ELSIF ZX AND ZY THEN  
match := sy * ABS(Z - Z_ant1);  
IF Relative THEN  
profY := sy * ABS(Y - Y_ant1);  
Y1 := profY ;  
ELSE  
profY := ABS(Y - Y_ant1);  
Y1 := Y ;
```

```

END_IF;
diffY_mm := profY;
END_IF;
END_IF;
END_IF;

IF PXY THEN
IF (NOT RETURN_0) OR (SENTIDO = SENTIDO_ant) THEN

```

```

IF XY AND ZX THEN
match := sx * ABS(X - X_ant1);
IF Relative THEN
profZ := sy * ABS(Z - Z_ant1);
Z1 := profZ;
ELSE
profZ := ABS(Z - Z_ant1);
Z1 := Z ;
END_IF;
diffZ_mm := profZ;
ELSIF XY AND ZY THEN
match := sy * ABS(Y - Y_ant1);
IF Relative THEN
profZ := sx * ABS(Z - Z_ant1);
Z1 := profZ ;
ELSE
profZ := ABS(Z - Z_ant1);
Z1 := Z;
END_IF;
diffZ_mm := profZ;
END_IF;
END_IF;
END_IF;
END_IF;

```

```

IF contadorCiclos = 0 THEN

```

```

IF (NOT RETURN_0) OR (SENTIDO = SENTIDO_ant) THEN

```

```

IF r1 > 0 THEN
r := r + match;
ELSE
match := deltaSS ;
END_IF;

```

```

IF PXY THEN
IF Q1 AND Q2 THEN X1C := X1C + match; Y0C := Y0C + match;
ELSIF Q1 AND Q3 THEN X1C := X1C + match; X0C := X0C - match;
ELSIF Q1 AND Q4 THEN Y0C := Y0C - match; X1C := X1C + match;
ELSIF Q2 AND Q3 THEN Y1C := Y1C + match; X0C := X0C - match;
ELSIF Q2 AND Q4 THEN Y1C := Y1C + match; Y0C := Y0C - match;
ELSIF Q3 AND Q4 THEN X1C := X1C - match; Y0C := Y0C - match;
ELSIF Q1 THEN X1C := X1C + match;
ELSIF Q2 THEN Y1C := Y1C + match;
ELSIF Q3 THEN X0C := X0C - match;

```

```
ELSIF Q4 THEN Y0C := Y0C - match;
END_IF;
END_IF;
```

```
IF PZY THEN
```

```
  IF Q1 AND Q2 THEN Y1C := Y1C + match; Z0C := Z0C + match;
  ELSIF Q1 AND Q3 THEN Y1C := Y1C + match; Y0C := Y0C - match;
  ELSIF Q1 AND Q4 THEN Y1C := Y1C + match; Z0C := Z0C - match;
  ELSIF Q2 AND Q3 THEN Z1C := Z1C + match; Y0C := Y0C - match;
  ELSIF Q2 AND Q4 THEN Z1C := Z1C + match; Z0C := Z0C - match;
  ELSIF Q3 AND Q4 THEN Y1C := Y1C - match; Z0C := Z0C - match;
  ELSIF Q1 THEN Y1C := Y1C + match;
  ELSIF Q2 THEN Z1C := Z1C + match;
  ELSIF Q3 THEN Y0C := Y0C - match;
  ELSIF Q4 THEN Z0C := Z0C - match;
  END_IF;
END_IF;
```

```
IF PZX THEN
```

```
  IF Q1 AND Q2 THEN X1C := X1C + match; Z0C := Z0C + match;
  ELSIF Q1 AND Q3 THEN X1C := X1C + match; X0C := X0C - match;
  ELSIF Q1 AND Q4 THEN Z0C := Z0C - match; X1C := X1C + match;
  ELSIF Q2 AND Q3 THEN Z1C := Z1C + match; X0C := X0C - match;
  ELSIF Q2 AND Q4 THEN Z1C := Z1C + match; Z0C := Z0C - match;
  ELSIF Q3 AND Q4 THEN X1C := X1C - match; Z0C := Z0C - match;
  ELSIF Q1 THEN X1C := X1C + match;
  ELSIF Q2 THEN Z1C := Z1C + match;
  ELSIF Q3 THEN X0C := X0C - match;
  ELSIF Q4 THEN Z0C := Z0C - match;
  END_IF;
END_IF;
END_IF; // return0
END_IF;
```

```
IF (contadorCiclos = totalPassos) AND (r < rmax) AND (ANGGFINAL <> 360.0) THEN
```

```
  SENTIDO := 1 - SENTIDO;
```

```
  contadorCiclos := 0;
```

```
  ang_i_tmp := ANGULOINICIAL;
```

```
  ang_f_tmp := ANGULOFINAL;
```

```
  ANGULOINICIAL := MOD(360.0 - ang_f_tmp, 360.0);
```

```
  ANGULOFINAL := MOD(360.0 - ang_i_tmp, 360.0);
```

```
IF SENTIDO = 1 THEN
```

```
  XC := X1C;
```

```
  YC := Y1C;
```

```
  ZC := Z1C;
```

```
ELSE
```

```
  XC := X0C;
```

```
  YC := Y0C;
```

```
  ZC := Z0C;
```

```
END_IF;
```

```
END_IF;
```

END_IF;

// --- Buffers ---

```
IF (WR_PDO_6040_X AND HALT_BIT) = 0 THEN
  eventosParada[eventoldx].indexBuffer := indexBuffer;
  eventosParada[eventoldx].halt := (WR_PDO_6040_X AND HALT_BIT) = 0;
  eventoldx := eventoldx + 1;
  IF eventoldx > 19 THEN eventoldx := 0; END_IF;
END_IF;
```

```
bufferX[indexBuffer] := X1;
bufferY[indexBuffer] := Y1;
bufferZ[indexBuffer] := Z1;
```

```
targetDINT_X := ConvertRealDiffToTargetDINT(diffX_mm);
targetDINT_Y := ConvertRealDiffToTargetDINT(diffY_mm);
targetDINT_Z := ConvertRealDiffToTargetDINT(diffZ_mm);
```

```
bufferTargetX_H[indexBuffer] := INT(SHR(UDINT(targetDINT_X),16) AND 65535);
bufferTargetX_L[indexBuffer] := INT(UDINT(targetDINT_X) AND 65535);
bufferTargetY_H[indexBuffer] := INT(SHR(UDINT(targetDINT_Y),16) AND 65535);
bufferTargetY_L[indexBuffer] := INT(UDINT(targetDINT_Y) AND 65535);
bufferTargetZ_H[indexBuffer] := INT(SHR(UDINT(targetDINT_Z),16) AND 65535);
bufferTargetZ_L[indexBuffer] := INT(UDINT(targetDINT_Z) AND 65535);
```

```
VELOCIDADE_X_Y_Z(diffX_mm, diffY_mm, diffZ_mm, rpmX, rpmY, rpmZ);
bufferVelX[indexBuffer] := rpmX;
bufferVelY[indexBuffer] := rpmY;
bufferVelZ[indexBuffer] := rpmZ;
```

```
IF ABS(rmax - r) <= 0.0005 THEN
  NF := TRUE;
END_IF;
```

```
contadorCiclos := ContadorCiclos + 1 ;
```

```
contador := contador + 1; // totalpassos
```

```
indexBuffer := indexBuffer + 1;
```

```
IF contadorCiclos > totalPassos THEN
  contadorCiclos := 0 ;
  contador := 1;
END_IF;
```

```
IF contadorCiclos >= 1 THEN
  X_ant := X; // posição segura de ataque
  Y_ant := Y; // posição de ataque
```

```

    Z_ant := Z; // posição de ataque
END_IF;

IF contadorCiclos1 >= 1 THEN
    X_ant1 := X; // posição segura de ataque
    Y_ant1 := Y; // posição de ataque
    Z_ant1 := Z; // posição de ataque
END_IF;

END_WHILE;

// ☒ DONE único: finalização consciente da lista

IF DONE THEN
    bufferFilled := TRUE;

    // Marca os índices finais dos eixos
    axisX.lastIndex := indexBuffer - 1;
    axisY.lastIndex := indexBuffer - 1;
    axisZ.lastIndex := indexBuffer - 1;

    // Marca eixos como prontos
    axisX.ready := TRUE;
    axisY.ready := TRUE;
    axisZ.ready := TRUE;

    // Zera índices ou ciclos se necessário
    indexBuffer := 0;
    totalCycles := 0;

END_IF;

END_FUNCTION;

```

@

17. FUNCTION INTERPOLACAO_BI_PLANAR_PUMPKIN

A função INTERPOLACAO_BI_PLANAR_PUMPKIN realiza a geração de trajetória interpolada em dois conjuntos de eixos sincronizados, com suporte a modos de projeção planar (XY, ZX, ZY e combinações), além de variação radial e helicoidal. Ela inicia com reset completo de estados, centros, raios, contadores e buffers, e em seguida configura a direção e o modo de execução. O núcleo do algoritmo percorre o buffer enquanto respeita limites geométricos e de ciclo, calculando discretização angular e radial a partir de condições de erro, tolerância e

resolução mínima, garantindo que o número de passos seja compatível com divisibilidade angular.

A cada ciclo, são atualizados incrementos angulares, deslocamentos radiais e projeções trigonométricas para X, Y e Z, com inversão de sinal conforme quadrante e sentido de rotação. Em paralelo, um segundo conjunto de variáveis mantém sincronismo geométrico independente, permitindo interpolação dupla. Quando habilitado, o modo helicoidal ajusta progressivamente o raio e acopla avanço axial à rotação. O sistema também implementa limitação dinâmica de velocidade via RPM e desaceleração baseada em incremento máximo permitido, evitando transições abruptas no final do arco.

Os resultados são armazenados em buffers de posição, diferença e velocidade, incluindo conversão para formato inteiro segmentado para transmissão. Há tratamento explícito de HALT com registro de eventos, controle de ciclos, reversão de sentido com transformação simétrica de ângulos e rebase de centros, além de finalização estruturada com marcação de DONE e consolidação dos índices finais dos eixos.

FuncoesOriginais[4]

:= 'INTERPOLACAO_BI_PLANAR_PUMPKIN';

FuncoesGCODE[4]

:= 'F4';

Variaveis[4]

:=

'EXTERNO,XY,ZY,ZX,PXY,PZX,PZY,LIM_POS,LIM_NEG,
SENTIDO,SENTIDO1,
X0C,Y0C,Z0C,X1C,Y1C,Z1C,
XC,YC,ZC,XC1,YC1,ZC1,
r,r1,r2,rmax,r1max,r2max,
deltaX,deltaX1,
ANGULOINICIAL,ANGULOFINAL,ANGULOINICIAL1,ANGULOFINAL1';

TiposVariaveis[4]

:=

'BOOL,BOOL,BOOL,BOOL,BOOL,BOOL,BOOL,BOOL,BOOL,
USINT,USINT,
REAL,REAL,REAL,REAL,REAL,REAL,
REAL,REAL,REAL,REAL,REAL,REAL,
REAL,REAL,REAL,REAL,REAL,REAL,
REAL,REAL,
REAL,REAL,REAL,REAL';

FUNCTION INTERPOLACAO_BI_PLANAR_PUMPKIN

VAR

deltaR,deltaR2 : REAL;

START_1 : BOOL := FALSE;

ang_i_tmp : REAL;

ang_f_tmp : REAL;

PI : REAL := 3.14159265;

delta : REAL;

deltaS : REAL;

```

deltaS1 : REAL;
PQ_min   : INT;
PQ       : INT;
theta    : REAL;
teste0   : REAL;
arred    : REAL;
erro     : REAL;
tolerancia : REAL;
PQ_min1  : INT;
PQ1      : INT;
theta1   : REAL;
teste1   : REAL;
arred1   : REAL;
erro1    : REAL;
tolerancia1 : REAL;
x_tmp    : REAL;
y_tmp    : REAL;
z_tmp    : REAL;
sx       : SINT;
sy       : SINT;
END_VAR

VAR_OUTPUT
// --- Diferenças de posição ---
diffX_mm, diffY_mm, diffZ_mm : REAL;
END_VAR

VAR_INPUT
// --- Velocidades ---
rpmX, rpmY, rpmZ : UDINT;
END_VAR

BEGIN

    SENTIDO_ant := SENTIDO;

IF Reset THEN
Q1 ,Q2,Q3,Q4 , Q1r,Q2r,Q3r,Q4r,Q1r2,Q2r2,Q3r2,Q4r2 := FALSE ;
START_1 := FALSE ;
PartialStart0 := 0.0;
PartialEnd0 := 100.0;
EXTERNO := FALSE;
XY := FALSE; ZY := FALSE; ZX := FALSE;
PXY := FALSE; PZX := FALSE; PZY := FALSE;
X0C := 0.0; Y0C := 0.0; Z0C := 0.0;
X1C := 0.0; Y1C := 0.0; Z1C := 0.0;
XC := 0.0; YC := 0.0; ZC := 0.0;
XC1 := 0.0; YC1 := 0.0; ZC1 := 0.0;
r := 0.0; r1 := 0.0; r2 := 0.0;
rmax := 0.0; r1max := 0.0; r2max := 0.0;
contadorCiclos := 0; contadorCiclos1 := 0;
contador := 1; contador1 := 1;
totalPassosTotal := 0;
END_IF;

IF NOT START_1 THEN
IF EXTERNO THEN

```

```

IF SENTIDO1 = 1 THEN
contador2 := 0;
ELSE
contador2 := 2;
END_IF;
ELSE
IF SENTIDO1 = 1 THEN
contador2 := 2;
ELSE
contador2 := 0;
END_IF;
END_IF;
START_1 := TRUE;
END_IF;

```

```

WHILE (indexBuffer < maxBuffer)
AND (contadorCiclos <= totalPassosTotal)
AND (
    (EXTERNO AND ((r >= rmax) OR (r1 >= r1max) OR (r2 > r2max)))
    OR
    ((NOT EXTERNO) AND
    ((r <= rmax) OR (r1 <= r1max) OR (r2 <= r2max)))
)
DO
IF (contadorCiclos = 0) AND (contadorCiclos1 = 0) THEN
    X_ant := X; // posição segura de cálculo
    Y_ant := Y; // posição de cálculo
    Z_ant := Z; // posição de cálculo
END_IF;

```

```

IF ((contadorCiclos = 0) OR (contadorCiclos = totalPassosTotal)) THEN

```

```

    passes := CEIL((2.0 * 3.14159265 * r2) / MIN(0.95 * d, passoR));

```

```

IF (r2 = r2max) AND (smooth > 0) THEN
    passes := CEIL((2.0 * 3.14159265 * r2max) / (0.95 * (d - smooth)));
END_IF;

```

```

    deltaX := (2.0 * 3.14159265 * r2) / REAL(passes);

```

```

s := 3.14159265 * r2 / 2.0; // r
PQ_min := CEIL(s / deltaX);
theta := ANGULOFINAL - ANGULOINICIAL;
IF theta < 0 THEN theta := theta + 360;
passosQuadrante := 0;
deltaS := 0.0;
anguloInc := 0.0;
totalPassos := 0;
tolerancia := 3.0;

```

```

FOR PQ := PQ_min TO 2000 DO

```



```

    teste0 := (theta * REAL(PQ)) / 90.0;
arred := REAL(ROUND(teste0));
erro := ABS(teste0 - arred);
    IF (erro < tolerancia) AND (INT(arred) MOD 2 = 0) THEN
passosQuadrante := PQ;
    deltaS := s / REAL(PQ);
    anguloInc := 90.0 / REAL(PQ);
    totalPassos := INT(arred);
EXIT;
    END_IF;
END_FOR;

```

```

totalPassosTotal := totalPassosTotal + totalPassos ;
contador := 1;
contador1 := 1;

```

```

ANGGFINAL := ANGULOFINAL - ANGULOINICIAL;
IF ANGGFINAL < 0 THEN ANGGFINAL := ANGGFINAL + 360;

```

```

//deltaS max 0.45mm // 3000 RPM // fuso 6mm// 1/2at²
// limites 1RPM

```

```

RPM := deltaS * 6666.7;

```

```

incremento_maximo := RPM * WR_PDO_6083_X * 0.00005;

```

```

ANG1 := (incremento_maximo /  $r^2$ ) * 57.32 ;

```

```

IF ANGGFINAL <= 2 * ANG1 THEN
ANG1 := ANGGFINAL * 0.5 ;
END_IF;

```

```

s1 := 3.14159265 * r1 / 2.0;
PQ_min1 := CEIL(s1 / deltaX1);
theta1 := ANGULOFINAL1 - ANGULOINICIAL1;
IF theta1 < 0 THEN theta1 := theta1 + 360;
passosQuadrante1 := 0;
deltaS1 := 0.0;
anguloInc1 := 0.0;
totalPassos1 := 0;
tolerancia1 := 3;

```

```

FOR PQ1 := PQ_min1 TO 2000 DO
teste1 := (theta1 * REAL(PQ1)) / 90.0;
arred1 := REAL(ROUND(teste1));
erro1 := ABS(teste1 - arred1);
    IF (erro1 < tolerancia1) AND (INT(arred1) MOD 2 = 0) THEN
passosQuadrante1 := PQ1;
    deltaS1 := s1 / REAL(PQ1);
    anguloInc1 := 90.0 / REAL(PQ1);
    totalPassos1 := INT(arred1);
EXIT;
    END_IF;
END_FOR;

```

```
ANGGFINAL1 := ANGULOFINAL1 - ANGULOINICIAL1;  
IF ANGGFINAL1 < 0 THEN ANGGFINAL1 := ANGGFINAL1 + 360;
```

```
RPM := deltaS1 * 6666.7;  
incremento_maximo1 := RPM * WR_PDO_6083_X * 0.00005;  
ANG11 := (incremento_maximo1 / r1) * 57.32;  
IF ANGGFINAL1 <= 2 * ANG11 THEN  
ANG11 := ANGGFINAL1 * 0.5;  
END_IF;  
END_IF; // ciclo = 0
```

```
angulo1 := ANGULOINICIAL1 + contador1 * anguloInc1;  
ANGULOTOTAL1 := contador1 * anguloInc1;  
IF ANGULOTOTAL1 >= ANGGFINAL1 - ANG11 THEN  
MAX_RPM := RPM - 30.0 * SQRT(RPM * (incremento_maximo1 - deltaS1) / (0.045 * WR_PDO_6083_X));  
END_IF;  
IF ANGULOTOTAL1 >= ANGGFINAL1 THEN  
ANGULOTOTAL1 := 0;  
END_IF;
```

```
angulo := ANGULOINICIAL + contador * anguloInc;  
ANGULOTOTAL := contador * anguloInc;  
IF ANGULOTOTAL >= ANGGFINAL - ANG1 THEN  
MAX_RPM := RPM - 30.0 * SQRT( RPM * (incremento_maximo - deltaS) / (0.045 * WR_PDO_6083_X) );  
END_IF;  
IF ANGULOTOTAL >= ANGGFINAL THEN  
ANGULOTOTAL := 0 ;  
END_IF;
```

```
rad := angulo * 0.0174533;  
rad1 := angulo1 * 0.0174533;
```

```
IF Relative THEN  
diffX_mm := 0;  
diffY_mm := 0;  
diffZ_mm := 0;  
END_IF;
```

```
IF SENTIDO = 1 THEN  
IF rad < PI/2 THEN  
sx := -1; sy := +1;  
ELSIF rad < PI THEN  
sx := -1; sy := -1;  
ELSIF rad < 3*PI/2 THEN  
sx := +1; sy := -1;  
ELSE  
sx := +1; sy := +1;  
END_IF;  
ELSE  
IF rad < PI/2 THEN  
sx := -1; sy := -1;  
ELSIF rad < PI THEN  
sx := -1; sy := +1;  
ELSIF rad < 3*PI/2 THEN
```

```

        sx := +1; sy := +1;
    ELSE
        sx := +1; sy := -1;
    END_IF;
END_IF;

```

```

IF (contador2 = 1) THEN
IF NOT RETURN_0 THEN
contadorCiclos := contadorCiclos + 1;
contador := contador + 1;
END_IF;
IF r = 0 THEN
contador2 := 2;
END_IF;
END_IF;

```

```

IF PZY THEN
IF ((XY AND ZX) OR (ZY AND ZX)) THEN
IF ( (contador2 = 0) OR (contador2 = 2) ) THEN
    contadorCiclos1 := contadorCiclos1 + 1;
    contador1 := contador1 + 1; // total passos

```

```

// Bloco original: mode1 AND mode3
IF (XY AND ZY) THEN
    IF SENTIDO1 = 1 THEN
x_tmp := r1 * (1 - COS(rad1));
y_tmp := r1 * SIN(rad1);
X := XC1 - x_tmp;
Y := YC1 + y_tmp;
x_tmp := X;
X := x_tmp * COS(rad1);
Z := x_tmp * SIN(rad1);
    ELSE
x_tmp := r1 * (1 - COS(rad1));
y_tmp := r1 * SIN(rad1);
X := XC1 - x_tmp;
Y := YC1 - y_tmp;
x_tmp := X;
X := x_tmp * COS(rad1);
Z := x_tmp * SIN(rad1);
    END_IF; END_IF;

```

```

// Bloco original: mode2 AND mode3
IF (ZX AND ZY) THEN
    IF SENTIDO1 = 1 THEN
x_tmp := r * (1 - COS(rad));
z_tmp := r * SIN(rad);
X := XC - x_tmp;
Z := ZC + z_tmp;
x_tmp := X;
X := x_tmp * COS(rad1);
Y := x_tmp * SIN(rad1);
    ELSE
x_tmp := r * (1 - COS(rad));
z_tmp := r * SIN(rad);
X := XC - x_tmp;
Z := ZC - z_tmp;

```

```

x_tmp := X;
X := x_tmp * COS(rad1);
Y := x_tmp * SIN(rad1);
END_IF;
END_IF;
END_IF;

IF (contador2 = 1) AND r > 0 THEN
  IF SENTIDO = 1 THEN
    z_tmp := r * (1 - COS(rad));
    y_tmp := r * SIN(rad);
    Z := ZC - x_tmp;
    Y := YC + y_tmp;
  ELSE
    z_tmp := r * (1 - COS(rad));
    y_tmp := r * SIN(rad);
    Z := ZC - z_tmp;
    Y := YC - y_tmp;
  END_IF;
  contadorCiclos := contadorCiclos + 1 ;
  contador := contador + 1 ;
  contador2 := 2;
END_IF;

```

```

IF (contador2 = 3) THEN
  IF SENTIDO = 1 THEN
    z_tmp := r2 * (1 - COS(rad));
    y_tmp := r2 * SIN(rad);
    Z := ZC2 - z_tmp; // x
    Y := YC2 + y_tmp;
  ELSE
    z_tmp := r2 * (1 - COS(rad));
    y_tmp := r2 * SIN(rad);
    Z := ZC2 - z_tmp;
    Y := YC2 - y_tmp;
  END_IF;
  contadorCiclos := contadorCiclos + 1 ;
  contador := contador + 1 ;
  contador2 := 0;
END_IF;
END_IF;

```

```

IF Relative THEN
  diffX_mm := sx * ABS(X - X_ant);
  X1 := diffX_mm;

```

```

  diffY_mm := sy * ABS(Y - Y_ant);
  Y1 := diffY_mm;

```

```

  diffZ_mm := sx * ABS(Z - Z_ant);
  Z1 := diffZ_mm;

```

```

ELSE

```

```

  diffX_mm := ABS(X - X_ant);
  X1 := X;

```

```
diffY_mm := ABS(Y - Y_ant);  
Y1 := Y;
```

```
diffZ_mm := ABS(Z - Z_ant);  
Z1 := Z;
```

```
END_IF;  
END_IF;  
END_IF;
```

```
IF (contador2 = 1) THEN  
IF NOT RETURN_0 THEN  
contadorCiclos := contadorCiclos + 1;  
contador := contador + 1;  
END_IF;  
IF r = 0 THEN  
contador2 := 2;  
END_IF;  
END_IF;
```

```
IF PZX THEN  
IF ((XY AND ZX) OR (ZY AND ZX)) THEN  
IF ( (contador2 = 0) OR (contador2 = 2) ) THEN  
contadorCiclos1 := contadorCiclos1 + 1;  
contador1 := contador1 + 1; // total passos
```

```
IF (XY AND ZX) THEN  
IF SENTIDO1 = 1 THEN  
x_tmp := r1 * (1 - COS(rad1));  
y_tmp := r1 * SIN(rad1);  
X := XC1 - x_tmp;  
Y := YC1 + y_tmp;  
x_tmp := X;  
X := x_tmp * COS(rad1);  
Z := x_tmp * SIN(rad1);  
ELSE  
x_tmp := r1 * (1 - COS(rad1));  
y_tmp := r1 * SIN(rad1);  
X := XC1 - x_tmp;  
Y := YC1 - y_tmp;  
x_tmp := X;  
X := x_tmp * COS(rad1);  
Z := x_tmp * SIN(rad1);  
END_IF; END_IF;
```

```
// Bloco original: mode3 AND mode2  
IF (ZY AND ZX) THEN  
IF SENTIDO1 = 1 THEN  
z_tmp := r1 * (1 - COS(rad1));  
y_tmp := r1 * SIN(rad1);  
Z := ZC1 - z_tmp;  
Y := YC1 + y_tmp;  
z_tmp := Z;  
Z := z_tmp * COS(rad1);
```

```

X := z_tmp * SIN(rad1);
ELSE
z_tmp := r1 * (1 - COS(rad1));
y_tmp := r1 * SIN(rad1);
Z := ZC1 - z_tmp;
Y := YC1 - y_tmp;
z_tmp := Z;
Z := z_tmp * COS(rad1);
X := z_tmp * SIN(rad1);
END_IF; END_IF;
END_IF;

```

```

IF (contador2 = 1) AND r > 0 THEN
IF SENTIDO = 1 THEN
x_tmp := r * (1 - COS(rad));
z_tmp := r * SIN(rad);
X := XC - x_tmp;
Z := ZC + z_tmp;
ELSE
x_tmp := r * (1 - COS(rad));
z_tmp := r * SIN(rad);
X := XC - x_tmp;
Z := ZC - z_tmp;
END_IF;
contadorCiclos := contadorCiclos + 1 ;
contador := contador + 1; // totalpassos
contador2 := 2 ;
END_IF;

```

```

IF (contador2 = 3) THEN
IF SENTIDO = 1 THEN
x_tmp := r2 * (1 - COS(rad));
z_tmp := r2 * SIN(rad);
X := XC2 - x_tmp;
Z := ZC2 + z_tmp;
ELSE
x_tmp := r2 * (1 - COS(rad));
z_tmp := r2 * SIN(rad);
X := XC2 - x_tmp;
Z := ZC2 - z_tmp;
END_IF;

```

```

contadorCiclos := contadorCiclos + 1 ;
contador := contador + 1; // total passos
contador2 := 0;
END_IF;
END_IF;

```

```

IF Relative THEN
diffX_mm := sx * ABS(X - X_ant);
X1 := diffX_mm;

```

```

diffY_mm := sy * ABS(Y - Y_ant);
Y1 := diffY_mm;

```

```
diffZ_mm := sy * ABS(Z - Z_ant);  
Z1 := diffZ_mm;
```

ELSE

```
diffX_mm := ABS(X - X_ant);  
X1 := X;
```

```
diffY_mm := ABS(Y - Y_ant);  
Y1 := Y;
```

```
diffZ_mm := ABS(Z - Z_ant);  
Z1 := Z;  
END_IF;  
END_IF;  
END_IF;
```

```
(* ----- MODE1 4 (XY) ----- *)
```

```
IF (contador2 = 1) THEN  
IF NOT RETURN_0 THEN  
contadorCiclos := contadorCiclos + 1;  
contador := contador + 1;  
END_IF;  
IF r = 0 THEN  
contador2 := 2;  
END_IF;  
END_IF;
```

```
IF PXY THEN  
IF ((XY AND ZX) OR (ZY AND ZX)) THEN  
IF ( (contador2 = 0) OR (contador2 = 2) ) THEN  
contadorCiclos1 := contadorCiclos1 + 1;  
contador1 := contador1 + 1; // total passos
```

```
// Bloco original: mode1 AND mode2  
IF (XY AND ZX) THEN  
IF SENTIDO1 = 1 THEN  
x_tmp := r1 * (1 - COS(rad1));  
z_tmp := r1 * SIN(rad1);  
X := XC1 - x_tmp;  
Z := ZC1 + z_tmp;  
x_tmp := X;  
X := x_tmp * COS(rad1);  
Y := x_tmp * SIN(rad1);  
ELSE  
x_tmp := r1 * (1 - COS(rad1));  
z_tmp := r1 * SIN(rad1);  
X := XC1 - x_tmp;  
Z := ZC1 - z_tmp;  
END_IF;  
x_tmp := X;  
X := x_tmp * COS(rad1);  
Y := x_tmp * SIN(rad1);  
END_IF;  
END_IF;
```

```
// Bloco original: mode1 AND mode3
```

```
IF (XY AND ZY) THEN
```

```
IF SENTIDO1 = 1 THEN
```

```
z_tmp := r1 * (1 - COS(rad1));
```

```
y_tmp := r1 * SIN(rad1);
```

```
Z := ZC1 - z_tmp;
```

```
Y := YC1 + y_tmp;
```

```
z_tmp := Z;
```

```
Z := z_tmp * COS(rad1);
```

```
X := z_tmp * SIN(rad1);
```

```
ELSE
```

```
z_tmp := r1 * (1 - COS(rad1));
```

```
y_tmp := r1 * SIN(rad1);
```

```
Z := ZC1 - z_tmp;
```

```
Y := YC1 - y_tmp;
```

```
z_tmp := Z;
```

```
Z := z_tmp * COS(rad1);
```

```
X := z_tmp * SIN(rad1);
```

```
END_IF; END_IF;
```

```
IF (contador2 = 1) AND r > 0 THEN
```

```
IF SENTIDO = 1 THEN
```

```
x_tmp := r * (1 - COS(rad));
```

```
y_tmp := r * SIN(rad);
```

```
X := XC - x_tmp;
```

```
Y := YC + y_tmp;
```

```
ELSE
```

```
x_tmp := r * (1 - COS(rad));
```

```
y_tmp := r * SIN(rad);
```

```
X := XC - x_tmp;
```

```
Y := YC - y_tmp;
```

```
END_IF;
```

```
contadorCiclos := contadorCiclos + 1 ;
```

```
contador := contador + 1; // totalpassos
```

```
contador2 := 2 ;
```

```
END_IF;
```

```
IF (contador2 = 3) THEN
```

```
IF SENTIDO = 1 THEN
```

```
x_tmp := r2 * (1 - COS(rad));
```

```
y_tmp := r2 * SIN(rad);
```

```
X := XC2 - x_tmp;
```

```
Y := YC2 + y_tmp;
```

```
ELSE
```

```
x_tmp := r2 * (1 - COS(rad));
```

```
y_tmp := r2 * SIN(rad);
```

```
X := XC2 - x_tmp;
```

```
Y := YC2 - y_tmp;
```

```
END_IF;
```

```
contadorCiclos := contadorCiclos + 1 ;
```

```
contador := contador + 1; // total passos
```

```
contador2 := 0;
```

```
END_IF;
```

```
END_IF;
```

```
IF Relative THEN
```



```
diffX_mm := sx * ABS(X - X_ant);  
X1 := diffX_mm;
```

```
diffY_mm := sy * ABS(Y - Y_ant);  
Y1 := diffY_mm;
```

```
diffZ_mm := sx * ABS(Z - Z_ant);  
Z1 := diffZ_mm;
```

```
ELSE
```

```
diffX_mm := ABS(X - X_ant);  
X1 := X;
```

```
diffY_mm := ABS(Y - Y_ant);  
Y1 := Y;
```

```
diffZ_mm := ABS(Z - Z_ant);  
Z1 := Z;
```

```
END_IF;  
END_IF;  
END_IF;
```

```
// Atualiza valores congelados nos ciclos iniciais  
IF (contadorCiclos >= 1) AND (contadorCiclos1 >= 1) THEN  
    X_ant := X; // posição segura de cálculo  
    Y_ant := Y; // posição de cálculo  
    Z_ant := Z; // posição de cálculo  
END_IF;
```

```
IF (contadorCiclos = totalPassosTotal) AND (ANGGFINAL <> 360.0) THEN
```

```
    SENTIDO := 1 - SENTIDO;  
    contadorCiclos := 0;  
    ang_i_tmp := ANGULOINICIAL;  
    ang_f_tmp := ANGULOFINAL;  
    ANGULOINICIAL := MOD(360.0 - ang_f_tmp, 360.0);  
    ANGULOFINAL := MOD(360.0 - ang_i_tmp, 360.0);  
    IF SENTIDO = 1 THEN  
        XC := X1C;  
        YC := Y1C;  
        ZC := Z1C;  
        XC2 := X1C2;  
        YC2 := Y1C2;  
        ZC2 := Z1C2;  
    ELSE  
        XC := X0C;  
        YC := Y0C;  
        ZC := Z0C;  
        XC2 := X0C2;  
        YC2 := Y0C2;  
        ZC2 := Z0C2;  
    END_IF;  
END_IF;
```

```
IF (contadorCiclos1 = totalPassos1) AND (r < rmax) THEN
```

```

SENTIDO1 := 1 - SENTIDO1;
contadorCiclos1 := 0;
  ang_i_tmp := ANGULOINICIAL1;
  ang_f_tmp := ANGULOFINAL1;
  ANGULOINICIAL1 := MOD(360.0 - ang_f_tmp, 360.0);
  ANGULOFINAL1 := MOD(360.0 - ang_i_tmp, 360.0);
IF SENTIDO1 = 1 THEN
  XC1 := X1C1;
  YC1 := Y1C1;
  ZC1 := Z1C1;
ELSE
  XC1 := X0C1;
  YC1 := Y0C1;
  ZC1 := Z0C1;
END_IF;
END_IF;

```

```

IF contadorCiclos1 = totalPassos1 THEN
contador2 := contador2 + 1 ;
contadorCiclos1 := 0 ;
END_IF ;

```

```

IF (contadorCiclos >= totalPassosTotal) AND
(
  (EXTERNO AND
    (ABS(r - rmax) <= 0.005) AND
    (ABS(r1 - r1max) <= 0.005) AND
    (ABS(r2 - r2max) <= 0.005))
  OR
  ((NOT EXTERNO) AND
    (ABS(r - rmax) <= 0.005) AND
    (ABS(r1 - r1max) <= 0.005) AND
    (ABS(r2 - r2max) <= 0.005)))
THEN
  NF := TRUE;
END_IF;

```

```

IF (contadorCiclos = totalPassosTotal) AND
  ((NOT RETURN_0) OR (SENTIDO = SENTIDO_ant)) THEN
IF EXTERNO THEN
IF r1 >= r1max THEN
r1 := r1 - passoR;
END_IF;
IF r1 < r1max THEN
  contadorCiclos := 0;
  r := 0.0;
  r1 := 0.0;
  r2 := 0.0;
END_IF;
ELSE
IF r1 <= r1max THEN
r1 := r1 + passoR;
END_IF;
IF r1 > r1max THEN
  contadorCiclos := 0;

```

```

r := 0.0;
r1 := 0.0;
r2 := 0.0;
END_IF;
END_IF;
IF SENTIDO1 = 1 THEN
r2 := r1 * ABS(COS(ANGULOINICIAL * 0.0174533));
r := r1 * ABS(COS(ANGULOFINAL * 0.0174533));
ELSE
r := r1 * ABS(COS(ANGULOINICIAL * 0.0174533));
r2 := r1 * ABS(COS(ANGULOFINAL * 0.0174533));
END_IF;
END_IF;

```

```

IF (contadorCiclos = totalPassosTotal) AND (r > 0) AND (r1 > 0) AND (r2 > 0) THEN

```

```

delta := passoR; // r1
IF EXTERNO THEN delta := -passoR; END_IF;

```

```

IF PXY THEN
IF Q1 AND Q2 THEN X1C1 := X1C1 + delta; Y0C1 := Y0C1 + delta;
ELSIF Q1 AND Q3 THEN X1C1 := X1C1 + delta; X0C1 := X0C1 - delta;
ELSIF Q1 AND Q4 THEN Y0C1 := Y0C1 - delta; X1C1 := X1C1 + delta;
ELSIF Q2 AND Q3 THEN Y1C1 := Y1C1 + delta; X0C1 := X0C1 - delta;
ELSIF Q2 AND Q4 THEN Y1C1 := Y1C1 + delta; Y0C1 := Y0C1 - delta;
ELSIF Q3 AND Q4 THEN X1C1 := X1C1 - delta; Y0C1 := Y0C1 - delta;
ELSIF Q1 AND NOT Q2 AND NOT Q3 AND NOT Q4 THEN
IF EXTERNO THEN X1C1 := X1C1 - delta; Y0C1 := Y0C1 - delta;
ELSE X1C1 := X1C1 + delta; Y0C1 := Y0C1 + delta;
END_IF;
END_IF;
END_IF;

```

```

IF PZY THEN
IF Q1 AND Q2 THEN Y1C1 := Y1C1 - delta; Z0C1 := Z0C1 + delta;
ELSIF Q1 AND Q3 THEN Y1C1 := Y1C1 - delta; Y0C1 := Y0C1 + delta;
ELSIF Q1 AND Q4 THEN Y1C1 := Y1C1 - delta; Z0C1 := Z0C1 - delta;
ELSIF Q2 AND Q3 THEN Z1C1 := Z1C1 + delta; Y0C1 := Y0C1 + delta;
ELSIF Q2 AND Q4 THEN Z1C1 := Z1C1 + delta; Z0C1 := Z0C1 - delta;
ELSIF Q3 AND Q4 THEN Y1C1 := Y1C1 + delta; Z0C1 := Z0C1 - delta;
ELSIF Q1 AND NOT Q2 AND NOT Q3 AND NOT Q4 THEN
IF EXTERNO THEN Y1C1 := Y1C1 + delta; Z0C1 := Z0C1 - delta;
ELSE Y1C1 := Y1C1 - delta; Z0C1 := Z0C1 + delta;
END_IF;
END_IF;
END_IF;

```

```

IF PZX THEN
IF Q1 AND Q2 THEN X1C1 := X1C1 + delta; Z0C1 := Z0C1 + delta;
ELSIF Q1 AND Q3 THEN X1C1 := X1C1 + delta; X0C1 := X0C1 - delta;
ELSIF Q1 AND Q4 THEN Z0C1 := Z0C1 - delta; X1C1 := X1C1 + delta;
ELSIF Q2 AND Q3 THEN Z1C1 := Z1C1 + delta; X0C1 := X0C1 - delta;
ELSIF Q2 AND Q4 THEN Z1C1 := Z1C1 + delta; Z0C1 := Z0C1 - delta;
ELSIF Q3 AND Q4 THEN X1C1 := X1C1 - delta; Z0C1 := Z0C1 - delta;
ELSIF Q1 AND NOT Q2 AND NOT Q3 AND NOT Q4 THEN
IF EXTERNO THEN X1C1 := X1C1 - delta; Z0C1 := Z0C1 - delta;
ELSE X1C1 := X1C1 + delta; Z0C1 := Z0C1 + delta;
END_IF;
END_IF;
END_IF;

```

```

deltaR := ABS(r - r_ant);
IF EXTERNO THEN deltaR := -deltaR; END_IF;

```

```

IF PXY THEN
IF Q1r AND Q2r THEN X1C := X1C + deltaR; Y0C := Y0C + deltaR;
ELSIF Q1r AND Q3r THEN X1C := X1C + deltaR; X0C := X0C - deltaR;

```

```

ELSIF Q1r AND Q4r THEN Y0C := Y0C - deltaR; X1C := X1C + deltaR;
ELSIF Q2r AND Q3r THEN Y1C := Y1C + deltaR; X0C := X0C - deltaR;
ELSIF Q2r AND Q4r THEN Y1C := Y1C + deltaR; Y0C := Y0C - deltaR;
ELSIF Q3r AND Q4r THEN X1C := X1C - deltaR; Y0C := Y0C - deltaR;
ELSIF Q1r AND NOT Q2r AND NOT Q3r AND NOT Q4r THEN
IF EXTERNO THEN X1C := X1C - deltaR; Y0C := Y0C - deltaR;
ELSE X1C := X1C + deltaR; Y0C := Y0C + deltaR;
END_IF;
END_IF;
END_IF;

```

```

IF PZY THEN
IF Q1r AND Q2r THEN Y1C := Y1C - deltaR; Z0C := Z0C + deltaR;
ELSIF Q1r AND Q3r THEN Y1C := Y1C - deltaR; Y0C := Y0C + deltaR;
ELSIF Q1r AND Q4r THEN Y1C := Y1C - deltaR; Z0C := Z0C - deltaR;
ELSIF Q2r AND Q3r THEN Z1C := Z1C + deltaR; Y0C := Y0C + deltaR;
ELSIF Q2r AND Q4r THEN Z1C := Z1C + deltaR; Z0C := Z0C - deltaR;
ELSIF Q3r AND Q4r THEN Y1C := Y1C + deltaR; Z0C := Z0C - deltaR;
ELSIF Q1r AND NOT Q2r AND NOT Q3r AND NOT Q4r THEN
IF EXTERNO THEN Y1C := Y1C + deltaR; Z0C := Z0C - deltaR;
ELSE Y1C := Y1C - deltaR; Z0C := Z0C + deltaR;
END_IF;
END_IF;
END_IF;

```

```

IF PZX THEN
IF Q1r AND Q2r THEN X1C := X1C + deltaR; Z0C := Z0C + deltaR;
ELSIF Q1r AND Q3r THEN X1C := X1C + deltaR; X0C := X0C - deltaR;
ELSIF Q1r AND Q4r THEN Z0C := Z0C - deltaR; X1C := X1C + deltaR;
ELSIF Q2r AND Q3r THEN Z1C := Z1C + deltaR; X0C := X0C - deltaR;
ELSIF Q2r AND Q4r THEN Z1C := Z1C + deltaR; Z0C := Z0C - deltaR;
ELSIF Q3r AND Q4r THEN X1C := X1C - deltaR; Z0C := Z0C - deltaR;
ELSIF Q1r AND NOT Q2r AND NOT Q3r AND NOT Q4r THEN
IF EXTERNO THEN X1C := X1C - deltaR; Z0C := Z0C - deltaR;
ELSE X1C := X1C + deltaR; Z0C := Z0C + deltaR;
END_IF;
END_IF;
END_IF;

```

```

deltaR2 := ABS(r2 - r2_ant);
IF EXTERNO THEN deltaR2 := -deltaR2; END_IF;

```

```

IF PXY THEN
IF Q1r2 AND Q2r2 THEN X1C2 := X1C2 + deltaR2; Y0C2 := Y0C2 + deltaR2;
ELSIF Q1r2 AND Q3r2 THEN X1C2 := X1C2 + deltaR2; X0C2 := X0C2 - deltaR2;
ELSIF Q1r2 AND Q4r2 THEN Y0C2 := Y0C2 - deltaR2; X1C2 := X1C2 + deltaR2;
ELSIF Q2r2 AND Q3r2 THEN Y1C2 := Y1C2 + deltaR2; X0C2 := X0C2 - deltaR2;
ELSIF Q2r2 AND Q4r2 THEN Y1C2 := Y1C2 + deltaR2; Y0C2 := Y0C2 - deltaR2;
ELSIF Q3r2 AND Q4r2 THEN X1C2 := X1C2 - deltaR2; Y0C2 := Y0C2 - deltaR2;
ELSIF Q1r2 AND NOT Q2r2 AND NOT Q3r2 AND NOT Q4r2 THEN
IF EXTERNO THEN X1C2 := X1C2 - deltaR2; Y0C2 := Y0C2 - deltaR2;
ELSE X1C2 := X1C2 + deltaR2; Y0C2 := Y0C2 + deltaR2;
END_IF;
END_IF;
END_IF;

```

```

IF PZY THEN
IF Q1r2 AND Q2r2 THEN Y1C2 := Y1C2 - deltaR2; Z0C2 := Z0C2 + deltaR2;
ELSIF Q1r2 AND Q3r2 THEN Y1C2 := Y1C2 - deltaR2; Y0C2 := Y0C2 + deltaR2;
ELSIF Q1r2 AND Q4r2 THEN Y1C2 := Y1C2 - deltaR2; Z0C2 := Z0C2 - deltaR2;
ELSIF Q2r2 AND Q3r2 THEN Z1C2 := Z1C2 + deltaR2; Y0C2 := Y0C2 + deltaR2;
ELSIF Q2r2 AND Q4r2 THEN Z1C2 := Z1C2 + deltaR2; Z0C2 := Z0C2 - deltaR2;
ELSIF Q3r2 AND Q4r2 THEN Y1C2 := Y1C2 + deltaR2; Z0C2 := Z0C2 - deltaR2;
ELSIF Q1r2 AND NOT Q2r2 AND NOT Q3r2 AND NOT Q4r2 THEN
IF EXTERNO THEN Y1C2 := Y1C2 + deltaR2; Z0C2 := Z0C2 - deltaR2;
ELSE Y1C2 := Y1C2 - deltaR2; Z0C2 := Z0C2 + deltaR2;
END_IF;
END_IF;
END_IF;

```

```

IF PZX THEN
IF Q1r2 AND Q2r2 THEN X1C2 := X1C2 + deltaR2; Z0C2 := Z0C2 + deltaR2;
ELSIF Q1r2 AND Q3r2 THEN X1C2 := X1C2 + deltaR2; X0C2 := X0C2 - deltaR2;
ELSIF Q1r2 AND Q4r2 THEN Z0C2 := Z0C2 - deltaR2; X1C2 := X1C2 + deltaR2;
ELSIF Q2r2 AND Q3r2 THEN Z1C2 := Z1C2 + deltaR2; X0C2 := X0C2 - deltaR2;
ELSIF Q2r2 AND Q4r2 THEN Z1C2 := Z1C2 + deltaR2; Z0C2 := Z0C2 - deltaR2;
ELSIF Q3r2 AND Q4r2 THEN X1C2 := X1C2 - deltaR2; Z0C2 := Z0C2 - deltaR2;
ELSIF Q1r2 AND NOT Q2r2 AND NOT Q3r2 AND NOT Q4r2 THEN
IF EXTERNO THEN X1C2 := X1C2 - deltaR2; Z0C2 := Z0C2 - deltaR2;
ELSE X1C2 := X1C2 + deltaR2; Z0C2 := Z0C2 + deltaR2;
END_IF;
END_IF;
END_IF;

```

```

r_ant := r;
r2_ant := r2;
END_IF;

```

```

IF (WR_PDO_6040_X AND HALT_BIT) = 0 THEN
    eventosParada[eventoldx].indexBuffer := indexBuffer;
    eventosParada[eventoldx].halt := (WR_PDO_6040_X AND HALT_BIT) = 0;
    eventoldx := eventoldx + 1;
    IF eventoldx > 19 THEN eventoldx := 0; END_IF;
END_IF;

```

```

bufferX[indexBuffer] := X1;
bufferY[indexBuffer] := Y1;
bufferZ[indexBuffer] := Z1;

```

```

targetDINT_X := ConvertRealDiffToTargetDINT(diffX_mm);
targetDINT_Y := ConvertRealDiffToTargetDINT(diffY_mm);
targetDINT_Z := ConvertRealDiffToTargetDINT(diffZ_mm);

```

```

bufferTargetX_H[indexBuffer] := INT(SHR(UDINT(targetDINT_X),16) AND 65535);
bufferTargetX_L[indexBuffer] := INT(UDINT(targetDINT_X) AND 65535);
bufferTargetY_H[indexBuffer] := INT(SHR(UDINT(targetDINT_Y),16) AND 65535);
bufferTargetY_L[indexBuffer] := INT(UDINT(targetDINT_Y) AND 65535);
bufferTargetZ_H[indexBuffer] := INT(SHR(UDINT(targetDINT_Z),16) AND 65535);
bufferTargetZ_L[indexBuffer] := INT(UDINT(targetDINT_Z) AND 65535);

```

```

VELOCIDADE_X_Y_Z(diffX_mm, diffY_mm, diffZ_mm, rpmX, rpmY, rpmZ);
    bufferVelX[indexBuffer] := rpmX;
    bufferVelY[indexBuffer] := rpmY;
    bufferVelZ[indexBuffer] := rpmZ;

```

```

indexBuffer := indexBuffer + 1;

```

```

END_WHILE;

```

```

// ☒ DONE único: finalização consciente da lista

```

```

IF DONE THEN
    bufferFilled := TRUE;

```

```

// Marca os índices finais dos eixos

```

```

axisX.lastIndex := indexBuffer - 1;
axisY.lastIndex := indexBuffer - 1;
axisZ.lastIndex := indexBuffer - 1;

// Marca eixos como prontos
axisX.ready := TRUE;
axisY.ready := TRUE;
axisZ.ready := TRUE;

// Zera índices ou ciclos se necessário
indexBuffer := 0;
totalCycles := 0;

END_IF;
END_FUNCTION;

```

@

18. FUNCTION INTERPOLACAO_CONICA

A função INTERPOLACAO_CONICA implementa uma interpolação geométrica de trajetória cônica com acoplamento entre avanço axial, variação radial e rotação angular, gerando pontos em buffers para execução sincronizada em múltiplos eixos.

Ela inicia com reset de estados, removendo referências de centro, raio, plano ativo, contadores e parâmetros de cone, garantindo uma condição inicial neutra. A partir disso, o sistema entra em um loop de geração enquanto houver espaço no buffer, enquanto um plano de operação estiver ativo (PXY, PZX ou PZY) e enquanto a profundidade atual ainda não atingir o perfil definido.

O núcleo do modelo calcula a discretização do cone a partir da altura total e da resolução longitudinal, derivando um passo axial (ΔS_1) e um incremento radial proporcional via tangente do ângulo do cone. Esse acoplamento define simultaneamente o avanço em profundidade e a expansão ou contração do raio, criando a geometria cônica progressiva.

A cada iteração, o sistema atualiza o avanço axial em função do plano ativo: no PXY altera Z, no PZX altera Y, no PZY altera X, mantendo coerência entre eixo de avanço e plano de interpolação. Em paralelo, calcula incremento radial e atualiza centros auxiliares (X0C, X1C, Y0C, Y1C, Z0C, Z1C), permitindo construção incremental da superfície.

A discretização angular é obtida por compatibilização entre ângulo total e resolução por quadrante, com busca de um número de passos inteiro que minimize erro de quantização angular. A partir disso são derivados ΔS , incremento angular e total de passos por arco, garantindo fechamento geométrico consistente.

O modelo converte o avanço angular em coordenadas circulares via seno e cosseno, com sinais definidos por quadrante e sentido de rotação. Cada plano (XY, ZX, ZY) projeta essas

componentes em pares distintos de eixos, permitindo que o mesmo modelo matemático gere diferentes superfícies conforme a configuração ativa.

Há controle de continuidade cinemática com inversão de sentido ao final de um ciclo completo ou quando limites geométricos são atingidos, recalculando ângulos iniciais e finais por simetria de 360°. Isso preserva continuidade da trajetória entre segmentos consecutivos do cone.

Os resultados são armazenados em buffers de posição e também convertidos para diferenças incrementais (diffX, diffY, diffZ), com suporte a modo relativo ou absoluto. Esses valores são posteriormente convertidos para formato DINT fracionado para comunicação com o sistema de acionamento, juntamente com velocidades calculadas por função dedicada baseada nos incrementos.

O encerramento ocorre quando a profundidade alvo e o raio final são atingidos dentro de tolerância, marcando a trajetória como completa, fixando índices finais dos buffers e sinalizando os eixos como prontos.

```
FuncoesOriginais[5]
```

```
:= 'INTERPOLACAO_CONICA';
```

```
FuncoesGCODE[5]
```

```
:= 'F5';
```

```
Variaveis[5]
```

```
:= 'PXY,PZX,PZY,SENTIDO,  
X0C,Y0C,Z0C,X1C,Y1C,Z1C,  
XC,YC,ZC,  
r,rmax,deltaX,hcone,angulocone,ANGULOINICIAL,ANGULOFINAL,LIM_POS,LIM_NEG';
```

```
TiposVariaveis[5]
```

```
:=  
BOOL,BOOL,BOOL,USINT,REAL,REAL,REAL,REAL,REAL,REAL,REAL,REAL,REAL,REAL,REAL,REAL,  
REAL,REAL,REAL,REAL  
';
```

```
FUNCTION INTERPOLACAO_CONICA
```

```
r0 : REAL ;
```

```
VAR
```

```
SENTIDO_ant : USINT ;
```

```
deltaS : REAL;
```

```
PI : REAL := 3.14159265;
```

```
ang_i_tmp : REAL;
```

```
ang_f_tmp : REAL;
```

```

PQ_min : UDINT;
PQ     : UDINT;
N_min  : UDINT;
theta  : REAL;
teste0 : REAL;
arred   : REAL;
erro   : REAL;
tolerancia : REAL;
x_tmp  : REAL;
y_tmp  : REAL;
z_tmp  : REAL;
sx     : SINT;
sy     : SINT;
deltaR : REAL
deltaS1 : REAL;
prof : REAL;

```

```

END_VAR

```

```

VAR_OUTPUT
// --- Diferenças de posição ---
diffX_mm, diffY_mm, diffZ_mm : REAL;
END_VAR

```

```

VAR_INPUT
// --- Velocidades ---
rpmX, rpmY, rpmZ : UDINT;
END_VAR

```

```

BEGIN
contadorCiclos := 0;
contador := 1;
SENTIDO_ant := SENTIDO;

```

```

IF Reset THEN
N := 0;
PartialStart0 := 0.0;
PartialEnd0 := 100.0;
PXY:=FALSE;
PZX:=FALSE;
PZY:=FALSE;
X0C:=0.0;Y0C:=0.0;Z0C:=0.0;
X1C:=0.0;Y1C:=0.0;Z1C:=0.0;
XC:=0.0;YC:=0.0;ZC:=0.0;
r:=0.0;
r0 := 0 ;
rmax:=0.0;
hcone:=0.0;
angulocone:=0.0;
END_IF;

```

```

WHILE
(indexBuffer < maxBuffer) AND
(PXY OR PZX OR PZY) AND
(ABS(hcone) > ABS(prof))
DO

```



```
IF contadorCiclos = 0 THEN
X_ant := X;
Y_ant := Y;
Z_ant := Z;
END_IF;
```

```
IF (contadorCiclos = 0) THEN
```

```
    N_min := CEIL(ABS(hcone) / deltaX);
```

```
    deltaS1 := hcone / REAL(N_min);
```

```
    deltaR := ABS(deltaS1) * TAN(angulocone / 2.0 * PI / 180.0);
```

```
IF (NOT RETURN_0) OR (SENTIDO = SENTIDO_ant) THEN
```

```
    r := r + deltaR;
    prof := prof + deltaS1;
```

```
    IF PXY THEN Z := Z + deltaS1; END_IF;
```

```
    IF PZX THEN Y := Y + deltaS1; END_IF;
```

```
    IF PZY THEN X := X + deltaS1; END_IF;
```

```
END_IF;
```

```
    rmax := r0 + hcone * TAN(angulocone / 2.0 * PI / 180.0);
```

```
END_IF;
```

```
// ----- PLANO XY -----
```

```
IF PXY THEN
```

```
    IF Q1 AND Q2 THEN X1C := X1C + deltaR; Y0C := Y0C + deltaR;
```

```
    ELSIF Q1 AND Q3 THEN X1C := X1C + deltaR; X0C := X0C - deltaR;
```

```
    ELSIF Q1 AND Q4 THEN Y0C := Y0C - deltaR; X1C := X1C + deltaR;
```

```
    ELSIF Q2 AND Q3 THEN Y1C := Y1C + deltaR; X0C := X0C - deltaR;
```

```
    ELSIF Q2 AND Q4 THEN Y1C := Y1C + deltaR; Y0C := Y0C - deltaR;
```

```
    ELSIF Q3 AND Q4 THEN X1C := X1C - deltaR; Y0C := Y0C - deltaR;
```

```
    ELSIF Q1 THEN X1C := X1C + deltaR;
```

```
    ELSIF Q2 THEN Y1C := Y1C + deltaR;
```

```
    ELSIF Q3 THEN X0C := X0C - deltaR;
```

```
    ELSIF Q4 THEN Y0C := Y0C - deltaR;
```

```
END_IF;
```

```
END_IF;
```

```
// ----- PLANO ZY -----
```

```
IF PZY THEN
```

```
    IF Q1 AND Q2 THEN Y1C := Y1C + deltaR; Z0C := Z0C + deltaR;
```

```
    ELSIF Q1 AND Q3 THEN Y1C := Y1C + deltaR; Y0C := Y0C - deltaR;
```

```
    ELSIF Q1 AND Q4 THEN Y1C := Y1C + deltaR; Z0C := Z0C - deltaR;
```

```
    ELSIF Q2 AND Q3 THEN Z1C := Z1C + deltaR; Y0C := Y0C - deltaR;
```

```
    ELSIF Q2 AND Q4 THEN Z1C := Z1C + deltaR; Z0C := Z0C - deltaR;
```

```
    ELSIF Q3 AND Q4 THEN Y1C := Y1C - deltaR; Z0C := Z0C - deltaR;
```

```
    ELSIF Q1 THEN Y1C := Y1C + deltaR;
```

```
    ELSIF Q2 THEN Z1C := Z1C + deltaR;
```

```
    ELSIF Q3 THEN Y0C := Y0C - deltaR;
```

```
    ELSIF Q4 THEN Z0C := Z0C - deltaR;
```

```

END_IF;
END_IF;
// ----- PLANO ZX -----
IF PZX THEN
  IF Q1 AND Q2 THEN X1C := X1C + deltaR; Z0C := Z0C + deltaR;
  ELSIF Q1 AND Q3 THEN X1C := X1C + deltaR; X0C := X0C - deltaR;
  ELSIF Q1 AND Q4 THEN Z0C := Z0C - deltaR; X1C := X1C + deltaR;
  ELSIF Q2 AND Q3 THEN Z1C := Z1C + deltaR; X0C := X0C - deltaR;
  ELSIF Q2 AND Q4 THEN Z1C := Z1C + deltaR; Z0C := Z0C - deltaR;
  ELSIF Q3 AND Q4 THEN X1C := X1C - deltaR; Z0C := Z0C - deltaR;
  ELSIF Q1 THEN X1C := X1C + deltaR;
  ELSIF Q2 THEN Z1C := Z1C + deltaR;
  ELSIF Q3 THEN X0C := X0C - deltaR;
  ELSIF Q4 THEN Z0C := Z0C - deltaR;
END_IF;
END_IF;

```

```

s := 3.14159265 * r / 2.0;
PQ_min := CEIL(s / deltaX);
theta := ANGULOFINAL - ANGULOINICIAL;
IF theta < 0 THEN theta := theta + 360;
passosQuadrante := 0;
deltaS := 0.0;
anguloInc := 0.0;
totalPassos := 0;
tolerancia := 3.0;

```

```

FOR PQ := PQ_min TO 2000 DO
  teste0 := (theta * REAL(PQ)) / 90.0;
  arred := REAL(ROUND(teste0));
  erro := ABS(teste0 - arred);
  IF (erro < tolerancia) AND (INT(arred) MOD 2 = 0) THEN
    passosQuadrante := PQ;
    deltaS := s / REAL(PQ);
    anguloInc := 90.0 / REAL(PQ);
    totalPassos := INT(arred);
  END_IF;
END_FOR;

```

```

ANGGFINAL := ANGULOFINAL - ANGULOINICIAL;
IF ANGGFINAL < 0 THEN ANGGFINAL := ANGGFINAL + 360;

```

```

//deltaS max 0.45mm // 3000 RPM // fuso 6mm// 1/2at²
// limites 1RPM

```

```

RPM := deltaS * 6666.7;

```

```

incremento_maximo := RPM * WR_PDO_6083_X * 0.00005;

```

```

ANG1 := (incremento_maximo / r) * 57.32 ;

```

```

IF ANGGFINAL <= 2 * ANG1 THEN
  ANG1 := ANGGFINAL * 0.5 ;
END_IF;

```

END_IF; // contadorciclo = 0

angulo := ANGULOINICIAL + contador * anguloInc;

ANGULOTOTAL := contador * anguloInc;

IF ANGULOTOTAL >= ANGGFINAL - ANG1 THEN

MAX_RPM := RPM - 30.0 * SQRT(RPM * (incremento_maximo - deltaS) / (0.045 * WR_PDO_6083_X));

END_IF;

IF ANGULOTOTAL >= ANGGFINAL THEN

ANGULOTOTAL := 0 ;

END_IF;

rad := angulo * 0.0174533;

IF Relative THEN

diffX_mm := 0;

diffY_mm := 0;

diffZ_mm := 0;

END_IF;

IF SENTIDO = 1 THEN

IF rad < PI/2 THEN

sx := -1; sy := +1;

ELSIF rad < PI THEN

sx := -1; sy := -1;

ELSIF rad < 3*PI/2 THEN

sx := +1; sy := -1;

ELSE

sx := +1; sy := +1;

END_IF;

ELSE

IF rad < PI/2 THEN

sx := -1; sy := -1;

ELSIF rad < PI THEN

sx := -1; sy := +1;

ELSIF rad < 3*PI/2 THEN

sx := +1; sy := +1;

ELSE

sx := +1; sy := -1;

END_IF;

END_IF;

(* ----- MODE 3 = 6 (ZY) ----- *)

IF PZY THEN

z_tmp := r * (1 - COS(rad));

y_tmp := r * SIN(rad);

IF SENTIDO = 1 THEN

Z := ZC - z_tmp; Y := YC + y_tmp;

ELSE

Z := ZC - z_tmp;

Y := YC - y_tmp;

END_IF;

```
contadorCiclos := contadorCiclos + 1 ;  
contador := contador + 1 ;  
END_IF;
```

```
IF Relative THEN  
diffX_mm := sx * ABS(X - X_ant);  
X1 := diffX_mm;
```

```
diffY_mm := sy * ABS(Y - Y_ant);  
Y1 := diffY_mm;
```

```
diffZ_mm := sx * ABS(Z - Z_ant);  
Z1 := diffZ_mm;
```

```
ELSE
```

```
diffX_mm := ABS(X - X_ant);  
X1 := X;
```

```
diffY_mm := ABS(Y - Y_ant);  
Y1 := Y;
```

```
diffZ_mm := ABS(Z - Z_ant);  
Z1 := Z;  
END_IF;  
END_IF;
```

```
(* ----- MODE2 = 5 (ZX) ----- *)
```

```
IF PZX THEN  
x_tmp := r * (1 - COS(rad));  
z_tmp := r * SIN(rad);  
IF SENTIDO = 1 THEN  
X := XC - x_tmp; Z := ZC + z_tmp;  
ELSE  
X := XC + x_tmp;  
Z := ZC - z_tmp;  
END_IF  
;contadorCiclos := contadorCiclos + 1 ;  
contador := contador + 1; // totalpassos  
END_IF;
```

```
IF Relative THEN  
diffX_mm := sx * ABS(X - X_ant);  
X1 := diffX_mm;
```

```
diffY_mm := sy * ABS(Y - Y_ant);  
Y1 := diffY_mm;
```

```
diffZ_mm := sy * ABS(Z - Z_ant);  
Z1 := diffZ_mm;
```

```
ELSE
```

```
diffX_mm := ABS(X - X_ant);  
X1 := X;
```

```
diffY_mm := ABS(Y - Y_ant);  
Y1 := Y;
```

```
diffZ_mm := ABS(Z - Z_ant);  
Z1 := Z;  
END_IF;  
    END_IF;
```

```
(* ----- MODE1 4 (XY) ----- *)  
IF PXY THEN  
    x_tmp := r * (1 - COS(rad));  
    y_tmp := r * SIN(rad);  
    IF SENTIDO = 1 THEN  
        X := XC - x_tmp; Y := YC + y_tmp;  
    ELSE  
        X := XC - x_tmp;  
        Y := YC - y_tmp;  
    END_IF;  
    contadorCiclos := contadorCiclos + 1 ;  
    contador := contador + 1; // totalpassos  
END_IF;
```

```
    IF Relative THEN  
        diffX_mm := sx * ABS(X - X_ant);  
        X1 := diffX_mm;
```

```
        diffY_mm := sy * ABS(Y - Y_ant);  
        Y1 := diffY_mm;
```

```
        diffZ_mm := sx * ABS(Z - Z_ant);  
        Z1 := diffZ_mm;
```

```
    ELSE
```

```
        diffX_mm := ABS(X - X_ant);  
        X1 := X;
```

```
        diffY_mm := ABS(Y - Y_ant);  
        Y1 := Y;
```

```
        diffZ_mm := ABS(Z - Z_ant);  
        Z1 := Z;
```

```
    END_IF;  
    END_IF;
```

```
IF (contadorCiclos > 0) THEN  
    X_ant := X;  
    Y_ant := Y;  
    Z_ant := Z;  
END_IF;
```

```
IF (contadorCiclos = totalPassos)  
    AND (r < rmax)
```

AND (ANGGFINAL <> 360.0) THEN

SENTIDO := 1 - SENTIDO ;
contadorCiclos := 0;

ang_i_tmp := ANGULOINICIAL;
ang_f_tmp := ANGULOFINAL;

ANGULOINICIAL := MOD(360.0 - ang_f_tmp, 360.0);
ANGULOFINAL := MOD(360.0 - ang_i_tmp, 360.0);

IF SENTIDO = 1 THEN

XC := X1C;
YC := Y1C;
ZC := Z1C;

ELSE

XC := X0C;
YC := Y0C;
ZC := Z0C;

END_IF;

END_IF;

IF (WR_PDO_6040_X AND HALT_BIT) = 0 THEN

eventosParada[eventoldx].indexBuffer := indexBuffer;
eventosParada[eventoldx].halt := (WR_PDO_6040_X AND HALT_BIT) = 0;
eventoldx := eventoldx + 1;
IF eventoldx > 19 THEN eventoldx := 0; END_IF;
END_IF;

bufferX[indexBuffer] := X1;
bufferY[indexBuffer] := Y1;
bufferZ[indexBuffer] := Z1;

targetDINT_X := ConvertRealDiffToTargetDINT(diffX_mm);
targetDINT_Y := ConvertRealDiffToTargetDINT(diffY_mm);
targetDINT_Z := ConvertRealDiffToTargetDINT(diffZ_mm);

bufferTargetX_H[indexBuffer] := INT(SHR(UDINT(targetDINT_X),16) AND 65535);
bufferTargetX_L[indexBuffer] := INT(UDINT(targetDINT_X) AND 65535);
bufferTargetY_H[indexBuffer] := INT(SHR(UDINT(targetDINT_Y),16) AND 65535);
bufferTargetY_L[indexBuffer] := INT(UDINT(targetDINT_Y) AND 65535);
bufferTargetZ_H[indexBuffer] := INT(SHR(UDINT(targetDINT_Z),16) AND 65535);
bufferTargetZ_L[indexBuffer] := INT(UDINT(targetDINT_Z) AND 65535);

VELOCIDADE_X_Y_Z(diffX_mm, diffY_mm, diffZ_mm, rpmX, rpmY, rpmZ);
bufferVelX[indexBuffer] := rpmX;
bufferVelY[indexBuffer] := rpmY;
bufferVelZ[indexBuffer] := rpmZ;

IF

(contadorCiclos >= totalPassosTotal) AND
(indexBuffer < maxBuffer) AND
(ABS(prof - hcône) <= 0.005) AND
(ABS(r - rmax) <= 0.005)

THEN

```

    NF := TRUE;
END_IF;

indexBuffer := indexBuffer + 1;

IF contadorCiclos = totalPassos THEN
    contadorCiclos := 0 ;
    contador := 1;
END_IF;
END_WHILE;

// ☒ DONE único: finalização consciente da lista

IF DONE THEN
    bufferFilled := TRUE;

    // Marca os índices finais dos eixos
    axisX.lastIndex := indexBuffer - 1;
    axisY.lastIndex := indexBuffer - 1;
    axisZ.lastIndex := indexBuffer - 1;

    // Marca eixos como prontos
    axisX.ready := TRUE;
    axisY.ready := TRUE;
    axisZ.ready := TRUE;

    // Zera índices ou ciclos se necessário
    indexBuffer := 0;
    totalCycles := 0;

END_IF;
END_FUNCTION ;

```

18A.. FUNCTION_REVOLUTION_POINTS

A função REVOLUTION_POINTS é o núcleo responsável pela geração completa da trajetória de usinagem. Ela recebe um conjunto de pontos cartesianos, aplica transformações geométricas e tecnológicas, converte o resultado em comandos para servoacionamentos e monta integralmente os buffers de movimento que serão executados pelos eixos.

As principais responsabilidades da função são:

Receber a trajetória cartesiana de entrada através dos buffers X, Y e Z.

Identificar blocos independentes de geometria utilizando o valor sentinela 7777.

Rediscretizar automaticamente cada segmento linear, subdividindo trechos longos conforme o fator máximo permitido, produzindo uma distribuição uniforme de pontos.

Calcular múltiplas passagens de escala entre os fatores inicial e final, permitindo contração ou expansão progressiva da geometria.

Executar escalonamento global ou por segmento, mantendo cada bloco referenciado por sua própria origem quando necessário.

Atualizar automaticamente referências de anti-colisão associadas ao término de cada bloco.
 Determinar a quantidade de passos necessária para revoluções e movimentos angulares a partir do raio, resolução linear e intervalo angular solicitado.
 Gerar movimentos circulares nos planos XY, ZX ou ZY.
 Incorporar deslocamento helicoidal sincronizado ao eixo perpendicular ao plano de revolução.
 Controlar sentidos horário e anti-horário da interpolação.
 Executar percursos alternados (ida e retorno) durante operações de desbaste, reduzindo deslocamentos improdutivos.
 Gerenciar inversão automática da trajetória quando configurado o retorno pelo caminho inverso.
 Calcular posições absolutas e diferenciais de cada eixo conforme o modo de operação configurado.
 Converter deslocamentos em posições de destino compatíveis com os servoacionamentos.
 Gerar palavras alta e baixa (High Word/Low Word) dos registradores de posição.
 Calcular automaticamente a velocidade sincronizada de cada eixo para todos os pontos produzidos.
 Preencher integralmente os buffers de posição, velocidade e destino dos três eixos.
 Registrar eventos relacionados ao estado de parada (HALT) durante a geração do buffer.
 Gerenciar índices de escrita, ciclos de interpolação e estados internos da trajetória.
 Inicializar e atualizar buffers auxiliares utilizados para percursos direto e reverso.
 Restaurar parâmetros de operação quando solicitado através da condição de reset.
 Finalizar a preparação do buffer indicando aos eixos o último índice válido e sinalizando que a trajetória está pronta para execução.
 A função concentra, em um único ciclo de preparação, toda a lógica de transformação geométrica, discretização, interpolação, escalonamento, cálculo cinemático e geração dos buffers utilizados posteriormente pelo sistema de controle de movimento em tempo real.

```
FuncoesOriginais[6] := 'REVOLUTION_POINTS';
```

```
FuncoesGCODE[6] := 'F6';
```

```
Variaveis[6] := 'PXY , PZY , PZX , SENTIDO , segmentos, ANGULOINICIAL, ANGULOFINAL ,
XC,YC,ZC,r,';
```

```
TiposVariaveis[6] := 'BOOL,BOOL,BOOL,USINT,USINT,REAL,REAL,REAL,REAL,REAL,REAL';
```

```
blockIdx : UINT := 0;
baseOffsetX : REAL ;
SCALE_BASE_X : REAL;
baseOffsetY : REAL;
SCALE_BASE_Y : REAL;
baseOffsetZ : REAL;
SCALE_BASE_Z : REAL;
```

```
DESBASTE : BOOL := FALSE;
FATOR_MAX : REAL := 1;
SCALE : BOOL := FALSE;
// SCALE_FACTOR : REAL := 1.0;
SCALE_INICIAL : REAL ;
SCALE_FINAL : REAL;
SCALE_BY_SEGMENT : BOOL := FALSE;
```

FUNCTION REVOLUTION_POINTS

```
VAR_INPUT // redundante
  BBufferX : ARRAY[1..100] OF REAL;
```



```
    BBufferY : ARRAY[1..100] OF REAL;  
    BBufferZ : ARRAY[1..100] OF REAL;  
    segmentos : INT;  
END_VAR
```

```
VAR  
ScaleX : ARRAY[1..100] OF REAL;  
ScaleY : ARRAY[1..100] OF REAL;  
ScaleZ : ARRAY[1..100] OF REAL;  
scalePass : INT := 1;  
scaleAtual : REAL := 1.0;  
totalScalePasses : INT := 1;
```

```
RefX : REAL;  
RefY : REAL;  
RefZ : REAL;
```

```
i : INT;  
k : INT;  
idx : INT;  
div : INT;  
PASSO : REAL;  
Lmax : REAL;  
L : ARRAY[1..100] OF REAL;  
dx : REAL;  
dy : REAL;  
dz : REAL;  
incX : REAL;  
incY : REAL;  
incZ : REAL;
```

```
    i : INT;  
    invert : BOOL := FALSE;  
PI : REAL := 3.141592;  
y_tmp : REAL;  
z_tmp : REAL;
```

```
PQ_min : UDINT;  
PQ : UDINT;  
deltaS : REAL;  
theta : REAL;  
teste0 : REAL;  
arred : REAL;  
erro : REAL;  
tolerancia : REAL;
```

```
rad : REAL;  
angulo : REAL;
```

```
sx : SINT;  
sy : SINT;
```

```
BufferX_F : ARRAY[1..100] OF REAL;  
BufferY_F : ARRAY[1..100] OF REAL;  
BufferZ_F : ARRAY[1..100] OF REAL;
```

```
BufferX_R : ARRAY[1..100] OF REAL;
```

```

BufferY_R : ARRAY[1..100] OF REAL;
BufferZ_R : ARRAY[1..100] OF REAL;

END_VAR;

BEGIN

totalScalePasses :=
    REAL_TO_INT(
        ABS(SCALE_FINAL - SCALE_INICIAL) / SCALE_RESOLUTION
    ) + 1;

idx:=1;
blocoIni:=1;
RefX:=ScaleX[1];
RefY:=ScaleY[1];
RefZ:=ScaleZ[1];
WHILE blocoIni<=segmentos DO
blocoFim:=blocoIni;
WHILE(blocoFim<=segmentos)AND(ScaleX[blocoFim]<>7777)AND(ScaleY[blocoFim]<>7777)AND(ScaleZ[blocoFim]<>7777)DO
blocoFim:=blocoFim+1;
END_WHILE;
nPontos:=blocoFim-blocoIni;
IF nPontos>0 THEN
Lmax:=0.0;
FOR i:=blocoIni TO blocoFim-2 DO
dx:=ScaleX[i+1]-ScaleX[i];
dy:=ScaleY[i+1]-ScaleY[i];
dz:=ScaleZ[i+1]-ScaleZ[i];
L[i]:=SQRT(dx*dx+dy*dy+dz*dz);
IF L[i]>Lmax THEN
Lmax:=L[i];
END_IF;
END_FOR;
IF FATOR_MAX<=0 THEN
FATOR_MAX:=1;
END_IF;
PASSO:=Lmax/FATOR_MAX;
IF PASSO<=0 THEN
PASSO:=1;
END_IF;
BufferX_F[idx]:=ScaleX[blocoIni]-RefX;
BufferY_F[idx]:=ScaleY[blocoIni]-RefY;
BufferZ_F[idx]:=ScaleZ[blocoIni]-RefZ;
idx:=idx+1;
FOR i:=blocoIni TO blocoFim-2 DO
dx:=ScaleX[i+1]-ScaleX[i];
dy:=ScaleY[i+1]-ScaleY[i];
dz:=ScaleZ[i+1]-ScaleZ[i];
div:=REAL_TO_INT(CEIL(L[i]/PASSO));
IF div<1 THEN
div:=1;
END_IF;
incX:=dx/div;
incY:=dy/div;
incZ:=dz/div;

```

```

FOR k:=1 TO div DO
BufferX_F[idx]:=(ScaleX[i]+incX*k)-RefX;
BufferY_F[idx]:=(ScaleY[i]+incY*k)-RefY;
BufferZ_F[idx]:=(ScaleZ[i]+incZ*k)-RefZ;
idx:=idx+1;
END_FOR;
END_FOR;
END_IF;
IF blocoFim<=segmentos THEN
BufferX_F[idx]:=7777;
BufferY_F[idx]:=7777;
BufferZ_F[idx]:=7777;
idx:=idx+1;
END_IF;
blocoIni:=blocoFim+1;
END_WHILE;
segmentos:=idx-1;

```

```

segmentos := 0;
FOR i := 1 TO 100 DO
  IF (BBufferX[i] = 7777) OR
    (BBufferY[i] = 7777) OR
    (BBufferZ[i] = 7777) THEN
    (* apenas ignora o separador *)
  ELSE
    segmentos := segmentos + 1;
  END_IF;

```

```

END_FOR;

```

```

IF Reset THEN
N := 0;
PartialStart1 := 0.0;
PartialEnd1  := 0.0;

PartialStart2 := 0.0;
PartialEnd2  := 0.0;

PartialStart3 := 0.0;
PartialEnd3  := 0.0;

PartialStart4 := 0.0;
PartialEnd4  := 0.0;

PartialStart5 := 0.0;
PartialEnd5  := 0.0;
LimitStart1 := 0.0;
LimitEnd1  := 0.0;

LimitStart2 := 0.0;
LimitEnd2  := 0.0;

LimitStart3 := 0.0;
LimitEnd3  := 0.0;

```

```
LimitStart4 := 0.0;  
LimitEnd4  := 0.0;
```

```
LimitStart5 := 0.0;  
LimitEnd5  := 0.0;
```

```
PXY := FALSE;  
PZY := FALSE;  
PZX := FALSE;  
segmentos := 0;  
r := 0.0;  
END_IF;
```

```
WHILE (indexBuffer < maxBuffer) AND  
(totalPassos >= contadorCiclos) DO
```

```
IF contadorCiclos = 0 THEN  
(* atualizar antigos *)  
X_ant := X;  
Y_ant := Y;  
Z_ant := Z;  
contador := 0;  
END_IF;
```

```
IF contadorCiclos = 0 THEN
```

```
IF SCALE THEN
```

```
IF SCALE_FACTOR <= 0.0 THEN  
SCALE_FACTOR := 1.0;  
END_IF;
```

```
blockIdx := 0;  
baseOffsetX := SCALE_BASE_X;  
baseOffsetY := SCALE_BASE_Y;  
baseOffsetZ := SCALE_BASE_Z;
```

```
i := 1;
```

```
WHILE i <= segmentos DO
```

```
(* ----- SENTINELA ----- *)  
IF (BBufferX[i] = 7777) OR  
(BBufferY[i] = 7777) OR  
(BBufferZ[i] = 7777) THEN
```

```
IF ANTI_COLISAO THEN
```

```
IF scalePass = totalScalePasses THEN  
BufferN[IndexBloco].indexAntiColisao := i - 1;  
END_IF;
```

```
END_IF;
```

```
IF SCALE_BY_SEGMENT THEN  
scalePass := scalePass + 1;
```

```
IF scalePass > totalScalePasses THEN
```

```

scalePass := 1;
contadorCiclos := contadorCiclos + 1;
END_IF;
END_IF;

```

```

i := i + 1; (* entra no primeiro ponto do bloco *)

```

```

blockIdx := blockIdx + 1;

```

```

(* âncora = origem do bloco *)
baseOffsetX := SCALE_BASE_X + BBufferX[i];
baseOffsetY := SCALE_BASE_Y + BBufferY[i];
baseOffsetZ := SCALE_BASE_Z + BBufferZ[i];
END_IF;
(* ----- PONTO NORMAL ----- *)

```

```

t := REAL(scalePass - 1) /
    REAL(totalScalePasses - 1);
scaleAtual :=
    SCALE_INICIAL +
    (SCALE_FINAL - SCALE_INICIAL) * t;

```

```

IF scaleAtual > 1.0 THEN
    scaleAtual := 1.0;
END_IF;

```

```

ScaleX[i] := (BBufferX[i] - baseOffsetX) * scaleAtual + baseOffsetX;
ScaleY[i] := (BBufferY[i] - baseOffsetY) * scaleAtual + baseOffsetY;
ScaleZ[i] := (BBufferZ[i] - baseOffsetZ) * scaleAtual + baseOffsetZ;

```

```

i := i + 1;

```

```

END_WHILE;
ELSE

```

```

    FOR i := 1 TO segmentos DO
        IF (BBufferX[i] = 7777) OR
            (BBufferY[i] = 7777) OR
            (BBufferZ[i] = 7777) THEN

```

```

                IF ANTI_COLISAO THEN
                    BufferN[IndexBloco].indexAntiColisao := i - 1;
                END_IF;

```

```

            ELSE
                ScaleX[i] := BBufferX[i];
                ScaleY[i] := BBufferY[i];
                ScaleZ[i] := BBufferZ[i];

```

```

            END_IF;

```

```

        END_FOR;
    END_IF;

```

```

s := 3.14159265 * r / 2.0;

```

```

PQ_min := CEIL(s / deltaX);
theta := ANGULOFINAL - ANGULOINICIAL;
IF theta < 0 THEN theta := theta + 360;
passosQuadrante := 0;

```

```
deltaS := 0.0;
anguloInc := 0.0;
totalPassos := 0;
tolerancia := 3.0;
```

```
FOR PQ := PQ_min TO 2000 DO
teste0 := (theta * REAL(PQ)) / 90.0;
arred := REAL(ROUND(teste0));
erro := ABS(teste0 - arred);
  IF (erro < tolerancia) AND (INT(arred) MOD 2 = 0) THEN
    passosQuadrante := PQ;
    deltaS := s / REAL(PQ);
    anguloInc := 90.0 / REAL(PQ);
    totalPassos := INT(arred);
    EXIT;
  END_IF;
END_FOR;
```

```
IF PXY THEN // XXXXX
IF PHELICAL THEN Z := (PH / (2 * PI)) * (anguloInc * PI / 180);
RelativeZ := TRUE ;
END_IF;
```

```
IF PZX THEN // ZX XXXX
IF PHELICAL THEN Y := (PH / (2 * PI)) * (anguloInc * PI / 180);
RelativeY := TRUE ;
END_IF;
```

```
IF PZY THEN // ZY YYYY
IF PHELICAL THEN X := (PH / (2 * PI)) * (anguloInc * PI / 180);
RelativeX := TRUE ;
END_IF;
```

```
END_IF ; // CICLO 0 FECHADO
```

```
angulo := ANGULOINICIAL + contadorCiclos * anguloInc ;
rad := angulo * 3.14159265 / 180.0;
```

```
IF (contadorCiclos = 0) AND (NOT initBuffers) THEN
  FOR i := 1 TO segmentos DO
    BufferX_R[i] := BufferX_F[segmentos - i + 1];
    BufferY_R[i] := BufferY_F[segmentos - i + 1];
    BufferZ_R[i] := BufferZ_F[segmentos - i + 1];
  END_FOR;
  initBuffers := TRUE;
END_IF;
```

```
(* LEITURA *)
```

```
IF DESBASTE THEN
  XX := BufferX_F[contador + 1];
  YY := BufferY_F[contador + 1];
  ZZ := BufferZ_F[contador + 1];
```

```

ELSE
  IF invert THEN
    XX := - BufferX_R[contador + 1];
    YY := - BufferY_R[contador + 1];
    ZZ := - BufferZ_R[contador + 1];
  ELSE
    XX := BufferX_F[contador + 1];
    YY := BufferY_F[contador + 1];
    ZZ := BufferZ_F[contador + 1];
  END_IF;
END_IF;
(* CONTROLE *)
IF contador >= segmentos THEN
  contador := 0;
  IF NOT DESBASTE THEN
    invert := NOT invert;
  END_IF;
  IF NOT (RETURN_0 AND invert) THEN
    IF SCALE_BY_SEGMENT THEN
      contadorCiclos := contadorCiclos + 1;
    ELSE
      IF SCALE THEN
        scalePass := scalePass + 1;
        IF scalePass > totalScalePasses THEN
          scalePass := 1;
          contadorCiclos := contadorCiclos + 1;
        END_IF;
      ELSE
        contadorCiclos := contadorCiclos + 1;
      END_IF;
    END_IF;
  END_IF;
ELSE
  contador := contador + 1;
END_IF;

(* ----- MODE = 6 (ZY) ----- *)
IF PZY THEN // ZY YYYy_
  // r := r + YY ;
  X := XX + X ;
  y_tmp := (r + YY) * (1 - COS(rad)); // inv
  z_tmp := (r + YY) * SIN(rad); // inv
  IF SENTIDO = 1 THEN
    Y := YC + YY ; // Y absoluto
    Z := ZC + z_tmp; // inv
    Y := YC - y_tmp; // inv
  ELSE
    Y := YC + YY ;
    Z := ZC - z_tmp;
    Y := YC - y_tmp;
  END_IF; END_IF;

(* ----- MODE = 5 (XZ) ----- *)
IF PZX THEN // ZX XXXX
  // r := r + XX ;
  Y := Y + YY ;
  x_tmp := (r + XX) * (1 - COS(rad));
  z_tmp := (r + XX) * SIN(rad);
  IF SENTIDO = 1 THEN
    XC := XC + XX ;
    X := XC - x_tmp;

```

```

Z := ZC + z_tmp;
ELSE
XC := XC + XX;
X := XC - x_tmp;
Z := ZC - z_tmp;
END_IF; END_IF;

```

```

(* ----- MODE = 4 (XY) -----
IF PXY THEN //XXXXX
Z := Z + ZZ ;
// r := r + XX ;
x_tmp := ( r + XX)* ( 1 - COS(rad));
y_tmp := (r + XX ) * SIN(rad);
IF SENTIDO = 1 THEN
XC := XC + XX ;
X := XC - x_tmp;
Y := YC + y_tmp;
ELSE
XC := XC + XX ;
X := XC - x_tmp;
Y := YC - y_tmp;
END_IF; END_IF;

```

```

(* ----- DIFERENCIAIS ----- *)

```

```

IF Relative THEN
diffX_mm := 0;
diffY_mm := 0;
diffZ_mm := 0;
END_IF;

```

```

IF SENTIDO = 1 THEN
IF rad < PI/2 THEN
sx := -1; sy := +1;
ELSIF rad < PI THEN
sx := -1; sy := -1;
ELSIF rad < 3*PI/2 THEN
sx := +1; sy := -1;
ELSE
sx := +1; sy := +1;
END_IF;
ELSE
IF rad < PI/2 THEN
sx := -1; sy := -1;
ELSIF rad < PI THEN
sx := -1; sy := +1;
ELSIF rad < 3*PI/2 THEN
sx := +1; sy := +1;
ELSE
sx := +1; sy := -1;
END_IF;
END_IF;

```

```

IF RelativeX THEN
diffX_mm := sx * ABS(X - X_ant);

```



```

    X1 := diffX_mm;
ELSE
    diffX_mm := X // absoluto puro
    X1 := X;
END_IF;

```

```

(* Y *)
IF RelativeY THEN
    diffY_mm := sy * ABS(Y - Y_ant);
    Y1 := diffY_mm;
ELSE
    diffY_mm := Y;
    Y1 := Y;
END_IF;

```

```

(* Z *)
IF RelativeZ THEN
    diffZ_mm := sy * ABS(Z - Z_ant);
    Z1 := diffZ_mm;
ELSE
    diffZ_mm := Z;
    Z1 := Z;
END_IF;

```

```

// --- Buffers ---
IF (WR_PDO_6040_X AND HALT_BIT) = 0 THEN
    eventosParada[eventoldx].indexBuffer := indexBuffer;
    eventosParada[eventoldx].halt := (WR_PDO_6040_X AND HALT_BIT) = 0;
    eventoldx := eventoldx + 1;
    IF eventoldx > 19 THEN eventoldx := 0; END_IF;
END_IF;

```

```

bufferX[indexBuffer] := X1;
bufferY[indexBuffer] := Y1;
bufferZ[indexBuffer] := Z1;

```

```

targetDINT_X := ConvertRealDiffToTargetDINT(diffX_mm);
targetDINT_Y := ConvertRealDiffToTargetDINT(diffY_mm);
targetDINT_Z := ConvertRealDiffToTargetDINT(diffZ_mm);

```

```

bufferTargetX_H[indexBuffer] := INT(SHR(UDINT(targetDINT_X),16) AND 65535);
bufferTargetX_L[indexBuffer] := INT(UDINT(targetDINT_X) AND 65535);
bufferTargetY_H[indexBuffer] := INT(SHR(UDINT(targetDINT_Y),16) AND 65535);
bufferTargetY_L[indexBuffer] := INT(UDINT(targetDINT_Y) AND 65535);
bufferTargetZ_H[indexBuffer] := INT(SHR(UDINT(targetDINT_Z),16) AND 65535);
bufferTargetZ_L[indexBuffer] := INT(UDINT(targetDINT_Z) AND 65535);

```

```

VELOCIDADE_X_Y_Z(diffX_mm, diffY_mm, diffZ_mm, rpmX, rpmY, rpmZ);
bufferVelX[indexBuffer] := rpmX;
bufferVelY[indexBuffer] := rpmY;
bufferVelZ[indexBuffer] := rpmZ;

```

```

indexBuffer := indexBuffer + 1;

```

```

IF (PZY AND PHELICAL) AND contador >= segmentos THEN
    diffX_mm := X;

```

```

X1 := diffX_mm ;
END_IF;

IF (PZX AND PHELICAL) AND contador >= segmentos THEN
diffY_mm := Y;
Y1 := diffY_mm ;
END_IF;

IF (PXY AND PHELICAL) AND contador >= segmentos THEN
diffZ_mm := Z;
Z1 := diffZ_mm;
END_IF;

bufferX[indexBuffer] := X1;
bufferY[indexBuffer] := Y1;
bufferZ[indexBuffer] := Z1;

targetDINT_X := ConvertRealDiffToTargetDINT(diffX_mm);
targetDINT_Y := ConvertRealDiffToTargetDINT(diffY_mm);
targetDINT_Z := ConvertRealDiffToTargetDINT(diffZ_mm);

bufferTargetX_H[indexBuffer] := INT(SHR(UDINT(targetDINT_X),16) AND 65535);
bufferTargetX_L[indexBuffer] := INT(UDINT(targetDINT_X) AND 65535);
bufferTargetY_H[indexBuffer] := INT(SHR(UDINT(targetDINT_Y),16) AND 65535);
bufferTargetY_L[indexBuffer] := INT(UDINT(targetDINT_Y) AND 65535);
bufferTargetZ_H[indexBuffer] := INT(SHR(UDINT(targetDINT_Z),16) AND 65535);
bufferTargetZ_L[indexBuffer] := INT(UDINT(targetDINT_Z) AND 65535);

VELOCIDADE_X_Y_Z(diffX_mm, diffY_mm, diffZ_mm, rpmX, rpmY, rpmZ);
bufferVelX[indexBuffer] := rpmX;
bufferVelY[indexBuffer] := rpmY;
bufferVelZ[indexBuffer] := rpmZ;

IF (indexBuffer >= maxBuffer) OR (contadorCiclos >= totalPassos) THEN
NF := TRUE;
END_IF;
indexBuffer := indexBuffer + 1;

IF contadorCiclos > 0 THEN
(* atualizar antigos *)
X_ant := X;
Y_ant := Y;
Z_ant := Z;
END_IF;

END_WHILE;

contadorCiclos := 0;
contador := 0;

IF DONE THEN
bufferFilled := TRUE;

// Marca os índices finais dos eixos
axisX.lastIndex := indexBuffer - 1;
axisY.lastIndex := indexBuffer - 1;

```

```

axisZ.lastIndex := indexBuffer - 1;

// Marca eixos como prontos
axisX.ready := TRUE;
axisY.ready := TRUE;
axisZ.ready := TRUE;

// Zera índices ou ciclos se necessário
indexBuffer := 0;
totalCycles := 0;

END_IF;
END_FUNCTION ;

```

@

(*

RPMReference: calcula a velocidade de referência com base no maior deslocamento (X, Y ou Z), escala para RPM e limita ao máximo permitido.

FUNCTION RPMReference

A função RPMReference converte a leitura analógica do potenciômetro em uma referência de velocidade para todo o sistema de movimentação. A entrada proveniente do conversor A/D (0 a 32767) é escalonada linearmente para a faixa de rotação permitida pelo equipamento, definida por MAX_RPM, produzindo a variável referenceRPM.

Além da referência absoluta em RPM, a função gera o fator normalizado POT, variando de 0,0 a 1,0, utilizado por outras rotinas como multiplicador de velocidade. Esse fator representa diretamente a posição relativa do potenciômetro, independentemente do valor máximo de rotação configurado.

Como medida de segurança, POT é limitado ao valor máximo de 1,0, impedindo que leituras acima da faixa nominal provoquem referências superiores ao limite permitido.

A função centraliza o tratamento da referência de velocidade do operador, mantendo um único ponto de escalonamento para todo o sistema de controle.

19. FUNCTION RPMReference

20. FUNCTION VELOCIDADE_X_Y_Z

21. FUNCTION CalcAccelDecelFromProfile

22. FUNCTION ConvertRealDiffToTargetDINT : DINT

*)

FUNCTION RPMReference

BEGIN

// Escalonamento do potenciômetro

referenceRPM := UDINT((DINT(Potenciometro) * MAX_RPM) / 32767);

// Gera um fator de 0 a 1 baseado na leitura do potenciômetro

POT := REAL(Potenciometro) / 32767;

IF POT > 1.0 THEN

POT := 1.0;

END_IF;

END_FUNCTION

@

(*

FUNCTION VELOCIDADE_X_Y_Z

Calcula RPM X, Y e Z proporcional ao maior deslocamento e limita cada eixo a MAX_RPM. Ajusta todos para referenceRPM se modo ativo e deslocamentos de qualquer valor

FUNCTION VELOCIDADE_X_Y_Z

A função VELOCIDADE_X_Y_Z calcula as velocidades individuais dos eixos X, Y e Z em RPM, preservando o sincronismo entre movimentos com deslocamentos diferentes.

Inicialmente é determinado o maior deslocamento entre os três eixos (diffMax), que passa a ser utilizado como referência para o escalonamento proporcional das velocidades. Dessa forma, o eixo de maior percurso opera na velocidade máxima permitida enquanto os demais recebem velocidades proporcionais aos seus respectivos deslocamentos, garantindo que todos concluem o movimento simultaneamente.

Quando não existe deslocamento em nenhum eixo, a função retorna imediatamente com todas as velocidades iguais a zero, evitando cálculos desnecessários.

Para impedir divisões por zero, caso o maior deslocamento seja nulo, diffMax é forçado para um valor mínimo antes da execução dos cálculos.

A função também adapta automaticamente o limite máximo de rotação conforme o comprimento do deslocamento. Para percursos iguais ou superiores a 0,45 mm, utiliza-se o limite nominal de 3000 RPM. Em deslocamentos menores, o limite passa a ser calculado proporcionalmente ao percurso, reduzindo a velocidade máxima para manter a mesma duração temporal do segmento e preservar a sincronização do sistema.

As velocidades finais de cada eixo são obtidas multiplicando-se o fator do potenciômetro (POT) pela velocidade máxima permitida e pela razão entre o deslocamento do eixo e o maior deslocamento do conjunto. Após o cálculo, cada valor é limitado ao respectivo MAX_RPM, garantindo que nenhuma referência exceda a capacidade operacional configurada do servoacionador.

Essa função constitui o núcleo do sincronismo de velocidade entre os eixos, permitindo movimentos coordenados independentemente das diferenças de percurso.

*)

```
VAR
referenceRPM : UDINT := 0;
  MAX_RPM : UDINT := 3000 ;
  POT : REAL;
  Potenciometro : UINT;
END_VAR;
```

FUNCTION VELOCIDADE_X_Y_Z

```
VAR_INPUT
  diffX_mm : REAL;    // Entrada X parâmetros
  diffY_mm : REAL;    // Entrada Y parâmetros
  diffZ_mm : REAL;    // Z
  POT : REAL;
END_VAR

VAR
  diffMax : REAL;
END_VAR

VAR_OUTPUT
  rpmX : UDINT;    // Saída RPM X
  rpmY : UDINT;    // Saída RPM Y
  rpmZ : UDINT;    // Saída RPM Z
END_VAR

BEGIN

  // Determina o maior deslocamento
  diffMax := MAX(diffX_mm, MAX(diffY_mm, diffZ_mm));

  // Se não há deslocamento
  IF (diffX_mm = 0.0) AND (diffY_mm = 0.0) AND (diffZ_mm = 0.0) THEN
    rpmX := 0;
    rpmY := 0;
    rpmZ := 0;
    VELOCIDADE_X_Y_Z := TRUE;
    RETURN;
  END_IF;

  // Evita divisão por zero
  IF diffMax = 0.0 THEN
```

```

    diffMax := 1.0;
END_IF;

(* Seleciona MAX_RPM de acordo com os deslocamentos *)

IF (diffX_mm >= 0.45) OR (diffY_mm >= 0.45) OR (diffZ_mm >= 0.45) THEN
    MAX_RPM := 3000 ;
ELSE
    MAX_RPM := MAX(MAX(diffX_mm, diffY_mm), diffZ_mm) * 6666.66 ;
END_IF;

// Calcula velocidades proporcionais ao maior eixo
rpmX := UDINT(REAL(MAX_RPM) * POT * (diffX_mm / diffMax) + 0.5);
rpmY := UDINT(REAL(MAX_RPM) * POT * (diffY_mm / diffMax) + 0.5);
rpmZ := UDINT(REAL(MAX_RPM) * POT * (diffZ_mm / diffMax) + 0.5);

// Limita os valores ao máximo permitido
IF rpmX > MAX_RPM THEN rpmX := MAX_RPM; END_IF;
IF rpmY > MAX_RPM THEN rpmY := MAX_RPM; END_IF;
IF rpmZ > MAX_RPM THEN rpmZ := MAX_RPM; END_IF;

END_FUNCTION

```

(*)

FUNÇÃO ACELERAÇÃO DESACELERAÇÃO / JERK

Calcula a aceleração máxima de cada eixo a partir de massa, torque e fator de sobrecarga, determina a aceleração comum limitada pelo eixo mais lento, converte para RPM/s² e calcula o tempo de aceleração/desaceleração necessário em milissegundos, aplicando o mesmo tempo para todos os eixos.

F2 – CalcAccelDecelFromProfile

A função CalcAccelDecelFromProfile calcula automaticamente os tempos de aceleração (0x6083) e desaceleração (0x6084) dos três eixos a partir das características mecânicas da máquina. O objetivo é determinar um perfil de aceleração comum que respeite o eixo mais crítico do sistema, garantindo sincronismo durante movimentos interpolados.

Inicialmente são calculados os torques efetivos de cada eixo, aplicando o fator de sobrecarga (OverdriveFactor) sobre o torque nominal informado para cada servoacionador.

Em seguida, a função calcula o denominador dinâmico de cada eixo, composto pela contribuição da inércia rotacional do conjunto mecânico e pela massa linear convertida para o sistema de transmissão através do passo do fuso (Pitch_m) e do raio equivalente (Radius_m).

Com esses valores é obtida a aceleração linear máxima disponível para cada eixo em m/s^2 .

Após o cálculo individual, a função determina a menor aceleração entre X, Y e Z. Esse valor passa a representar a aceleração comum do sistema, impedindo que qualquer eixo seja solicitado acima de sua capacidade física durante movimentos sincronizados.

A aceleração linear comum é convertida para aceleração angular e posteriormente para RPM/s^2 , unidade utilizada pelos servoacionadores. Caso ocorra uma condição de divisão por zero ou aceleração nula, a função aplica um valor mínimo de segurança para preservar a estabilidade do cálculo.

Utilizando a velocidade de referência configurada (referenceRPM), é determinado o tempo necessário para atingir essa velocidade segundo a aceleração calculada.

Embora o cálculo seja executado para os três eixos, todos utilizam a mesma aceleração comum, resultando em tempos idênticos. Ainda assim, a função determina o maior tempo obtido, garantindo que nenhum eixo opere com uma rampa inferior à exigida pelo conjunto mecânico.

O tempo final é convertido para milissegundos, arredondado para cima e limitado ao intervalo permitido pelo tipo UDINT.

Por fim, os valores calculados são gravados simultaneamente na estrutura AccelDecelLimits e diretamente nos objetos de comunicação dos servoacionadores:

0x6083 – Acceleration

0x6084 – Deceleration

Os três eixos recebem exatamente os mesmos tempos de aceleração e desaceleração, assegurando perfis dinâmicos compatíveis e sincronismo durante toda a execução dos movimentos.

*)

```
TYPE AccelDecelLimits : STRUCT
```

```
  ACCEL_6083_X : UDINT;
```

```
  DECEL_6084_X : UDINT;
```

```
  ACCEL_6083_Y : UDINT;
```

```
  DECEL_6084_Y : UDINT;
```

```
  ACCEL_6083_Z : UDINT;
```

```
  DECEL_6084_Z : UDINT;
```

```
END_STRUCT
```

```
END_TYPE
```

```
VAR
```

```
  result : AccelDecelLimits;
```

```
END_VAR
```

```
FuncoesOriginais[2] := 'CalcAccelDecelFromProfile';
```

```
FuncoesGCODE[2] := 'F2';
```

```

Variaveis[2]                                     :=
'J_X,J_Y,J_Z,Mass_X_kg,Mass_Y_kg,Mass_Z_kg,TorqueMax_X_Nm,TorqueMax_Y_Nm,TorqueMax_Z_Nm,OverdriveFactor_X,OverdriveFactor_Y,OverdriveFactor_Z,Pitch_m,Radius_m';

TiposVariaveis[2]                                 :=
'REAL,REAL,REAL,REAL,REAL,REAL,REAL,REAL,REAL,REAL,REAL,REAL,REAL,REAL';

```

FUNCTION CalcAccelDecelFromProfile

VAR

```

    PulsesPerRev_local : REAL := 65536.0;    // pulsos por volta

    // intermediários por eixo
    torque_effective_X : REAL;
    torque_effective_Y : REAL;
    torque_effective_Z : REAL;

    denom_X      : REAL;
    denom_Y      : REAL;
    denom_Z      : REAL;
    a_linear_max_X : REAL;
    a_linear_max_Y : REAL;
    a_linear_max_Z : REAL;
    a_linear_common : REAL;

    // conversões
    alpha_rad_s2 : REAL;
    accel_rpm_s2 : REAL;

    // tempos de aceleração por eixo
    rpm_real : REAL;
    tempo_accel_X_s : REAL;
    tempo_accel_Y_s : REAL;
    tempo_accel_Z_s : REAL;
    tempo_max_s : REAL;
    tempo_ms : UDINT;
END_VAR

BEGIN
    // torques efetivos por eixo
    torque_effective_X := TorqueMax_X_Nm * OverdriveFactor_X;
    torque_effective_Y := TorqueMax_Y_Nm * OverdriveFactor_Y;
    torque_effective_Z := TorqueMax_Z_Nm * OverdriveFactor_Z;

    // denominadores: inércia + massa convertida
    denom_X := J_X * (2.0 * 3.14159265 / Pitch_m) + Mass_X_kg * Radius_m;
    denom_Y := J_Y * (2.0 * 3.14159265 / Pitch_m) + Mass_Y_kg * Radius_m;
    denom_Z := J_Z * (2.0 * 3.14159265 / Pitch_m) + Mass_Z_kg * Radius_m;

    // acelerações máximas por eixo
    IF denom_X <= 0.0 THEN
        a_linear_max_X := 0.0;
    ELSE

```



```

    a_linear_max_X := torque_effective_X / denom_X; // m/s²
END_IF;

IF denom_Y <= 0.0 THEN
    a_linear_max_Y := 0.0;
ELSE
    a_linear_max_Y := torque_effective_Y / denom_Y; // m/s²
END_IF;

IF denom_Z <= 0.0 THEN
    a_linear_max_Z := 0.0;
ELSE
    a_linear_max_Z := torque_effective_Z / denom_Z; // m/s²
END_IF;

// aceleração comum (gargalo entre os 3 eixos)
a_linear_common := a_linear_max_X;
IF a_linear_max_Y < a_linear_common THEN
    a_linear_common := a_linear_max_Y;
END_IF;
IF a_linear_max_Z < a_linear_common THEN
    a_linear_common := a_linear_max_Z;
END_IF;

// aceleração angular em rad/s² e RPM/s²
alpha_rad_s2 := a_linear_common * (2.0 * 3.14159265 / Pitch_m);
accel_rpm_s2 := alpha_rad_s2 * 60.0 / (2.0 * 3.14159265);

// evitar divisão por zero
IF accel_rpm_s2 <= 0.0 THEN
    accel_rpm_s2 := 1.0;
END_IF;

// conversão de velocidade de referência para REAL
rpm_real := referenceRPM;

// tempo = velocidade / aceleração
tempo_accel_X_s := rpm_real / accel_rpm_s2;
tempo_accel_Y_s := rpm_real / accel_rpm_s2;
tempo_accel_Z_s := rpm_real / accel_rpm_s2;

// tempo máximo entre os três eixos
tempo_max_s := tempo_accel_X_s;
IF tempo_accel_Y_s > tempo_max_s THEN
    tempo_max_s := tempo_accel_Y_s;
END_IF;

IF tempo_accel_Z_s > tempo_max_s THEN
    tempo_max_s := tempo_accel_Z_s;
END_IF;

// conversão para milissegundos com saturação
tempo_ms := TO_UDINT(CEIL(tempo_max_s * 1000.0));
IF tempo_ms > 4294967295 THEN
    tempo_ms := 4294967295;
END_IF;

```

```

// saída: mesmo tempo para aceleração e desaceleração em todos os eixos
result.ACCEL_6083_X := tempo_ms;
result.DECEL_6084_X := tempo_ms;
result.ACCEL_6083_Y := tempo_ms;
result.DECEL_6084_Y := tempo_ms;
result.ACCEL_6083_Z := tempo_ms;
result.DECEL_6084_Z := tempo_ms;

// Atualiza diretamente os PDOs globais com os valores calculados
WR_PDO_6083_X := result.ACCEL_6083_X;
WR_PDO_6084_X := result.DECEL_6084_X;
WR_PDO_6083_Y := result.ACCEL_6083_Y;
WR_PDO_6084_Y := result.DECEL_6084_Y;
WR_PDO_6083_Z := result.ACCEL_6083_Z;
WR_PDO_6084_Z := result.DECEL_6084_Z;

END_FUNCTION

```

FUNÇÃO CONVERSORA

Converte uma diferença em milímetros (diff_mm) para um valor DINT de destino combinando parte inteira em blocos de 6 mm e parte fracionária em 16 bits, aplicando saturação e preservando o sinal.

A função ConvertRealDiffToTargetDINT é responsável pela conversão de deslocamentos lineares, expressos em milímetros, para o formato numérico utilizado internamente pelo sistema de controle. Essa conversão estabelece a interface entre a representação geométrica adotada pelas rotinas de programação e a representação binária empregada pelos buffers de movimentação.

O algoritmo recebe como entrada um deslocamento em milímetros e inicialmente determina seu valor absoluto, preservando o sinal para posterior reconstrução do resultado. Em seguida, o deslocamento é decomposto em duas partes distintas: uma parcela inteira, correspondente ao número de blocos completos de 6 mm, e uma parcela fracionária, correspondente ao deslocamento remanescente dentro desse intervalo.

A parcela inteira é armazenada nos 16 bits mais significativos do valor de saída, enquanto a parcela fracionária é normalizada para uma resolução de 16 bits, ocupando os 16 bits menos significativos. Dessa forma, cada intervalo de 6 mm é subdividido em 65 536 níveis, proporcionando elevada resolução para a representação dos deslocamentos.

Durante o processamento são aplicados mecanismos de saturação para ambas as parcelas da conversão. A parte inteira é limitada ao intervalo representável em 16 bits com sinal, enquanto a parte fracionária é restringida ao intervalo correspondente a 16 bits sem sinal. Essas limitações evitam transbordamentos numéricos e asseguram que o valor produzido permaneça compatível com a estrutura utilizada pelo sistema de controle.

Após a conversão, as parcelas inteira e fracionária são combinadas em um único valor de 32 bits por meio de operações de deslocamento e composição de palavras. Finalmente, o sinal original do deslocamento é reaplicado ao resultado, permitindo representar movimentos tanto no sentido positivo quanto no sentido negativo dos eixos.

O valor produzido por essa função é posteriormente utilizado pelas rotinas responsáveis pelo preenchimento dos buffers de trajetória, sendo armazenado no formato exigido pelas estruturas internas do controlador para posterior processamento pelas tarefas determinísticas de controle de movimento.

FUNCTION ConvertRealDiffToTargetDINT : DINT

VAR_INPUT

diff_mm : REAL;

END_VAR

VAR

abs_diff : REAL;

full_blocks : DINT;

remainder : REAL;

fractional : DINT;

assembled : UDINT;

result : DINT;

END_VAR

BEGIN

// Valor absoluto

IF diff_mm < 0.0 THEN

abs_diff := -diff_mm;

ELSE

abs_diff := diff_mm;

END_IF;

// Parte inteira em blocos de 6 mm

full_blocks := DINT(TRUNC(abs_diff / 6.0));

// Saturação da parte alta (16 bits com sinal)

IF full_blocks > 32767 THEN

full_blocks := 32767;

END_IF;

// Parte fracionária

remainder := abs_diff - (REAL(full_blocks) * 6.0);

fractional := DINT(TRUNC((remainder * 65536.0) / 6.0));

```

// Saturação da parte baixa (16 bits sem sinal)
IF fractional > 65535 THEN
    fractional := 65535;
END_IF;

// Monta UDINT: parte alta = full_blocks, parte baixa = fractional
assembled := UDINT(SHL(UDINT(full_blocks AND 65535), 16))
            OR UDINT(fractional AND 65535);

// Aplica sinal negativo se necessário
IF diff_mm < 0.0 THEN
    result := DINT(-DINT(assembled));
ELSE
    result := DINT(assembled);
END_IF;

// Retorno da função
ConvertRealDiffToTargetDINT := result;
END_FUNCTION

```

@

(*

FUNÇÃO DRIVE INPUT OUTPUT

A função implementa um sequenciador robusto de setpoints para movimentos de eixo com controle bidirecional e suporte a ciclos parciais e completos.

A função DRIVE_INPUT_OUTPUT implementa a camada central de execução dos eixos servo, sendo responsável pela preparação das janelas de processamento, seleção dinâmica das regiões válidas do buffer, carregamento de posições e velocidades para o drive, controle completo do handshake CiA-402, sincronização entre múltiplos eixos, gerenciamento dos ciclos de repetição, aplicação de elevações tridimensionais e avanço coordenado dos índices de trajetória.

O processamento inicia determinando os limites ativos de operação do eixo. Em modo parcial, as janelas locais são calculadas a partir das regiões configuradas, podendo operar tanto em segmentação fixa quanto em expansão progressiva controlada pelo número de passadas. Durante essa expansão, os centros das regiões permanecem preservados enquanto suas larguras convergem gradualmente até ocupar todo o intervalo definido pelo buffer. Todas as regiões calculadas são posteriormente limitadas aos extremos físicos válidos e regiões desabilitadas são automaticamente excluídas do processamento.

Após a definição das janelas, a função inicializa o ciclo de execução quando o eixo é habilitado, posicionando o índice inicial conforme o modo de operação e preparando o estado interno necessário para o envio da trajetória.

Durante a execução, apenas os pontos pertencentes à janela ativa são carregados para o drive. Cada posição recebe simultaneamente seu alvo absoluto e sua velocidade

correspondente, preservando o sincronismo entre posição e perfil cinemático antes da emissão do comando de movimentação.

O envio dos comandos utiliza integralmente o protocolo de handshake do CiA-402. A função escreve o novo alvo, solicita sua aceitação através do bit de novo setpoint, aguarda a confirmação do drive, remove automaticamente o comando após o reconhecimento, monitora o consumo do acknowledge e somente considera o ponto concluído quando o drive libera novamente a interface ou, em operações de ponto único, quando a posição final é efetivamente alcançada.

O avanço global somente ocorre quando todos os eixos participantes atingem simultaneamente o estado de liberação. Dessa forma, nenhum eixo progride isoladamente, mantendo sincronismo absoluto durante toda a execução da trajetória.

O mecanismo de janelas percorre sequencialmente todas as regiões configuradas, permitindo que diferentes segmentos do buffer sejam executados de forma ordenada. A mudança de janela somente acontece após a conclusão sincronizada da janela anterior, preservando a coerência temporal entre todos os eixos.

Na última passagem de escala, a função também controla o acionamento das elevações programadas. Esse acionamento pode ocorrer automaticamente ao término da janela correspondente ou em um índice específico definido pelo mecanismo de anticolisão, permitindo a inserção de movimentos tridimensionais sobre a trajetória principal sem alterar sua sequência lógica.

Quando uma elevação é ativada, a função calcula a maior amplitude entre os deslocamentos dos três eixos e utiliza esse valor como referência para normalizar proporcionalmente as velocidades individuais. Em seguida, reconstrói os alvos absolutos dos eixos, aplica os deslocamentos adicionais convertidos para pulsos e envia os novos valores ao drive, preservando a proporcionalidade espacial da elevação independentemente da direção predominante do movimento.

Cada elevação possui controle próprio de progressão e encerramento. O índice interno evolui até o último elemento programado e, ao final, a elevação é automaticamente finalizada, impedindo reaplicações indevidas durante o restante do ciclo.

Após a conclusão de cada ponto, a função atualiza os índices da trajetória conforme o modo configurado. São suportados percursos unidirecionais, movimentos alternados de ida e volta, múltiplas repetições, meia passagem final e inversões automáticas de direção ao atingir os limites definidos para cada janela. Todo o gerenciamento dos ciclos é realizado mantendo os contadores internos consistentes até que o número programado de repetições seja completamente executado.

Como resultado, a função concentra toda a lógica responsável pela transferência determinística da trajetória calculada para os servoacionadores, coordenando segmentação, expansão de janelas, sincronização entre eixos, comunicação CiA-402, aplicação de elevações, controle de repetições e avanço seguro da execução sem perda de sincronismo entre os movimentos.

23. FUNCTION DRIVE_INPUT_OUTPUT

*)

TYPE tElevationPoint :
STRUCT

```
X : REAL;  
Y : REAL;  
Z : REAL;  
END_STRUCT  
END_TYPE
```

```
CONST  
    MAX_PTS : INT := 256;  
END_CONST
```

```
// Estruturas globais  
TYPE ServoFlags : STRUCT  
    initCycle      : BOOL;  
    loadedTarget   : BOOL;  
    wroteSetpoint   : BOOL;  
    acknowledged   : BOOL;  
    clearedSetpoint : BOOL;  
    readyToAdvance : BOOL;  
    singlePoint     : BOOL;  
END_STRUCT  
END_TYPE
```

```
TYPE ServoAxis : STRUCT
```

```
    scalePassIndex : INT := 1;
```

```
    SENTIDO : INT;  
    elevIndex : ARRAY[1..5] OF INT;
```

```
    LimitStart1 : REAL;  
    LimitEnd1   : REAL;
```

```
    LimitStart2 : REAL;  
    LimitEnd2   : REAL;
```

```
    LimitStart3 : REAL;  
    LimitEnd3   : REAL;
```

```
    LimitStart4 : REAL;  
    LimitEnd4   : REAL;
```

```
    LimitStart5 : REAL;  
    LimitEnd5   : REAL;
```

```
    elevationDone : ARRAY[1..5] OF BOOL;  
    elevation     : ARRAY[1..5] OF BOOL;  
    invert        : BOOL;  
    PartialStart1 : REAL;  
    PartialEnd1   : REAL;
```

```
PartialStart2 : REAL;  
PartialEnd2   : REAL;
```

```
PartialStart3 : REAL;  
PartialEnd3   : REAL;
```

```
PartialStart4 : REAL;  
PartialEnd4   : REAL;
```

```
PartialStart5 : REAL;  
PartialEnd5   : REAL;  
NFcount       : INT;
```

```
  flags       : ServoFlags;  
  index       : INT;  
  WR_PDO_607A_H: INT;  
  WR_PDO_607A_L: INT;  
  WR_PDO_6081  : UDINT;  
  WR_PDO_6040  : INT;  
  ready       : BOOL;  
  lastIndex   : INT;  
  StartBuffer  : DINT;  
  EndBuffer    : DINT;  
  N           : INT;  
  PR, PR1, P2  : BOOL;  
  PX, PY, PZ   : REAL;  
  cycleCount   : DINT;  
  direction    : INT;  
  RD_PDO_6041  : INT;  
  buffer_H     : ARRAY[0..1256000] OF INT;  
  buffer_L     : ARRAY[0..1256000] OF INT;  
  bufferVel    : ARRAY[0..1256000] OF UDINT;  
END_STRUCT  
END_TYPE
```

```
// Variáveis globais DRIVE
```

```
VAR  
  flagsX : ServoFlags := (OTHERS := FALSE);  
  flagsY : ServoFlags := (OTHERS := FALSE);  
  flagsZ : ServoFlags := (OTHERS := FALSE);  
  
  axisX : ServoAxis;  
  axisY : ServoAxis;  
  axisZ : ServoAxis;  
END_VAR
```

FUNCTION DRIVE_INPUT_OUTPUT

VAR_IN_OUT

axis : ServoAxis;

END_VAR

VAR

eMax : REAL;

startLocal1 : INT;

endLocal1 : INT;

startLocal2 : INT;

endLocal2 : INT;

startLocal3 : INT;

endLocal3 : INT;

startLocal4 : INT;

endLocal4 : INT;

startLocal5 : INT;

endLocal5 : INT;

t : REAL;

controlword : INT;

statusword : INT;

lowerLimit : INT;

upperLimit : INT;

readyToAdvanceGlobal : BOOL;

lx_low, lx_up : INT;

ly_low, ly_up : INT;

lz_low, lz_up : INT;

END_VAR

BEGIN

(* --- determino limites do eixo atual (para uso local) --- *)

IF PARTIAL THEN

lowerLimit := axis.StartBuffer;

upperLimit := axis.EndBuffer;


```
IF axis.N = 0 THEN
```

```
  (* MODO SEGMENTAÇÃO *)
```

```
  startLocal1 := axis.PartialStart1;  
  endLocal1   := axis.PartialEnd1;
```

```
  startLocal2 := axis.PartialStart2;  
  endLocal2   := axis.PartialEnd2;
```

```
  startLocal3 := axis.PartialStart3;  
  endLocal3   := axis.PartialEnd3;
```

```
  startLocal4 := axis.PartialStart4;  
  endLocal4   := axis.PartialEnd4;
```

```
  startLocal5 := axis.PartialStart5;  
  endLocal5   := axis.PartialEnd5;
```

```
ELSE
```

```
// IF (axis.N > 0) THEN
```

```
  lowerLimit := axis.startbuffer;  
  upperLimit := axis.endbuffer;
```

```
t := 1.0;
```

```
IF axis.N > 0 THEN
```

```
  t := REAL(axis.NFcount) / REAL(axis.N);  
END_IF;
```

```
globalStart := axis.StartBuffer;  
globalEnd   := axis.EndBuffer;
```

```
H := REAL_TO_INT((globalEnd - globalStart) * 0.5 * t);
```

```
IF (axis.SENTIDO = 0) OR (axis.invert = FALSE) THEN
```

```
  center1 :=  
    axis.LimitStart1 +  
    REAL_TO_INT(  
      (axis.PartialStart1 / 100.0) *  
      (axis.LimitEnd1 - axis.LimitStart1)  
    );
```

```
  center2 :=  
    axis.LimitStart2 +  
    REAL_TO_INT(  
      (axis.PartialStart2 / 100.0) *  
      (axis.LimitEnd2 - axis.LimitStart2)  
    );
```

```
  center3 :=  
    axis.LimitStart3 +  
    REAL_TO_INT(  
      (axis.PartialStart3 / 100.0) *  
      (axis.LimitEnd3 - axis.LimitStart3)  
    );
```

```

        (axis.PartialStart3 / 100.0) *
        (axis.LimitEnd3 - axis.LimitStart3)
    );

center4 :=
    axis.LimitStart4 +
    REAL_TO_INT(
        (axis.PartialStart4 / 100.0) *
        (axis.LimitEnd4 - axis.LimitStart4)
    );

center5 :=
    axis.LimitStart5 +
    REAL_TO_INT(
        (axis.PartialStart5 / 100.0) *
        (axis.LimitEnd5 - axis.LimitStart5)
    );

ELSE

center1 :=
    axis.LimitStart1 +
    REAL_TO_INT(
        ((100.0 - axis.PartialStart1) / 100.0) *
        (axis.LimitEnd1 - axis.LimitStart1)
    );

center2 :=
    axis.LimitStart2 +
    REAL_TO_INT(
        ((100.0 - axis.PartialStart2) / 100.0) *
        (axis.LimitEnd2 - axis.LimitStart2)
    );

center3 :=
    axis.LimitStart3 +
    REAL_TO_INT(
        ((100.0 - axis.PartialStart3) / 100.0) *
        (axis.LimitEnd3 - axis.LimitStart3)
    );

center4 :=
    axis.LimitStart4 +
    REAL_TO_INT(
        ((100.0 - axis.PartialStart4) / 100.0) *
        (axis.LimitEnd4 - axis.LimitStart4)
    );

center5 :=
    axis.LimitStart5 +
    REAL_TO_INT(
        ((100.0 - axis.PartialStart5) / 100.0) *
        (axis.LimitEnd5 - axis.LimitStart5)
    );

END_IF;
(* força convergência global *)

```

IF t >= 1.0 THEN

startLocal1 := axis.StartBuffer;
endLocal1 := axis.EndBuffer;

startLocal2 := axis.StartBuffer;
endLocal2 := axis.EndBuffer;

startLocal3 := axis.StartBuffer;
endLocal3 := axis.EndBuffer;

startLocal4 := axis.StartBuffer;
endLocal4 := axis.EndBuffer;

startLocal5 := axis.StartBuffer;
endLocal5 := axis.EndBuffer;

ELSE

startLocal1 := center1 - H;
endLocal1 := center1 + H;

startLocal2 := center2 - H;
endLocal2 := center2 + H;

startLocal3 := center3 - H;
endLocal3 := center3 + H;

startLocal4 := center4 - H;
endLocal4 := center4 + H;

startLocal5 := center5 - H;
endLocal5 := center5 + H;

END_IF;

(* ===== *)
(* CLAMP FÍSICO NO DOMÍNIO VÁLIDO *)
(* ===== *)

startLocal1 := MAX(axis.StartBuffer, MIN(axis.EndBuffer, startLocal1));
endLocal1 := MAX(axis.StartBuffer, MIN(axis.EndBuffer, endLocal1));

startLocal2 := MAX(axis.StartBuffer, MIN(axis.EndBuffer, startLocal2));
endLocal2 := MAX(axis.StartBuffer, MIN(axis.EndBuffer, endLocal2));

startLocal3 := MAX(axis.StartBuffer, MIN(axis.EndBuffer, startLocal3));
endLocal3 := MAX(axis.StartBuffer, MIN(axis.EndBuffer, endLocal3));

startLocal4 := MAX(axis.StartBuffer, MIN(axis.EndBuffer, startLocal4));
endLocal4 := MAX(axis.StartBuffer, MIN(axis.EndBuffer, endLocal4));

startLocal5 := MAX(axis.StartBuffer, MIN(axis.EndBuffer, startLocal5));
endLocal5 := MAX(axis.StartBuffer, MIN(axis.EndBuffer, endLocal5));

(* ===== *)
(* DESABILITA FAIXAS 0 / 0 *)

```
(* ===== *)
```

```
IF axis.PartialStart1 = 0 AND axis.PartialEnd1 = 0 THEN  
    startLocal1 := axis.EndBuffer;  
    endLocal1 := axis.StartBuffer;  
END_IF;
```

```
IF axis.PartialStart2 = 0 AND axis.PartialEnd2 = 0 THEN  
    startLocal2 := axis.EndBuffer;  
    endLocal2 := axis.StartBuffer;  
END_IF;
```

```
IF axis.PartialStart3 = 0 AND axis.PartialEnd3 = 0 THEN  
    startLocal3 := axis.EndBuffer;  
    endLocal3 := axis.StartBuffer;  
END_IF;
```

```
IF axis.PartialStart4 = 0 AND axis.PartialEnd4 = 0 THEN  
    startLocal4 := axis.EndBuffer;  
    endLocal4 := axis.StartBuffer;  
END_IF;
```

```
IF axis.PartialStart5 = 0 AND axis.PartialEnd5 = 0 THEN  
    startLocal5 := axis.EndBuffer;  
    endLocal5 := axis.StartBuffer;  
END_IF;
```

```
ELSE
```

```
    lowerLimit := 0;  
    upperLimit := axis.lastIndex;
```

```
//END_IF;
```

```
(* --- inicialização de ciclo (mantida) --- *)
```

```
IF axis.ready AND NOT axis.flags.initCycle THEN
```

```
IF (axis.N > 0) OR PARTIAL THEN
```

```
    axis.index := axis.startbuffer;
```

```
ELSE
```

```
    axis.index := 0;
```

```
END_IF;
```

```
axis.flags.initCycle := TRUE;
```

```
axis.direction := +1;
```

```
END_IF;
```

```
(* --- leio status atual do drive --- *)
```

```
statusword := axis.RD_PDO_6041;
```

```
IF axis.flags.initCycle AND NOT axis.flags.loadedTarget THEN
```

```
IF axis.index <= upperLimit THEN
```

```
IF (activeWindow = 1 AND axis.index >= startLocal1 AND axis.index <= endLocal1) OR  
   (activeWindow = 2 AND axis.index >= startLocal2 AND axis.index <= endLocal2) OR  
   (activeWindow = 3 AND axis.index >= startLocal3 AND axis.index <= endLocal3) OR  
   (activeWindow = 4 AND axis.index >= startLocal4 AND axis.index <= endLocal4) OR
```

```
(activeWindow = 5 AND axis.index >= startLocal5 AND axis.index <= endLocal5)
THEN
```

```
axis.WR_PDO_607A_H := axis.buffer_H[axis.index];
axis.WR_PDO_607A_L := axis.buffer_L[axis.index];
axis.WR_PDO_6081 := axis.bufferVel[axis.index];
```

```
axis.flags.loadedTarget := TRUE;
```

```
IF (activeWindow = 1 AND axis.index = endLocal1 + 1) OR
   (activeWindow = 2 AND axis.index = endLocal2 + 1) OR
   (activeWindow = 3 AND axis.index = endLocal3 + 1) OR
   (activeWindow = 4 AND axis.index = endLocal4 + 1) OR
   (activeWindow = 5 AND axis.index = endLocal5 + 1)
```

```
THEN
```

```
IF axis.scalePassIndex = BlocoN.totalScalePasses THEN
```

```
IF ANTI_COLISAO THEN
```

```
IF axis.index = BlocoN.indexAntiColisao THEN
    axis.elevation := TRUE;
END_IF;
```

```
ELSE
```

```
IF (activeWindow = 1 AND axis.index = endLocal1) OR
   (activeWindow = 2 AND axis.index = endLocal2) OR
   (activeWindow = 3 AND axis.index = endLocal3) OR
   (activeWindow = 4 AND axis.index = endLocal4) OR
   (activeWindow = 5 AND axis.index = endLocal5)
```

```
THEN
```

```
axis.elevation := TRUE;
END_IF;
```

```
END_IF;
```

```
END_IF;
```

```
END_IF;
END_IF;
END_IF;
```

```
END_IF;
```

```
(* --- enviar setpoint (mantido) --- *)
```

```
IF axis.flags.loadedTarget AND NOT axis.flags.wroteSetpoint THEN
    controlword := axis.WR_PDO_6040;
    controlword := controlword OR NEW_SETPOINT;
    axis.WR_PDO_6040 := controlword;
    axis.flags.wroteSetpoint := TRUE;
END_IF;
```

```
(* --- CALCULO do readyToAdvanceGlobal usando readyToAdvance (ACK consumido) --- *)
```

```
(* IMPORTANTE: usa flags.readyToAdvance, não acknowledged *)
```

```
readyToAdvanceGlobal :=
    axisX.flags.readyToAdvance AND
    axisY.flags.readyToAdvance AND
    axisZ.flags.readyToAdvance;
```

```

(* --- esperar acknowledge: quando status indica ACK (bit), setamos acknowledged --- *)
IF axis.flags.wroteSetpoint AND NOT axis.flags.acknowledged THEN
  IF (statusword AND ACKNOWLEDGE) <> 0 THEN
    axis.flags.acknowledged := TRUE;
  END_IF;
END_IF;

(* --- limpar setpoint quando acknowledged e não limpou ainda --- *)
IF axis.flags.acknowledged AND NOT axis.flags.clearedSetpoint THEN
  controlword := axis.WR_PDO_6040;
  controlword := controlword AND NOT NEW_SETPOINT;
  axis.WR_PDO_6040 := controlword;
  axis.flags.clearedSetpoint := TRUE;
END_IF;

(* --- pronto para avançar: ACK caiu para 0 (consumido) -> readyToAdvance TRUE --- *)
IF axis.flags.clearedSetpoint AND NOT axis.flags.readyToAdvance THEN
  IF (statusword AND ACKNOWLEDGE) = 0 THEN
    axis.flags.readyToAdvance := TRUE;
  END_IF;

  (* caso singlePoint (target reached) também pode forçar readyToAdvance *)
  IF axis.flags.singlePoint THEN
    IF ((statusword AND TARGET_REACHED) <> 0) AND ((statusword AND ACKNOWLEDGE) = 0) THEN
      axis.flags.readyToAdvance := TRUE;
    END_IF;
  END_IF;
END_IF;

IF t = 0.0 THEN
  FOR i := 1 TO 5 DO
    axis.elevIndex[i] := 1;
  END_FOR;
END_IF;

FOR i := 1 TO 5 DO
  IF (startLocal[i] <= center[i]) AND
    (endLocal[i] >= center[i]) THEN
    IF invert THEN
      IF NOT ONE_DIR THEN
        axis.elevation[i] := TRUE;
      END_IF;
    ELSE
      axis.elevation[i] := TRUE;
    END_IF;
  END_IF;
END_FOR;

FOR i := 1 TO 5 DO
  IF axis.elevation[i] THEN

    IF (startLocal[i] <= axis.EndBuffer) AND
      (endLocal[i] >= axis.StartBuffer) THEN

      eMax := ABS(BufferN[IndiceBloco]. Elevation[i,elevIndex[i]].X);

    IF ABS(BufferN[IndiceBloco]. Elevation[i,elevIndex[i]].Y) > eMax THEN
      eMax := ABS(BufferN[IndiceBloco]. Elevation[i,elevIndex[i]].Y);
    END_IF;
  END_IF;
END_FOR;

```

```

END_IF;

IF ABS(BufferN[IndiceBloco].ElevationZ[i]) > eMax THEN
    eMax := ABS(BufferN[IndiceBloco].ElevationZ[i]);
END_IF;

IF eMax > 0.0 THEN

    axis.WR_PDO_6081_X :=
        UDINT(3000.0 * REAL(BufferN[IndiceBloco]. Elevation[i,elevIndex[i]].X) / eMax);

    axis.WR_PDO_6081_Y :=
        UDINT(3000.0 * REAL(BufferN[IndiceBloco]. Elevation[i,elevIndex[i]].Y) / eMax);

    axis.WR_PDO_6081_Z :=
        UDINT(3000.0 * REAL(BufferN[IndiceBloco]. Elevation[i,elevIndex[i]].Z) / eMax);

END_IF;

(* X *)
targetX := DINT(SHL(UDINT(axis.WR_PDO_607A_H_X),16))
           + DINT(UDINT(axis.WR_PDO_607A_L_X));

targetX := targetX +    REAL_TO_DINT(BufferN[IndiceBloco]. Elevation[i,elevIndex[i]].X * 10923.0);
axis.WR_PDO_607A_H_X := INT(SHR(UDINT(targetX),16) AND 65535);
axis.WR_PDO_607A_L_X := INT(UDINT(targetX) AND 65535);

(* Y *)
targetY := DINT(SHL(UDINT(axis.WR_PDO_607A_H_Y),16))
           + DINT(UDINT(axis.WR_PDO_607A_L_Y));

(* Y *)
targetY := targetY +    REAL_TO_DINT(BufferN[IndiceBloco]. Elevation[i,elevIndex[i]].Y * 10923.0);
axis.WR_PDO_607A_H_Y := INT(SHR(UDINT(targetY),16) AND 65535);
axis.WR_PDO_607A_L_Y := INT(UDINT(targetY) AND 65535);

(* Z *)
targetZ := DINT(SHL(UDINT(axis.WR_PDO_607A_H_Z),16))
           + DINT(UDINT(axis.WR_PDO_607A_L_Z));

(* Z *)
targetZ := targetZ +    REAL_TO_DINT(BufferN[IndiceBloco]. Elevation[i,elevIndex[i]].Z * 10923.0);
axis.WR_PDO_607A_H_Z := INT(SHR(UDINT(targetZ),16) AND 65535);
axis.WR_PDO_607A_L_Z := INT(UDINT(targetZ) AND 65535);

END_IF;
axis.elevIndex[i] := axis.elevIndex[i] + 1;

IF axis.elevIndex[i] >
    BufferN[IndiceBloco].ElevationCount[i] THEN

    axis.elevation[i] := FALSE;

    axis.elevIndex[i] :=
        BufferN[IndiceBloco].ElevationCount[i];

END_IF;

```

```

// axis.elevation[i] := FALSE;
IF axis.elevIndex[i] >
  BufferN[IndiceBloco].ElevationCount[i]
THEN
  axis.elevation[i] := FALSE;
axis.elevationDone[i] := TRUE;
END_IF;
END_IF;
END_FOR;
END_IF;

(* -----
BLOCO CENTRAL DE AVANÇO — só executa quando TODOS os eixos estiverem prontos
Este bloco centraliza TODO o incremento/decremento (ou inversão) de índices,
garantindo sincronismo absoluto e preservando N / PR / PR1.
----- *)

IF readyToAdvanceGlobal
AND NOT elevationBusy THEN
elevationBusy := FALSE;
FOR i := 1 TO 5 DO
  IF axisX.elevation[i] OR
    axisY.elevation[i] OR
    axisZ.elevation[i] THEN
    elevationBusy := TRUE;
  END_IF;
END_FOR;

(* encerra janela atual *)
IF activeWindow < 5 THEN
  activeWindow := activeWindow + 1;
ELSE
  activeWindow := 1; (* loop sequencial obrigatório *)
END_IF;
END_IF;

(* --- preparar limites por eixo (usamos os limites definidos por cada axis global) --- *)
lx_low := axisX.startbuffer; lx_up := axisX.endbuffer;
IF axisX.N = 0 THEN lx_low := 0; lx_up := axisX.lastIndex; END_IF;

ly_low := axisY.startbuffer; ly_up := axisY.endbuffer;
IF axisY.N = 0 THEN ly_low := 0; ly_up := axisY.lastIndex; END_IF;

lz_low := axisZ.startbuffer; lz_up := axisZ.endbuffer;
IF axisZ.N = 0 THEN lz_low := 0; lz_up := axisZ.lastIndex; END_IF;

(* ----- Avanço por eixo: X ----- *)
IF axisX.PR = FALSE THEN
  (* somente ida: incrementa *)
  axisX.index := axisX.index + 1;

IF BufferN[axisX.index].NFtrigger THEN
  axisX.NFcount := axisX.NFcount + 1;
END_IF;
(* ----- Atualização do eixo X com PX ----- *)
IF axisX.PR OR axisX.PR1 THEN
  IF (axisX.direction = +1) OR axisX.P2 THEN
    axisX.WR_PDO_607A_L := axisX.WR_PDO_607A_L + REAL_TO_INT(axisX.PX * 10923);

```



```

(* Tratamento de overflow *)
IF axisX.WR_PDO_607A_L > 65535 THEN
    axisX.WR_PDO_607A_L := axisX.WR_PDO_607A_L - 65536;
    axisX.WR_PDO_607A_H := axisX.WR_PDO_607A_H + 1;
END_IF;

ELSIF axisX.P2 AND (axisX.direction = -1) THEN
    axisX.WR_PDO_607A_L := axisX.WR_PDO_607A_L + REAL_TO_INT(axisX.PX * 10923);

    (* Tratamento de overflow *)
    IF axisX.WR_PDO_607A_L > 65535 THEN
        axisX.WR_PDO_607A_L := axisX.WR_PDO_607A_L - 65536;
        axisX.WR_PDO_607A_H := axisX.WR_PDO_607A_H + 1;
    END_IF;

END_IF;
END_IF;

IF axisX.index > lx_up THEN
    (* ciclo X terminou (ida) *)
    axisX.flags.initCycle := FALSE;
    axisX.ready := FALSE;
    axisX.cycleCount := axisX.cycleCount + 1;
    IF axisX.cycleCount < axisX.N THEN
        axisX.ready := TRUE;
    ELSIF (axisX.cycleCount = axisX.N) AND axisX.PR1 THEN
        axisX.ready := TRUE; (* meio avanço extra *)
    ELSE
        axisX.ready := FALSE; (* terminou geral *)
    END_IF;
END_IF;
ELSE
    (* ida e volta *)
    IF axis.direction = -1 THEN

        IF axis.index <= startLocal1 AND axis.index >= endLocal1 THEN
            axis.direction := +1;

            ELSIF axis.index <= startLocal2 AND axis.index >= endLocal2
            THEN
                axis.direction := +1;

                ELSIF axis.index <= startLocal3 AND axis.index >= endLocal3
                THEN
                    axis.direction := +1;

```

```
ELSIF axis.index <= startLocal4 AND axis.index >= endLocal4  
THEN  
    axis.direction := +1;
```

```
ELSIF axis.index <= startLocal5 AND axis.index >= endLocal5  
THEN  
    axis.direction := +1;
```

```
END_IF;
```

```
END_IF;
```

```
IF axis.direction = +1 THEN
```

```
    IF axis.index >= endLocal1 THEN  
        axis.direction := -1;
```

```
    ELSIF axis.index >= endLocal2 THEN  
        axis.direction := -1;
```

```
    ELSIF axis.index >= endLocal3 THEN  
        axis.direction := -1;
```

```
    ELSIF axis.index >= endLocal4 THEN  
        axis.direction := -1;
```

```
    ELSIF axis.index >= endLocal5 THEN  
        axis.direction := -1;
```

```
END_IF;
```

```
END_IF;
```

```
IF axisX.direction = +1 THEN  
    IF axisX.index < lx_up THEN  
        axisX.index := axisX.index + 1;
```

```

ELSIF axisX.index = lx_up THEN
  IF lx_up > lx_low THEN
    axisX.direction := -1;
    axisX.index := lx_up - 1; (* começa a voltar *)
  ELSE
    (* lastIndex = 0 caso especial *)
    axisX.flags.initCycle := FALSE;
    axisX.ready := FALSE;
    axisX.cycleCount := axisX.cycleCount + 1;
    IF axisX.cycleCount < axisX.N THEN
      axisX.ready := TRUE;
    ELSIF (axisX.cycleCount = axisX.N) AND axisX.PR1 THEN
      axisX.ready := TRUE;
    ELSE
      axisX.ready := FALSE;
    END_IF;
  END_IF;
END_IF;
ELSE
  (* voltando *)
  IF axisX.index > lx_low THEN
    axisX.index := axisX.index - 1;
  ELSIF axisX.index = lx_low THEN
    (* ciclo completo *)
    axisX.flags.initCycle := FALSE;
    axisX.ready := FALSE;
    axisX.cycleCount := axisX.cycleCount + 1;
    IF axisX.cycleCount < axisX.N THEN
      axisX.ready := TRUE;
      axisX.direction := +1;
    ELSIF (axisX.cycleCount = axisX.N) AND axisX.PR1 THEN
      axisX.ready := TRUE;
      axisX.direction := +1;
    ELSE
      axisX.ready := FALSE;
    END_IF;
  END_IF;
END_IF;
END_IF;

```

(* ----- Avanço por eixo: Y (mesma lógica) ----- *)

```

IF axisY.PR = FALSE THEN
axisY.index := axisY.index + 1;

```

```

IF BufferN[axisY.index].NFtrigger THEN
  axisY.NFcount := axisY.NFcount + 1;
END_IF;

```

```

IF axisY.PR OR axisY.PR1 THEN
  IF (axisY.direction = +1) OR axisY.P2 THEN
    axisY.WR_PDO_607A_L := axisY.WR_PDO_607A_L + REAL_TO_INT(axisY.PY * 10923);
    IF axisY.WR_PDO_607A_L > 65535 THEN
      axisY.WR_PDO_607A_L := axisY.WR_PDO_607A_L - 65536;
      axisY.WR_PDO_607A_H := axisY.WR_PDO_607A_H + 1;
    END_IF;
  ELSIF axisY.P2 AND (axisY.direction = -1) THEN

```

```
axisY.WR_PDO_607A_L := axisY.WR_PDO_607A_L + REAL_TO_INT(axisY.PY * 10923);
IF axisY.WR_PDO_607A_L > 65535 THEN
    axisY.WR_PDO_607A_L := axisY.WR_PDO_607A_L - 65536;
    axisY.WR_PDO_607A_H := axisY.WR_PDO_607A_H + 1;
END_IF;
END_IF;
END_IF;
```

```
IF axisY.index > ly_up THEN
    axisY.flags.initCycle := FALSE;
    axisY.ready := FALSE;
    axisY.cycleCount := axisY.cycleCount + 1;
    IF axisY.cycleCount < axisY.N THEN
        axisY.ready := TRUE;
    ELSIF (axisY.cycleCount = axisY.N) AND axisY.PR1 THEN
        axisY.ready := TRUE;
    ELSE
        axisY.ready := FALSE;
    END_IF;
END_IF;
ELSE
```

```
IF axis.direction = -1 THEN
```

```
IF axis.index <= startLocal1 AND axis.index >= endLocal1 THEN
    axis.direction := +1;
```

```
ELSIF axis.index <= startLocal2 AND axis.index >= endLocal2
THEN
    axis.direction := +1;
```

```
ELSIF axis.index <= startLocal3 AND axis.index >= endLocal3
THEN
    axis.direction := +1;
```

```
ELSIF axis.index <= startLocal4 AND axis.index >= endLocal4
THEN
    axis.direction := +1;
```

```
ELSIF axis.index <= startLocal5 AND axis.index >= endLocal5
THEN
    axis.direction := +1;
```

```
END_IF;
```

END_IF;

IF axis.direction = +1 THEN

IF axis.index >= endLocal1 THEN
axis.direction := -1;

ELSIF axis.index >= endLocal2 THEN
axis.direction := -1;

ELSIF axis.index >= endLocal3 THEN
axis.direction := -1;

ELSIF axis.index >= endLocal4 THEN
axis.direction := -1;

ELSIF axis.index >= endLocal5 THEN
axis.direction := -1;

END_IF;

END_IF;

```
IF axisY.direction = +1 THEN
  IF axisY.index < ly_up THEN
    axisY.index := axisY.index + 1;
  ELSIF axisY.index = ly_up THEN
    IF ly_up > ly_low THEN
      axisY.direction := -1;
      axisY.index := ly_up - 1;
    ELSE
      axisY.flags.initCycle := FALSE;
      axisY.ready := FALSE;
      axisY.cycleCount := axisY.cycleCount + 1;
      IF axisY.cycleCount < axisY.N THEN
        axisY.ready := TRUE;
      ELSIF (axisY.cycleCount = axisY.N) AND axisY.PR1 THEN
        axisY.ready := TRUE;
      ELSE
        axisY.ready := FALSE;
      END IF;
    END IF;
  END IF;
END IF;
```

```

        END_IF;
    END_IF;
END_IF;
ELSE
    IF axisY.index > ly_low THEN
        axisY.index := axisY.index - 1;
    ELSIF axisY.index = ly_low THEN
        axisY.flags.initCycle := FALSE;
        axisY.ready := FALSE;
        axisY.cycleCount := axisY.cycleCount + 1;
        IF axisY.cycleCount < axisY.N THEN
            axisY.ready := TRUE;
            axisY.direction := +1;
        ELSIF (axisY.cycleCount = axisY.N) AND axisY.PR1 THEN
            axisY.ready := TRUE;
            axisY.direction := +1;
        ELSE
            axisY.ready := FALSE;
        END_IF;
    END_IF;
END_IF;
END_IF;

```

(* ----- Avanço por eixo: Z (mesma lógica) ----- *)

```

IF axisZ.PR = FALSE THEN
    axisZ.index := axisZ.index + 1;

```

```

IF BufferN[axisZ.index].NFtrigger THEN
    axisZ.NFcount := axisZ.NFcount + 1;
END_IF;

```

(* Eixo Z *)

```

IF axisZ.PR OR axisZ.PR1 THEN
    IF (axisZ.direction = +1) OR axisZ.P2 THEN
        axisZ.WR_PDO_607A_L := axisZ.WR_PDO_607A_L + REAL_TO_INT(axisZ.PZ * 10923);
        IF axisZ.WR_PDO_607A_L > 65535 THEN
            axisZ.WR_PDO_607A_L := axisZ.WR_PDO_607A_L - 65536;
            axisZ.WR_PDO_607A_H := axisZ.WR_PDO_607A_H + 1;
        END_IF;
    ELSIF axisZ.P2 AND (axisZ.direction = -1) THEN
        axisZ.WR_PDO_607A_L := axisZ.WR_PDO_607A_L + REAL_TO_INT(axisZ.PZ * 10923);
        IF axisZ.WR_PDO_607A_L > 65535 THEN
            axisZ.WR_PDO_607A_L := axisZ.WR_PDO_607A_L - 65536;
            axisZ.WR_PDO_607A_H := axisZ.WR_PDO_607A_H + 1;
        END_IF;
    END_IF;
END_IF;
END_IF;

```

```

IF axisZ.index > lz_up THEN
    axisZ.flags.initCycle := FALSE;
    axisZ.ready := FALSE;
    axisZ.cycleCount := axisZ.cycleCount + 1;
    IF axisZ.cycleCount < axisZ.N THEN
        axisZ.ready := TRUE;
    ELSIF (axisZ.cycleCount = axisZ.N) AND axisZ.PR1 THEN
        axisZ.ready := TRUE;
    ELSE

```

```
axisZ.ready := FALSE;  
END_IF;  
END_IF;  
ELSE
```

```
IF axis.direction = -1 THEN
```

```
IF axis.index <= startLocal1 AND axis.index >= endLocal1 THEN  
axis.direction := +1;
```

```
ELSIF axis.index <= startLocal2 AND axis.index >= endLocal2  
THEN  
axis.direction := +1;
```

```
ELSIF axis.index <= startLocal3 AND axis.index >= endLocal3  
THEN  
axis.direction := +1;
```

```
ELSIF axis.index <= startLocal4 AND axis.index >= endLocal4  
THEN  
axis.direction := +1;
```

```
ELSIF axis.index <= startLocal5 AND axis.index >= endLocal5  
THEN  
axis.direction := +1;
```

```
END_IF;
```

```
END_IF;
```

```
IF axis.direction = +1 THEN
```

```
IF axis.index >= endLocal1 THEN  
axis.direction := -1;
```

```
ELSIF axis.index >= endLocal2 THEN
```

axis.direction := -1;

ELSIF axis.index >= endLocal3 THEN
axis.direction := -1;

ELSIF axis.index >= endLocal4 THEN
axis.direction := -1;

ELSIF axis.index >= endLocal5 THEN
axis.direction := -1;

END_IF;

END_IF;

```
IF axisZ.direction = +1 THEN
  IF axisZ.index < lz_up THEN
    axisZ.index := axisZ.index + 1;
  ELSIF axisZ.index = lz_up THEN
    IF lz_up > lz_low THEN
      axisZ.direction := -1;
      axisZ.index := lz_up - 1;
    ELSE
      axisZ.flags.initCycle := FALSE;
      axisZ.ready := FALSE;          axisZ.cycleCount := axisZ.cycleCount + 1;
      IF axisZ.cycleCount < axisZ.N THEN
        axisZ.ready := TRUE;
      ELSIF (axisZ.cycleCount = axisZ.N) AND axisZ.PR1 THEN
        axisZ.ready := TRUE;
      ELSE
        axisZ.ready := FALSE;
      END_IF;
    END_IF;
  END_IF;
ELSE
  IF axisZ.index > lz_low THEN
    axisZ.index := axisZ.index - 1;
  ELSIF axisZ.index = lz_low THEN
    axisZ.flags.initCycle := FALSE;
    axisZ.ready := FALSE;
    axisZ.cycleCount := axisZ.cycleCount + 1;
    IF axisZ.cycleCount < axisZ.N THEN
      axisZ.ready := TRUE;
      axisZ.direction := +1;
    ELSIF (axisZ.cycleCount = axisZ.N) AND axisZ.PR1 THEN
      axisZ.ready := TRUE;
      axisZ.direction := +1;
    ELSE
```



```
        axisZ.ready := FALSE;
    END_IF;
END_IF;
END_IF;
END_IF;
```

```
(* --- Após ajustar índices, limpar as flags que controlam o ciclo do ponto
(estas serão re-ativadas quando cada eixo carregar e escrever o novo setpoint) --- *)
```

```
axisX.flags.loadedTarget := FALSE;
axisX.flags.wroteSetpoint := FALSE;
axisX.flags.acknowledged := FALSE;
axisX.flags.clearedSetpoint := FALSE;
axisX.flags.readyToAdvance := FALSE;
axisX.flags.singlePoint := FALSE;
```

```
axisY.flags.loadedTarget := FALSE;
axisY.flags.wroteSetpoint := FALSE;
axisY.flags.acknowledged := FALSE;
axisY.flags.clearedSetpoint := FALSE;
axisY.flags.readyToAdvance := FALSE;
axisY.flags.singlePoint := FALSE;
```

```
axisZ.flags.loadedTarget := FALSE;
axisZ.flags.wroteSetpoint := FALSE;
axisZ.flags.acknowledged := FALSE;
axisZ.flags.clearedSetpoint := FALSE;
axisZ.flags.readyToAdvance := FALSE;
axisZ.flags.singlePoint := FALSE;
```

```
END_IF; (* fim readyToAdvanceGlobal *)
```

```
(* --- a máquina local do eixo (fora do bloco global) NÃO deve mais alterar index.
Assim evitamos incrementos duplicados. --- *)
```

```
END_FUNCTION
```

@

@

(*

CAMADA EXTERNA PID

Função **6064_LAG_TORQUE_FORÇA_VELOCIDADE**: ajusta velocidades dos eixos X, Y, Z para compensar atraso (lag) e aciona parada se erro persistir.

FUNCTION LAG_6064_TORQUE_FORCA_VELOCIDADE

A função LAG_6064_TORQUE_FORCA_VELOCIDADE implementa o sistema de monitoramento e compensação dinâmica de posição dos eixos utilizando o valor real de posição fornecido pelo objeto 6064 (Position Actual Value) do protocolo CiA 402. Seu objetivo é acompanhar continuamente o deslocamento executado pelos servoacionadores, calcular o erro instantâneo entre o movimento previsto e o movimento efetivamente realizado (lag), ajustar a velocidade dos eixos em tempo real e, quando necessário, executar uma parada segura.

Inicialmente, a função converte a posição recebida em pulsos para milímetros utilizando os parâmetros mecânicos do sistema, preservando a posição absoluta acumulada de cada eixo e determinando automaticamente o sentido de deslocamento (CW ou CCW). Também realiza o tratamento de estouro do contador de 16 bits, garantindo continuidade da medição mesmo após transições entre valores máximos e mínimos do encoder.

Durante cada ciclo do PLC é calculado o incremento teórico que deveria ser percorrido em função da velocidade instantânea, aceleração configurada e tempo de amostragem. Esse incremento é comparado com a distância restante do segmento atualmente em execução, limitando automaticamente o avanço quando ocorre o término de um segmento do buffer de interpolação.

Sempre que um segmento é concluído, a função troca automaticamente a velocidade de referência para a velocidade programada do próximo segmento, recalculando a aceleração correspondente e preservando a continuidade dinâmica do movimento. Dessa forma, a transição entre segmentos ocorre sem descontinuidades de velocidade.

Os incrementos previstos são armazenados em buffers internos independentes para cada eixo. Quando o encoder informa novo deslocamento real, esses incrementos são consumidos

e comparados com o deslocamento efetivamente realizado pelo servoacionador, produzindo continuamente os valores LagX, LagY e LagZ, expressos em milímetros.

A função também implementa um observador de parada (STP Observer). Quando o buffer indica velocidade programada igual a zero, qualquer deslocamento residual detectado pelo encoder é acumulado, permitindo medir a distância percorrida após o comando de parada. O maior deslocamento residual observado é registrado em STPMax, possibilitando avaliação do desempenho de frenagem do sistema.

Além do monitoramento absoluto, a rotina mantém o controle de posição relativa dos eixos, reiniciando automaticamente os acumuladores sempre que ocorre mudança do índice do buffer de movimento. Esse mecanismo permite acompanhar deslocamentos relativos independentemente da posição absoluta da máquina.

Após o cálculo dos erros individuais, a função identifica automaticamente o eixo que apresenta o maior erro absoluto de sincronismo. Esse eixo passa a definir o valor global LAGXYZ, utilizado como referência para todo o algoritmo de compensação.

Quando o erro ultrapassa os limites configurados (LIM_POS ou LIM_NEG), é ativado o controle automático de lag (CONTROL_LAGXYZ). Nesse modo, a função modifica continuamente a velocidade dos servoacionadores através do objeto 6081 (Profile Velocity). O ajuste é proporcional ao erro medido e ocorre de forma progressiva, utilizando um filtro de convergência que evita alterações bruscas de velocidade e reduz oscilações durante a correção.

Cada eixo possui um estado interno de velocidade que converge gradualmente para a velocidade nominal programada, enquanto fatores de ganho calculados em função da magnitude do erro aumentam ou reduzem temporariamente a velocidade conforme o servo esteja adiantado ou atrasado em relação ao movimento previsto.

Ao final do processamento, todas as velocidades calculadas são limitadas ao intervalo operacional permitido do sistema, preservando a integridade do acionamento.

Caso o algoritmo determine que a velocidade necessária para correção se aproxima de zero, a função interpreta essa condição como impossibilidade de manter o sincronismo de forma segura e aciona automaticamente o Quick Stop, escrevendo o respectivo bit no objeto 6040 (Controlword) dos três eixos simultaneamente.

Como resultado, a função constitui um sistema completo de observação e compensação de movimento, integrando monitoramento do encoder, cálculo de posição absoluta e relativa, previsão dinâmica de deslocamento, medição contínua de lag, compensação automática de velocidade e proteção por parada segura quando os limites operacionais são excedidos.

24. LAG_6064_TORQUE_FORCA_VELOCIDADE := BOOL

*)

```
TYPE tEventoLimite :  
  STRUCT  
    indexBuffer : UDINT;  
    LIM_POS    : REAL;  
    LIM_NEG    : REAL;  
  END_STRUCT  
END_TYPE
```

FUNCTION

LAG_6064_TORQUE_FORCA_VELOCIDADE := BOOL ;

VAR

fator : REAL;

incremento_consumido : REAL;

dt : REAL := 0.001;

eixo : INT;

omega_t : ARRAY[0..2] OF REAL;

omega_alvo : ARRAY[0..2] OF REAL;

a : ARRAY[0..2] OF REAL;

vel_rpm_alvo : ARRAY[0..2] OF REAL;

dt_evento : ARRAY[0..2] OF REAL;

dt_restante : ARRAY[0..2] OF REAL;

incremento_mm_ms_original : ARRAY[0..2] OF REAL;

incremento_mm_ms : ARRAY[0..2] OF REAL;

falta_mm : ARRAY[0..2] OF REAL;

ABSOLUTO : ARRAY[0..2] OF REAL;

LAGXYZ : REAL;

vel_state : ARRAY[0..2] OF REAL;

refVel : REAL;

k : REAL;

alpha : REAL := 0.000005;

bufferSTPX : BOOL;

bufferSTPY : BOOL;

bufferSTPZ : BOOL;

lagError : BOOL;

PULSOS_POR_VOLTAX : REAL := 65536.0;

MM_POR_VOLTA : REAL := 6.0;

PULSOS_POR_MM : REAL;

eventosLimite : ARRAY[0..19] OF tEventoLimite;

buffer_incremento_mm_ms : ARRAY[0..2] OF ARRAY[0..19] OF REAL;

eventoLimitIdx : INT := 0;

delta_raw : ARRAY[0..2] OF DINT;

pos_prev_raw : ARRAY[0..2] OF DINT;

indice_anterior : ARRAY[0..2] OF DINT;

buf_rd : ARRAY[0..2] OF INT;

buf_wr : ARRAY[0..2] OF INT;

incremento_total_mm : ARRAY[0..2] OF REAL;

omega_atual : ARRAY[0..2] OF REAL;

tempo_decorrido : ARRAY[0..2] OF REAL;

ramoNegativo : ARRAY[0..2] OF BOOL;

ramoPositivo : ARRAY[0..2] OF BOOL;

ajusteFinal : ARRAY[0..2] OF BOOL;

POS_REL_X, POS_REL_Y, POS_REL_Z : REAL;

velocidadeNominal : ARRAY[0..2] OF REAL;

END_VAR;

BEGIN

// Conversão de pulsos para mm

PULSOS_POR_MM := PULSOS_POR_VOLTAX / MM_POR_VOLTA;

```

// Verifica erro de following // em aberto
lagError := ((RD_PDO_6041_X AND FOLLOWING_ERROR)<>0)
           OR ((RD_PDO_6041_Y AND FOLLOWING_ERROR)<>0)
           OR ((RD_PDO_6041_Z AND FOLLOWING_ERROR)<>0);

// Calcula delta_raw e ABSOLUTO
FOR eixo := 0 TO 2 DO
    CASE eixo OF
        0: delta_raw[eixo] := RD_PDO_6064_X - pos_prev_raw[eixo];
        1: delta_raw[eixo] := RD_PDO_6064_Y - pos_prev_raw[eixo];
        2: delta_raw[eixo] := RD_PDO_6064_Z - pos_prev_raw[eixo];
    END_CASE;

FOR eixo := 0 TO 2 DO
    CASE eixo OF
        0:
            eventosLimite[eventoLimitIdx].indexBuffer := axisX.index;
            eventosLimite[eventoLimitIdx].LIM_POS := LIM_POS;
            eventosLimite[eventoLimitIdx].LIM_NEG := LIM_NEG;
        1:
            eventosLimite[eventoLimitIdx].indexBuffer := axisY.index;
            eventosLimite[eventoLimitIdx].LIM_POS := LIM_POS;
            eventosLimite[eventoLimitIdx].LIM_NEG := LIM_NEG;
        2:
            eventosLimite[eventoLimitIdx].indexBuffer := axisZ.index;
            eventosLimite[eventoLimitIdx].LIM_POS := LIM_POS;
            eventosLimite[eventoLimitIdx].LIM_NEG := LIM_NEG;
    END_CASE;

    eventoLimitIdx := eventoLimitIdx + 1;
    IF eventoLimitIdx > 19 THEN
        eventoLimitIdx := 0;
    END_IF;
END_FOR;

IF delta_raw[eixo] > 32767 THEN delta_raw[eixo] := delta_raw[eixo]-65536;
ELSIF delta_raw[eixo] < -32768 THEN delta_raw[eixo] := delta_raw[eixo]+65536; END_IF;

CASE eixo OF
    0: sentidoX_CW := delta_raw[eixo]>=0;
    1: sentidoY_CW := delta_raw[eixo]>=0;
    2: sentidoZ_CW := delta_raw[eixo]>=0;
END_CASE;

ABSOLUTO[eixo] := delta_raw[eixo]/PULSOS_POR_MM;

CASE eixo OF
    0: IF sentidoX_CW THEN ABSOLUTOX := ABSOLUTOX + ABSOLUTO[eixo] ELSE ABSOLUTOX :=
ABSOLUTOX - ABSOLUTO[eixo]; END_IF;
    1: IF sentidoY_CW THEN ABSOLUTOY := ABSOLUTOY + ABSOLUTO[eixo] ELSE ABSOLUTOY :=
ABSOLUTOY - ABSOLUTO[eixo]; END_IF;
    2: IF sentidoZ_CW THEN ABSOLUTOZ := ABSOLUTOZ + ABSOLUTO[eixo] ELSE ABSOLUTOZ :=
ABSOLUTOZ - ABSOLUTO[eixo]; END_IF;
END_CASE;
END_FOR;

// Cálculo de incrementos e atualização de índices

```

```

FOR eixo := 0 TO 2 DO
    omega_t[eixo] := omega_atual[eixo] + a[eixo]*tempo_decorrido[eixo];

CASE eixo OF

    0: IF bufferX[axisX.index] > 0.45 THEN
        falta_mm[0] := bufferX[axisX.index];
    ELSE
        falta_mm[0] := axisX.bufferVel[axisX.index] * 0.00015;
    END_IF;

    1: IF bufferY[axisY.index] > 0.45 THEN
        falta_mm[1] := bufferY[axisY.index];
    ELSE
        falta_mm[1] := axisY.bufferVel[axisY.index] * 0.00015;
    END_IF;

    2: IF bufferZ[axisZ.index] > 0.45 THEN
        falta_mm[2] := bufferZ[axisZ.index];
    ELSE
        falta_mm[2] := axisZ.bufferVel[axisZ.index] * 0.00015;
    END_IF;

END_CASE;

incremento_mm_ms_original[eixo] := omega_t[eixo]*dt;
incremento_mm_ms[eixo] := incremento_mm_ms_original[eixo];

IF incremento_mm_ms[eixo] >= falta_mm[eixo] THEN
    incremento_mm_ms[eixo] := falta_mm[eixo];
    incremento_total_mm[eixo] := incremento_total_mm[eixo] + incremento_mm_ms[eixo];
    dt_evento[eixo] := incremento_mm_ms[eixo]/omega_t[eixo];
    dt_restante[eixo] := dt - dt_evento[eixo];

    IF ((eixo=0 AND axisX.index<=axisX.lastIndex) OR
        (eixo=1 AND axisY.index<=axisY.lastIndex) OR
        (eixo=2 AND axisZ.index<=axisZ.lastIndex)) THEN

        CASE eixo OF

            0: vel_rpm_alvo[eixo] := axisX.bufferVel[axisX.index];

            1: vel_rpm_alvo[eixo] := axisY.bufferVel[axisY.index];

            2: vel_rpm_alvo[eixo] := axisZ.bufferVel[axisZ.index];

        END_CASE;

        omega_alvo[eixo] := vel_rpm_alvo[eixo]*0.104719755;

CASE
eixo OF

    0: a[eixo] :=
        (omega_alvo[eixo]-omega_t[eixo]) /
        (WR_PDO_6083_X * 0.001);

    1: a[eixo] :=

```

```

        (omega_alvo[eixo]-omega_t[eixo]) /
        (WR_PDO_6083_Y * 0.001);

2: a[eixo] :=
        (omega_alvo[eixo]-omega_t[eixo]) /
        (WR_PDO_6083_Z * 0.001);

END_CASE;

omega_atual[eixo] := omega_t[eixo];
tempo_decorrido[eixo] := 0;
END_IF;

tempo_decorrido[eixo] := tempo_decorrido[eixo] + dt_restante[eixo];
ELSE
    incremento_total_mm[eixo] := incremento_total_mm[eixo] + incremento_mm_ms[eixo];
    tempo_decorrido[eixo] := tempo_decorrido[eixo] + dt;
END_IF;

buffer_incremento_mm_ms[eixo][buf_wr[eixo]] := incremento_mm_ms[eixo];
buf_wr[eixo] := buf_wr[eixo] + 1;
END_FOR;

```

// Consumo do buffer e cálculo de lag

```

IF (ABS(delta_raw[0]) <> 0) OR
   (ABS(delta_raw[1]) <> 0) OR
   (ABS(delta_raw[2]) <> 0) THEN

```

```

FOR eixo := 0 TO 2 DO
    IF buf_rd[eixo] < buf_wr[eixo] THEN
        incremento_consumido := buffer_incremento_mm_ms[eixo][buf_rd[eixo]];
        CASE eixo OF
            0: LagX := ABSOLUTO[0] - incremento_consumido;
            1: LagY := ABSOLUTO[1] - incremento_consumido;
            2: LagZ := ABSOLUTO[2] - incremento_consumido;
        END_CASE;
        buf_rd[eixo] := buf_rd[eixo]+1;
    END_IF;

    IF buf_rd[eixo]=buf_wr[eixo] THEN
        buf_rd[eixo] := 0; buf_wr[eixo] := 0;
    END_IF;
END_FOR;
END_IF;

```

```

(* BLOCO INDEPENDENTE - STP OBSERVER *)
bufferSTPX := (axisX.bufferVel[axisX.index] = 0);
bufferSTPY := (axisY.bufferVel[axisY.index] = 0);
bufferSTPZ := (axisZ.bufferVel[axisZ.index] = 0);

```

```

IF bufferSTPX AND (delta_raw[0] <> 0) THEN

```

```
    STPX := STPX + (delta_raw[0] / PULSOS_POR_MM);  
END_IF;
```

```
IF bufferSTPY AND (delta_raw[1] <> 0) THEN  
    STPY := STPY + (delta_raw[1] / PULSOS_POR_MM);  
END_IF;
```

```
IF bufferSTPZ AND (delta_raw[2] <> 0) THEN  
    STPZ := STPZ + (delta_raw[2] / PULSOS_POR_MM);  
END_IF;
```

```
IF ABS(STPX) > STPMax THEN STPMax := ABS(STPX); END_IF;  
IF ABS(STPY) > STPMax THEN STPMax := ABS(STPY); END_IF;  
IF ABS(STPZ) > STPMax THEN STPMax := ABS(STPZ); END_IF;
```

```
IF (bufferSTPX AND delta_raw[0] <> 0) OR  
   (bufferSTPY AND delta_raw[1] <> 0) OR  
   (bufferSTPZ AND delta_raw[2] <> 0) THEN
```

```
    CONTROL_LAGXYZ := FALSE;
```

```
END_IF;
```

```
// Atualiza posição anterior
```

```
FOR eixo := 0 TO 2 DO
```

```
    CASE eixo OF
```

```
        0: pos_prev_raw[eixo] := RD_PDO_6064_X;
```

```
        1: pos_prev_raw[eixo] := RD_PDO_6064_Y;
```

```
        2: pos_prev_raw[eixo] := RD_PDO_6064_Z;
```

```
    END_CASE;
```

```
END_FOR;
```

```
// Controle de posição relativa
```

```
FOR eixo := 0 TO 2 DO
```

```
    IF axisX.index<>indice_anterior[0] OR axisY.index<>indice_anterior[1] OR axisZ.index<>indice_anterior[2]  
    THEN
```

```
        CASE eixo OF
```

```
            0: indice_anterior[eixo] := axisX.index;
```

```
            1: indice_anterior[eixo] := axisY.index;
```

```
            2: indice_anterior[eixo] := axisZ.index;
```

```
        END_CASE;
```

```
        CASE eixo OF
```

```
            0: pos_rel_X_mm := 0; POS_REL_X := 0;
```

```
            1: pos_rel_Y_mm := 0; POS_REL_Y := 0;
```

```
            2: pos_rel_Z_mm := 0; POS_REL_Z := 0;
```

```
        END_CASE;
```

```
        CASE eixo OF
```

```
            0: IF RelativeX THEN pos_rel_X_mm := pos_rel_X_mm + ABSOLUTO[eixo]; POS_REL_X :=  
ABSOLUTO[eixo]; END_IF;
```

```
            1: IF RelativeY THEN pos_rel_Y_mm := pos_rel_Y_mm + ABSOLUTO[eixo]; POS_REL_Y :=  
ABSOLUTO[eixo]; END_IF;
```



```

2: IF RelativeZ THEN pos_rel_Z_mm := pos_rel_Z_mm + ABSOLUTO[eixo]; POS_REL_Z :=
ABSOLUTO[eixo]; END_IF;
END_CASE;
END_IF;
END_FOR;

```

```

// Determina se controle de lag é necessário
CONTROL_LAGXYZ := FALSE;

```

```

IF ABS(LagX) >= ABS(LagY) AND ABS(LagX) >= ABS(LagZ) THEN
  IF LagX > LIM_POS OR LagX < LIM_NEG THEN
    CONTROL_LAGXYZ := TRUE;
  END_IF;
  LAGXYZ := LagX;

```

```

ELSIF ABS(LagY) >= ABS(LagZ) THEN
  IF LagY > LIM_POS OR LagY < LIM_NEG THEN
    CONTROL_LAGXYZ := TRUE;
  END_IF;
  LAGXYZ := LagY;

```

```

ELSE
  IF LagZ > LIM_POS OR LagZ < LIM_NEG THEN
    CONTROL_LAGXYZ := TRUE;
  END_IF;
  LAGXYZ := LagZ;
END_IF;

```

```

// Atualiza velocidade nominal
velocidadeNominal[0] := axisX.bufferVel[axisX.index];

```

```

velocidadeNominal[1] := axisY.bufferVel[axisY.index];

```

```

velocidadeNominal[2] := axisZ.bufferVel[axisZ.index];

```

```

// Ajusta velocidades em função do lag

```

```

IF CONTROL_LAGXYZ THEN

```

```

  FOR eixo := 0 TO 2 DO

```

```

    CASE eixo OF

```

```

      0:

```

```

        IF LagX<LIM_NEG AND NOT ramoNegativo[eixo] THEN ramoNegativo[eixo]:=TRUE;
        ramoPositivo[eixo]:=FALSE;

```

```

        ELSIF LagX>LIM_POS AND NOT ramoPositivo[eixo] THEN ramoPositivo[eixo]:=TRUE;
        ramoNegativo[eixo]:=FALSE; END_IF;

```

```

      1:

```

```

        IF LagY<LIM_NEG AND NOT ramoNegativo[eixo] THEN ramoNegativo[eixo]:=TRUE;
        ramoPositivo[eixo]:=FALSE;

```

```

        ELSIF LagY>LIM_POS AND NOT ramoPositivo[eixo] THEN ramoPositivo[eixo]:=TRUE;
        ramoNegativo[eixo]:=FALSE; END_IF;

```

```

      2:

```

```

        IF LagZ<LIM_NEG AND NOT ramoNegativo[eixo] THEN ramoNegativo[eixo]:=TRUE;
        ramoPositivo[eixo]:=FALSE;

```

```
        ELSIF LagZ>LIM_POS AND NOT ramoPositivo[eixo] THEN ramoPositivo[eixo]:=TRUE;
ramoNegativo[eixo]:=FALSE; END_IF;
```

```
END_CASE;
```

```
IF RPM_max = 0 THEN
```

```
    k := 0.0;
```

```
ELSE
```

```
    k := ABS(LAGXYZ) * 10000.0 / RPM_max;
```

```
END_IF;
```

```
CASE eixo OF
```

```
0: refVel := axisX.bufferVel[axisX.index];
```

```
1: refVel := axisY.bufferVel[axisY.index];
```

```
2: refVel := axisZ.bufferVel[axisZ.index];
```

```
END_CASE;
```

```
vel_state[eixo] := vel_state[eixo] + k * (refVel - vel_state[eixo]);
```

```
IF vel_state[eixo] < 0.9 * refVel THEN vel_state[eixo] := vel_state[eixo] + alpha * (refVel - vel_state[eixo]);
```

```
END_IF;
```

```
CASE eixo OF
```

```
0:
```

```
axisX.WR_PDO_6081 := vel_state[0];
```

```
    IF LAGXYZ < 0 THEN
```

```
        axisX.WR_PDO_6081 := axisX.WR_PDO_6081 * (1 + k);
```

```
    ELSE
```

```
        axisX.WR_PDO_6081 := axisX.WR_PDO_6081 * (1 - k);
```

```
    END_IF;
```

```
1:
```

```
axisY.WR_PDO_6081 := vel_state[1];
```

```
IF LAGXYZ < 0 THEN
```

```
    axisY.WR_PDO_6081 := axisY.WR_PDO_6081 * (1 + k);
```

```
ELSE
```

```
    axisY.WR_PDO_6081 := axisY.WR_PDO_6081 * (1 - k);
```

```
END_IF;
```

```
2:
```

```
axisZ.WR_PDO_6081 := vel_state[2];
```

```
IF LAGXYZ < 0 THEN
```

```
    axisZ.WR_PDO_6081 := axisZ.WR_PDO_6081 * (1 - k);
```

```
ELSE
```

```
    axisZ.WR_PDO_6081 := axisZ.WR_PDO_6081 * (1 + k);
```

```
END_IF;
```

```
END_CASE;
```

```
// Marca ajustes finais
```

```
FOR eixo := 0 TO 2 DO
```

```
    CASE eixo OF
```

```
        0: IF axisX.WR_PDO_6081 <= 0.01 THEN ajusteFinal[0] := TRUE; END_IF;
```

```
        1: IF axisY.WR_PDO_6081 <= 0.01 THEN ajusteFinal[1] := TRUE; END_IF;
```

```

2: IF axisZ.WR_PDO_6081 <= 0.01 THEN ajusteFinal[2] := TRUE; END_IF;

END_CASE;

END_FOR;

// Limita WR_PDO_6081 entre 0 e 3000
axisX.WR_PDO_6081 := MAX(0, MIN(axisX.WR_PDO_6081, 3000));
axisY.WR_PDO_6081 := MAX(0, MIN(axisY.WR_PDO_6081, 3000));
axisZ.WR_PDO_6081 := MAX(0, MIN(axisZ.WR_PDO_6081, 3000));

VELX := axisX.WR_PDO_6081;
VELY := axisY.WR_PDO_6081;
VELZ := axisZ.WR_PDO_6081;

VELX0 := axisX.bufferVel[axisX.index];
VELY0 := axisY.bufferVel[axisY.index];
VELZ0 := axisZ.bufferVel[axisZ.index];

// Verifica QUICK_STOP
QUICK_STOP := ajusteFinal[0] OR ajusteFinal[1] OR ajusteFinal[2];

IF QUICK_STOP THEN
    WR_PDO_6040_X := WR_PDO_6040_X OR 16#004;
    WR_PDO_6040_Y := WR_PDO_6040_Y OR 16#004;
    WR_PDO_6040_Z := WR_PDO_6040_Z OR 16#004;
END_IF;

// Retorna resultado da função
LAG_6064_TORQUE_FORCA_VELOCIDADE := QUICK_STOP;
END_FUNCTION;

```

(*

PROGRAM Main

25. PROGRAM Main

O programa principal (Main) constitui o nível superior de coordenação da aplicação, sendo responsável pela execução sequencial dos módulos que compõem o sistema de controle numérico desenvolvido neste trabalho. Sua função consiste em integrar as rotinas de interface com o operador, interpretação dos programas, geração de trajetórias, comunicação industrial e acionamento dos servoacionamentos, estabelecendo o fluxo lógico de execução do controlador.

Inicialmente, são processados os comandos provenientes do painel físico por meio da função ControlPushButtons, responsável pela leitura dos dispositivos de entrada e pela atualização dos objetos de controle dos servoacionadores. Em seguida, o programa seleciona a origem dos comandos de operação manual, permitindo que o controle dos movimentos seja realizado tanto pela Interface Homem-Máquina (HMI) quanto pelos botões físicos instalados no painel da máquina.

Após a seleção da origem dos comandos, a função JOG_X_Y_Z_PULSADO executa o gerenciamento dos movimentos manuais, produzindo os buffers de deslocamento utilizados

pelos servoacionamentos durante a operação em modo manual, incluindo o procedimento de homing e os deslocamentos incrementais dos eixos.

Na sequência, o programa executa o Interpretador GRACIA_code, responsável pela interpretação dos programas CNC previamente carregados, convertendo suas instruções em estruturas de dados organizadas para processamento. As trajetórias geradas são então encaminhadas às funções de interpolação, que calculam os pontos intermediários necessários para a execução dos movimentos lineares, circulares, biplanares, cônicos e demais geometrias implementadas pelo sistema.

Quando os buffers de trajetória encontram-se disponíveis para execução, o programa aciona a rotina DRIVE_INPUT_OUTPUT para cada um dos servoacionadores dos eixos X, Y e Z, realizando a atualização dos objetos de processo utilizados na comunicação em tempo real. Paralelamente, são executadas as rotinas de monitoramento responsáveis pela aquisição de posição, torque, velocidade e erro de seguimento, permitindo o acompanhamento contínuo do comportamento dos servoacionamentos.

Determinadas funções de usinagem requerem etapas adicionais de processamento. Nesses casos, o programa principal identifica automaticamente a função atualmente em execução e aciona os algoritmos específicos necessários, como o cálculo dos perfis de aceleração e desaceleração empregados em determinadas estratégias de movimentação.

Por fim, o programa realiza as tarefas de comunicação com a Interface Homem-Máquina, executando simultaneamente a recepção de comandos, a transmissão das variáveis de supervisão e o carregamento de novos programas para o interpretador. Essa organização estabelece uma arquitetura modular, na qual cada função desempenha uma responsabilidade específica, permitindo que a comunicação, o processamento geométrico e o controle dos servoacionamentos operem de forma coordenada e independente, preservando o comportamento determinístico exigido pelas aplicações de controle numérico computadorizado.*)

PROGRAM Main

ControlPushButtons();

IF HMI_ativo THEN

JogAtivo_X := JogX;

JogAtivo_Y := JogY;

JogAtivo_Z := JogZ;

JogAtivo_XN := JogNX;

JogAtivo_YN := JogNY;

JogAtivo_ZN := JogNZ;

HomingBtn := HOMING;

ELSE

JogAtivo_X := BtnX_JOG;

JogAtivo_Y := BtnY_JOG;

JogAtivo_Z := BtnZ_JOG;

JogAtivo_XN := BtnX_JOGN;

JogAtivo_YN := BtnY_JOGN;

JogAtivo_ZN := BtnZ_JOGN;

HomingBtn := BtnXYZ_HOMING;

END_IF;

JOG_X_Y_Z_PULSADO(
 BtnX_JOG := JogAtivo_X,
 BtnY_JOG := JogAtivo_Y,
 BtnZ_JOG := JogAtivo_Z,
 BtnX_JOGN := JogAtivo_XN,
 BtnY_JOGN := JogAtivo_YN,
 BtnZ_JOGN := JogAtivo_ZN,
 BtnXYZ_HOMING := HomingBtn
);

Interpretador_GRACIA_CODE(ProgramaGCODE);

INTERPOLACAO_LINEAR_CIRCULAR();
INTERPOLACAO_BI_PLANAR();
INTERPOLACAO_BI_PLANAR_PUMPKIN();
INTERPOLACAO_CONICA();

IF bufferFilled OR bufferFilling THEN
 DRIVE_INPUT_OUTPUT(axis := axisX);
 DRIVE_INPUT_OUTPUT(axis := axisY);
 DRIVE_INPUT_OUTPUT(axis := axisZ);
 LAG_6064_TORQUE_FORCA_VELOCIDADE();
END_IF;

IF FunAtivaNome = 'F2' THEN
 CalcAccelDecelFromProfile();
END_IF;

HMI_USB_Receiver_Transmitter(
 tx_step := tx_step,
 rx_buffer := rx_buffer,
 Enable := TRUE,
 LastLineReceived => last_line
);

Loader_ProgGCODE(
 rxString := rxString,
 rxValid := rxValid,
 rxDone := rxDone,
 ProgramaGCODE := ProgramaGCODE,
 IndexWrite := IndexWrite,
 rxAck := rxAck,
 rxDoneAck := rxDoneAck
);

END_PROGRAM

(*

Exemplo de Exposição do Ganho Proporcional (Kp) no Contexto PID

Parâmetros PID no SCA06

1. Controle de Posição (Position Controller)

- **P00159 – Proportional Gain of the Position Controller (Kp)**
 - Faixa ajustável: 0 a 32767
 - Valor de fábrica: **50**

2. Controle de Velocidade (Speed Controller)

- **P00161 – Proportional Gain of the Speed Controller (Kp)**
 - Faixa: 0 a 3276.7
 - Valor de fábrica: **25.0**
- **P00162 – Integral Gain of the Speed Controller (Ki)**
 - Faixa: 0 a 327.67
 - Valor de fábrica: **1.50**
- **P00163 – Derivative Gain of the Speed Controller (Kd)**
 - Faixa: 0 a 32767
 - Valor de fábrica: **0**

3. Controle de Corrente (Current PID)

P00385 – MOTOR - PARAMETRIZAÇÃO AUTOMÁTICA ABAIXO INDICADO :

- **P00392 – Proportional Gain of the Iq Current PID (Kp)**
 - Valor de fábrica: **1343**
- **P00393 – Integral Gain of the Iq Current PID (Ki)**
 - Valor de fábrica: **75**
- **P00395 – Proportional Gain of the Id Current PID (Kp)**
 - Valor de fábrica: **1959**
- **P00396 – Integral Gain of the Id Current PID (Ki)**
 - Valor de fábrica: **597**
- **P00398 – Phase Compensation with wr**
 - Compensa atraso de fase devido à velocidade
 - Valor de fábrica: **8192**

Panorama geral dos parâmetros

Tipo de Controle	Parâmetros Ajustáveis	Faixa e Valor de Fábrica
Posição	P00159 (Kp)	0 a 32767 (factory = 50)
Velocidade	P00161 (Kp), P00162 (Ki), P00163 (Kd)	0–3276.7 / 0–327.67 / 0–32767
Corrente (Torque)	P00392, P00393, P00395, P00396, P00398	Valores conforme acima/nos itens

LAG_error (Kp)max 3000 rpm 3N.m 5 kg

FREQUENCIA DO LAÇO DE CORRENTE DO DRIVE .

Kp máx _ 32767 de 32767 163 graus/18ms _ 2.71 mm

Lag error_ 7290 de 16383_ 2.71 mm
Fuso 6 mm
55 HZ

Kp_2730 de 32767 18.3graus/2ms_0.3 mm
Lag error_ 819 de 16383_0.3mm
Fuso 6 mm
500 HZ

Kp_306 de 32767 2 graus/0.225ms_0.0337 mm
Lag error_ 92 de 16383_0.0337 mm
Fuso 6 mm
4.444 HZ

Kp_226 de 32767 1.5graus/0.166ms_0.025 mm
Lag error_68 de 16383_0.025mm
Fuso 6 mm
6.000 HZ

Kp_34 de 32767 0.224graus/0.025 ms_0.0037 mm
Lag error_ 10 de 16383_0.0037 mm
Fuso 6 mm
40.000 HZ

@

